



Arm[®] CoreLink[™] MMU-720AE System Memory Management Unit

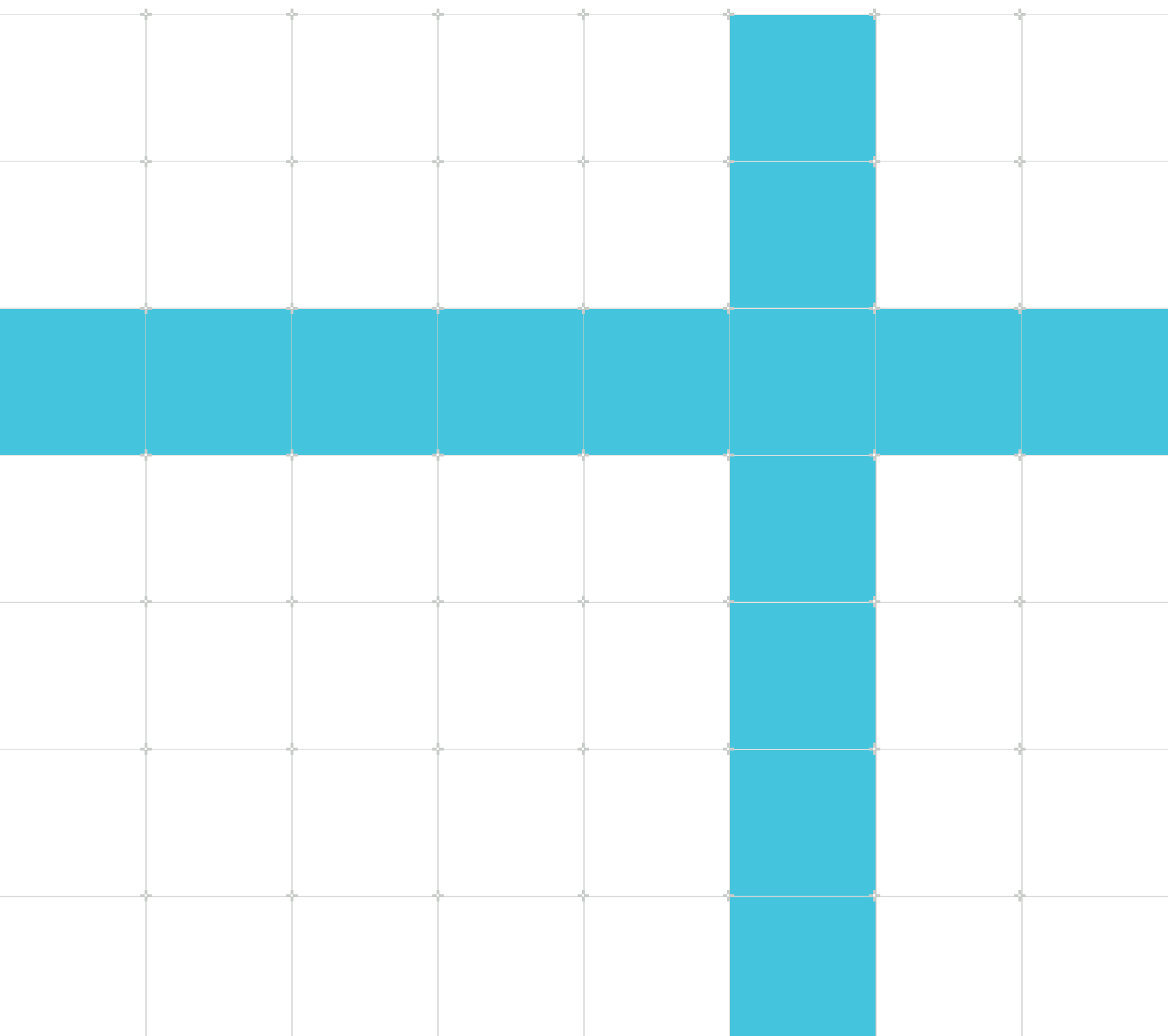
Revision r0p2

Technical Reference Manual

Non-Confidential

Issue 04

Copyright © 2024–2025 Arm Limited (or its affiliates). 109745_0002_04_en
All rights reserved.



Arm® CoreLink™ MMU-720AE System Memory Management Unit Technical Reference Manual

This document is Non-Confidential.

Copyright © 2024–2025 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.

Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary Notice](#) found at the end of this document.

This document (109745_0002_04_en) was issued on 2025-08-08. There might be a later issue at <https://developer.arm.com/documentation/109745>

The product revision is r0p2.

See also: [Proprietary Notice](#) | [Product and document information](#) | [Useful resources](#)

Start reading

If you prefer, you can skip to [the start of the content](#).

Intended audience

This book is written for system designers, system integrators, and programmers who are designing or programming a System-on-Chip (SoC) that uses the CoreLink™ MMU-720AE System Memory Management Unit.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

We believe that this document contains no offensive language. To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on <https://support.developer.arm.com>.

To provide feedback on the document, fill the following survey: <https://developer.arm.com/documentation-feedback-survey>.

Contents

1. About the MMU-720AE System Memory Management Unit.....	12
1.1 Compliance.....	13
1.2 Features.....	14
1.3 TCU and TBU common interfaces.....	17
1.4 Configurable options.....	17
1.5 Product documentation and design flow.....	18
1.5.1 Documentation.....	18
1.5.2 Design flow.....	19
2. Functional description of MMU-720AE Components.....	21
2.1 Translation Control Unit.....	23
2.2 Translation Buffer Unit.....	26
2.3 Distributed Translation Interface interconnect.....	30
2.4 Interfaces.....	32
2.4.1 TCU interfaces.....	32
2.4.2 TBU interfaces.....	36
2.4.3 DTI interconnect interfaces.....	42
2.5 Operation.....	45
2.5.1 Distributed Translation Interface.....	45
2.5.2 Performance Monitoring Unit.....	47
2.5.3 Multiple LTI interface TBU.....	57
2.5.4 Main TLB direct indexing and main TLB direct partitioning.....	57
2.5.5 RAS implementation.....	59
2.5.6 Quality of Service.....	65
2.5.7 Distributed Virtual Memory messages.....	66
2.5.8 TCU transaction handling.....	67
2.5.9 TCU prefetch.....	69
2.5.10 Error responses.....	71
2.5.11 Conversion between ACE-Lite and Armv8 attributes.....	72
2.5.12 AXI USER bits that the SMMU TBU TBM defines.....	75
2.5.13 NoStreamID.....	75
2.6 Constraints and limitations of use.....	76

2.6.1 SMMUv3 implementation.....	76
2.6.2 AMBA implementation.....	82
2.6.3 Local Translation Interface implementation.....	92
2.6.4 MPAM implementation.....	93
2.7 Configuration parameters and methodology.....	103
2.7.1 TCU I/O and buffer configuration parameters.....	103
2.7.2 TCU debug configuration parameters.....	105
2.7.3 Common ACE-Lite and LTI TBU configuration parameters.....	105
2.7.4 ACE-Lite TBU register slice configuration parameters.....	106
2.7.5 ACE-Lite TBU I/O configuration parameters.....	107
2.7.6 LTI TBU configuration parameters.....	108
2.7.7 Common TBU debug configuration parameters.....	108
2.8 Debug capability.....	109
3. Fault detection and control mechanisms.....	110
3.1 Interface protection.....	110
3.2 Logic duplication.....	112
3.2.1 DTI duplication.....	112
3.3 RAM protection.....	113
3.4 Error reporting.....	114
3.4.1 Access the FMU.....	114
3.4.2 Control which Safety Mechanisms report errors to the FMU.....	114
3.4.3 FMU interrupt handling.....	115
3.4.4 FMU error injection.....	117
3.4.5 FMU warm reset.....	117
3.5 FuSa specific configuration parameter.....	118
4. Programmers model for MMU-720AE.....	119
4.1 Clearing ERRSTATUS registers.....	120
4.1.1 Realm Management Extension Register Principles.....	121
4.2 SMMU architectural registers.....	121
4.3 SMMU memory map.....	127
4.3.1 TCU memory map.....	128
4.3.2 TBU memory map.....	130
4.4 SMMU registers summary.....	131
4.4.1 TCU identification registers summary.....	131
4.4.2 TCU and TBU PMU identification registers summary.....	132

4.4.3 TCU Reliability, Availability, and Serviceability registers summary.....	132
4.4.4 TCU microarchitectural registers summary.....	133
4.4.5 TCU system discovery registers summary.....	134
4.4.6 TCU AUX registers summary.....	135
4.4.7 TBU identification registers summary.....	136
4.4.8 TBU Reliability, Availability, and Serviceability registers summary.....	137
4.4.9 TBU microarchitectural registers summary.....	137
4.4.10 TBU system discovery registers summary.....	138
4.4.11 TBU integration registers summary.....	139
4.5 TCU component and peripheral ID registers.....	139
4.6 TCU PMU registers.....	140
4.6.1 Registers.....	140
4.6.2 Events.....	141
4.6.3 SMMU_PMCG_CFGR.....	142
4.6.4 SMMU_PMCG_CEID{0-1} registers.....	142
4.6.5 PMU ID registers.....	142
4.7 TCU microarchitectural registers.....	143
4.7.1 TCU_CTRL register.....	146
4.7.2 TCU_QOS register.....	147
4.7.3 TCU_CFG register.....	149
4.7.4 TCU_STATUS register.....	149
4.7.5 TCU_SCR register.....	150
4.7.6 TCU_CTRL2 register.....	152
4.7.7 TCU_ROOT_CTRL register.....	153
4.7.8 TCU_ROOT_CTRL2 register.....	154
4.7.9 TCU_RCR register.....	155
4.7.10 TCU_NODE_CTRLn register.....	157
4.7.11 TCU_NODE_STATUSn register.....	160
4.7.12 TCU_WC_SxLy_CMAX registers.....	161
4.7.13 TCU_AGW_GC_Lx_CMAX registers.....	162
4.7.14 TCU_DGW_GC_Lx_CMAX registers.....	162
4.8 TCU RAS registers.....	163
4.8.1 TCU_ERRFR register.....	163
4.8.2 TCU_ERRCTLR register.....	164
4.8.3 TCU_ERRSTATUS register.....	165
4.8.4 TCU_ERRGEN register.....	170

4.9 TCU system discovery registers.....	175
4.9.1 TCU_SYSDISC0 system discovery register.....	176
4.9.2 TCU_SYSDISC1 system discovery register.....	177
4.9.3 TCU_SYSDISC2 system discovery register.....	178
4.9.4 TCU_SYSDISC3 system discovery register.....	179
4.9.5 TCU_SYSDISC4 system discovery register.....	180
4.9.6 TCU_SYSDISC5 system discovery register.....	181
4.9.7 TCU_SYSDISC6 system discovery register.....	182
4.9.8 TCU_SYSDISC7 system discovery register.....	183
4.9.9 TCU_SYSDISC8 system discovery register.....	184
4.9.10 TCU_SYSDISC9 system discovery register.....	184
4.9.11 TCU_SYSDISC10 system discovery register.....	185
4.9.12 TCU_SYSDISC11 system discovery register.....	186
4.9.13 TCU_SYSDISC12 system discovery register.....	187
4.9.14 TCU_SYSDISC13 system discovery register.....	188
4.9.15 TCU_SYSDISC14 system discovery register.....	189
4.9.16 TCU_SYSDISC15 system discovery register.....	190
4.9.17 TCU_SYSDISC16 system discovery register.....	191
4.9.18 TCU_SYSDISC17 system discovery register.....	192
4.9.19 TCU_SYSDISC18 system discovery register.....	193
4.9.20 TCU_SYSDISC19 system discovery register.....	194
4.9.21 TCU_SYSDISC20 system discovery register.....	195
4.9.22 TCU_SYSDISC21 system discovery register.....	196
4.9.23 TCU_SYSDISC22 system discovery register.....	197
4.9.24 TCU_SYSDISC23 system discovery register.....	198
4.9.25 TCU_SYSDISC24 system discovery register.....	199
4.9.26 TCU_SYSDISC25 system discovery register.....	199
4.9.27 TCU_SYSDISC26 system discovery register.....	200
4.9.28 TCU_SYSDISC27 system discovery register.....	201
4.9.29 TCU_SYSDISC28 system discovery register.....	202
4.9.30 TCU_SYSDISC29 system discovery register.....	203
4.9.31 TCU_SYSDISC30 system discovery register.....	204
4.9.32 TCU_SYSDISC31 System Discovery Register.....	205
4.9.33 TCU_SYSDISC32 System Discovery Register.....	206
4.9.34 TCU_SYSDISC33 system discovery register.....	206
4.9.35 TCU_SYSDISC34 System Discovery Register.....	207

4.9.36 TCU_SYSDISC35 system discovery register.....	208
4.9.37 TCU_SYSDISC36 system discovery register.....	209
4.9.38 TCU_SYSDISC37 system discovery register.....	210
4.10 TCU_CTRL_AUX<n> registers.....	211
4.11 TCU integration registers.....	212
4.11.1 ITEN register for the TCU.....	212
4.12 TCU PIU integration registers.....	213
4.12.1 ITOP_PIU register for the TCU Programmer Interface Unit.....	213
4.12.2 ITIN_PIU register for the TCU Programmer Interface Unit.....	218
4.13 TCU TMU integration registers.....	219
4.13.1 ITOP_TMU register for the TCU Translation Management Unit.....	219
4.13.2 ITIN_TMU register for the TCU Translation Management Unit.....	220
4.14 TBU component and peripheral ID registers.....	221
4.15 TBU PMU registers.....	222
4.15.1 Registers.....	222
4.15.2 Events.....	222
4.15.3 SMMU_PMCGR_CFGR.....	223
4.15.4 SMMU_PMCGR_CEID{0-1} registers.....	223
4.15.5 PMU ID registers.....	223
4.16 TBU microarchitectural registers.....	224
4.16.1 TBU_CTRL register.....	225
4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register.....	226
4.16.3 TBU_SCR register.....	229
4.16.4 TBU_ROOT_CTRL register.....	231
4.16.5 TBU_RCR register.....	232
4.17 TBU RAS registers.....	233
4.17.1 TBU_ERRFR register.....	234
4.17.2 TBU_ERRCTRL register.....	235
4.17.3 TBU_ERRSTATUS register.....	236
4.17.4 TBU_ERRGEN register.....	239
4.18 TBU system discovery registers.....	243
4.18.1 TBU_SYSDISC0 system discovery register.....	243
4.18.2 TBU_SYSDISC1 system discovery register.....	244
4.18.3 TBU_SYSDISC2 system discovery register.....	245
4.18.4 TBU_SYSDISC3 system discovery register.....	246
4.18.5 TBU_SYSDISC4 system discovery register.....	247

4.18.6 TBU_SYSDISC5 system discovery register.....	248
4.18.7 TBU_SYSDISC6 system discovery register.....	249
4.18.8 TBU_SYSDISC7 system discovery register.....	250
4.18.9 TBU_SYSDISC8 system discovery register.....	251
4.18.10 TBU_SYSDISC9 system discovery register.....	252
4.18.11 TBU_SYSDISC10 system discovery register.....	253
4.18.12 TBU_SYSDISC11 system discovery register.....	254
4.18.13 TBU_SYSDISC12 system discovery register.....	255
4.18.14 TBU_SYSDISC13 system discovery register.....	256
4.18.15 TBU_SYSDISC14 system discovery register.....	257
4.18.16 TBU_SYSDISC15 system discovery register.....	258
4.18.17 TBU_SYSDISC16 system discovery register.....	259
4.18.18 TBU_SYSDISC17 system discovery register.....	260
4.18.19 TBU_SYSDISC18 system discovery register.....	261
4.18.20 TBU_SYSDISC19 system discovery register.....	262
4.18.21 TBU_SYSDISC20 system discovery register.....	263
4.18.22 TBU_SYSDISC21 system discovery register.....	264
4.18.23 TBU_SYSDISC22 system discovery register.....	265
4.19 TBU integration registers.....	266
4.19.1 ITEN register for the TBU.....	266
4.19.2 ITOP register for the TBU.....	267
4.19.3 ITIN register for the TBU.....	270
4.20 TCU FMU registers summary.....	271
4.20.1 TCU_FMU_ERRFR register.....	272
4.20.2 TCU_FMU_ERRCTLR register.....	273
4.20.3 TCU_FMU_ERRSTATUS register.....	274
4.20.4 TCU_FMU_ERRGSR register.....	276
4.20.5 TCU_FMU_SMEN register.....	277
4.20.6 TCU_FMU_ERRGEN register.....	278
4.21 TBU FMU registers summary.....	280
4.21.1 TBU_FMU_ERRFR register.....	281
4.21.2 TBU_FMU_ERRCTLR register.....	282
4.21.3 TBU_FMU_ERRSTATUS register.....	284
4.21.4 TBU_FMU_ERRGSR register.....	285
4.21.5 TBU_FMU_SMEN register.....	286
4.21.6 TBU_FMU_ERRGEN register.....	287

A. Signal descriptions.....	290
A.1 TCU signals.....	290
A.1.1 TCU clock and reset signals.....	290
A.1.2 TCU QTW/DVM interface signals.....	290
A.1.3 TCU PTW interface signals.....	293
A.1.4 TCU programming interface signals.....	296
A.1.5 TCU SYSCO interface signals.....	297
A.1.6 TCU LPI_PD interface signals.....	297
A.1.7 TCU LPI_CG interface signals.....	297
A.1.8 TCU DTI interface signals.....	298
A.1.9 TCU interrupt signals.....	299
A.1.10 TCU Message Signaled Interrupt interface signals.....	301
A.1.11 TCU event interface.....	302
A.1.12 TCU tie-off signals.....	303
A.1.13 TCU ELA debug signals.....	305
A.2 TBU signals.....	306
A.2.1 TBU clock and reset signals.....	306
A.2.2 TBU TBS interface signals.....	307
A.2.3 TBU TBM interface signals.....	312
A.2.4 TBU LPI_PD interface signals.....	317
A.2.5 TBU LPI_CG interface signals.....	318
A.2.6 TBU DTI interface signals.....	318
A.2.7 TBU LTI interface signals.....	319
A.2.8 TBU interrupt signals.....	320
A.2.9 TBU tie-off signals.....	321
A.2.10 TBU ELA debug signals.....	323
A.3 TCU and TBU shared signals.....	324
A.3.1 TCU and TBU test and debug signals.....	324
A.3.2 PMU Snapshot signals.....	324
A.4 DTI signals.....	326
A.4.1 DTI interconnect switch DN_Sn interface signals.....	326
A.4.2 DTI interconnect switch UP_Sn interface signals.....	326
A.4.3 DTI interconnect switch DN_M interface signals.....	327
A.4.4 DTI interconnect switch UP_M interface signals.....	327
A.4.5 DTI interconnect sizer LPI_CG interface signals.....	327
A.4.6 DTI interconnect sizer DN_S interface signals.....	328

A.4.7 DTI interconnect sizer UP_S interface signals.....	328
A.4.8 DTI interconnect sizer DN_M interface signals.....	328
A.4.9 DTI interconnect sizer UP_M interface signals.....	329
A.4.10 DTI interconnect register slice LPI_CG interface signals.....	329
A.4.11 DTI interconnect register slice DN_S interface signals.....	330
A.4.12 DTI interconnect register slice UP_S interface signals.....	330
A.4.13 DTI interconnect register slice DN_M interface signals.....	330
A.4.14 DTI interconnect register slice UP_M interface signals.....	331
A.5 LTI signals.....	331
A.5.1 LTI request channel signals.....	331
A.5.2 LTI response channel signals.....	333
A.5.3 LTI completion channel signals.....	333
A.6 I/O Port protection.....	334
A.6.1 TCU protection types.....	334
A.6.2 TBU protection types.....	346
A.6.3 ADB protection types.....	360
A.6.4 LPD protection types.....	365
B. ELA signal descriptions.....	368
B.1 TCU observation interfaces.....	368
B.2 ACE-Lite TBU observation interfaces.....	372
B.3 LTI TBU observation interfaces.....	375
C. Software initialization examples.....	379
C.1 Initializing the SMMU.....	379
C.1.1 Initializing Granule Protection Checks.....	379
C.1.2 Allocating the Command queue.....	380
C.1.3 Allocating the Event queue.....	381
C.1.4 Allocating the PRI queue.....	381
C.1.5 Configuring the Stream table.....	381
C.1.6 Initializing the Command queue.....	382
C.1.7 Initializing the Event queue.....	382
C.1.8 Invalidating TLBs and configuration caches.....	383
C.1.9 Creating a basic Context Descriptor.....	384
C.1.10 Creating a Stream Table Entry.....	385
C.2 Enabling the SMMU.....	386

Proprietary Notice.....387

Product and document information..... 389

Product status.....389

Revision history..... 389

Conventions.....392

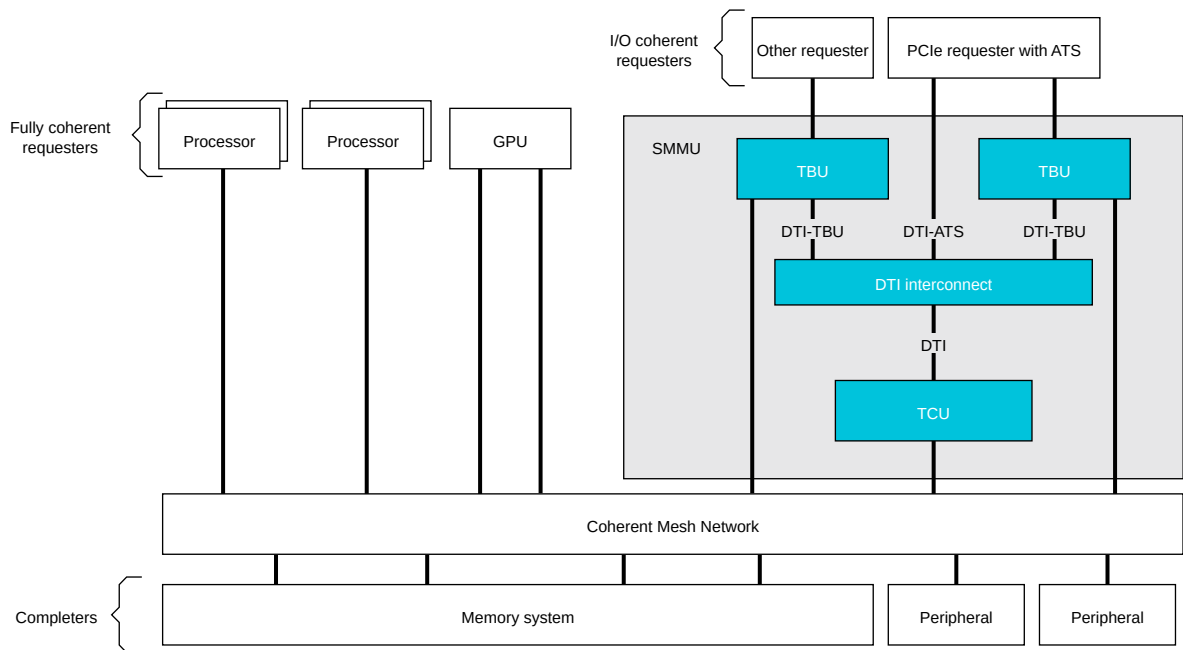
Useful resources..... 394

1. About the MMU-720AE System Memory Management Unit

MMU-720AE is a System Memory Management Unit (SMMU) designed for Functional Safety (FuSa) applications up to ASIL D. MMU-720AE also implements the SMMUv3.3 architecture.

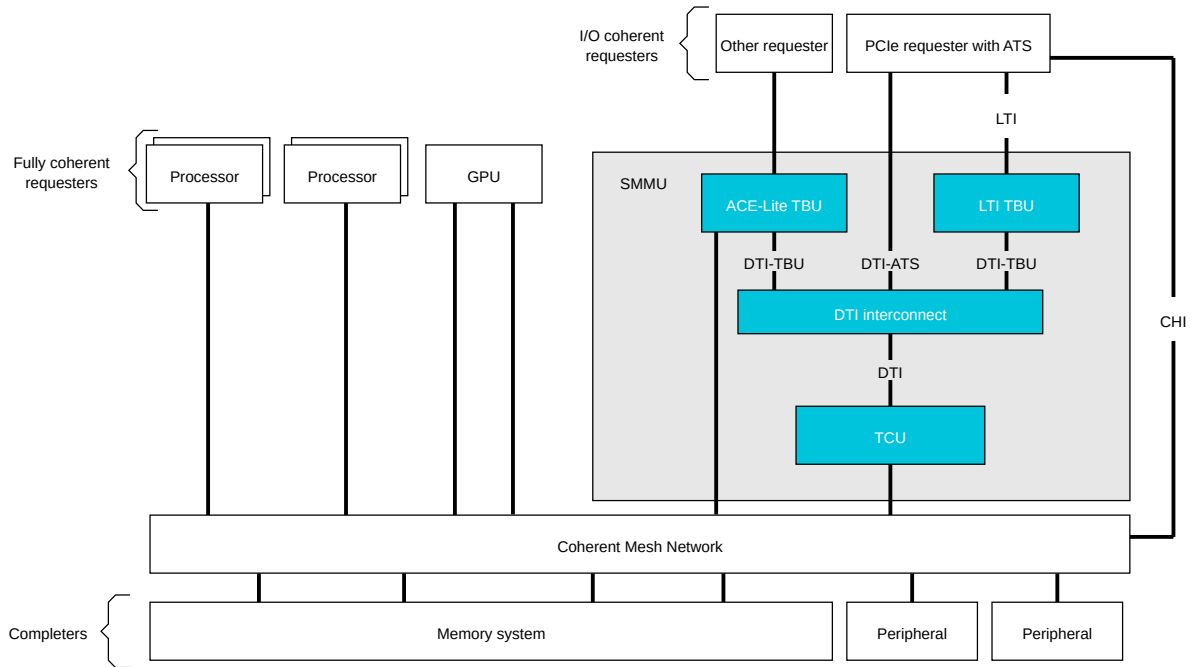
The following figure shows an example system where SMMU represents MMU-720AE.

Figure 1-1: Example system containing MMU-720AE



The following figure shows an example system that uses MMU-720AE, and includes a Local Translation Interface (LTI) Translation Buffer Unit (TBU).

Figure 1-2: Example system with MMU-720AE and LTI TBU



MMU-720AE adds the following features:

- Duplicated logic protection
- Interface protection on all ports
- Error logging and reporting

1.1 Compliance

MMU-720AE complies with, or implements, the specifications that this section describes.

This Technical Reference Manual (TRM) complements architecture reference manuals, architecture specifications, protocol specifications, and relevant external standards. It does not duplicate information from these sources.

Arm® architecture

MMU-720AE implements parts of the Arm v8.6 and v9.1 Virtual Memory System Architecture (VMSA), as the [Arm® Architecture Reference Manual for A-profile architecture](#) defines. The SMMUv3.3 architecture describes the parts of VMSA that apply to MMU-720AE and describes support for Realm Management Extension (RME).

SMMU architecture

MMU-720AE implements the SMMUv3.3 architecture. See the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

AMBA® Distributed Translation Interface protocol

MMU-720AE implements v3 of the Distributed Translation Interface (DTI) protocol, as the [AMBA® DTI Protocol Specification](#) defines. The DTI interfaces use the AXI5-Stream transport layer interface, with Wakeup_Signal enabled and Check_Type not enabled, as the [AMBA® AXI-Stream Protocol Specification](#) defines.

AMBA ACE5-Lite and AMBA AXI5 protocol

MMU-720AE complies with the AMBA® ACE5-Lite protocol. For more information, see the [AMBA® AXI Protocol Specification](#).

AMBA APB protocol

MMU-720AE complies with the AMBA® APB5 protocol, as the [AMBA® APB Protocol Specification](#) defines.

AMBA® LTI protocol

MMU-720AE complies with the AMBA® LTI protocol, as the [AMBA® LTI Protocol Specification](#) defines.

LPI Q-Channel protocol

MMU-720AE complies with the LPI Q-Channel, as the [AMBA® Low Power Interface Specification](#) defines.

1.2 Features

MMU-720AE provides features such as compliance with architectures, support for architecture extensions, support for flexible integration, high speed translation, and trace debugging.

The MMU-720AE features are as follows:

Compliance with the SMMUv3.3 architecture

- Support for stage 1 translation, stage 2 translation, and stage 1 followed by stage 2 translation
- Support for Armv8 AArch64 translation table format
- Support for 4 KB, 16 KB, and 64 KB granule sizes in AArch64 format
- Support for PCI Express (PCIe) integration, including the following:
 - Address Translation Services (ATS), including full and split-stage ATS
 - Process Address Space IDs (PASIDs)
 - Access Control Services (ACS)
- Support for Page Request Interface (PRI), as SMMUv3 defines. PRI is an optional PCIe ATS extension that enables support for unpinned memory in PCIe.
- Support for Memory System Resource Partitioning and Monitoring (MPAM).
- Support for the Secure-EL2 translation regime.
- Requesters can be stalled while a processor handles translation faults, enabling software support for on-demand paging.

- Configuration tables in memory.
- Queues in memory that perform MMU-720AE management. There is no requirement to stall a processor when it accesses MMU-720AE.
- A Performance Monitoring Unit (PMU) in each TBU and TCU that enables MMU-720AE performance to be investigated.
- Reliability, Availability, and Serviceability (RAS) features for RAM corruption detection and correction.
- EOPD, PTWNNC, ATSRECERR, and MPAM_NS features from SMMUv3.3.
- Granule Protection Checks (GPCs) for SMMU-originated and client-originated accesses.
- Root PAS registers for GPC.

Support for Realm Management Extensions

- Reporting of GPC faults.
- Support for GPC invalidations.

Support for Realm Management Extensions Device Assignment

- Realm translation regime.
- Realm queues.
- Realm programmers model.
- Memory Encryption Context (MEC) support.

Support for AMBA interfaces

- ACE5-Lite TBU transaction interfaces that support cache stash transactions, deallocating transactions, and Cache Maintenance Operations (CMOs).
- Support for AXI5 untranslated transactions version 3.
- An ACE5-Lite+Distributed Virtual Memory (DVM) TCU table walk interface that enables Armv8.6 and Realm Management Extension (RME)-capable processors to perform shared TLB invalidate operations without accessing MMU-720AE directly.
- An ACE5-Lite TCU table walk interface that enables Armv8.6 and RME-capable processors to perform shared TLB invalidate operations without accessing MMU-720AE directly.
- An ACE5 Low-Power extension that enables the TCU to subscribe to DVM TLB invalidate requests on powerup and powerdown.
- AMBA DTI communication between the TCU and TBUs, enabling requesters to request translations and implement TBU functionality internally.
- Support for the AMBA Low-Power Interface (LPI) Q-Channel so that standard controllers can control power and clock gating.
- WAKEUP signaling on all interfaces, including DTI and APB interfaces.
- Support for ACE5-Lite atomic transactions in the ACE-Lite TBU.

- Support for Local Translation Interface (LTI). LTI is a point-to-point protocol that enables devices to directly request a translation for each transaction while leaving the TBU to manage the Translation Lookaside Buffer (TLB).
- Support for a dedicated Generic Interrupt Controller (GIC) integration, with Message Signaled Interrupts (MSIs) supported for common interrupt types.

Support for flexible integration

- You can place a configurable number of TBUs close to the requesters being translated.
- Communication between TBU and TCU over AXI5-Stream is supported using the supplied DTI interconnect components, or any other AXI5-Stream interconnect, with Wakeup_Signal enabled and Check_Type not enabled.
- DTI interconnect components support hierarchical topologies and control the tradeoff between the number of wires and the DTI bandwidth.

Support for high-performance translation

- Scalable configurable MicroTLB and Main TLB (MTLB) in the TBU can reduce the number of translation requests to the TCU.
- TBU direct indexing and MTLB partitioning enable the use of MTLB entries to be managed outside the TBU, improving real-time translation performance.
- Optimization enables storage of all architecturally-defined page and block sizes, including contiguous page and block entries, as a single entry in the TBU TLBs and TCU walk caches.
- Per-TBU prioritization in the TCU enables high-priority transaction streams to be translated before low-priority streams.
- TCU prefetch of translation tables, which can be enabled on a per-context basis, improves translation performance for real-time requesters that access memory linearly.
- Hit-Under-Miss (HUM) support in the TBU enables transactions with different AXI IDs to be propagated out of order, when a translation is available. Reordering of transactions is possible only if the transactions are also in different ordering groups.
- TBU detects multiple transactions that require the same translation so that only one TBU request to the TCU is required.
- TCU detects multiple translations that require the same table in memory so that only one TCU memory request is required.
- Multi-level, multi-stage walk caches in the TCU reduce translation cost by performing only part of the table walk process on a miss.
- Granule Protection Table (GPT) caches which cache GPT information, covering both SMMU-originated and client- originated accesses.
- A configurable number of concurrent translations in the TBU and TCU promotes high translation throughput.

Translation logging using an Embedded Logic Analyzer

- Using a CoreSight™ ELA-600 Embedded Logic Analyzer to log port activities

1.3 TCU and TBU common interfaces

TCUs and TBUs support some common interfaces.

Both TCUs and TBUs support the following common interfaces:

- Clocks and resets
- Distributed Translation Interface (DTI)
- Tie-offs
- Interrupts
- Performance Monitoring Unit (PMU) snapshot
- Test and debug
- Low Power Interface (LPI) clock gating
- LPI powerdown
- ELA observation interface

The TCU also supports the following interfaces:

- Programming interface, APB5 PROG
- Primary ACE-Lite+DVM Queue and Table Walk (QTW)/DVM interface
- Secondary ACE-Lite Page Table Walk (PTW) interface, in dual AXI mode only
- Generic Interrupt Controller (GIC) Message Signaled Interrupt (MSI) interface
- Event interface, eventoreq, or eventoack
- System Coherency interface (SYSCO)

The ACE-Lite TBU also supports the following interfaces:

- Transaction subordinate (TBS)
- Transaction manager (TBM)

The LTI TBU also supports the Local Translation Interface (LTI). For more information on TCU and TBU interfaces see [2.4 Interfaces](#) on page 31.

1.4 Configurable options

MMU-720AE is a highly configurable System Memory Management Unit that provides configuration options for each of the main components.

For the TCU, you can configure the following:

- Size and structure of each cache
- Data width of the Queue and Table Walk (QTW)/DVM and Page Table Walk (PTW) interface

- Number of translations that can be performed at the same time
- Number of Granule Protection Checks (GPCs) that can be performed at the same time for client-originated accesses. You cannot configure the number of parallel GPCs for SMMU-originated accesses. The number of possible SMMU-originated accesses determines the number of parallel GPCs.
- Number of translation requests that can be accepted from all DTI requesters including TBU and ATS requesters
- Type of some RAMs

For the TBU, you can configure the following:

- Size of each cache
- Number of transactions that can be translated at the same time
- Register slices
- Type of some RAMs

For the ACE-Lite TBU, you can configure the following:

- Write data buffer depth
- Number of outstanding read and write transactions that the TBM interface supports
- Width of data, ID, User, StreamID, and SubstreamID signals on the TBS and TBM interfaces



Depths are specified as a discrete number of entries.

You can also configure the DTI interconnect components to meet your system requirements.

See [2.7 Configuration parameters and methodology](#) on page 102.

1.5 Product documentation and design flow

MMU-720AE documents are used during different parts of the design flow.

1.5.1 Documentation

We deliver MMU-720AE with a Technical Reference Manual (TRM) and a Configuration and Integration Manual (CIM).

The MMU-720AE documentation is as follows:

Technical Reference Manual

The Technical Reference Manual (TRM) describes the functionality and the effects of functional options on the behavior of MMU-720AE. It is required at all stages of the design

flow. The choices that are made in the design flow can mean that some behaviors that are described in the TRM are not relevant. If you are programming MMU-720AE, then contact:

- The implementer to determine:
 - The build configuration of the implementation
 - The integration, if any, that was performed before implementing MMU-720AE
- The integrator to determine the pin configuration of the device that you are using.

Configuration and Integration Manual

The Configuration and Integration Manual (CIM) describes:

- The available build configuration options and related issues in selecting them.
- How to integrate MMU-720AE into an SoC. This section describes the pins that the integrator must tie off to configure the macrocells for the required integration.
- The processes to sign off on the configuration, integration, and implementation of the design.

The CIM is a confidential book that is only available to licensees.

1.5.2 Design flow

MMU-720AE is delivered as synthesizable RTL. Before it can be used in a product, it must go through the following processes:

Implementation

The implementer configures and synthesizes the RTL to produce a hard macrocell. This process might include integrating RAMs into the design.

Integration

The integrator connects the implemented design into an SoC. Integration includes connecting the design to a memory system and peripherals.

Programming

The system programmer develops the software to configure and initialize MMU-720AE, and tests the required application software.

Each process is separate, and can include implementation and integration choices that affect the behavior and features of MMU-720AE.

The operation of the final device depends on:

Build configuration

The implementer chooses the options that affect how the RTL source files are pre-processed. These options usually include or exclude logic that affects one or more of the following:

- Area
- Maximum frequency
- Features of the resulting macrocell

Configuration inputs

The integrator configures some features of MMU-720AE by tying inputs to specific values. These configurations affect the start-up behavior before any software configuration is made.

Software configuration

The programmer configures MMU-720AE by programming particular values into registers. This configuration affects the behavior of MMU-720AE.

See also:

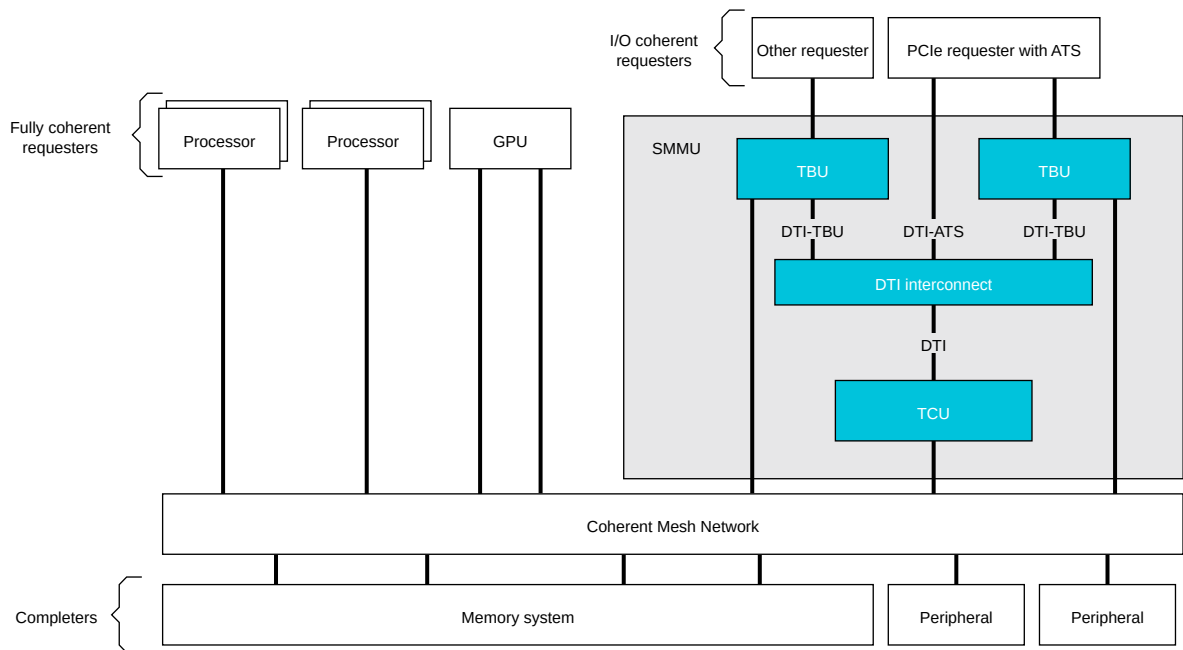
- [1.1 Compliance](#) on page 13
- [1.4 Configurable options](#) on page 17
- [2.7 Configuration parameters and methodology](#) on page 102

2. Functional description of MMU-720AE Components

The major functional blocks of MMU-720AE are the Translation Buffer Unit (TBU), Translation Control Unit (TCU), and Distributed Translation Interface (DTI) interconnect.

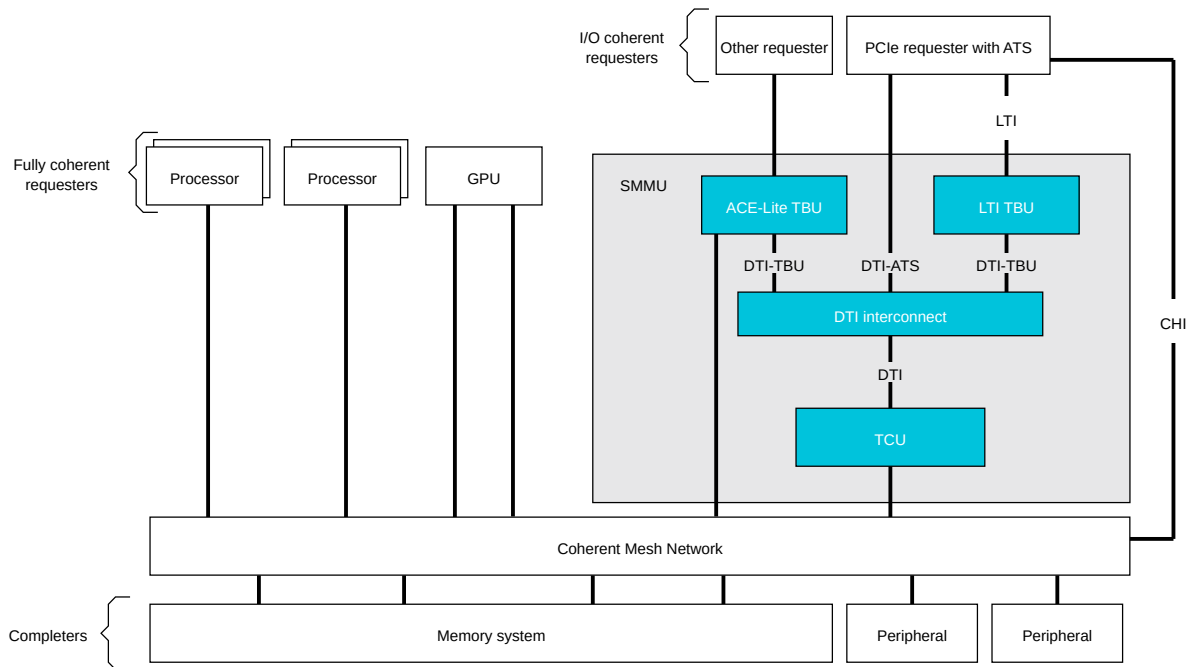
The following figure shows an example system that uses MMU-720AE.

Figure 2-1: Example system containing MMU-720AE



The following figure shows an example system that uses MMU-720AE, and includes a Local Translation Interface (LTI) TBU.

Figure 2-2: Example system with MMU-720AE and LTI TBU



MMU-720AE contains the following key components:

Translation Buffer Unit (TBU)

The TBU contains Translation Lookaside Buffers (TLBs) that cache translation tables. MMU-720AE implements a TBU that can be connected to a single requester or multiple requesters. It is also possible to connect multiple TBUs to a single requester to improve performance. These TBUs are local to the corresponding requester and can be one of the following:

- ACE-Lite TBU.
- LTI TBU.

Translation Control Unit (TCU)

The TCU controls and manages the address translations. MMU-720AE implements a single TCU. In MMU-720AE-based systems, the AMBA® DTI protocol defines the standard for communicating with the TCU. See the [AMBA® DTI Protocol Specification](#).

DTI interconnect

The DTI interconnect connects multiple TBUs to the TCU.

When an MMU-720AE TBU receives a transaction on the TBS or LA channel of the LTI interface, it looks for a matching translation in its TLBs. If it has a matching translation, it uses it to translate the transaction and outputs the transaction on the TBM interface, or LR channel of the LTI interface. If it does not have a matching translation, it requests a new translation from the TCU using the DTI interface.

When the TCU receives a DTI translation request, it uses the QTW/PTW interfaces to perform the following:

- Configuration table walks, that return configuration information for the translation context.
- Translation table walks, that return translation information that is specific to the transaction address.
- Granule Protection Table (GPT) walk, that returns a set of GPT tables that are used to check the physical address space with which each granule is associated.

The TCU contains caches that reduce the number of configuration, translation, and Granule Protection Table (GPT) walks that are to be performed. Sometimes no walks are required.

When the TBU receives the translation from the TCU, it stores it in its TLBs. If the translation was successful, the TBU uses it to translate the transaction, otherwise it terminates it.

A processor controls the TCU by performing the following:

- Writing commands to a Command queue in memory.
- Receiving events from an Event queue in memory.
- Writing to its configuration registers using the programming interface.

The [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) describes the following:

- Translation.
- Translation for Realm devices.
- How software communicates with the TCU.
- Granule protection checks.
- How software communicates with Granule Protection Check (GPC) related aspects of the TCU.
- Memory Encryption Context (MEC).

2.1 Translation Control Unit

A typical SMMUv3-based system includes a single Translation Control Unit (TCU). The TCU is usually the largest block in the system, and performs several roles.

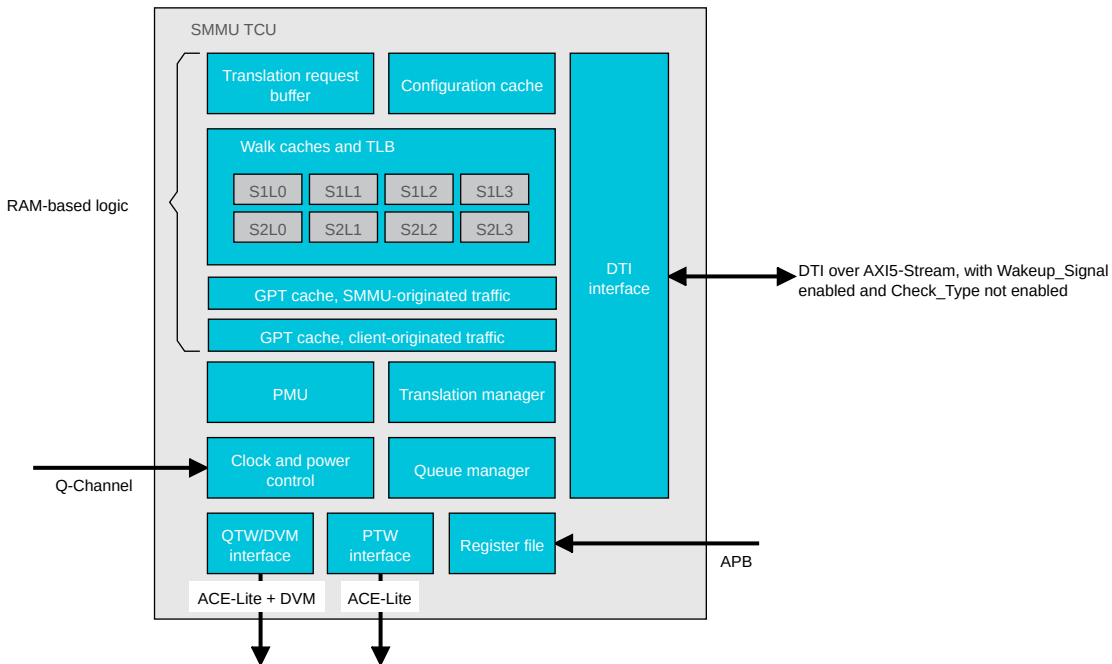
The TCU performs the following roles:

- Manages the memory queues
- Performs the following:
 - Translation table walks
 - Configuration table walks
 - Granule Protection Table (GPT) walks
- Implements the following:

- Backup caching structures
- The SMMU programmers model

The following figure shows the TCU with its main interfaces. The figure is illustrative of the functions that the block performs but might not match exactly with the block names inside the RTL.

Figure 2-3: MMU-720AE TCU



The ACE-Lite PTW interface is optional.

The TCU consists of:

Walk cache

The walk cache is a set-associative walk cache that has a configurable number of banks and ways and stores the results of translation table walks. During runtime, the cache line entries can be programmatically split to create separate walk caches that are reserved for:

- Stage 1 level 0 table entries.
- Stage 1 level 1 table and block entries.
- Stage 1 level 2 table and block entries.
- Stage 1 level 3 block entries.

- Stage 2 level 0 table entries.
- Stage 2 level 1 table and block entries.
- Stage 2 level 2 table and block entries.
- Stage 2 level 3 block entries.

To enable and disable the walk cache for a particular stage and level of translation, use the [4.7.1 TCU_CTRL register](#) on page 146. If an error occurs for a cache line entry, the [4.8.3 TCU_ERRSTATUS register](#) on page 165 identifies the affected entry.

The walk cache is useful in cases where a translation request results in a miss in other TCU caches. A subsequent hit in the walk cache requires only a single memory access to complete the translation table walk and fetch the required descriptor.

Configuration cache

The configuration caches are set-associative cache structures that store configuration information. Each entry stores the Context Descriptor (CD) and Stream Table Entry (STE) contents for a translation context.



The configuration cache does not cache the contents of intermediate configuration tables.

Granule Protection Table (GPT) cache

The GPT cache is a set-associative granule protection walk cache that has a configurable number of banks and ways and stores the results of GPT walks. Two GPT caches exist in the TCU for the following:

- Client-originated AXI Granule Protection Check (GPC) checks, DTI GPC Wrapper (DGW).
- SMMU-originated GPC checks, AXI GPC Wrapper (AGW).

During runtime, the cache line entries can be programmatically split to create separate walk caches that are reserved for the following:

- L0 block and table entries.
- L1 contiguous and granule entries.

To enable and disable the GPT cache for a particular stage and level of translation, use the [4.7.7 TCU_ROOT_CTRL register](#) on page 153. If an error occurs for a cache line entry, use the [4.8.3 TCU_ERRSTATUS register](#) on page 165 to identify the affected entry.

Translation manager

The translation manager manages translation requests that are in progress.

Translation request buffer

The translation request buffer stores translation requests from TBUs when all translation manager slots are full. The translation request buffer supports more slots than the translation manager. When correctly configured, this buffer has enough space to store all translation

requests that TBUs can issue simultaneously. This buffer therefore prevents the DTI interface from becoming blocked.

PMU

The PMU counts TCU performance-related events and has a configurable number of counters to count the events.

Clock and power control

The TCU has its own clock and power control, that the Q-Channels provide.

Queue manager

The queue manager manages all SMMUv3 queues that are stored in memory as follows:

- Command queues, Secure and Non-secure.
- Event queues, Secure and Non-secure.
- PRI queue, Non-secure.

QTW/DVM and PTW interfaces

The Queue and Table Walk (QTW)/Distributed Virtual Memory (DVM) is an ACE-Lite+DVM interface. The Page Table Walk (PTW) interface is an ACE-Lite interface.

Register file

The register file implements the SMMUv3 programmers model, as the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) defines.

DTI interface

The completer (TCU) DTI interface uses the DTI protocol, over AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, to enable the TCU to communicate with a requester (TBU) component. For MMU-720AE, the requester (TBU) component is either a TBU or a PCIe requester.

See also:

- [2.4 Interfaces](#) on page 31
- [2.5 Operation](#) on page 45
- [2.5.8 TCU transaction handling](#) on page 67
- [2.5.9 TCU prefetch](#) on page 69
- [4.2 SMMU architectural registers](#) on page 121

2.2 Translation Buffer Unit

A typical SMMUv3-based system includes multiple Translation Buffer Units (TBUs). Each TBU is located close to the component for which it provides address translation.

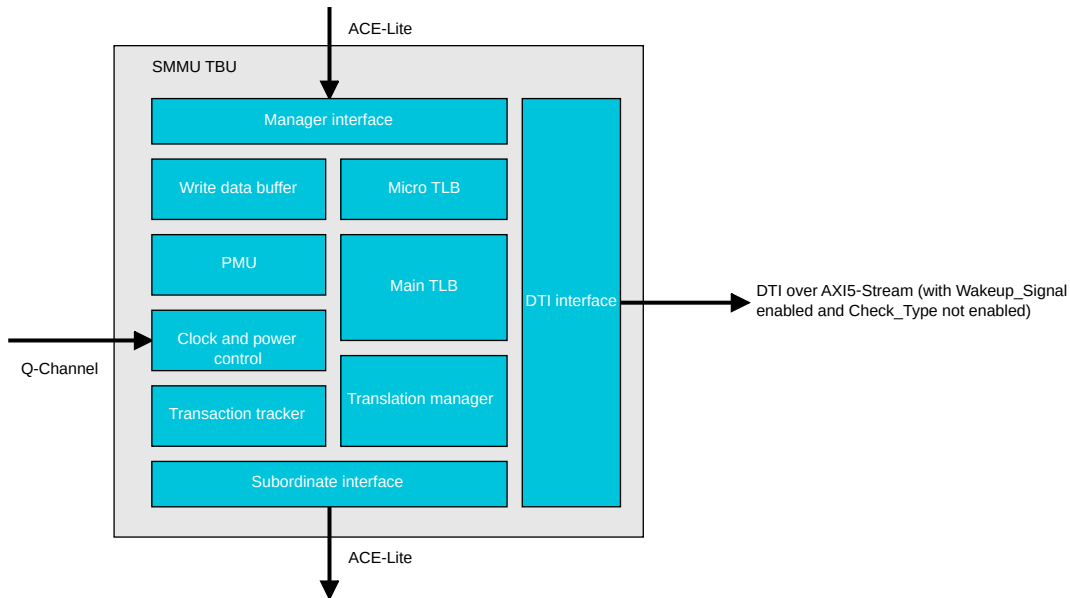
A TBU can be one of the following:

- ACE-Lite TBU
- Local Translation Interface (LTI) TBU

A TBU intercepts transactions and provides the required translation from a Translation Lookaside Buffer (TLB) if possible. If a TLB does not contain the required translation, the TBU requests translations from the TCU, and then caches the translation in one of the TLBs.

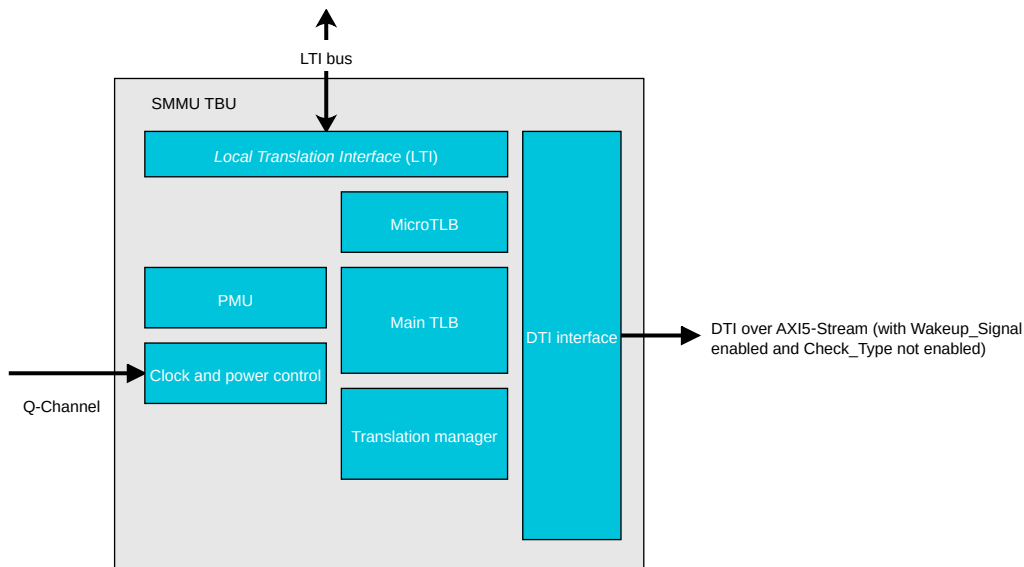
The following figure shows the ACE-Lite TBU. The figure is illustrative of the functions that the block performs but might not match exactly with the block names inside the RTL.

Figure 2-4: MMU-720AEACE-Lite TBU



The following figure shows the LTI TBU. The figure is illustrative of the functions that the block performs but might not match exactly with the block names inside the RTL.

Figure 2-5: MMU-720AE LTI TBU



The TBU consists of:

Manager and subordinate interfaces

The TBU manager and subordinate interfaces can contain the following ACE-Lite TBU and LTI TBU interfaces:

ACE-Lite TBU

The ACE-Lite TBU contains the following interfaces:

TBS interface

For receiving untranslated transactions from the requesting manager

TBM interface

For issuing translated transactions to the rest of the system.

LTI TBU

For LTI translation requests and responses.

MicroTLB

The TBU compares incoming transactions with translations that are cached in the MicroTLB before looking in the Main TLB (MTLB). The MicroTLB provides end-to-end translation from an input address to an output address in addition to Granule Protection Check (GPC) information. You can use a tie-off signal to configure the cache replacement policy as either round-robin or Pseudo Least Recently Used (PLRU).

Main TLB

Each TBU includes an optional Main TLB (MTLB) that caches translation table walk entries from:

- Stage 1 translations

- Stage 2 translations
- Stage 1 combined with stage 2 translations

GPC information is also cached in each Main TLB entry.

The MTLB is a set associative cache structure with a configurable number of ways and banks.

If multiple translation sizes are in use, a single transaction might require multiple lookups.

TBU direct indexing enables MMU-720AE to manage MTLB entries externally to the TBU. Direct indexing improves the predictability of TBU performance, for bus managers that have real-time performance requirements.

TBU hazarding

If the MicroTLB lookup results in a miss, the transaction checks whether there are any pending transactions from which it can use the translation. This method is called forming a hazard. If the transaction hazards on any pending transactions, then the transaction waits until the response is available for the hazarded transaction and uses that response. The number of unique addresses that form a hazard is limited. The `TBUCFG_HZRD_ENTRIES` parameter controls the number of unique addresses. For information about `TBUCFG_HZRD_ENTRIES`, see [2.7.5 ACE-Lite TBU I/O configuration parameters](#) on page 107 and [2.7.6 LTI TBU configuration parameters](#) on page 108.

For more information about TBU hazarding, see the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

Translation manager

The translation manager manages translation requests that are in progress. Each transaction occupies a translation slot until it is propagated downstream through the ACE-Lite TBM, or an LTI translation response is returned. All transactions are hazard-checked to reduce the possibility of duplicate translation requests being sent to the TCU.

Restrictions can exist on the ordering of transactions with different AXI IDs/LTI Order Groups (OGs), if different AXI IDs map to the same ordering group. Transactions with different AXI IDs can be propagated downstream out-of-order.

All transactions with a given AXI ID/LTI OG value must remain ordered. The translation manager propagates such transactions when the translation is ready, provided no other transaction with the same AXI ID/LTI OG is already waiting.

For more information about AXI transaction identifiers, see the [AMBA® AXI Protocol Specification](#).

For more information about LTI OGs, see the [AMBA® LTI Protocol Specification](#).

Write data buffer

The write data buffer is available in the ACE-Lite TBU only. The optional write data buffer enables write transactions with different AXI IDs to progress through the TBU out-of-order. It reorders the data to match the downstream transaction order.

Performance Monitoring Unit (PMU)

The PMU counts TBU performance-related events.

Clock and power control

The TBU has its own clock and power control, that the Q-Channels provide.

DTI interface

The manager DTI interface uses the DTI protocol over AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, to enable the TBU to communicate with a subordinate component. For MMU-720AE, the subordinate component is the TCU. Although you can implement DTI over different transport protocols, the MMU-720AE interfaces use AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled.

Transaction tracker

The transaction trackers manage outstanding read and write transactions, permitting invalidation and synchronization to take place without stalling the AXI interfaces.



The transaction tracker is available in the ACE-Lite TBU only.

See also:

- [2.5 Operation](#) on page 45
- [2.5.4 Main TLB direct indexing and main TLB direct partitioning](#) on page 57
- [4.2 SMMU architectural registers](#) on page 121

2.3 Distributed Translation Interface interconnect

The TCU and TBUs use a Distributed Translation Interface (DTI) to communicate. The DTI interconnect enables the DTI interface to use the AXI5-Stream transport protocol, with Wakeup_Signal enabled and Check_Type not enabled.

The DTI interconnect can connect any components that conform to the AXI5-Stream protocol, with Wakeup_Signal enabled and Check_Type not enabled, as the [AMBA® DTI Protocol Specification](#) defines.

The DTI interconnect contains internal components that are hierarchically composable, that is, they can be connected in different ways to suit your system requirements. For example, within an MMU-720AE system, you can use the switch component to combine the DTI interfaces of multiple TBUs into a single DTI interface. You can then connect the combined DTI interface to another DTI interconnect that is closer to the TCU.

The DTI interconnect includes switch, sizer, and register slice components.

Switch

The switch connects multiple DTI TBUs, to a DTI TCU. The switch implements the following parallel networks:

TBU to TCU traffic

Network connecting multiple AXI5-Stream TBUs to a single AXI5-Stream TCU, with Wakeup_Signal enabled and Check_Type not enabled

TCU to TBU traffic

Network connecting a single AXI5-Stream TCU to multiple AXI5-Stream TBUs, with Wakeup_Signal enabled and Check_Type not enabled



The switch does not store any data, and therefore does not require a Q-Channel clock gating interface.

Sizer

The sizer connects channels that have different data widths, enabling different tradeoffs of bandwidth to area. The sizer supports conversion between the supported AXI5-Stream data widths, with Wakeup_Signal enabled and Check_Type not enabled:

- 1 byte
- 4 bytes
- 10 bytes
- 20 bytes
- 24 bytes

The sizer includes a Q-Channel interface to provide clock gating control.

Register slice

Use the register slice to improve timing. The register slice includes a Q-Channel interface to provide clock gating control.



MMU-720AE DTI interconnect components do not include a component to connect different clock and power domains. You can connect DTI interfaces in different clock and power domains by using the Bidirectional AXI5-Stream (BAS) configuration of ADB-400 AMBA® Domain Bridge.

See also [2.5 Operation](#) on page 45.

2.4 Interfaces

MMU-720AE includes interfaces for each of the TCU, TBU, and DTI interconnect components.

The DTI interconnect consists of switch, sizer, and register slice components that you can connect separately, and these components therefore have their own interfaces.

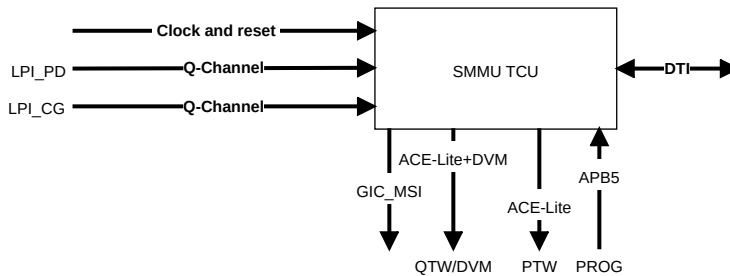
The PMU snapshot interface is common to both TCU and TBU.

2.4.1 TCU interfaces

The MMU-720AE TCU includes several requester and completer interfaces.

The following figure shows the TCU interfaces.

Figure 2-6: TCU interfaces



The ACE-Lite PTW interface is optional.

See also:

- [2.5.7 Distributed Virtual Memory messages](#) on page 66
- [2.5.10 Error responses](#) on page 71
- [2.6.2 AMBA implementation](#) on page 81
- [A.1.2 TCU QTW/DVM interface signals](#) on page 290

2.4.1.1 TCU Queue and Table Walk/Distributed Virtual Memory interface

The Queue and Table Walk/Distributed Virtual Memory (QTW/DVM) interface is an ACE-Lite +DVM manager interface.

The QTW/DVM interface can issue the following transaction types:

- ReadNoSnoop

- WriteNoSnoop
- ReadOnce
- WriteUnique
- DVM Complete
- AtomicCompare, but only if only a single port is present, that is, TCU x1

The QTW/DVM interface uses the write address transaction ID signal `awid_qtw`, and the read address transaction ID signal, `arid_qtw`.

The AXI ID width on this interface is calculated as follows:

$$(\text{ceil}(\log_2(\max(\text{TCUCFG_PTW_SLOTS}, \text{TCUCFG_DGW_SLOTS}, 8))) + 3)$$

The minimum possible TCU ID width value is 6.

For information about the possible values of `awid_qtw` and `arid_qtw`, see [2.5.8 TCU transaction handling](#) on page 67.

To support GPT TLBI, the interface provides DVM_9.2 support.

The interface does not issue cache maintenance operations or exclusive accesses.

2.4.1.2 TCU Page Table Walk interface

In the `tcu_x2` variant of the TCU, the Page Table Walk (PTW) interface is present. The dedicated PTW interface is required for high-performance applications where a single AXI interface cannot provide enough bandwidth to handle the combined table walks.

If the SMMU is configured to be in legacy mode, by setting the `TCUCFG_LEGACY_TZ_EN` parameter is set to 1 or setting the `legacy_tz_en` tie-off signal to 1, the single QTW/DVM interface is usually sufficient. The PTW interface is used only for PTW traffic, including HTTU updates, and DPT traffic, and these are not performed on the QTW/DVM interface. The PTW interface can issue the following transaction types:

- ReadNoSnoop
- ReadOnce
- AtomicCompare

The external ID width is calculated as follows:

$$\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3.$$

External ID Width defines the width of the `awid_ptw`, `arid_ptw`, `bid_ptw`, and `rid_ptw` ports on this interface.

2.4.1.3 TCU PROG interface

The programming, PROG interface is an AMBA APB5 completer interface. It enables software to program the MMU-720AE internal registers and read the Performance Monitoring Unit (PMU) registers and the Debug registers.

This interface runs synchronously with the other TCU interfaces.

The applicable address width for this interface is 32 bits.

Transactions are Read-As-Zero, Writes Ignored (RAZ/WI) when any of the following apply:

- An unimplemented register is accessed
- PSTRB[3:0] is not 0b1111 for write transfers
- {PNSE, PPROT[1]} does not encode a PAS which has access to the register being addressed
- In Legacy mode when PNSE == 1

For more information, see:

- [A.1.4 TCU programming interface signals](#) on page 296
- [AMBA® APB Protocol Specification](#)

2.4.1.4 TCU LPI_PD interface

The TCU LPI_PD Q-Channel completer interface manages Low Power Interface (LPI) powerdown (PD) for the TCU.

See also:

- [A.1.6 TCU LPI_PD interface signals](#) on page 297
- [AMBA® Low Power Interface Specification](#)

2.4.1.5 TCU LPI_CG interface

The TCU LPI_CG Q-Channel requester interface enables Low Power Interface (LPI) Clock Gating (CG) for the TCU.

See also:

- [A.1.7 TCU LPI_CG interface signals](#) on page 297
- [AMBA® Low Power Interface Specification](#)

2.4.1.6 TCU DTI interface

The DTI interface manages communication between the TBUs and the TCU, using the DTI protocol. The DTI protocol can be conveyed over different transport layer mediums, including AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled.

The TCU includes a DTI interface, in the same way as each TBU does. To permit bidirectional communication, each DTI interface includes:

Downstream interface

One AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, interface

Upstream interface

One AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, interface

See also:

- [2.5.1 Distributed Translation Interface](#) on page 45
- [A.1.8 TCU DTI interface signals](#) on page 298
- [AMBA® DTI Protocol Specification](#)
- [AMBA® AXI-Stream Protocol Specification](#)

2.4.1.7 TCU MSI interface

The Message Signaled Interrupt (MSI) interface provides global, per-context, and performance interrupts. A direct MSI connection to a Generic Interrupt Controller (GIC) is supported, to avoid complex dependencies in the system.

For more information, see [A.1.10 TCU Message Signaled Interrupt interface signals](#) on page 301.

2.4.1.8 TCU SYSCO signaling

MMU-720AE provides a hardware system coherency interface. The SYSCO requester interface permits the TCU to remove itself from a coherency domain in response to an LPI request.

The SYSCO signals include the syscoreq_qtw and syscoack_qtw handshake signals to enter or exit a coherency domain.

If the sup_btm signal is tied HIGH, the TCU does not assert ACREADY until SYSCOACK has been received and the interface has entered the CONNECTED state.



To prevent a deadlock, the connected interconnect must be capable of driving SYSCOACK without requiring progress on the AC channel.

If the sup_btm signal is tied LOW, the syscoreq_qtw signal is always driven LOW and the syscoack_qtw signal is ignored.

For more information, see [A.1.5 TCU SYSCO interface signals](#) on page 296.

2.4.1.9 TCU tie-off signals

The TCU tie-off signals enable you to initialize various operating parameters on exit from reset state.

For more information, see [A.1.12 TCU tie-off signals](#) on page 302.

2.4.1.10 TCU ELA observation interface

This Embedded Logic Analyzer (ELA) observation requester interface drives the signal group, signal qualifier, and signal clock enable ELA signals to the on-chip ELA module, if present.

When `TCUCFG_USE_ELA_DEBUG` is 0, these signals are tied to 0. See [2.7.2 TCU debug configuration parameters](#) on page 105.

See also:

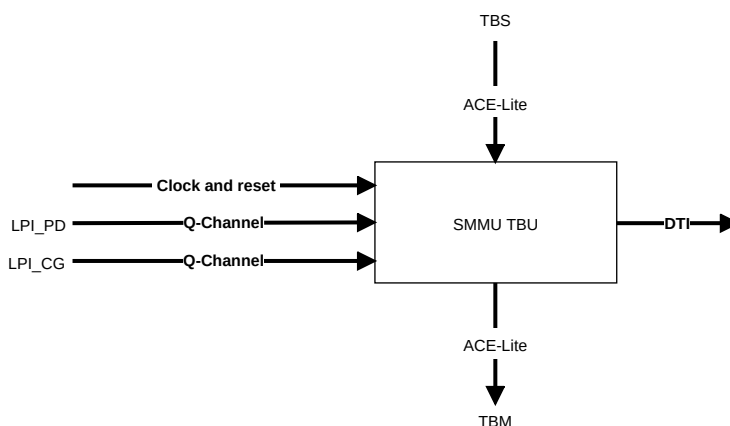
- [B.1 TCU observation interfaces](#) on page 368
- [Arm® CoreSight™ ELA-600 Embedded Logic Analyzer Technical Reference Manual](#)

2.4.2 TBU interfaces

MMU-720AE can contain both ACE-Lite and LTI Translation Buffer Units (TBUs). Each MMU-720AE TBU includes several requester and completer interfaces.

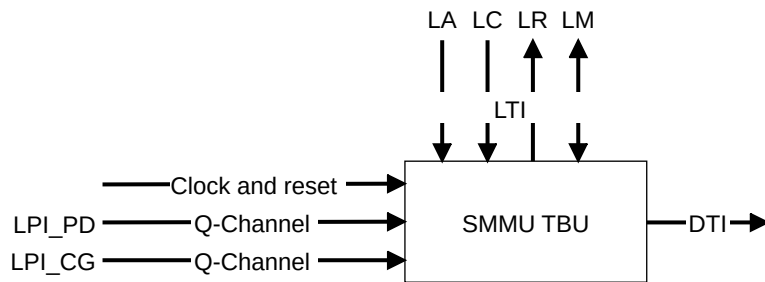
The following figure shows the ACE-Lite TBU interfaces.

Figure 2-7: ACE-Lite TBU interfaces



The following figure shows the LTI TBU interfaces.

Figure 2-8: LTI TBU interfaces



LTI TBUs can have variants with 1, 2, 4, and 8 LTI interfaces. The figure shows a TBU with one LTI interface.

2.4.2.1 ACE-Lite TBU TBS interface

The transaction subordinate interface, TBS, is an ACE5-Lite interface on which the ACE-Lite TBU receives incoming untranslated memory accesses.

This interface supports a 64-bit address width.

The interface implements optional signals to support the following AXI5 extensions:

- Atomic_Transactions
- Cache_Stash_Transactions
- CMO_On_Read, Persist_CMO
- DeAllocation_Transactions
- Exclusive_Accesses
- InvalidateHint_Transaction
- Loopback_Signals
- MEC_Support
- MTE_Support basic
- PBHA_Support
- Poison
- Read_Data_Chunking
- RME_Support
- Shareable_Transactions
- Unique_ID_Support

- Untranslated_Transactions v3
- Wakeup_Signals

The TBS interface supports ACE-Lite Exclusive accesses.

If a transaction is terminated in the TBU, the transaction tracker returns the transaction with the user-defined AXI RUSER and BUSER bits set to 0.

See also:

- [2.5.10 Error responses](#) on page 71
- [2.6.2 AMBA implementation](#) on page 81
- [A.2.2 TBU TBS interface signals](#) on page 306

2.4.2.2 ACE-Lite TBU TBM interface

The transaction manager, TBM, interface is an ACE5-Lite interface on which the ACE-Lite TBU sends outgoing translated memory accesses.

The AXI ID of a transaction on this interface is the same as the AXI ID of the corresponding transaction on the transaction subordinate, TBS interface.

This interface supports a 52-bit address width, and `TBUCFG_DATA_WIDTH` defines the data width. See:

- [2.7.5 ACE-Lite TBU I/O configuration parameters](#) on page 107
- [2.7.6 LTI TBU configuration parameters](#) on page 108

This interface can issue read and write transactions until the outstanding transaction limit is reached. MMU-720AE provides parameters that permit you to configure:

- The outstanding read transactions limit
- The outstanding write transactions limit
- The total outstanding read and write transactions limit

The interface implements optional signals to support the following AXI5 extensions:

- Atomic_Transactions
- Cache_Stash_Transactions
- CMO_On_Read, Persist_CMO
- DeAllocation_Transactions
- Exclusive_Accesses
- InvalidateHint_Transaction
- Loopback_Signals
- MEC_Support
- MPAM_Support

- MTE_Support basic
- Ordered Write Observation
- PBHA_Support
- Poison
- Read_Data_Chunking
- Read_Interleaving_Disabled



Note

If terminated transaction responses are not interleaved, the BIU supports the Read_Interleaving_Disabled property.

-
- RME_Support
 - Shareable_Transactions
 - Unique_ID_Support
 - Untranslated_Transactions v3



Note

The TBM interface does not support the Untranslated_Transactions property. The TBM contains the axmmusecsid and axmmusid signals for backward-compatibility with other SMMU products. These signals are not required for normal operation of MMU-720AE and you can ignore them.

-
- Wakeup_Signals

When receiving an SLVERR or DECERR response to a downstream transaction, the TBM interface propagates the same response to the TBS interface.

See also:

- [2.5.10 Error responses](#) on page 71
- [2.6.2 AMBA implementation](#) on page 81
- [A.2.3 TBU TBM interface signals](#) on page 312

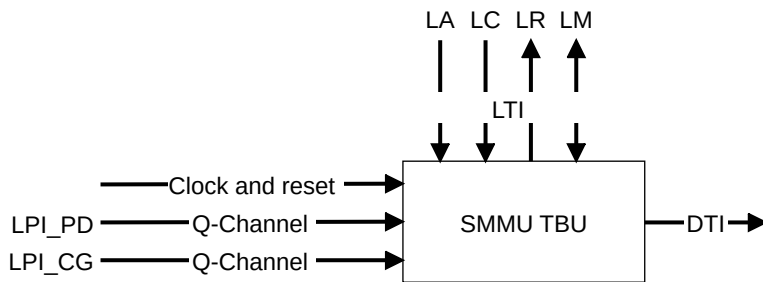
2.4.2.3 LTI TBU LTI interface

There are multiple LTI TBU variants, with 1, 2, 4, and 8 LTI interfaces. Each LTI interface is a complete interface. Most of the parameters that you can use to configure an LTI interface are shared between all of them on the same TBU.

An exception to this sharing is the register slice modes on the LA and LR channels. For more information, see the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

The following figure shows the LTI TBU interfaces.

Figure 2-9: LTI TBU interfaces



LTI TBUs can have variants with 1, 2, 4, and 8 LTI interfaces. The figure shows a TBU with one LTI interface.

The interface contains the following channels:

LA	Request channel. Address and attributes that require translation are sent to the TBU.
LR	Response channel. Provides the translated address and attributes to the LTI device.
LC	Completion channel. LTI devices must provide information about completion to the TBU.
LM	Link Management channel. Contains: • LMOPENREQ • LMOPENACK • LMASKCLOSE • LMACTIVE

See also:

- [2.6.3 Local Translation Interface implementation](#) on page 92
- [2.7 Configuration parameters and methodology](#) on page 102
- [A.2.7 TBU LTI interface signals](#) on page 319
- [AMBA® LTI Protocol Specification](#)

2.4.2.4 TBU LPI_PD interface

The TBU LPI_PD Q-Channel completer interface manages Low Power Interface (LPI) powerdown (PD) for the TBU.

See also:

- [A.2.4 TBU LPI_PD interface signals](#) on page 317
- [AMBA® Low Power Interface Specification](#)

2.4.2.5 TBU LPI_CG interface

The TBU LPI_CG Q-Channel completer interface enables Low Power Interface (LPI) clock gating (CG) for the TBU.

See also:

- [A.2.5 TBU LPI_CG interface signals](#) on page 318
- [AMBA® Low Power Interface Specification](#)

2.4.2.6 TBU DTI interface

The TBU DTI interface enables the TBU to request translations from the TCU. This interface uses the DTI-TBU protocol for communication between the TBU and the TCU.

The TCU includes a TCU DTI interface and each TBU includes a TBU DTI interface. To permit bidirectional communication, each DTI interface includes:

Downstream interface

One AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, interface

Upstream interface

One AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, interface.

See also:

- [2.5.1 Distributed Translation Interface](#) on page 45
- [A.2.6 TBU DTI interface signals](#) on page 318
- [AMBA® DTI Protocol Specification](#)
- [AMBA® AXI-Stream Protocol Specification](#)

2.4.2.7 TBU interrupt interface

The TBU interrupt interface provides edge-triggered and level-triggered global, per-context, and performance interrupt signals.

See also:

- [2.5.1 Distributed Translation Interface](#) on page 45
- [A.2.9 TBU tie-off signals](#) on page 320
- [A.2.8 TBU interrupt signals](#) on page 319

2.4.2.8 TBU tie-off signals

The TBU tie-off signals enable you to initialize various operating parameters on exit from reset state.

See also:

- [A.2.9 TBU tie-off signals](#) on page 320
- [A.2.8 TBU interrupt signals](#) on page 319

2.4.2.9 TBU ELA observation interface

This Embedded Logic Analyzer (ELA) observation requester interface drives the signal group, signal qualifier, and signal clock enable ELA signals to the on-chip ELA module, if present.

When `TBUCFG_USE_ELA_DEBUG` is 0, these signals are tied to 0. See [2.7.7 Common TBU debug configuration parameters](#) on page 108.

See also:

- [B.2 ACE-Lite TBU observation interfaces](#) on page 372
- [B.3 LTI TBU observation interfaces](#) on page 375
- *Arm® CoreSight™ ELA-600 Embedded Logic Analyzer Technical Reference Manual*

2.4.3 DTI interconnect interfaces

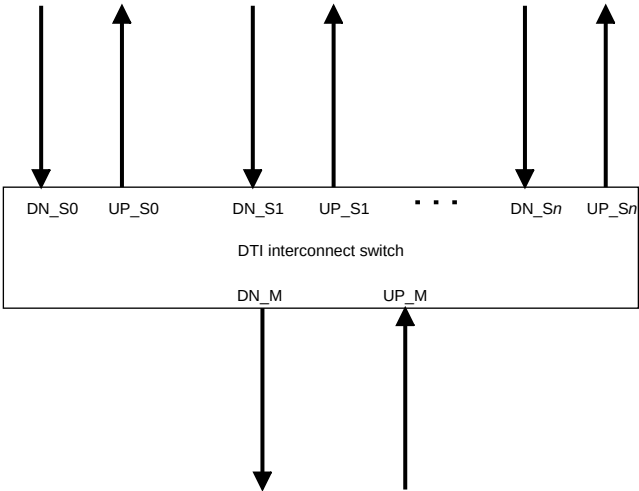
The DTI interconnect includes interfaces for each of the switch, sizer, and register slice components.

2.4.3.1 DTI interconnect switch interfaces

The DTI interconnect switch component includes dedicated interfaces.

The following figure shows the DTI interconnect switch interfaces.

Figure 2-10: DTI interconnect switch interfaces



The following table provides more information about the switch interfaces.

Table 2-1: DTI interconnect switch interfaces

Interface	Data flow	Side	Description
DN_Sn	Downstream	TBU side	TCU downstream interface. One DN_Sn interface is present for each TBU interface.
UP_Sn	Upstream	TBU side	TCU upstream interface. One UP_Sn interface is present for each TBU interface.
DN_M	Downstream	TCU side	TBU downstream interface
UP_M	Upstream	TCU side	TBU upstream interface



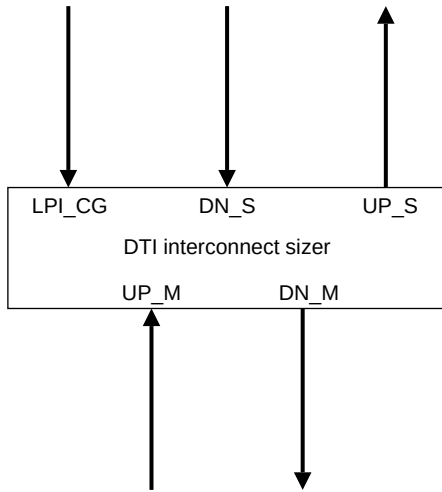
The interconnect switch does not store any data, and therefore does not require a Q-Channel clock gating interface.

2.4.3.2 DTI interconnect sizer interfaces

The DTI interconnect sizer component includes dedicated interfaces.

The following figure shows the DTI interconnect sizer interfaces.

Figure 2-11: DTI interconnect sizer interfaces



The following table provides more information about the sizer interfaces.

Table 2-2: DTI interconnect sizer interfaces

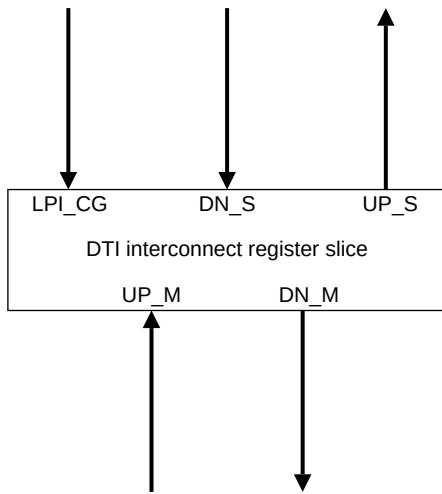
Interface	Data flow	Facing	Description
LPI_CG	-	Facing the clock controller	Clock gating interface
DN_S	Downstream	TBU side	TCU downstream interface
UP_S	Upstream	TBU side	TCU upstream interface
DN_M	Downstream	TCU side	TBU downstream interface
UP_M	Upstream	TCU side	TBU upstream interface

2.4.3.3 DTI interconnect register slice interfaces

The DTI interconnect register slice component includes dedicated interfaces.

The following figure shows the DTI interconnect register slice interfaces.

Figure 2-12: DTI interconnect register slice interfaces



The following table provides more information about the register slice interfaces.

Table 2-3: DTI interconnect register slice interfaces

Interface	Data flow	Facing	Description
LPI_CG	-	Facing the clock controller	Clock gating interface
DN_S	Downstream	TBU side	TCU downstream interface
UP_S	Upstream	TBU side	TCU upstream interface
DN_M	Downstream	TCU side	TBU downstream interface
UP_M	Upstream	TCU side	TBU upstream interface

2.5 Operation

MMU-720AE contains components that operate together.

2.5.1 Distributed Translation Interface

In an MMU-720AE-based system, the AMBA® Distributed Translation Interface (DTI) protocol defines the standard for communicating with a TCU.

The AMBA® DTI protocol includes the following:

DTI-TBU protocol

For communication between a TBU and a TCU.

DTI-ATS protocol

For communication between a PCIe Root Port and a TCU.

The DTI protocol is a point-to-point protocol. Each channel consists of the following:

- Link
- DTI-TBU
- DTI-TCU

The DTI requesters in the respective protocols are as follows:

- The TBU, in the DTI-TBU protocol
- The PCIe Root Port, in the DTI-ATS protocol

The DTI completer, in both DTI-TBU and DTI-ATS is the TCU.

DTI requesters, which are TBUs, and completers, which are TCUs, communicate using defined DTI messages. The DTI protocol defines the following message groups:

- Page request.
- Register access.
- Translation request.
- Connection and disconnection.
- Invalidation and synchronization.

A DTI requester, a TBU, uses a DTI_TBU_CONDIS_REQ or a DTI_ATS_CONDIS_REQ message to initiate a connection handshake. If the requester (TBU) provides a TID value that is greater than the maximum supported TID that `TCUCFG_NUM_TBU` defines, the completer, the TCU, sends a Connect Deny message.

For the TBU, the `max_tok_trans` signal defines the number of translation tokens that the TBU requests. The TBU must request a minimum of two translation tokens per LTI port.

The TBU uses the `TOK_INV_GNT` field to grant invalidation tokens. The TBU grants only one invalidation token, and the TCU can only issue one invalidate message at a time.

The DTI_TBU_CONDIS_REQ message initiates a TBU connection or disconnection handshake. The TBU uses this message to connect to the TCU. This handshake process also applies to ATS connections. During connection, the `TOK_TRANS_REQ` field of this message specifies the number of requested translation tokens.

A translation request to the TCU where $\text{StreamID} \geq 2^{32}$ results in a fault and an SMMUv3 `C_BAD_STREAMID` event. If the TBU receives an invalidation request where $\text{StreamID} \geq 2^{32}$, any comparisons with a StreamID value fail. No TLB entries are invalidated, but other effects, that do not consider the supplied StreamID, occur as normal.



- The TBU never generates translation requests with $\text{StreamID} \geq 2^{32}$
- The TCU never generates invalidation requests with $\text{StreamID} \geq 2^{32}$

For more information, see the [AMBA® DTI Protocol Specification](#).

2.5.2 Performance Monitoring Unit

MMU-720AE includes a PMU for the TCU and a PMU for each TBU. The PMU events and counters indicate the runtime performance of MMU-720AE.

MMU-720AE includes logic to gather various statistics on the operation of MMU-720AE during runtime, using events and counters. These events, which the SMMUv3 architecture defines, provide useful information about the behavior of MMU-720AE. You can use this information when debugging or profiling traffic.

2.5.2.1 SMMUv3 architectural performance events

Both the TCU and the TBU implement performance events that the SMMUv3 Performance Monitor extension defines.

The SMMU_PMCG_SMR0 register can filter some events so that only events with a particular StreamID are counted. This event filtering includes:

- Speculative transactions and translations
- Transactions and translations that result in a terminated transaction or a translation fault

The following table shows the architecturally defined MMU-720AE TCU performance events.

Table 2-4: SMMUv3 performance events for the TCU

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
Clock cycle	0x0	No	Counts clock cycles. Cycles where the clock is gated after a clock Q-Channel handshake are not counted.
Transaction	0x1	Yes	Counts translation requests that originate from a DTI-TBU or DTI-ATS requester
TLB miss caused by incoming transaction or translation request	0x2	Yes	Counts translation requests where the translation walks new translation table entries
Configuration cache miss caused by transaction or translation request	0x3	Yes	Counts translation requests where the translation walks new configuration table entries
Translation table walk access	0x4	Yes	Counts translation table walk accesses
Configuration structure access	0x5	Yes	Counts configuration table walk accesses
PCIe ATS Translation Request received	0x6	Yes	Counts translation requests that originate from a DTI-ATS requester

The following table shows the architecturally defined MMU-720AE TBU performance events.

Table 2-5: SMMUv3 performance events for the TBU

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
Clock cycle	0x0	No	Counts clock cycles. Cycles where the clock is gated after a clock Q-Channel handshake are not counted.
Transaction	0x1	Yes	Counts transactions that are issued on the TBM interface
TLB miss caused by incoming transaction or translation request	0x2	Yes	Counts speculative and non-speculative translation requests that are issued to the TCU
PCIe ATS Translation Request received	0x7	Yes	Counts ATS-translated transactions that are issued on the TBM interface

For more information, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

2.5.2.2 SMMU TCU events

You can configure the MMU-720AE PMU to monitor a range of **IMPLEMENTATION DEFINED** TCU performance events.

The SMMU_PMCG_SMR0 register can filter some TCU performance events so that only events with a particular StreamID are counted. This event filtering includes:

- Speculative transactions and translations.
- Transactions and translations that result in a terminated transaction or a translation fault.

The following table shows the TCU performance events.

Table 2-6: MMU-720AE TCU performance events

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
S1LOWC lookup	0x80	Yes	This event counts translation requests that access S1LOWC entries in the walk cache.
S1LOWC miss	0x81	Yes	This event counts translation requests that access S1LOWC entries in the walk cache and do not result in a hit.
S1L1WC lookup	0x82	Yes	This event counts translation requests that access S1L1WC entries in the walk cache.
S1L1WC miss	0x83	Yes	This event counts translation requests that access S1L1WC entries in the walk cache and do not result in a hit.
S1L2WC lookup	0x84	Yes	This event counts translation requests that access S1L2WC entries in the walk cache.
S1L2WC miss	0x85	Yes	This event counts translation requests that access S1L2WC entries in the walk cache and do not result in a hit.
S1L3WC lookup	0x86	Yes	This event counts translation requests that access S1L3WC entries in the walk cache.

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
S1L3WC miss	0x87	Yes	This event counts translation requests that access S1L3WC entries in the walk cache and do not result in a hit.
S2LOWC lookup	0x88	Yes	This event counts translation requests that access S2LOWC entries in the walk cache.
S2LOWC miss	0x89	Yes	This event counts translation requests that access S2LOWC entries in the walk cache and do not result in a hit.
S2L1WC lookup	0x8A	Yes	This event counts translation requests that access S2L1WC entries in the walk cache.
S2L1WC miss	0x8B	Yes	This event counts translation requests that access S2L1WC entries in the walk cache and do not result in a hit.
S2L2WC lookup	0x8C	Yes	This event counts translation requests that access S2L2WC entries in the walk cache.
S2L2WC miss	0x8D	Yes	This event counts translation requests that access S2L2WC entries in the walk cache and do not result in a hit.
S2L3WC lookup	0x8E	Yes	This event counts translation requests that access S2L3WC entries in the walk cache.
S2L3WC miss	0x8F	Yes	This event counts translation requests that access S2L3WC entries in the walk cache and do not result in a hit.
WC read	0x90	Yes	This event counts reads from the walk cache RAMs, excluding reads that invalidation requests cause. Note: A single walk cache lookup might result in multiple RAM reads. This behavior permits contiguous entries to be located.
Buffered translation	0x91	Yes	This event counts translations that are written to the translation request buffer because either all the configuration table walk slots or all the page table walk slots are occupied.
CC lookup	0x92	Yes	This event counts lookups into the configuration cache.
CC read	0x93	Yes	This event counts reads from the Configuration Cache (CC) RAMs, excluding reads that invalidation requests cause. Note: A single cache lookup might result in multiple RAM reads. This behavior permits contiguous entries to be located.
CC miss	0x94	Yes	This event counts lookups into the CC that do not result in a hit.
WMB is blocked from sending a request to WCB because of backpressure	0x95	No	Walk Management Block (WMB) is blocked from sending a request to Walk Cache Block (WCB) because of backpressure.
WC lookup slots full	0x96	No	Walk cache lookup slots are full.
WC update slots full	0x97	No	Walk cache RAM update slots are full.
CMB is blocked from sending a request to CCB because of back pressure	0x98	No	Configuration Management Block (CMB) is blocked from sending a request to the CCB because of back pressure.
PTW slots full	0x99	No	Page table walk slots are full.
CTW slots full	0x9A	No	Configuration table walk slots are full.

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
CC lookup slots full	0x9B	No	Lookup slots in the CC are full.
CC update slots full	0x9C	No	Update slots in the CC are full.
Speculative translation	0xA0	Yes	This event counts translation requests that are marked as speculative.
STE prefetch	0xA1	Yes	Stream Table Entry prefetch has occurred.
HTTU	0xA2	Yes	The Hardware Translation Table Update has occurred.
AGW GPC Cache (GC) lookup	0xB0	Yes	AXI Granule Protection Check (GPC) Wrapper (AGW) GC lookup.
AGW GC miss	0xB1	Yes	AGW GC miss.
AGW GC read	0xB2	Yes	AGW GC.
AGW GMB request	0xB5	Yes	AGW GPC Management Block (GMB) request.
AGW is blocked from sending a request to GCB because of back pressure	0xB6	No	AGW is blocked from sending a request to GPT cache block because of back pressure.
AGW GPC Cache Block (GCB) lookup slots full	0xB7	No	Lookup slots in the AGW GPT cache block are full.
AGW GCB update slots full	0xB8	No	Update slots in the AGW GPT cache block are full.
DGW GC lookup	0xC0	Yes	DTI GPT Wrapper (DGW) GC lookup.
DGW GC miss	0xC1	Yes	DGW GC miss.
DGW GC read	0xC2	Yes	DGW GC read.
DGW GMB request	0xC5	Yes	DGW GMB request.
DGW is blocked from sending a request to GCB because of back pressure	0xC6	No	DGW is blocked from sending a request to GCB because of back pressure.
DGW GC lookup slots full	0xC7	No	DGW GC lookup slots full.
DGW GC update slots full	0xC8	No	DGW GC update slots full.
DGW out of credits	0xC9	No	All DGW slots are reserved for use.
Primary hazard hit	0xD0	No	Primary hazard hit.
Hazard created, new primary hazard entered into primary hazard cache	0xD1	No	Hazard created, new primary hazard entered into primary hazard cache.
Primary hazard cache full, all primary hazard slots in use	0xD2	No	Primary hazard cache full, all primary hazard slots in use.
Hazard hit in secondary HZU	0xD3	No	Hazard hit in secondary HZU.
Hazard created, new hazard entered into secondary hazard cache	0xD4	No	Hazard created, new hazard entered into secondary hazard cache.
Secondary hazard cache full, all secondary hazard slots in use	0xD5	No	Secondary hazard cache full, all secondary hazard slots in use.
As event 2 but only counts page table misses	0xE2	Yes	As event 2 but only counts page table misses.
As event 4 but only counts page table walks, not GPC	0xE4	Yes	As event 4 but only counts page table walks, not GPC.
HIT_MAP_1_4	0xE8	Yes	CCB update received per category, with SID cont ≤ 2.
HIT_MAP_8_128	0xE9	Yes	CCB update received per category, with SID cont >2 and ≤ 7.

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
HIT_MAP_256_MAX	0xEA	Yes	CCB update received per category, with SID cont ≥ 8.
HIT_MAP_04K_L0_C0_S1	0xEB	Yes	WCB update received per category, 04K_L0_C0_S1 update.
HIT_MAP_04K_L1_C0_S1	0xEC	Yes	WCB update received per category, 04K_L1_C0_S1 update.
HIT_MAP_04K_L1_C1_S1	0xED	Yes	WCB update received per category, 04K_L1_C1_S1 update.
HIT_MAP_04K_L2_C0_S1	0xEE	Yes	WCB update received per category, 04K_L2_C0_S1 update.
HIT_MAP_04K_L2_C1_S1	0xEF	Yes	WCB update received per category, 04K_L2_C1_S1 update.
HIT_MAP_04K_L3_C0_S1	0xF0	Yes	WCB Update received per category, 04K_L3_C0_S1 update.
HIT_MAP_04K_L3_C1_S1	0xF1	Yes	WCB update received per category, 04K_L3_C1_S1 update.
HIT_MAP_16K_L0_C0_S1	0xF2	Yes	WCB update received per category, 16K_L0_C0_S1 update.
HIT_MAP_16K_L1_C0_S1	0xF3	Yes	WCB update received per category, 16K_L1_C0_S1 update.
HIT_MAP_16K_L2_C0_S1	0xF4	Yes	WCB update received per category, 16K_L2_C0_S1 update.
HIT_MAP_16K_L2_C1_S1	0xF5	Yes	WCB update received per category, 16K_L2_C1_S1 update.
HIT_MAP_16K_L3_C0_S1	0xF6	Yes	WCB update received per category, 16K_L3_C0_S1 update.
HIT_MAP_16K_L3_C1_S1	0xF7	Yes	WCB update received per category, 16K_L3_C1_S1 update.
HIT_MAP_64K_L1_C0_S1	0xF8	Yes	WCB update received per category, 64K_L1_C0_S1 update.
HIT_MAP_64K_L2_C0_S1	0xF9	Yes	WCB update received per category, 64K_L2_C0_S1 update.
HIT_MAP_64K_L2_C1_S1	0xFA	Yes	WCB update received per category, 64K_L2_C1_S1 update.
HIT_MAP_64K_L3_C0_S1	0xFB	Yes	WCB update received per category, 64K_L3_C0_S1 update.
HIT_MAP_64K_L3_C1_S1	0xFC	Yes	WCB update received per category, 64K_L3_C1_S1 update.
HIT_MAP_04K_L0_C0_S2	0xFD	Yes	WCB update received per category, 04K_L0_C0_S2 update.
HIT_MAP_04K_L1_C0_S2	0xFE	Yes	WCB update received per category, 04K_L1_C0_S2 update.
HIT_MAP_04K_L1_C1_S2	0xFF	Yes	WCB update received per category, 04K_L1_C1_S2 update.
HIT_MAP_04K_L2_C0_S2	0x100	Yes	WCB update received per category, 04K_L2_C0_S2 update.
HIT_MAP_04K_L2_C1_S2	0x101	Yes	WCB update received per category, 04K_L2_C1_S2 update.
HIT_MAP_04K_L3_C0_S2	0x102	Yes	WCB update received per category, 04K_L3_C0_S2 update.
HIT_MAP_04K_L3_C1_S2	0x103	Yes	WCB update received per category, 04K_L3_C1_S2 update.
HIT_MAP_16K_L0_C0_S2	0x104	Yes	WCB update received per category, 16K_L0_C0_S2 update.
HIT_MAP_16K_L1_C0_S2	0x105	Yes	WCB update received per category, 16K_L1_C0_S2 update.
HIT_MAP_16K_L2_C0_S2	0x106	Yes	WCB update received per category, 16K_L2_C0_S2 update.
HIT_MAP_16K_L2_C1_S2	0x107	Yes	WCB update received per category, 16K_L2_C1_S2 update.
HIT_MAP_16K_L3_C0_S2	0x108	Yes	WCB update received per category, 16K_L3_C0_S2 update.
HIT_MAP_16K_L3_C1_S2	0x109	Yes	WCB update received per category, 16K_L3_C1_S2 update.
HIT_MAP_64K_L1_C0_S2	0x10A	Yes	WCB update received per category, 64K_L1_C0_S2 update.
HIT_MAP_64K_L2_C0_S2	0x10B	Yes	WCB update received per category, 64K_L2_C0_S2 update.
HIT_MAP_64K_L2_C1_S2	0x10C	Yes	WCB update received per category, 64K_L2_C1_S2 update.
HIT_MAP_64K_L3_C0_S2	0x10D	Yes	WCB update received per category, 64K_L3_C0_S2 update.
HIT_MAP_64K_L3_C1_S2	0x10E	Yes	WCB update received per category, 64K_L3_C1_S2 update.
Active Invalidate Mode Active TMU DGW GMB	0x111	Yes	This event counts per cycle the instance is in active invalidate mode.

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
Active Invalidate Mode Active TMU AGW GMB	0x112	Yes	This event counts per cycle the instance is in active invalidate mode.
Active Invalidate Mode Active TMU WMB	0x113	Yes	This event counts per cycle the instance is in active invalidate mode.
DGW_HM_UPDATE_L1_GRAN	0x116	Yes	One GCB update received per category, L1_GRAN update.
DGW_HM_UPDATE_L1_2M	0x117	Yes	One GCB update received per category, L1_2M update.
DGW_HM_UPDATE_L1_32M	0x118	Yes	One GCB update received per category, L1_32M update.
DGW_HM_UPDATE_L1_512M	0x119	Yes	One GCB update received per category, L1_512M update.
DGW_HM_UPDATE_L0_CONT	0x11A	Yes	One GCB update received per category; L0 leaf update.
DGW_HM_UPDATE_L0_TBL	0x11B	Yes	One GCB update received per category, L0 table update.
AGW_HM_UPDATE_L1_GRAN	0x11E	Yes	One GCB update received per category, L1_GRAN update.
AGW_HM_UPDATE_L1_2M	0x11F	Yes	One GCB update received per category, L1_2M update.
AGW_HM_UPDATE_L1_32M	0x120	Yes	One GCB update received per category, L1_32M update.
AGW_HM_UPDATE_L1_512M	0x121	Yes	One GCB update received per category, L1_512M update.
AGW_HM_UPDATE_L0_CONT	0x122	Yes	One GCB update received per category L0 leaf update.
AGW_HM_UPDATE_L0_TBL	0x123	Yes	One GCB update received per category, L0 table update.



Note

A single DTI translation request might correspond to multiple translation request events in either of the following circumstances:

- A translation results in a stall fault event and is restarted.
- If a translation results in a stall fault event, because the Event queue is full, the translation is retried when an Event queue slot becomes available.

2.5.2.3 SMMU TBU events

The MMU-720AE PMU can be configured to monitor a range of **IMPLEMENTATION DEFINED** TBU performance events.

The SMMU_PMCG_SMR0 register can filter the TBU performance events so that only events with a particular StreamID are counted. This event filtering includes:

- Speculative transactions and translations
- Transactions and translations that result in a terminated transaction or a translation fault

The following table shows the TBU performance events.

Table 2-7: MMU-720AE TBU performance events

Event	Event ID	SMMU_PMCG_SMR0 filterable	Description
Main TLB lookup	0x80	Yes	This event counts Main TLB lookups.

Event	Event ID	SMMU_PMCG_SMRO filterable	Description
Main TLB miss	0x81	Yes	This event counts translation requests that miss in the Main TLB.
Main TLB read	0x82	Yes	This event counts once per access to the Main TLB RAMs, excluding reads that invalidation requests cause. Note: A transaction might access the Main TLB multiple times to look for different page sizes.
MicroTLB lookup	0x83	Yes	This event counts MicroTLB lookups.
MicroTLB miss	0x84	Yes	This event counts translation requests that miss in the MicroTLB.
Translation slots full	0x85	No	This event counts once per cycle when all slots are occupied and not ready to issue transactions downstream. This Secure event is visible only when the SMMU_PMCG_SCR.SO bit is set to 1.
Out of translation tokens	0x86	No	This event counts once per cycle when a translation request cannot be issued because all translation tokens are in use. This Secure event is visible only when the SMMU_PMCG_SCR.SO bit is set to 1.
Write data buffer full	0x87	No	This event counts once per cycle when a transaction is blocked because the write data buffer is full. This Secure event is visible only when the SMMU_PMCG_SCR.SO bit is set to 1.
DCMO downgrade	0x8B	Yes	For the ACE-Lite TBU, this event counts when either: - A MakeInvalid transaction on the TBS interface is output as CleanInvalid on the TBM interface. - A ReadOnceMakeInvalid transaction on the TBS interface is output as ReadOnceCleanInvalid on the TBM interface. For the LTI TBU, this event counts once per cycle when an LTI DCMO or R-DCMO transaction on the LA channel is responded to with a downgrade on the LR channel.

Event	Event ID	SMMU_PMCG_SMRO filterable	Description
Stash fail	0x8C	Yes	For the ACE-Lite TBU, this event counts when either: - A WriteUniquePtlStash or WriteUniqueFullStash transaction on TBS is output as a WriteNoSnoop or WriteUnique transaction on the TBM interface. - A StashOnceShared or StashOnceUnique transaction on the TBS interface has a valid translation, but is terminated in the TBU. For the LTI TBU, this event counts once whenever either an: - LTI WDCP transaction on the LA channel is downgraded as W on the LR channel. - LTI DCP transaction on the LA channel that is responded to as FaultRAZWI on the LR channel is counted. This can be because of: - Memory attributes or DCP, R, W, or X permission check failure in the Translation Lookaside Buffer Unit (TLBU). - DTI fault response with Non-Abort. The transaction that is responded to with FaultAbort because of DTI StreamDisable or GlobalDisable is not counted. Note: A StashOnceShared or StashOnceUnique transaction that is terminated because of a StreamDisable or GlobalDisable translation response does not cause this event to count.
Fastpath taken	0x8D	Yes	This event counts each time the fastpath is taken. The fastpath is a lower latency path through the TBU. You can use a fastpath when the following conditions are true: - The TBU is an ACE-Lite or LTI TBU with only one interface. - The request is unordered, that is, AxIDUNQ == 1 or LAOGV == 0. - A uTLB hit occurs. - Responses are being accepted. - No existing transactions are currently using the slow path.
GPC only request	0x8E	Yes	This event counts each GPC only request. A GPC only request occurs when a request arrives with AxMMUVALID == 0 or LAMMUV == 0.
Fixed burst terminated in TBU	0x8F	Yes	This event counts each time a fixed burst is terminated in the TBU. Fixed bursts are terminated in the TBU if the fixed burst would otherwise be output as cacheable and outer shareable. AXI does not support shareable fixed bursts.
MTLB lookup slots full	0x96	No	This event indicates that the MTLB lookup slots are full.
MTLB update slots full	0x97	No	This event indicates that the MTLB update slots are full.
LTI port slots full	0xD0 - 0xD7	No	LTI port event (0xD0 + N) corresponds to LTI port N. This event counts once per cycle when the slots that are allocated to the LTI port are all occupied and not ready to issue downstream.
LTI port out of translation tokens	0xE0 - 0xE7	No	LTI port event (0xD0 + N) corresponds to LTI port N. This event counts once per cycle when a translation request cannot be issued for an LTI port because all its allocated translation tokens are in use.

Event	Event ID	SMMU_PMCG_SMRO filterable	Description
Populate with event Hazard hit, request not sent downstream because of hazard	0xF0	Yes	This event counts each hazard hit. This event occurs if the hazarding mechanism in the TBU has been able to prevent a translation request going to the TCU because it matches an existing request outstanding to the TCU. When the existing translation response returns, it is also used to satisfy this translation request.
Populate with event Hazard created, new hazard entered into hazard cache	0xF1	Yes	This event counts each time a hazard is created and indicates that a new hazard cache entry has been created. This event means that the translation request has been sent to the TCU and a hazard has been set up to enable future translation requests with the same request information to hazard against it, and potentially share the response.
Populate with event Hazard cache full, all hazard slots in use	0xF2	Yes	This event counts each time a hazard is created when all hazard slots are in use, thereby discarding an existing hazard, and indicates that the maximal number of hazards have been set up in the hazard cache. New hazards are still created, but only by removing existing hazards. This can mean missing opportunities to benefit from hazarding. If hazarding is low, this is expected. However, if the potential for hazarding in your traffic is high, increasing the value of the <code>TBUCFG_HZRD_ENTRIES</code> parameter can be beneficial. See the <code>TBUCFG_HZRD_ENTRIES</code> parameter in the <i>Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual</i> .
upd_trng_004k	0xF3	Yes	MTLB update request with <code>TRANS_RNG</code> = 4k.
upd_trng_016k	0xF4	Yes	MTLB Update request with <code>TRANS_RNG</code> = 16k.
upd_trng_064k	0xF5	Yes	MTLB Update request with <code>TRANS_RNG</code> = 64k.
upd_trng_002m	0xF6	Yes	MTLB Update request with <code>TRANS_RNG</code> = 2M.
upd_trng_032m	0xF7	Yes	MTLB Update request with <code>TRANS_RNG</code> = 32M.
upd_trng_512m	0xF8	Yes	MTLB Update request with <code>TRANS_RNG</code> = 512M.
upd_trng_001g	0xF9	Yes	MTLB Update request with <code>TRANS_RNG</code> = 1G.
upd_trng_016g	0xFA	Yes	MTLB Update request with <code>TRANS_RNG</code> = 16G.
upd_trng_064g	0xFB	Yes	MTLB Update request with <code>TRANS_RNG</code> = 64G.
upd_trng_512g	0xFC	Yes	MTLB Update request with <code>TRANS_RNG</code> = 512G.
upd_trng_004t	0xFD	Yes	MTLB Update request with <code>TRANS_RNG</code> = 4T.
upd_trng_full	0xFE	Yes	MTLB Update request with <code>TRANS_RNG</code> = FULL.
MTLB Plim RD	0xFF	No	MTLB PLIM reads.
MTLB Plim WR	0x100	No	MTLB PLIM writes.
MTLB Pcnt RD	0x101	No	MTLB PCNT reads.
MTLB Pcnt WR	0x102	No	MTLB PCNT writes.
MTLB Repl RD	0x103	No	MTLB REPL reads.
MTLB Repl WR	0x104	No	MTLB REPL writes.
MTLB Tgdt RD	0x105	No	MTLB TAGS reads.
MTLB Tgdt WR	0x106	No	MTLB TAGS writes.

2.5.2.4 SMMUv3 PMU register architectural options

The SMMUv3 architecture defines the Performance Monitor Counter Group (PMCG) configuration register, SMMU_PMCG_CFGR. An MMU-720AE implementation assumes fixed values for SMMU_PMCG_CFGR, and these values define behavioral aspects of the implementation.

The following table shows the SMMU_PMCG_CFGR register options that the MMU-720AE TCU and TBU use.

Table 2-8: MMU-720AE SMMU_PMCG_CFGR register architectural options

Field		Default value	Description for default value
SID_FILTER_TYPE		1	A single StreamID filter applies to all PMCG counters
CAPTURE		1	Capture of counter values into SVRn registers is supported
MSI		0	The counter group does not support Message Signaled Interrupts (MSIs)
RELOC_CTRS		1	The PMCG registers are relocated to page 1 of the PMU address map
SIZE		0x31	The counter group implements 32-bit counters
MPAM		0	Memory System Resource Partitioning and Monitoring (MPAM)
NCTR	NCTR for the TCU	See Description	The counter group includes TCUCFG_PMU_COUNTERS counters. See 2.7.1 TCU I/O and buffer configuration parameters on page 103. The default value is TCUCFG_PMU_COUNTERS - 1
	TCTR for the TBU	See Description	The counter group includes TBUCFG_PMU_COUNTERS counters. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105. The default value is TBUCFG_PMU_COUNTERS - 1

See also [4.3 SMMU memory map](#) on page 127.

2.5.2.5 PMU snapshot interface

The Performance Monitoring Unit (PMU) snapshot interface is included on the TCU and on each TBU. You can use this asynchronous interface to initiate a PMU snapshot. A simultaneous snapshot of each counter register is created and copied to the respective SMMU_PMCG_SVRn register.

The PMU snapshot sequence is a 4-phase handshake. Both pmusnapshot_req and pmusnapshot_ack are LOW after reset. A snapshot occurs on the rising edge of pmusnapshot_req, and is equivalent to writing the value 1 to SMMU_PMCG_CAPR.CAPTURE.

The pmusnapshot_req signal is sampled using synchronizing registers. A register drives pmusnapshot_ack so that the connected component can sample the signal asynchronously.

See also:

- [2.5.5 RAS implementation](#) on page 59
- [A.3.2 PMU Snapshot signals](#) on page 324

2.5.3 Multiple LTI interface TBU

There are variants of the LTI TBU that have 1, 2, 4, or 8 LTI interfaces. The traffic through these interfaces is multiplexed together and they share a single translation manager, TLB, and DTI interface.

Registers are provided to enable you to set limits on the maximum proportion of these shared resources any individual LTI interface can use. This limiting applies to entries in the translation managing structure and DTI translation tokens. See the [4. Programmers model for MMU-720AE](#) on page 119.

A given LTI interface is not permitted to issue more translation requests into the TBU if it already uses the limit or more than the limit of any of the types of managed resource. If the limits are changed while an interface has outstanding translation requests, and this results in the interface using more than the new permitted proportion, it is unable to issue translation requests until the resource usage returns to below the permitted maximum. To avoid deadlock, each LTI interface must be guaranteed to be able to have at least 2 outstanding DTI requests and 2 outstanding LTI translation requests, one on each virtual channel on the LTI interface.

As a result, when there is more than one LTI interface, the maximum number of translation requests that any individual LTI interface might have outstanding is less than the total number possible across all interfaces. An LTI transaction is considered to be outstanding for these purposes for its full life according to the [AMBA® LTI Protocol Specification](#). An LTI transaction is only considered to be using a DTI translation token until the translation manager determines that it is not required to perform a DTI translation request in the future. This determination can be because it has completed a DTI translation request or because it is not necessary to perform one because it gets a hit response to its TLB lookup or it receives a hazard response from another transaction ahead of it.

In all cases, a DTI translation token is required for a transaction to enter the LTI TBU because at the point that it enters the TBU, it cannot be known whether it requires a DTI translation request or not. Because the LTI TBU cannot guarantee deadlock freedom unless it receives at least 2 DTI credits per LTI interface, it fails to exit the Q_STOPPED LPI state if the max_trans_tok tie off signal does not indicate at least that number of tokens should be requested at connection. Similarly, on connection, if the TCU does not grant the minimum number of credits, then the TBU does not unfence its LTI interface, and instead initiates a DTI disconnection when this connection condition is detected.

2.5.4 Main TLB direct indexing and main TLB direct partitioning

Main TLB direct indexing can help your system to meet real-time translation requirements by enabling management of Main TLB (MTLB) entries from outside the TBU.



Note

If you use the Main TLB direct indexing or Main TLB direct partitioning features, the MPAM information is not used to control the allocation of the Main TLB.

Direct indexing enables real-time translation requirements to be met, as follows:

- You can guarantee that prefetched entries are not overwritten.
- The MTLB can be partitioned into different sets of entries that different streams use.

When direct indexing is enabled for a TBU:

- Lookups and updates to the MTLB use the mtlbidx field as the index.
- Updates to the MTLB use the way that mtlbway specifies.
- Lookups to the MTLB operate on all ways simultaneously.

If you configure your system to not use MTLB direct indexing, you can select MTLB direct partitioning. MTLB direct partitioning has similar behavior, but only the most significant TLB index bits must be provided externally, and the other bits are generated internally.

Direct indexing is enabled for a TBU when the `TBUCFG_DIRECT_IDX` parameter is set to 1.

When `TBUCFG_DIRECT_IDX` is set to 1, or when an MTLB is partitioned, the width of the `AxUSER` signals on the TBS interface is extended to convey the indexing information that is required for MTLB direct indexing or MTLB direct partitioning.

If `TBUCFG_DIRECT_IDX` is set to 0 and `TBUCFG_MTLB_PART` is set to 1 then `LATLBLOC` is ignored and can be tied to 0.

Indexing information is sent using the `LATLBLOC` signal for the LTI TBU. See [A.5.1 LTI request channel signals](#) on page 331.



The table shows the extended bits in the order MSB first.

Table 2-9: Extended aruser_s and awuser_s bits for MTLB partitioning

Field name	Width	Description
mtlbidx	When direct indexing is enabled, the width of this field is $\log_2(\text{TBUCFG_MTLB_DEPTH}) - \log_2(\text{TBUCFG_MTLB_WAYS})$. When direct indexing is not enabled, the width of this field is 0.	MTLB index
mtlbway	When direct indexing is enabled, the width of this field is $\log_2(\text{TBUCFG_MTLB_WAYS})$. When direct indexing is not enabled, the width of this field is 0.	MTLB way
mtlbpart	$\log_2(\text{TBUCFG_MTLB_PARTS})$	MTLB partition
-	<code>TBUCFG_AWUSER_WIDTH</code> for <code>awuser_s</code> . <code>TBUCFG_ARUSER_WIDTH</code> for <code>aruser_s</code> .	Regular <code>AxUSER</code> signals

If an MTLB is partitioned:

- The MTLB size is divided into a number of equal segments that the value of the `TBUCFG_MTLB_PARTS` parameter defines.
- The `mtlbp` field defines the $\log_2(\text{TBUCFG_MTLB_PARTS})$ most significant index bits.

To maintain system performance, we recommend that you disable DVM invalidation on TBUs on which direct indexing is enabled. Disable DVM invalidation by setting the appropriate `TCU_NODE_CTRLn.DIS_DVM` bit. See [4.7.10 TCU_NODE_CTRLn register](#) on page 156.

2.5.5 RAS implementation

Reliability, Availability, and Serviceability (RAS) features enable memory, normally SRAM, corruption to be detected and corrected, optionally generating interrupts into the system. All MMU-720AE RAM-based caches support either RAS error detection, or RAS error detection and correction.

MMU-720AE implements a combination of Single-Error-Correct-Double-Error-Detect (SECEDED) and Double-Error-Detect (DED) error correction mechanisms.

SECEDED is used in RAMs where a double error usually means that the SMMU cannot contain this error and must raise a Critical Error Interrupt. DED is used in TLB TAGS or DATA, where a single or double error can be recovered by fetching data from System Memory.

The SMMU also raises the following, based on a contained error or an uncontained error respectively:

- Fault Handling Interrupts (FHIs)
- Error Recovery Interrupts (ERIs)
- CRITICAL Error Interrupts (CRIs)

In addition to detection of errors in memories, MMU-720AE includes the following sources of error detection:

Flop parity

The value of the `TCUCFG_PARITY_ON` parameter controls whether or not parity is enabled for large flop structures and some software-accessible registers in the TCU.

The `TBUCFG_PARITY_ON` parameter controls the same behavior for structures and some software-accessible registers in the TBU.

All Software-accessible registers, except for the following registers, are unaffected by the `TCUCFG_PARITY_ON` and `TBUCFG_PARITY_ON` parameter settings:

- `SMMU_PMCG_*`
- `*_ITEN`, that is:
 - [4.11.1 ITEN register for the TCU](#) on page 212
 - [4.19.1 ITEN register for the TBU](#) on page 266
- `*_ITIN`, that is:

- [4.12.2 ITIN_PIU register for the TCU Programmer Interface Unit](#) on page 218
- [4.13.2 ITIN_TMU register for the TCU Translation Management Unit](#) on page 220
- [4.19.3 ITIN register for the TBU](#) on page 270
- *_ITOP, that is:
 - [4.12.1 ITOP_PIU register for the TCU Programmer Interface Unit](#) on page 213
 - [4.13.1 ITOP_TMU register for the TCU Translation Management Unit](#) on page 219
 - [4.19.2 ITOP register for the TBU](#) on page 267

Register that are not affected by the `TCUCFG_PARITY_ON` and `TBUCFG_PARITY_ON` parameters are considered to be critical configuration registers and are protected by parity logic regardless of the parameter value settings.

A parity error on one of these sets of flops means that the SMMU cannot contain this error and must raise a CRITICAL Error Interrupt. In parallel, a Fault Handling Interrupt (FHI) and an Error Recovery Interrupt (ERI) are also raised.

Poison

If the `poison_support` tie-off signal is tied HIGH, see [A.2.9 TBU tie-off signals](#) on page 320, the TCU QTW and PTW interfaces support the poison functionality. The poison function signals that a piece of data received has previously been corrupted and cannot be used. MMU-720AE cannot correct a poison error. Software can potentially recover data by re-writing the affected piece of data in memory. If corruption of write data occurs while in the Write Data Buffer, and poison functionality is enabled, the ACE-Lite TBU also generates poison signals.

The following tables show the MMU-720AE error sources, and the actions that are taken when errors occur.

In the tables, the following abbreviations apply:

CE

Corrected Error

CRI

Critical Error Interrupt

DE

Deferred Error

DED

Double-Error-Detect

ERI

Error Recovery Interrupt

FHI

Fault Handling Interrupt

SEC

Single-Error-Correct

SECDED

Single-Error-Correct-Double-Error-Detect

UC

Uncontainable error

UE

Uncorrected Error

UER

Recoverable Uncorrected Error

The following table shows the RAM RAS error sources in MMU-720AE, and actions that are taken when errors occur.

Table 2-10: RAM RAS error sources

RAM name	Error correction and detection mechanism	RAS error triggered		RAS interrupts triggered
BIU WDB ROBUFF D	SEC	CE		FHI
	DED	Poison supported:	DE	FHI
		Poison not supported:	UE (UC)	ERI, FHI, and CRI
BIU WDB ROBUFF C	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
BIU WDB ROBUFF P	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU OGQ	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU UOQ	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU DTIQ	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU REQ	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU RSP	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU LB	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU HLB ENTRY LEFT	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU HLB ENTRY RIGHT	SEC	CE		FHI
	DED	UE (UC)		FHI, ERI, and CRI
TLBU TOU HLB PTR LEFT	SEC	CE		FHI

RAM name	Error correction and detection mechanism	RAS error triggered	RAS interrupts triggered
	DED	UE (UC)	FHI, ERI, and CRI
TLBU TOU HLB PTR RIGHT	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TLBU DCU MTLB PLIM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TLBU DCU MTLB PCNT	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TLBU DCU MTLB REPL	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TLBU DCU MTLB TAGS	SEC	CE	FHI
	DED	CE	FHI
TMU TWB BSU	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU HZU PTR	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU TWB WMB LKP STATUS	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU TWB WMB WLK STATUS	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU TWB WMB SCRATCH	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU HTTU RAM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU WCB MWC PLIM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU WCB MWC PCNT	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU WCB MWC REPL	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU WCB MWC TAGS	SEC	CE	FHI
	DED	CE	FHI
TMU WCB MWC DATA	SEC	CE	FHI
	DED	CE	FHI
TMU CCB MCC PLIM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU CCB MCC PCNT	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU CCB MCC REPL	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU CCB MCC TAGS	SEC	CE	FHI

RAM name	Error correction and detection mechanism	RAS error triggered	RAS interrupts triggered
	DED	CE	FHI
TMU CCB MCC DATA	SED	CE	FHI
	DED	CE	FHI
TMU HZUSEC PTR	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
PIU AGW GMB WLK STATUS	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
PIU AGW GMB LOOP	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
PIU AGW GCB TAGS+DATA	SED	CE	FHI
	DED	CE	FHI
PIU AGW GCB REPL	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW ID RAM PARITY	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW STATUS RAM PARITY	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW RESPONSE RAM PARITY	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW GMB SCRATCH	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW GMB WLK STATUS	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW GMB LOOP	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW RAM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW GMB SCRATCH	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW GMB WLK STATUS	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW GMB LOOP	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW GCB TAGS+DATA	SED	CE	FHI
	DED	CE	FHI
TMU DGW GCB TAGS+DATA	SED	CE	FHI
	DED	CE	FHI
TMU AGW GCB REPL	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW GCB PCNT	SEC	CE	FHI

RAM name	Error correction and detection mechanism	RAS error triggered	RAS interrupts triggered
	DED	UE (UC)	FHI, ERI, and CRI
TMU AGW GCB PLIM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW GCB REPL	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW GCB PCNT	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI
TMU DGW GCB PLIM	SEC	CE	FHI
	DED	UE (UC)	FHI, ERI, and CRI

The following table shows the non-RAM RAS error sources in MMU-720AE, and actions that are taken when errors occur.

Table 2-11: Non-RAM RAS error sources

Source name	Error correction and detection mechanism	RAS error triggered	RAS interrupts triggered
PIU CMD RPOISON	RPOISON received, no correction	UE (UER)	FHI and ERI
PIU AGW GMB GPT WLK POISON	RPOISON received, no correction	UE (UER)	FHI and ERI
TMU AGW GMB GPT WLK POISON	RPOISON received, no correction	UE(UER)	FHI and ERI
DGW GMB GPT WLK POISON	RPOISON received, no correction	UE(UER)	FHI and ERI
TMU TWB WMB POISON	RPOISON received, no correction	UE (UER)	FHI and ERI
TMU TWB CMB POISON	RPOISON received, no correction	UE (UER)	FHI and ERI
PIU QTW FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU PIB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU AGW GMB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU AGW GCB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU AGW FF PARITY	FF parity error/FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU INV FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU CMD FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU EVT FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU PRI FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI and CRI
PIU MGB FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU CPB FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU FAR FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU PRG FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PIU RAS FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU HZU FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU PIB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU DMD FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU AGW GMB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU AGW GCB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI

Source name	Error correction and detection mechanism	RAS error triggered	RAS interrupts triggered
TMU AGW FF PARITY	FF parity error/FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU DGW GMB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU DGW GCB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU DGW FF PARITY	FF parity error/FF ctrl parity error r, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU TWB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU WCB FF PARITY	FF parity error/FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU CCB FF PARITY	FF parity error/FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU PRG FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TMU PMU FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
BIU MIB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
BIU SIB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
DIB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
DCU DCB FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
DCU TLB FF PARITY	FF parity error/FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
TOU FF PARITY	FF parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
BIU CPB FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
PMU FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI
RRB FF PARITY	FF ctrl parity error, uncorrectable	UE (UC)	FHI, ERI, and CRI

2.5.6 Quality of Service

You can program the TCU with a priority level for each LTI TBU interface. The priority level is applied to every translation from that TBU interface.

The TCU uses this priority level to:

- Arbitrate between translations that are waiting in the translation request buffer when translation manager slots become available
- Determine the AXI AxQOS value for translation table walks, configuration table walks and Granule Protection Table (GPT) walks, that the TCU issues on the QTW/DVM interface or Page Table Walk (PTW) interface

The arbiters contain starvation avoidance mechanisms to prevent transactions from being stalled indefinitely.

The TBU does not implement any prioritization between transactions. However, if a TBU has multiple LTI ports, the resources that a given port uses can be capped using the [4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register](#) on page 226. In most use cases, this limiting functionality, combined with the per-LTI interface priority level provides the required QoS management functionality. However, sometimes it might be necessary to use separate TBUs for requesters with different QoS requirements.

See also:

- [4.7.2 TCU_QOS register](#) on page 147
- [4.7.10 TCU_NODE_CTRLn register](#) on page 156

2.5.7 Distributed Virtual Memory messages

The QTW/DVM interface supports Distributed Virtual Memory (DVM) messages. MMU-720AE supports DVMv9.2.

The interface supports DVM transactions of message types TLB Invalidate and Synchronization. The interface accepts all other DVM transaction message types, and sends a snoop response, but otherwise ignores such transactions.

Tie the sup_btm input signal HIGH when the system supports Broadcast TLB Maintenance.

When Broadcast TLB Maintenance is supported, you can use SMMU_CR2, SMMU_R_CR2, and SMMU_S_CR2 to control how the SMMU handles TLB Invalidate operations as follows:

SMMU_CR2.PTM = 0

Non-secure TLB Invalidate operations are applied to the TLBs

SMMU_CR2.PTM = 1

Non-secure TLB Invalidate operations have no effect

SMMU_S_CR2.PTM = 0

Secure TLB Invalidate operations are applied to the TLBs

SMMU_S_CR2.PTM = 1

Secure TLB Invalidate operations have no effect

SMMU_R_CR2.PTM = 0

Realm TLB Invalidate operations are applied to the TLBs

SMMU_R_CR2.PTM = 1

Realm TLB Invalidate operations have no effect



Invalidation by PA is applied regardless of the above register values.



When sup_btm is tied HIGH, the reset value of SMMU_CR2.PTM, SMMU_S_CR2.PTM, and SMMU_R_CR2.PTM is 1.



Although TLB Invalidate operations have no effect when PTM = 1, the QTW/DVM interface still returns the appropriate response.

The QTW/DVM interface might receive DVM Sync transactions without receiving a DVM TLB Invalidate transaction, or when the PTM bits have masked a TLB Invalidate. If no DVM TLB Invalidate operations have occurred since the most recent DVM Sync transaction, subsequent DVM Sync transactions result in an immediate DVM Complete transaction. This behavior ensures that the TCU does not affect system DVM performance unless TLB Invalidate operations are performed.

The ACE-Lite interface allocates the access permissions and Shareability of DVM Complete transactions as follows:

- ARPROT = 0b000, indicating Unprivileged, Secure, Data access
- ARDOMAIN = 0b10, indicating Outer Shareable

See also [4.2 SMMU architectural registers](#) on page 121.

2.5.8 TCU transaction handling

The transaction width, burst length, and transfer size that the TCU supports depend on the transaction type.

The following table shows the TCU support for read transactions.



In the table, the value of the `TCU_ID_WIDTH` parameter is equal to the external ID width that [2.4.1.1 TCU Queue and Table Walk/Distributed Virtual Memory interface](#) on page 32 defines.

Table 2-12: TCU support for read transactions

Transaction type	<code>arid_qtw[TCU_ID_WIDTH - 1:6]</code>	<code>arid_qtw[5:0]</code>	Interface used
Command Queue read	Any additional bits are 0	6'b00_0000	QTW/DVM
Distributed Virtual Memory (DVM) Complete	Any additional bits are 0	6'b00_1000	QTW/DVM
Granule Protection Table (GPT) walk for CMDQ read	Any additional bits are 0	6'b10_0000	QTW/DVM
GPT walk for PRIQ write	Any additional bits are 0	6'b10_1000	QTW/DVM
GPT walk for EVTQ write	Any additional bits are 0	6'b11_0000	QTW/DVM
GPT walk for Message Signaled Interrupt (MSI) write	Any additional bits are 0	6'b11_1000	QTW/DVM

Transaction type	arid_qtw[TCU_ID_WIDTH - 1:6]	arid_qtw[5:0]	Interface used
GPT walk for CFG fetch	Indicates the configuration table walk slot requesting the page table walk, including bits marked as x in range [5:0]	6'bxx_x010 In addition to bits arid_qtw[TCU_ID_WIDTH - 1:6], bits marked as x in arid_qtw[5:0] are also used as part of the GPT walk ID.	QTW/DVM
Configuration table walk	Indicates the configuration table walk slot requesting the page table walk, including bits marked as x in range [5:0]	6'bxx_x011 In addition to bits arid_qtw[TCU_ID_WIDTH - 1:6], bits marked as x in arid_qtw[5:0] are also used as part of the configuration walk ID.	QTW/DVM
GPT walk for PTE fetch / Hardware Translation Table Update (HTTU) write	Indicates the page table walk slot requesting the page table walk, including bits marked as x in range [5:0]	6'bxx_x100 In addition to bits arid_qtw[TCU_ID_WIDTH - 1:6], bits marked as x in arid_qtw[5:0] are also used as part of the GPT walk ID.	QTW/DVM
Page table walk	Indicates the page table walk slot requesting the page table walk, including bits marked as x in range [5:0]	6'bxx_x101 In addition to bits arid_qtw[TCU_ID_WIDTH - 1:6], bits marked as x in arid_qtw[5:0] are also used as part of the page table walk ID.	PTW if present, otherwise QTW/DVM
GPT walk for final response	Indicates the DGW slot number of the final request, including bits marked as x in range [5:0]	6'bxx_x110 In addition to bits arid_qtw[TCU_ID_WIDTH - 1:6], bits marked as x in arid_qtw[5:0] are also used as part of the GPT walk ID.	QTW/DVM
HTTU access R response	Indicates the page table walk slot requesting the page table walk, including bits marked as x in range [5:0]	6'bxx_x111 In addition to bits arid_qtw[TCU_ID_WIDTH - 1:6], bits marked as x in arid_qtw[5:0] are also used as part of the HTTU update ID.	PTW if present, otherwise QTW/DVM

- DVM Complete transactions are always one beat of full data width.
- Command queue reads and DVM Complete transactions are independent of translation slots.

Therefore, the maximum number of read transactions that the TCU can issue at any time as follows:

- When TCUCFG_LEGACY_TZ_EN is set to 0, the maximum number of read transactions is as follows:

$$(2 \times \text{TCUCFG_PTW_SLOTS}) + \text{TCUCFG_DGW_SLOTS} + 5.$$

The PIU can generate an additional five outstanding read transactions as follows:

- One GPC check for a PRIQ queue write
- One GPC check for an EVENTQ write
- One GPC check for a MSI write
- One GPC check for a CMDQ read or one actual CMDQ read, which are serialized so there is only one
- One outstanding DVM Complete message on the AR channel

These extra five outstanding read transactions are low bandwidth, and having all five transactions outstanding at the same time is difficult.

- When TCUCFG_LEGACY_TZ_EN is set to 1, the maximum number of read transactions is as follows:

$TCUCFG_PTW_SLOTS + 2$

See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

The following table shows the TCU support for write transactions.



In the table, `TCU_ID_WIDTH` is equal to the external ID width that [2.4.1.1 TCU Queue and Table Walk/Distributed Virtual Memory interface](#) on page 32 defines.

Table 2-13: TCU support for write transactions

Transaction type	<code>awid_qtw[TCU_ID_WIDTH - 1:5]</code>	<code>awid_qtw[4:0]</code>
Page Request Interface (PRI) Queue Write	0 if present	5'b01000
Event Queue write	0 if present	5'b10000
MSI write	0 if present	5'b11000
HTTU Write	Indicates the page table walk slot requesting the HTTU write, including bits marked as x in range [4:0]	5'bxx111

There can be a maximum of five outstanding writes at one time, which includes:

- Two Hardware Translation Table Updates (HTTUs)
- One Event Queue write
- One Message Signaled Interrupt (MSI) write
- One Page Request Interface (PRI) queue write

All read and write transactions are aligned to the transaction size.

2.5.9 TCU prefetch

The TCU can prefetch translations on a per-context basis to improve translation performance for real-time requesters that access memory linearly. If TCU prefetch is enabled, a second translation request occurs after the original request, and is initiated and terminated entirely within the TCU.

This second translation request is regarded as the prefetch because it is an advance request of the next translation that is expected to be requested. This second request is speculative and is used to allocate into the caches of the TCU.

Software can enable TCU prefetch for a particular translation context by programming the Stream Table Entry (STE). Bits [121:120] are **IMPLEMENTATION DEFINED** in the SMMUv3 architecture. See the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

MMU-720AE uses these bits for the PF field as follows:

PF, bits [121:120]

This field determines whether prefetch is enabled or disabled for the translation context that this STE defines as follows:

0b00	Prefetching disabled
0b01	Reserved
0b10	Forward prefetching
0b11	Backward prefetching

Prefetching disabled

TCU prefetch does not occur

Reserved

Reserved values must not be used

Forward prefetching

The address to be prefetched is the first address following the end of the translation range, as DTI_TBU_TRANS_RESP.TRANS_RNG indicates

Backward prefetching

The address to be prefetched is the last address before the beginning of the translation range, as DTI_TBU_TRANS_RESP.TRANS_RNG indicates

Whenever a miss occurs in the MicroTLB and Main TLB of the TBU, the TBU sends a translation request to the TCU. If the STE for the translation is programmed to enable prefetch, each translation request to the TCU can also potentially result in a prefetch that occurs after the original request is complete.

When each incoming translation request completes its translation in the TCU, the STE.PF field indicates whether TCU prefetch is enabled. If TCU prefetch is enabled, a second translation request, the prefetch request, is then issued into the same TCU translation slot. This prefetch request is speculative, and only allocates into the TCU walk caches. A translation response for the prefetch is not returned to the TBU.

When the TCU handles each incoming translation request from the TBU, translation table walks can either occur, or not occur depending on whether there is a hit in each level of walk cache that is looked up. Translation table walks also might or might not occur for the subsequent prefetch request. The number of memory accesses that are performed for this prefetch are unrelated to the number of memory accesses that are performed for the original translation request.

Consider the following examples:

1. An incoming translation request might hit in the lowest level of walk cache, but the subsequent prefetch request might still require at least one translation table walk to memory.
2. The original translation request might require multiple translation table walks, but the subsequent prefetch request might hit in the lowest level of walk cache and not require any memory accesses. If the prefetch request hits in the lowest level of walk cache, then the walk caches are not updated and no memory accesses are performed.



The walk cache uses a Re-Reference Interval Prediction (RRIP) replacement policy.

The prefetch can only occur when the original request is complete irrespective of whether translation table walks were required. Waiting for completion of the original request means that by the time it becomes possible for the prefetch to be initiated, the TCU might have already received a non-speculative request for the next translation and begun to handle this request using a separate translation slot.

Therefore, TCU prefetch results in a performance advantage only if the number of cycles between each sequential translation request from the TBU is greater than the number of cycles that is taken for the TCU to handle the original translation request and to start the subsequent prefetch.

Even if TCU prefetch is enabled, a prefetch does not occur if one of the following caused the original request:

- A speculative translation request, that is, `DTI_TBU_TRANS_REQ.PERM[1:0] != 2'b11`, because a TBU receives a `StashOnceShared`, `StashOnceUnique`, or `StashTranslation` transaction
- A translation request for an atomic transaction that provides a data response, that is, `DTI_TBU_TRANS_REQ.PERM[1:0] != 2'b10`, because a TBU receives an `AtomicLoad`, `AtomicSwap`, or `AtomicCompare` transaction

If the original translation request returns one the following, prefetch also does not occur:

- Fault response.
- Global bypass response.
- Stream bypass response.



Prefetches can occur only with non-ATS translation requests because ATS itself is already a prefetch mechanism.

2.5.10 Error responses

The AMBA protocol defines external AXI subordinate error, `SLVERR`, and external AXI DECode ERRor, `DECERR`, and `TRANSFAULT`. The MMU-720AE error response behavior depends on the interface.

The TCU ACE-Lite interface treats `SLVERR` and `DECERR` identically, as an abort.

When terminating a transaction, the TBS interface generates an `OKAY`, `SLVERR`, or `TRANSFAULT` response depending on the reason for the termination and the value of `AxMMUFLOW`.

If the TBU TBM interface receives a DECERR or SLVERR response to a downstream transaction, it propagates the same abort type to the TBS interface.

2.5.11 Conversion between ACE-Lite and Armv8 attributes

The SMMUv3 architecture defines attributes in terms of the Arm®v8 architecture. The MMU-720AE components are therefore required to perform conversion between ACE-Lite and Arm®v8 attributes.

See the [Arm® Architecture Reference Manual for A-profile architecture](#).

The TBU must convert:

- ACE-Lite attributes to Arm®v8 attributes when it receives transactions on the Transaction Subordinate (TBS) interface
- Arm®v8 attributes to ACE-Lite attributes when it outputs transactions on the Transaction Manager (TBM) interface

The TCU must convert Arm®v8 attributes to ACE-Lite attributes when it outputs transactions on the QTW/DVM and PTW interfaces.

2.5.11.1 Subordinate interface memory type attribute handling

The AxCACHE and AxDOMAIN signals contain the memory attributes that apply to the TBS interface.

The following table shows the ACE-Lite to Armv8 attribute conversions that the TBU TBS interface performs.

Table 2-14: MMU-720AE ACE-Lite to Armv8 memory attribute conversions

AxCACHE attribute	AxDOMAIN attribute	Armv8 memory type	Armv8 Shareability
Device Non-bufferable	System	Device-nGnRnE	Outer Shareable
Device Bufferable	System	Device-nGnRE	Outer Shareable
Normal Non-cacheable Bufferable	Any	Normal Inner Non-cacheable Outer Non-cacheable	Outer Shareable
Normal Non-cacheable Non-bufferable			
Write-Through No Allocate			
Write-Through Read-Allocate			
Write-Through Write-Allocate			
Write-Through Read and Write-Allocate			

AxCACHE attribute	AxDOMAIN attribute	Armv8 memory type	Armv8 Shareability
Write-Back No Allocate	Non-shareable	Normal Inner Write-Back Outer Write-Back	Non-shareable
Write-Back Read-Allocate	Inner Shareable		Non-shareable
Write-Back Write-Allocate	Outer Shareable		Outer Shareable
Write-Back Read-Allocate Write-Allocate			



- Write-Back transactions are always treated as non-transient.
- The Armv8-A Read-Allocate and Write-Allocate hints are the same as the hints that the AxCACHE Write-Back type provides.
- The TBU TBS interface converts instruction writes into data writes, that is, it treats awprot_s[2] as 0.

2.5.11.2 Manager interface memory type attribute handling

The AxCACHE and AxDOMAIN signals contain the memory attributes that apply to the TBM and the QTW/DVM and PTW interfaces.

The TBU TBM interface can also use the AxLOCK signal to indicate an Exclusive access. The QTW/DVM and PTW interfaces do not use the AxLOCK signal.

On the TBU TBM interface, a bit on AxUSER indicates whether the memory type before the conversion is Outer Cacheable.

The following table shows the Armv8 to ACE-Lite attribute conversions that the manager interfaces perform.

Table 2-15: MMU-720AE Armv8 to ACE-Lite memory attribute conversions

Armv8 memory type	AxCACHE attribute	AxDOMAIN attribute	AxLOCK attribute	AxUSER Outer Cacheable
Device-nGnRnE	Device Non-bufferable	System	As Transaction Subordinate (TBS) AxLOCK value	0
Device-GRE	Device Bufferable	System	As TBS AxLOCK value	0
Device-nGRE				
Device-nGnRE				

Armv8 memory type	AxCACHE attribute	AxDOMAIN attribute	AxLOCK attribute	AxUSER Outer Cacheable
Normal Inner Non-cacheable Outer Non-cacheable	Normal Non-cacheable Bufferable	System	As TBS AxLOCK value	0
Normal Inner Write-Through Outer Non-cacheable				
Normal Inner Write-Back Outer Non-cacheable				
Normal Inner Non-cacheable Outer Write-Through	Normal Non-cacheable Bufferable	System	As TBS AxLOCK value	1
Normal Inner Write-Through Outer Write-Through				
Normal Inner Write-Back Outer Write-Through				
Normal Inner Non-cacheable Outer Write-Back				
Normal Inner Write-Through Outer Write-Back				
Normal Inner Write-Back Outer Write-Back	Write-Back No Allocate	If AxBURST == FIXED, an abort is generated and the transaction does not propagate downstream. The failure to propagate is because AXI does not support shareable FIXED bursts.	0	1
	Write-Back Read-Allocate	If AxBURST != FIXED, the attribute reflects the Armv8 Shareability:		
	Write-Back Write-Allocate	- Non-shareable - Outer Shareable		
	Write-Back Read and Write-Allocate	An Armv8 shareability attribute of Inner Shareable is always output with an AxDOMAIN value of Outer Shareable because Inner Shareable is deprecated in ACE and ACE-Lite.		

See also [2.5.12 AXI USER bits that the SMMU TBU TBM defines](#) on page 74.

2.5.12 AXI USER bits that the SMMU TBU TBM defines

The TBU TBM interface AxUSER signals, aruser_m, and awuser_m, have 1 bit more than TBUCFG_AxUSER_WIDTH defines. This extra bit is output in higher-order bits of aruser_m and awuser_m.

The following table shows the MMU-720AE-defined aruser_m and awuser_m bits, where *w* represents the AXI USER bus width that TBUCFG_AxUSER_WIDTH defines.

Table 2-16: MMU-720AE-defined aruser_m and awuser_m bits

Bit position	Value
[<i>w</i>]	Outer Cacheable The Outer cacheable is added to the upper most bit of AxUSER on the TBM interface.

See also [2.5.11.2 Manager interface memory type attribute handling](#) on page 73.

2.5.12.1 Page Based Hardware Attribute (PBHA) in SMMUs

The Arm® architecture defines that 4 bits in both stage 1 and stage 2 leaf page table entry formats are reserved for software use. Arm®v8.5 and SMMUV3 define a mechanism where software can declare that it does not require them, on a per bit basis.

See also:

- [Arm® Architecture Reference Manual for A-profile architecture](#)
- [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#)

The Page Based Hardware Attribute (PBHA) mechanism effectively hands over control of those bits to **IMPLEMENTATION DEFINED** hardware purposes. These bits are called PBHA bits.

For more information about PBHA bits, see the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

2.5.13 NoStreamID

The SMMU can propagate transactions from devices that are not associated with a StreamID. These devices are called NoStreamID devices.

Transactions from NoStreamID devices are not translated, but are still subject to Granule Protection Checks (GPC), if enabled.

ACE-Lite TBU

When using the ACE-Lite TBU, setting AxMMUVALID = 0 indicates a NoStreamID access.

LTI TBU

When using the LTI TBU, setting LAMMUV = 0 indicates a NoStreamID access.

NoStreamID accesses are also called GPC only requests.

The 0x8E TBU performance event can count GPC only requests. See performance event 0x8E that the table in [2.5.2.4 SMMUv3 PMU register architectural options](#) on page 55 describes. For more information about NoStreamID, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

NoStreamID transactions are typically used in systems that have upstream devices which directly access the Physical Address space without requiring translation using an ACE-Lite TBU or LTI TBU. However, in systems that use Realm Management Extension (RME), these transactions must still go through the SMMU so that GPCs can be performed.

2.6 Constraints and limitations of use

Certain usage constraints and limitations apply to MMU-720AE.

Unless otherwise specified, an **IMPLEMENTATION DEFINED** field in a structure that MMU-720AE:

- Generates is 0
- Reads is ignored

2.6.1 SMMUv3 implementation

MMU-720AE implements SMMUv3 features.

See also:

- [2.5.11.2 Manager interface memory type attribute handling](#) on page 73
- [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#)

2.6.1.1 ID register architectural options

MMU-720AE uses ID register architectural options that the SMMUv3 ID registers expose.

The following table shows the architectural options for MMU-720AE from the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) that the SMMUv3 ID registers expose.

Table 2-17: SMMUv3 ID register options

Register	Field	Value	Description
SMMU_ROOT_IDRO	ROOT_IMPL	1b1	Root implemented.
	BGPTM	sup_btm	Participates in broadcast TLBI PA operations.
	RGPTM	1b1	TLBI registers present.
	REALM_IMPL	1b1	Realm programming interface is present.
	BA_REALM	0x6	Base address of Realm register page 0 is 0x20000 + (6 × 0x10000).

Register	Field	Value	Description
SMMU_ROOT_IIDR	Implementer	0x43B	Arm implementation.
	Revision	MAX(p_level, ecorevnum)	Where p_level is: 1 For p1
	Variant	1b0	r0.
	ProductID	0x498	Third SMMUv3 implementation from Arm.
SMMU_IDR0	S2P	1b1	Stage 2 translation supported.
	S1P	1b1	Stage 1 translation supported.
	TTF	2b10	AArch64 translation supported, required for Realm Management Extension (RME), but does not change in legacy mode.
	COHACC	sup_cohacc or 1, see the description	Coherent accesses supported, system configuration option. The value is set as follows: sup_cohacc If the TCUCFG_LEGACY_TZ_EN parameter is set to 1 or the legacy_tz_en tie-off signal is set to 1. 1 If the TCUCFG_LEGACY_TZ_EN parameter is set to 0 and the legacy_tz_en tie-off signal is set to 0.
	BTM	sup_btm	Broadcast TLB maintenance, system configuration option. BTM can be set to 0 by writing 1 to TCU_ROOT_CTRL.DIS_DVM.
	HTTU[1:0]	{sup_httu, 1b0}	Access and dirty flag update supported. The sup_httu tie-off input signal configures the support. as follows: {sup_httu, 1b0} If the TCUCFG_LEGACY_TZ_EN parameter is set to 1 or the legacy_tz_en tie-off signal is set to 1
	DORMHINT	1b0	Dormant hint is not supported.
	Hyp	1b1	EL2-E2H is supported.
	ATS	1b1	ATS is supported.
	NS1ATS	1b0	Stage 1-only ATS is supported.
	ASID16	1b1	16-bit ASID is supported.
	MSI	1b1	Message Signaled Interrupts (MSIs) are supported.
	SEV	sup_sev	Send event is supported, system configuration option.
	ATOS	1b0	ATOS is not supported.
	PRI	1b1	PRI is supported.
	VMW	1b1	VMID wildcard matching supported.
	VMID16	1b1	16-bit VMIDs are supported.

Register	Field	Value	Description
	CD2L	1b1	2-level context descriptor tables are supported.
	VATOS	1b0	Virtual ATOS is not supported.
	TTENDIAN	0b00	Mixed-endian translation walks are supported.
	ATSRECERR	1b1	Supports recording some events for ATS translation requests.
	STALL_MODEL	{0b0, SMMU_S_CR0.NSSTALLD}	Stall and terminate models that are supported unless the Secure world disables Non-secure stalling.
	TERM_MODEL	1b0	Terminating a transaction with RAZ/WI is supported.
	ST_LEVEL	2b01	2-level stream table is supported.
	RME_IMPL	~legacy_tz_en && ~TCUCFG_LEGACY_TZ_EN	RME supported unless in legacy TZ mode.
SMMU_IDR1	SIDSIZE	32	32-bit StreamIDs are supported.
	SSIDSIZE	20	20-bit SubstreamIDs are supported.
	PRIQS	5b10011	2 ¹⁹ PRI queue entries are supported.
	EVENTQS	5b10011	2 ¹⁹ Event queue entries are supported.
	CMDQS	5b10011	2 ¹⁹ Command queue entries are supported.
	ATTR_PERMS_OVR	1b1	Incoming permission attributes can be overridden.
	ATTR_TYPES_OVR	1b1	Incoming memory attributes can be overridden.
	REL	1b0	Reported base addresses are absolute addresses.
	QUEUES_PRESET	1b0	Not fixed queue base addresses.
	TABLES_PRESET	1b0	Not fixed table base addresses.
	ECMDQ	1b0	Enhanced Command Queue is not supported.
SMMU_IDR2	BA_VATOS	1b0	No VATOS support.
SMMU_IDR3	HAD	1b1	Hierarchical attribute disable is supported.
	PBHA	1b1	Page-Based Hardware Attributes (PBHA) are supported.
	XNX	1b1	ELO/EL1 stage 2 execute control is supported.
	PPS	1b1	PASID, when present always used in PRI auto-generated response.
	MPAM	1b1	MPAM is supported.
	FWB	1b1	S2 control of memory type is supported.
	STT	1b1	Small translation tables are supported.
	RIL	1b1	Range-based invalidation and Level hint are supported.
	BBML	2	Break before Make level 2 is supported.
	EOPD	1b1	EOPD mechanism is implemented.

Register	Field	Value	Description
	PTWNNC	1b1	Treat stage 1 translation walks mapped as Device memory as Normal Non-cacheable.
	DPT	1b0	Non-secure DPT is not supported.
SMMU_IDR4	IMPDEF	1b0	No IMPLEMENTATION DEFINED features.
SMMU_IDR5	OAS	sup_oas	Output address size, system configuration option.
	GRAN4K	1b1	4K translation granule is supported.
	GRAN16K	1b1	16K translation granule is supported.
	GRAN64K	1b1	64K translation granule is supported.
	VAX	2b01	Virtual addresses of 52 bits per CD.TTBx are supported.
	STALL_MAX	TCUCFG_XLATE_SLOTS	Maximum number of outstanding stalled transactions.
SMMU_IDR6	CMDQ_CONTROL_PAGE_LOG2NUMQ	RAZ/WI8 for r0	Always 256 queues per page.
	CMDQ_CONTROL_PAGE_LOG2NUMP	RAZ/WITCUCFG_ECMDQ_PAGES_LOG2 for r0	Number of pages set by the parameter.
SMMU_IIDR	Implementor	0x43B	Arm® implementation.
	Revision	MAX(p_level, ecorevnum)	Where p_level is 1 for p1.
	Variant	1b0	Product variant, or major revision is r0.
	ProductID	0x498	Arm® ID.
SMMU_AIDR	ArchMinorRev	3	Architectural minor revision is SMMUv3.3.
	ArchMajorRev	1b0	Architectural Major Revision is SMMUv3.
SMMU_S_IDR0	MSI	1b1	Secure MSIs are supported.
	STALL_MODEL	2'b00	Stall and terminate model is supported.
	ECMDQ	1b0	Extended command queue is not supported.
SMMU_S_IDR1	S_SIDSIZE	32	32-bit Secure StreamIDs are supported.
	SEL2	1b1	Secure EL2 is supported.
	SECURE_IMPL	1b1	Two Security states are implemented.
SMMU_S_IDR3	SAMS	1b1	Secure ATS maintenance is not implemented.
SMMU_S_IDR4	IMPDEF	1b0	No IMPLEMENTATION DEFINED features.
SMMU_S_IDR6	CMDQ_CONTROL_PAGE_LOG2NUMQ	RAZ/WI	Always 256 queues per page.
	CMDQ_CONTROL_PAGE_LOG2NUMP	RAZ/WI	Number of pages set by parameter.
SMMU_R_IDR0	ATS	1	PCIe ATS supported for Realm.
	MSI	1	Realm MSIs are supported.
	PRI	1	PRI is supported.
	STALL_MODEL	0'b01	Stall not supported for Realm.
	ECMDQ	0	Enhanced Command queue is not supported.
SMMU_R_IDR1	RME_DA_IMPL	1	Realm support is implemented.
SMMU_R_IDR2	Res0	0	RES0.
SMMU_R_IDR3	DPT	0	DPT is not supported.

Register	Field	Value	Description
	MEC	(TCUCFG_MECID_WIDTH != 0 0)	Memory Encryption Contexts are supported.
SMMU_R_IDR4	Res0	0	RES0.
SMMU_R_MECIDR	MECIDSIZE	TCUCFG_MECID_WIDTH == 0 ? 0 : TCUCFG_MECID_WIDTH - 1;	Number of MECID bits present.

2.6.1.2 Non-implemented commands and events

MMU-720AE does not implement all the SMMUv3 commands and events.

Event queue

MMU-720AE does not generate the following events:

- F_UUT
- F_TLB_CONFLICT
- F_CFG_CONFLICT
- E_PAGE_REQUEST
- IMPDEF_EVENTn

Command queue

The following commands are accepted but silently ignored:

- CMD_PREFETCH_CONFIG
- CMD_PREFETCH_ADDR
- CMD_CFGI_VMS_PIDM

Only the Non-secure and Realm Command queues support the `CMD_ATC_INV` command. If the TCU encounters the `CMD_ATC_INV` command in the Secure Command queue, a Secure Command queue error with reason code `CERROR_ILL` occurs.

2.6.1.3 IMPLEMENTATION DEFINED fields

MMU-720AE contains **IMPLEMENTATION DEFINED** fields.

Memory structures

For memory structure, when:

- Using the IMPDEF field, reset value
- Not using the IMPDEF field, write ignored, treated as zero

Register structures

For register structure, when:

- Using the IMPDEF field, reset value
- Not using the IMPDEF field, write ignored, treated as zero

2.6.1.4 Non-implemented registers

MMU-720AE does not implement all registers.

The following optional registers are not implemented and are RAZ/WI:

- SMMU_IDR4
- SMMU_S_IDR4
- SMMU_IDR6
- SMMU_S_IDR6
- SMMU_R_IDR6
- SMMU_STATUSR
- SMMU_CMDQ_CONTROL_*
- SMMU_S_CMDQ_CONTROL_*
- SMMU_R_CMDQ_CONTROL_*
- SMMU_ECMDQ_*
- SMMU_DPT_*
- SMMU_R_DPT_*
- SMMU_GATOS_*
- SMMU_S_GATOS_*
- SMMU_VATOS_*
- SMMU_S_VATOS_*

The following PMCG registers are not implemented and are RAZ/WI:

- SMMU_PMCG_IIDR
- SMMU_PMCG_IRQ_CFG0
- SMMU_PMCG_IRQ_CFG1
- SMMU_PMCG_IRQ_CFG2
- SMMU_PMCG_IRQ_STATUS
- SMMU_PMCG_GMPAM
- SMMU_PMCG_MPAMIDR
- SMMU_PMCG_S_MPAMIDR

2.6.2 AMBA implementation

MMU-720AE implements parts of the AMBA protocols.

2.6.2.1 ACE-Lite property support

MMU-720AE supports many ACE-Lite properties.

The following table shows the ACE-Lite properties that MMU-720AE supports, and their values for the TBU TBS and TBM interfaces, and the TCU QTW/DVM and PTW interfaces. The table also shows the issue of the [AMBA® AXI Protocol Specification](#), that is, E, F, G, H, J, or K, to which the particular property was first added.

Table 2-18: AXI property support

Specification issue	Interface properties	ACE-Lite TBU		TCU	
		TBS	TBM	QTW/DVM	PTW
E	DVM_v8	False	False	True	False
	Multi_Copy_Atomicity	True	True	True	True
	Ordered_Write_Observation	True	True	False	False
F	Atomic_Transactions	True	True	True	True
	Barrier_Transactions	False	False	False	False
	Cache_Stash_Transactions	True, if TBUCFG_STASH_SUPPORT is 1	True, if TBUCFG_STASH_SUPPORT is 1	False	False
	Check_Type (Odd_Parity_Byte_All)	False	False	False	False
	Coherency_Connection_Signals	False	False	True	False
	DeAllocation_Transactions	True	True	False	False
	DVM_v8.1	False	False	True	False
	Loopback_Signals	True	True	False	False
	NSAccess_Identifiers	False	False	False	False
	Persist_CMO	True	True	False	False
	Poison	True	True	True	True
	QoS_Accept	False	False	False	False
	Trace_Signals	False	False	False	False
	Untranslated_Transactions	v3	v3 The TBM implements only certain signals of this property	False	False
	Wakeup_Signals	True	True	True	True
	WriteEvict_Transaction	False	False	False	False
G	CMO_On_Read	True	True	False	False
	CMO_On_Write	False	False	False	False
	Read_Data_Chunking	True	True	False	False

Specification issue	Interface properties	ACE-Lite TBU		TCU	
		TBS	TBM	QTW/DVM	PTW
	Read_Interleaving_Disabled	False The TBU is agnostic to this property	False The TBU is agnostic to this property	False The TCU is agnostic to this property	False The TCU is agnostic to this property
	Unique_ID_Support	True	True	True	True
H	Consistent_DECERR	False	False	False	False
	DVM_Message_Support	False	False	Receiver	False
	DVM_v8.4	False	False	True	False
	Exclusive_Accesses	True	True	False	False
	Max_Transaction_Bytes	IMPDEF	IMPDEF	64	64
	Prefetch_Transaction	False	False	False	False
	Regular_Transactions_Only	False	False	False	False
	Shareable_Transactions	True	True	True	True
	Write_Plus_CMO	False	False	False	False
	WriteZero_Transaction	False	False	False	False
J	BURST_Present	True	True	True	True
	Busy_Support	False	False	False	False
	CACHE_Present	True	True	True	True
	CMO_On_Write and Write_Plus_CMO (No CleanInvalidPoPA)	False	False	False	False
	DVM_v9.2	False	False	True	False
	InvalidateHint_Transaction	True	True	False	False
	LEN_Present	True	True	True	True
	Memory_Cache_Support	False	False	False	False
	MMUFLOW_Present	True	False	False	False
	PBHA_Support	True	True	True	True
	PROT_Present	True	True	True	True
	REGION_Present	True	True	False	False
	QOS_Present	True	True	True	True
	RLAST_Present	True	True	True	True
	RME_Support	True, if TBUCFG_LEGACY_TZ_EN is 0	True, if TBUCFG_LEGACY_TZ_EN is 0	True, if TCUCFG_LEGACY_TZ_EN is 0	True, if TCUCFG_LEGACY_TZ_EN is 0
	Shareable_Cache_Support	False	False	False	False
	SIZE_present	True	True	True	True
	STASHLPID_Present	True	True	False	False
	STASHNID_Present	True	True	False	False
	UnstashTranslation_Transaction	False	False	False	False
	WLAST_Present	True	True	True	True
	WSTRB_Present	True	True	True	True

Specification issue	Interface properties	ACE-Lite TBU		TCU	
		TBS	TBM	QTW/DVM	PTW
	WriteDeferrable_Transaction	False	False	False	False
K	MEC_Support	True, if TBUCFG_MECID_WIDTH > 0	True, if TBUCFG_MECID_WIDTH > 0	True, if TCUCFG_MECID_WIDTH > 0	True, if TCUCFG_MECID_WIDTH > 0
	Cache_Line_Size	IMPDEF	IMPDEF	64	64
	Device_Normal_Independence	False	False	False	False
	Fixed_Burst_Disable	False	False	False	False
	MPAM_Support	MPAM_9_1	MPAM_9_1	MPAM_9_1	MPAM_9_1
	MTE_Support	Basic	Basic	False	False
	WriteNoSnoopFull_Transaction	False	False	False	False

2.6.2.2 SLVERR and DECERR

MMU-720AE uses SLVERR and DECERR responses. The TCU QTW and PTW interfaces treat SLVERR and DECERR identically, as an abort.

The TBU TBS interface generates SLVERR when terminating a transaction that requires an abort response.

If the TBU TBM interface receives an SLVERR or DECERR response to a downstream transaction, the same abort type is propagated to the TBS interface.

2.6.2.3 Poison

The MMU-720AE TCU supports poison responses on the QTW and PTW interfaces. These are treated as an abort, but an Uncorrected error, Signaled or Recoverable error (UER) is raised.

2.6.2.4 Attribute handling

MMU-720AE handles untranslated or non-ATS transactions, fully-translated or full ATS transactions, and partially-translated or Split-stage ATS transactions.

When translation is enabled and a PCIe Root Port issues transactions to a TBU, the following apply, depending on the type of transaction:

Untranslated (non-ATS) transaction

The SMMU applies attributes that a combination of the input attributes, Stream Table Entry (STE) overrides, and translation table descriptors determine.

Fully-translated (full ATS) transaction

The SMMU does not modify the attributes that are encoded in the fully translated transaction. The unmodified attributes are used as the output attributes.

Partially-translated (Split-stage ATS) transaction

The ATS translation response from the TCU to the PCIe Root Port includes Stage 1 and Stage 2 attributes.

The Stage-1-translated transaction to the TBU encodes these Stage 1 and Stage 2 attributes. The SMMU performs Stage 2 translation and combines the Stage 2 attributes a second time, but this does not affect the output attributes. The output attributes remain the same as the attributes that the TBU receives for the Stage-1-translated transaction.

NoStreamID (AxMMUVALID = 0) transaction

The SMMU does not modify the attributes that are encoded in the NoStreamID transaction. The unmodified attributes are used as output attributes.

For information about the preceding transactions and their attributes, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).



TBUs that are connected to a PCIe Root Port must have the pcie_mode input signal tied HIGH, as the table in [A.2.9 TBU tie-off signals](#) on page 320 describes.

2.6.2.4.1 TBU subordinate interface attribute handling

MMU-720AE handles attribute in the subordinate interface. The TBU TBS interface converts the incoming ACE-Lite attributes into Armv8 attributes.

The following table shows how the subordinate interface handles attributes in MMU-720AE. The following terms are used in the table:

ISH

Inner-shareable

NSH

Non-shareable

OSH

Outer-shareable

SH

Shareable

SYS

System

MMU-720AE uses the 0b10 encoding for Shareable on AXI as recommended in Domain signaling in the AMBA® AXI Protocol Specification.

Table 2-19: TBU subordinate interface attribute handling

ACE-Lite AxCACHE	ACE-Lite AxDOMAIN	Armv8 memory type	Armv8 shareability	Description
Device Non-bufferable	SYS	Device-nGnRnE	OSH	-
Device Bufferable	SYS	Device-nGnRE	OSH	-
Normal Non-cacheable Bufferable, Normal Non-cacheable Non-bufferable,	Any	Normal-iNC-oNC	OSH	Normal Non-cacheable Non-bufferable is a deprecated AxCACHE type and is converted to Normal Non-cacheable Bufferable.
Write-Through No-allocate, Write-Through Read-Allocate, Write-Through Write-Allocate, Write-Through Read and Write-Allocate	Any	Normal-iNC-oNC	OSH	Write-Through types are converted to Non-cacheable on input to match the normalization step on output.
Write-back No-allocate, Write-back Read-Allocate, Write-back Write-Allocate, Write-back Read and Write-Allocate	NSH/ISH/ OSH	Normal-iWB-oWB	NSH/OSH	The Armv8 RA and WA hints depend on the Write-Back type. The transaction is always treated as non-transient. All ISH Shareability types are converted to OSH.

Subordinate interface AxPROT handling

Instruction writes are converted into data writes on the TBU TBS interface. In effect, AWPROT[2] is ignored and always treated as 0.

2.6.2.4.2 Manager interface attribute handling

MMU-720AE handles attributes in the ACE-Lite manager interfaces.

Normalization

The TBU TBM interface, TCU QTW interface, and TCU PTW interface carry normalized attributes using a standard conversion scheme:

- Memory that is marked as Inner Write-Back Cacheable and Outer Write-Back Cacheable is output as Write-Back Cacheable

- Memory that is marked as Inner Non-cacheable or Write-Through Cacheable, or Outer Non-cacheable or Write-Through Cacheable, is output as Non-cacheable, Outer Shareable

On the TBU TBM interface, a bit on AxUSER indicates whether the output memory type before this conversion is outer cacheable.

The following table shows how the Armv8 transaction types are translated into AMBA ACE-Lite signals.

Table 2-20: Armv8 transaction types translated into AMBA ACE-Lite signals

Armv8 Memory Type	AxCACHE (TBM, QTW)	AxDOMAIN (TBM, QTW)	AxLOCK (TBM)	AxUSER outer cacheable bit (TBM)
Device-nGnRnE	Device No-bufferable	SY	TBS AxLOCK value	0
Device-GRE, Device-nGRE, Device-nGnRE	Device Bufferable	SY	TBS AxLOCK value	0
Normal-iNC-oNC, Normal-iWT-oNC, Normal-iWB-oNC	Normal Non-cacheable Bufferable	SY	TBS AxLOCK value	0
Normal-iNC-oWT, Normal-iWT-oWT, Normal-iWB-oWT, Normal-iNC-oWB, Normal-iWT-oWB	Normal Non-cacheable Bufferable	SY	TBS AxLOCK value	1
Normal-iWB-oWB	Write-back No-allocate / Write-back Read-Allocate / Write-back Write-Allocate / Write-back Read and Write-Allocate, depending on the Armv8 outer allocate hints.	NSH or OSH, depending on the Armv8 Shareability Note: If AxBURST == FIXED and the Armv8 shareability is not NSH, an abort is generated and the transaction does not propagate downstream. The failure to propagate is because AXI does not support shareable FIXED bursts.	0	1

AxCACHE encodings

Where there are multiple legal values for AxCACHE as the [AMBA® AXI Protocol Specification](#) describes, the canonical, non-bracketed, one is used. Therefore, only the AxCACHE encodings that the following table shows are used.

Table 2-21: AxCACHE encodings

AMBA memory type	ARCACHE	AWCACHE
Device Non-bufferable	0000	0000
Device Bufferable	0001	0001
Normal Non-cacheable Bufferable	0011	0011
Write-back No-allocate	1011	0111
Write-back Read-Allocate	1111	0111
Write-back Write-Allocate	1011	1111
Write-back Read and Write-Allocate	1111	1111

2.6.2.5 AxREGION

MMU-720AE does not require AxREGION signals.

The AxREGION signals are not required because on the:

QTW and PTW interfaces

No AxREGION signals exist

TBM interface

The AxREGION signals are passed through from the corresponding TBS interface

2.6.2.6 DVM interface

MMU-720AE has a DVM interface that handles DVM operations and DVM complete.

Supported DVM operations

All DVM operations are handled in a protocol-compliant manner because the interconnect does not know that the TCU does not require DVM operations other than TLB invalidate. Any DVM operation with a DVM Message Type in ACADDR[14:12] other than TLB Invalidate or Synchronization is accepted and responded to on the CR channel but otherwise ignored.

DVM complete

DVM Complete messages are presented with:

- ARPROT[2:0] = 0b000, that is, Unprivileged Secure Data
- ARDOMAIN[1:0] = 0b10, that is, Outer Shareable
- ARNSE = 0b0, that is, not Root or Realm
- ARCACHE = 0b0010, that is, Modifiable and Non-cacheable

- ARQOS is driven by the value written to the QOS_DVMSYNC field of the [4.7.2 TCU_QOS register](#) on page 147

2.6.2.7 Internally terminated transactions

In MMU-720AE, transactions that are terminated inside the TBU are returned with all RUSER and BUSER bits set to zero.

2.6.2.8 Transaction types

MMU-720AE supports several special transaction types, distinguished by a nonzero encoding of AxSNOOP. We describe how each transaction type is handled.

Unless otherwise specified, transactions are propagated on TBM with the same transaction type that was presented on TBS.

An ordinary read or ordinary write is one with AxSNOOP = 0b0000, that is, depending on AxDOMAIN, and whether it is a read or a write, one of the following:

- ReadNoSnoop
- ReadOnce
- WriteNoSnoop
- WriteUnique

For more information about the mapping of LTI transactions to AXI types and the handling of LTI responses to convert them to AXI responses, see Considerations for AXI5 in the [AMBA® LTI Protocol Specification](#).

MMU-720AE always treats the following as Normal Write-Back:

- ReadOnceCleanInvalid (ROCI)
- ReadOnceMakeInvalid (ROMI)
- WriteUniquePtlStash
- WriteUniqueFullStash
- StashOnceShared
- StashOnceUnique

If the incoming memory type is not Normal Write-Back then MMU-720AE treats the incoming memory type as Normal Write-Back Allocate.

2.6.2.9 Transactions that can result in a translation fault

In an MMU-720AE system, some transactions can result in a translation fault, and certain behavior is associated with such transactions.

MMU-720AE treats the following transactions as ordinary reads when calculating translation faults:

- CleanShared
- CleanInvalid
- MakeInvalid
- CleanSharedPersist
- ReadOnceMakeInvalid
- ReadOnceCleanInvalid

Therefore, these transactions might require either read permission or execute permission at the appropriate privilege level.

MMU-720AE treats the following transactions as ordinary writes when calculating translation faults:

- WriteUniquePtlStash
- WriteUniqueFullStash

Therefore, these transactions require write permission at the appropriate privilege level.

The following transactions do not have a memory type:

- CleanShared
- CleanInvalid
- MakeInvalid
- CleanSharedPersist
- InvalidateHint

For these transactions, the following input and output transaction memory type and allocation hints are used:

- Normal
- Inner Write-Back
- Outer Write-Back
- Read Allocate
- Write Allocate

This behavior means that the ARDOMAIN output on the TBM interface is never System Shareable for these transactions, because they are never Non-cacheable or Device.

MMU-720AE treats transactions that pass the translation fault check as follows:

MakelInvalid transactions

MMU-720AE converts MakelInvalid transactions to CleanInvalid transactions, unless the translation also grants write permission and Destructive Read Enable (DRE) permission.

InvalidateHint transactions

MMU-720AE terminates InvalidateHint transactions with a RAZ/WI response, unless the translation also grants write permission and DRE permission.

ReadOnceMakelInvalid and ReadOnceCleanInvalid transactions

MMU-720AE outputs ReadOnceMakelInvalid transactions as ReadOnceCleanInvalid transactions, unless the translation also granted write permission and DRE permission.

If the final transaction attributes on the TBU TBM interface are not Outer Shareable Write-Back, MMU-720AE converts ReadOnceMakelInvalid and ReadOnceCleanInvalid transactions into ordinary reads.

WriteUniquePtlStash and WriteUniqueFullStash transactions

If they pass the translation fault check, MMU-720AE converts WriteUniquePtlStash and WriteUniqueFullStash transactions to ordinary write transactions if either:

- The translation did not grant Directed Cache Prefetch (DCP) permission
- The final transaction attributes on the TBU TBM interface are not Outer Shareable Write-Back

If such a conversion occurs, AWSTASH* is driven as 0.

2.6.2.10 Transactions that cannot result in a translation fault

In an MMU-720AE system, certain transactions cannot result in a translation fault, and certain behavior is associated with such transactions.

The following transactions never result in a translation fault:

- StashOnceShared
- StashOnceUnique
- StashTranslation
- UnstashTranslation
- InvalidateHint

If any of these transactions require a translation request to the TCU, MMU-720AE issues a speculative translation request on the DTI interconnect. StashOnceShared and StashOnceUnique transactions are terminated in the TBU, with a BRESP value of OKAY, when any of the following cases apply:

- The translation did not grant Directed Cache Prefetch (DCP) permission
- The final transaction attributes on the TBM interface are not Outer Shareable Write-Back
- The translation did not grant any of read, write, or execute permission at the appropriate privilege level



Only one of these permissions is required for the stash transaction to be permitted.

A BRESP value of OKAY indicates transaction success. MMU-720AE always generates this value when a StashOnceShared or a StashOnceUnique transaction is terminated in the TBU. This behavior applies even when a StreamDisable or GlobalDisable translation response causes the transaction to be terminated.

MMU-720AE never propagates StashTranslation transactions downstream, and uses StashTranslation only to prefetch Main TLB contents. MMU-720AE always terminates StashTranslation transactions with a BRESP value of OKAY, even if no translation could be stored in the Main TLB.

The TBU ignores AWPROT[0] and AWPROT[2] for StashTranslation transactions, because they do not affect speculative translation requests.



A StashTranslation transaction can be used to prefetch translations into the Main TLB of MMU-720AE. However, for this prefetching to be useful, any subsequent transactions that intend to take advantage of the translations that have been prefetched into the Main TLB must use the same StreamID as the original prefetch. The StreamID identifies a translation context. Using a different StreamID, SECSID, SSV or SubstreamID for a subsequent transaction means that this subsequent transaction uses a different translation context to the translation that has been prefetched into the Main TLB and might lead to a TLB miss.

2.6.3 Local Translation Interface implementation

MMU-720AE implements the Local Translation Interface (LTI), with certain properties.

For more information, see the [AMBA® LTI Protocol Specification](#).

The following table shows the values of the LTI properties.

Table 2-22: LTI properties

Name	Value	Description
LTI_VC_COUNT	2	Two LTI Virtual channels are chosen, one for read and one for write
LTI_ID_WIDTH	TBUCFG_ID_WIDTH	Equal to ID width of incoming transaction

Name	Value	Description
LTI_MECID_WIDTH	See description	Width of the MECID that is supported. The value of LTI_MEC_WIDTH is calculated as follows: $(TBUCFG_MECID_WIDTH == 0) ? 0 : 16$ Note: When LTI_MECID_WIDTH == 0, the lrmecid signals are 1-bit, unused, and tied-off internally.
LTI_SID_WIDTH	TBUCFG_SID_WIDTH	Equal to width of incoming SID
LTI_SSID_WIDTH	TBUCFG_SSID_WIDTH	Equal to width of incoming SSID
LTI_OG_WIDTH	TBUCFG_LTI_OG_WIDTH	Equal to width of incoming ordering groups
LTI_TLBLOC_WIDTH	See Description	Width of TLB location, in bits. The value of the LTI_TLBLOC_WIDTH parameter, which is a local parameter, is calculated as follows: $LTI_TLBLOC_WIDTH == \text{MAX}(1, ((TBUCFG_DIRECT_IDX == 1) ? ((TBUCFG_MTLB_DEPTH > 0) ? \log_2(TBUCFG_MTLB_DEPTH) : \log_2(4)) : \log_2(TBUCFG_MTLB_PARTS)))$
LTI_LOOP_WIDTH	See Description	Width of the LTI loopback signals, in bits. For the LTI TBU, the value is equal to TBUCFG_LTI_LOOP_WIDTH. For the ACE-Lite TBU, the value is calculated automatically.
LTI_LAUSER_WIDTH	0	LTI user signals are not used
LTI_LRUSER_WIDTH	0	
LTI_LCUSER_WIDTH	0	
LTI_GPC	~TBUCFG_LEGACY_TZ_EN	GPC is supported when not in legacy mode.
LTI_MMU	True	Stage 1 and 2 translation always supported.
LTI_LRADDR_WIDTH	52	Width of translated address.
LTI_LAHWATTR_PRESENT	True	The LAHWATTR signal is present.

2.6.4 MPAM implementation

MMU-720AE implements Memory System Resource Partitioning and Monitoring (MPAM) with some constraints and limitations. MMU-720AE implements some MPAM registers. Registers that are not implemented are not described.

For more information, see the [Arm® Memory System Resource Partitioning and Monitoring \(MPAM\) System Component Specification](#).

MPAM cache maximum capacity partitioning manages the following:

- TBU MTLB

- TCU configuration cache
- TCU Walk cache
- Granule Protection Table (GPT) caches

No other mechanism from the [Arm® Memory System Resource Partitioning and Monitoring \(MPAM\) System Component Specification](#) is implemented.

MPAM and NoStreamID

For NoStreamID transactions, where AxMMUVALID = 0, the following values are used for MPAM:

PARTID	0
PMG	0
MPAM_SP	Match the target PAS of the access

2.6.4.1 TCU MPAM

MMU-720AE enables you to implement TCU Memory System Resource Partitioning and Monitoring (MPAM). Internally, Resource Instance Selector (RIS) is truncated to different values depending on whether Realm Management Extension (RME) mode is enabled or not, and on the security state. However, externally RIS is the normal 4 bits. Because RIS is internally truncated, there is no illegal RIS handling.

RME mode not enabled

When RME mode is not enabled, the following RIS values are used, and internally, RIS is truncated to 1 bit and the maximum legal RIS value is 1.

The RIS value cache is as follows:

0b0	WCB
0b1	CCB

RME mode enabled

When Realm Management Extension (RME) mode is enabled:

0b00	Walk Cache Block (WCB)
0b01	Configuration Cache Block (CCB)
0b10	GPT Cache Block (GCB) in DTI GPC Wrapper (DGW), where GPC is Granule Protection Check (GPC) and GPT is Granule Protection Table (GPT)

0b11

GCB in AXI GPC Wrapper (AGW)

TCU MPAM registers

Four copies of these registers exist in the TCU memory map. See the table named TCU PMCG, RAS, and MPAM register allocation to regions of TCU address space in [4.3.1 TCU memory map](#) on page 128. The following tables show the values of these registers as they appear in the Register page relating to each PAS. When the `TCUCFG_LEGACY_TZ_EN` parameter is set to 1 or the `legacy_tie_off` signal is set to 1, the Root and Realm copies of these registers are RAZ/WI. The following tables show the TCU MPAM registers and register fields that are implemented. Each table describes a TCU MPAM register and shows the bit fields for each, and the values for Secure, Non-Secure, Realm, and Root.

Table 2-23: MPAMF_IDR_LO register, 0x0000, Shared

Field	Value			Description
	S, NS	Root	Realm	
PARTID_MAX	1, 63, 511	0	1, 63, 511	Set to: $(2^{\text{TCUCFG_PARTID_WIDTH}} - 1)$
PMG_MAX	1	0	1	Two Non-secure Performance Monitoring groups are supported per PARTID.
HAS_CCAP_PART	1	1	1	Cache maximum capacity partitioning is supported.
HAS_CPOR_PART	0	0	0	Cache portion partitioning is not supported.
HAS_MBW_PART	0	0	0	Memory Bandwidth partitioning is not supported.
HAS_PRI_PART	0	0	0	Priority partitioning is not supported.
EXT	1	1	1	EXTended MPAMF_IDR.
HAS_IMPL_IDR	0	0	0	Does not have IMPLEMENTATION-SPECIFIC partitioning features.
HAS_MSMON	1	1	1	Supports performance monitoring by matching a combination of PARTID and PMG.
HAS_PARTID_NRW	0	0	0	Does not have MPAMF_PARTID_NRW_IDR, MPAMCFG_INTPARTID, or intPARTID mapping support.

Table 2-24: MPAMF_IDR_HI register, 0x0004, Shared

Field	Value			Description
	S, NS	Root	Realm	
HAS_RIS	1	0	1	Has Resource Instance Selector (RIS).
HAS_EXTD_ESR	0	0	0	MPAMF_ESR is 64-bits. Not relevant because HAS_ESR is 0.
HAS_ESR	0	0	0	MPAMF_ESR and MPAMF_ECR are not implemented.
HAS_NFU	0	0	0	-
HAS_ENDIS	0	0	0	-
RIS_MAX	0, 1, 3	0, 1, 3	0, 1, 3	Maximum RIS value that is used in the Memory-System Component (MSC): Set to 3 Set to 1 in legacy mode

Field	Value			Description
	S, NS	Root	Realm	
SP4	0, 1	1	1	We support four security states: 0 If the TCUCFG_LEGACY_TZ_EN parameter is set to 1 or the legacy_tz_en tie-off signal is set to 1. 1 Otherwise.

Table 2-25: MPAMF_SIDR register, 0x0008, Secure only

Field	Value for Secure	Description
S_PARTID_MAX	1, 63, 511	Set to: $(2^{\text{TCUCFG_PARTID_WIDTH}} - 1)$
S_PMG_MAX	1	Two Secure Performance Monitoring groups are supported per PARTID.

Table 2-26: MPAMF_IIDR register, 0x0018, Shared

Field	Value			Description
	S, NS	Root	Realm	
All fields	- product ID = 0x498 - variant = 0 - revision = MAX (ecorevnum, P_LEVEL), where P_LEVEL is 1.			Implementation ID Register.

Table 2-27: MPAMF_AIDR register, 0x0020, Shared

Field	Value			Description
	S, NS	Root	Realm	
ArchMajorRev	1	1	1	MPAM architecture v1.1
ArchMinorRev	1	1	1	MPAM architecture v1.1

Table 2-28: MPAMF_CCAP_IDR register, 0x0038, Shared

Field	Value			Description
	S, NS	Root	Realm	
CMAW_WD	min ($\log_2(\text{Depth/Banks})$, 8)			Up to 256 fractions are supported.

Table 2-29: MPAMF_MSMON_IDR register, 0x0080, Shared

Field	Value			Description
	S, NS	Root	Realm	
MSMON_CSU	1	1	1	Performance monitor is supported for Cache Storage Usage by PARTID and PMG.
MSMON_MBWU	0	0	0	No performance monitor for Memory Bandwidth Usage by PARTID and PMG.
HAS_LOCAL_CAPT_EVNT	1	1	1	Has the local capture event generator and the MSMON_CAPT_EVNT register.
NO_HW_OFLW_INTR	1	1	1	-
HAS_OFLW_MSI	0	0	0	-
HAS_OFLOW_SR	0	0	0	-

Table 2-30: MPAMF_CSUMON_IDR register, 0x0088, Shared

Field	Value			Description
	S, NS	Root	Realm	
NUM_MON	4	1	4	Four monitoring counters are implemented. For the Root world, one monitoring counter is implemented.
HAS_CAPTURE	1	1	1	Has an MSMON_CSU_CAPTURE register for every MSMON_CSU and supports the capture event behavior.
CSU_RO	1	1	1	-

Table 2-31: MPAMCFG_PART_SEL register, 0x0100, Banked

Field	Value			Description
	S, NS	Root	Realm	
PARTID_SEL	[TCUCFG_PARTID_WIDTH - 1:0] bits are valid	This field is ignored	[TCUCFG_PARTID_WIDTH - 1:0] bits are valid	You can select up to 512 partitions to configure, based on the value of the TCUCFG_PARTID_WIDTH parameter.
RIS	For information about the RIS value, see the information earlier in this section.			Resource Instance Selector (RIS).

Table 2-32: MPAMCFG_CMAX register, 0x0108, Banked

Field	Value			Description
	S, NS	Root	Realm	
CMAX	Width depends on configuration, value is 16-bit aligned. Configure PARTID_SEL before you attempt to update CMAX information for any PARTID.			You can select up to 256 fractions.

Table 2-33: MSMON_CFG_MON_SEL register, 0x0800, Banked

Field	Value			Description
	S, NS	Root	Realm	
MON_SEL	Bits [1:0] are valid, and other bits are ignored.	This field is ignored.	Bits [1:0] are valid, and other bits are ignored.	Selects the monitor to configure.
RIS	For information about the RIS value, see the information earlier in this section.	This field is ignored	For information about the RIS value, see the information earlier in this section.	Resource Instance Selector (RIS).

Table 2-34: MSMON_CFG_CSU_FLT register, 0x0810, Banked

Field	Value			Description
	S, NS	Root	Realm	
PARTID	[TCUCFG_PARTID_WIDTH - 1:0] bits are valid	A value of 0 is supported.	[TCUCFG_PARTID_WIDTH - 1:0] bits are valid	You can select up to 512 partitions to configure, based on the value of the TCUCFG_PARTID_WIDTH parameter.
PMG	Bit [16] are valid	A value of 0 is supported.	Bit [16] are valid	You can select up to PMG number 1. Note: PMG does not exist for the Root world.

Table 2-35: MSMON_CFG_CSU_CTL register, 0x0818, Banked

Field	Value			Description
	S, NS	Root	Realm	
EN	Valid field			The monitor instance is enabled or disabled to collect information.
CAPT_EVNT	0b111			Capture occurs when a MSMON_CAPT_EVNT register is written. No other values are supported.
CAPT_RESET	RES0			There is never a reason to reset a CSU monitor.
OFLOW_STATUS	RES0			Overflow is not possible for a CSU monitor.
OFLOW_INTR	RES0			This MPAM implementation does not support OFLOW_INTR.
OFLOW_FRZ	RES0			Overflow is not possible for a CSU monitor.
SUBTYPE	RES0			This field is reserved for future use.

Table 2-36: MSMON_CSU register, 0x0840, Banked

Field	Value			Description
	S, NS	Root	Realm	
All fields	All fields are valid			Cache storage usage value.

Table 2-37: MSMON_CSU_CAPTURE register, 0x0848, Banked

Field	Value			Description
	S, NS	Root	Realm	
All fields	All fields are valid			Capture cache storage usage.

Table 2-38: MSMON_CAPT_EVNT register, 0x0808, Banked

Field	Value			Description
	S, NS	Root	Realm	
All fields	All fields are valid			Capture event.

2.6.4.2 TBU MPAM

MMU-720AE enables you to implement TBU Memory System Resource Partitioning and Monitoring (MPAM). The TBU Memory-System Component (MSC) has only one resource, MTLB, and does not require a Resource Instance Selector (RIS).

If the `TBUCFG_MTLB_DEPTH` parameter is set to 0, the resource is not present, and if the `TBUCFG_DIRECT_IDX` parameter is set to 1, the resource is present but does not have MPAM controls. The associated ID registers must report values of limited control under these circumstances. Therefore, many non-ID control registers are RAZ/WI in such circumstances.

The following tables show the TBU MPAM registers and register fields that are implemented. Each table describes a TBU MPAM register and shows the bit fields and the permitted values for each. Four copies of these registers exist in the [4.3.2 TBU memory map](#) on page 130. The following tables show the values of these registers as they appear in the Register page relating to each Physical Address Space (PAS). If the value is different in a page relating to a certain PAS, the values

are listed for each PAS. For registers that have different values depending on whether the resource is present and has MPAM controls, the different values are shown in different columns.

All MPAM registers apart from the following are RAZ/WI if the resource is not present or does not have MPAM controls:

- MPAMF_IDR_LO
- MPAMF_IDR_HI
- MPAMF_SIDR
- MPAMF_AIDR
- MPAMF_IIDR

If the `TBUCFG_LEGACY_TZ_EN` parameter is set to 1 or the `legacy_tz_en` tie-off signal is set to 1, the register pages containing the copies of these registers, that relate to the Root and Realm worlds, are RAZ/WI.

Table 2-39: MPAMF_IDR_LO, 0x0000, Shared

Field	Value		Description
	Resource present and has MPAM controls	Resource not present or has no MPAM controls	
PARTID_MAX	1, 63, 511	1, 63, 511	Set to: $(2^{\text{TBUCFG_PARTID_WIDTH}} - 1)$ The value is 0 for Root.
PMG_MAX	1 (0 for Root)	1 (0 for Root)	Two Non-secure Performance Monitoring groups are supported for each PARTID.
HAS_CCAP_PART	1	0	Cache maximum capacity partitioning is supported.
HAS_CPOR_PART	0	0	Cache portion partitioning is not supported.
HAS_MBW_PART	0	0	Memory bandwidth partitioning is not supported.
HAS_PRI_PART	0	0	Priority partitioning is not supported.
EXT	1	1	EXTended MPAMF_IDR.
HAS_IMPL_IDR	0	0	Does not have IMPLEMENTATION-SPECIFIC partitioning features.
HAS_MSMON	1	0	Supports performance monitoring by matching a combination of PARTID and PMG.
HAS_PARTID_NRW	0	0	Does not have MPAMF_PARTID_NRW_IDR, MPAMCFG_INTPARTID, or intPARTID mapping support.

Table 2-40: MPAMF_IDR_HI, 0x0004, Shared

Field	Value		Description
	Resource present and has MPAM controls	Resource not present or has no MPAM controls	
HAS_RIS	0	0	Does not have a Resource Instance Selector (RIS).
HAS_EXTD_ESR	0	0	MPAMF_ESR is 64-bits. Not relevant because HAS_ESR is 0.

Field	Value		Description
	Resource present and has MPAM controls	Resource not present or has no MPAM controls	
HAS_ESR	0	0	MPAMF_ESR and MPAMF_ECR are not implemented.
HAS_NFU	0	0	-
HAS_ENDIS	0	0	-
SP4	0 or 1	0	The value is 1, if not in legacy mode.

Table 2-41: MPAMF_SIDR, 0x0008, Secure only

Field	Value	Description
S_PARTID_MAX	1, 63, 511	Set to: $(2^{\text{TBUCFG_PARTID_WIDTH}} - 1)$
S_PMG_MAX	1	Two Secure Performance Monitoring groups supported per PARTID.

Table 2-42: MPAMF_IIDR, 0x0018, Shared

Field	Value	Description
product ID	0x499	Implementation ID Register field.
variant	0	Implementation ID Register field.
revision	MAX (ecorevnum, P_LEVEL), where P_LEVEL is 1.	Implementation ID Register field.

Table 2-43: MPAMF_AIDR, 0x0020, Shared

Field	Value	Description
ArchMajorRev	1	MPAM architecture v1.1.
ArchMinorRev	1	MPAM architecture v1.1.

The following registers are all RAZ/WI if the resource is not present or does not have MPAM controls.

Table 2-44: MPAMF_CCAP_IDR, 0x0038, Shared

Field	Value	Description
CMAX_WD	$\min(\log_2(\text{Depth/Banks}), 8)$	Implementation ID Register.

Table 2-45: MPAMF_MSMON_IDR, 0x0080, Shared

Field	Value	Description
MSMON_CSU	1	Performance monitor is supported for Cache Storage Usage by PARTID and PMG.
MSMON_MBWU	0	No performance monitor for Memory Bandwidth Usage by PARTID and PMG.
HAS_LOCAL_CAPT_EVNT	1	Has the local capture event generator and the MSMON_CAPT_EVNT register.
NO_HW_OFLW_INTR	1	-
HAS_OFLW_MSI	0	-
HAS_OFLOW_SR	0	-

Table 2-46: MPAMF_CSUMON_IDR, 0x0088, Shared

Field	Value	Description
NUM_MON	4, or 1 for Root	Four monitoring counters are implemented. For the Root world, one monitoring counter is implemented.
HAS_CAPTURE	1	Has an MSMON_CSU_CAPTURE register for every MSMON_CSU and supports the capture event behavior.
CSU_RO	1	-

Table 2-47: MPAMCFG_PART_SEL, 0x0100, Banked

Field	Value	Description
PARTID_SEL	Non-secure, Secure, and Realm Bits [(TBUCFG_PARTID_WIDTH - 1):0] valid. Root This field is ignored.	You can select up to 512 partitions to configure, based on the value of the TBUCFG_PARTID_WIDTH parameter. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-48: MPAMCFG_CMAX, 0x0108, Banked

Field	Value	Description
CMAX	The width depends on the configuration, and the value is 16-bit aligned.	You can select up to 256 fractions. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-49: MSMON_CFG_MON_SEL, 0x0800, Banked

Field	Value	Description
MON_SEL	Non-secure, Secure, and Realm Bits [1:0] are valid, and other bits are ignored. Root This field is ignored.	Selects the monitor to configure. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-50: MSMON_CAPT_EVTNT, 0x0808, Banked

Field	Value	Description
All fields	All fields are valid.	Capture event. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-51: MSMON_CFG_CSU_FLT, 0x0810, Banked

Field	Value			Description
	S, NS	Root	Realm	
PARTID	Bits [(TBUCFG_PARTID_WIDTH - 1):0] are valid.	A value of 0 is supported.	Bits [(TBUCFG_PARTID_WIDTH - 1):0] are valid.	You can select up to 512 partitions to configure, based on the value of the TBUCFG_PARTID_WIDTH parameter. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Field	Value			Description
	S, NS	Root	Realm	
PMG	Bit [16] is valid.	A value of 0 is supported.	Bit [16] is valid.	You can select up to PMG number 1. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-52: MSMON_CFG_CSU_CTL, 0x0818, Banked

Field	Value	Description
EN	Valid field.	The monitor instance is enabled or disabled to collect information. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.
CAPT_EVNT	3'b111	Capture occurs when a MSMON_CAPT_EVNT register is written. No other values are supported. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.
CAPT_RESET	RES0	There is never a reason to reset a CSU monitor. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.
OFLOW_STATUS	RES0	Overflow is not possible for a CSU monitor. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.
OFLOW_INTR	RES0	This MPAM implementation does not support OFLOW_INTR. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.
OFLOW_FRZ	RES0	Overflow is not possible for a CSU monitor. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.
SUBTYPE	RES0	This field is reserved for future use. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-53: MSMON_CSU, 0x0840, Banked

Field	Value	Description
All fields	All fields are valid.	Cache storage usage value. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

Table 2-54: MSMON_CSU_CAPTURE, 0x0848, Banked

Field	Value	Description
All fields	All fields are valid.	Capture cache storage usage. When direct indexing or direct partitioning is enabled, this field does not reflect any meaningful value.

2.7 Configuration parameters and methodology

The TBU, TCU, and BAS components in MMU-720AE are delivered as SystemVerilog that you can parameterize. Use the Generate script to configure these components. There are several versions of the switch component, part of the BAS components that are delivered, to accommodate different numbers of subordinate interfaces.

For more information about the Generate script and detailed descriptions of parameters, see the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

2.7.1 TCU I/O and buffer configuration parameters

You can configure the Translation Control Unit (TCU) I/O and buffer.



For more detailed descriptions of these configuration parameters, see the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

The following table shows the TCU buffer configuration parameters.

Table 2-55: TCU buffer configuration parameters

Parameter name	Values	Description
TCUCFG_QTW_DATA_WIDTH	64, 128, 256, 512	ACE-Lite+DVM Queue and Table Walk (QTW) and ACE-Lite Page Table Walk (PTW) interface data width.
TCUCFG_CC_DEPTH	4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096	Configuration cache depth, in entries.
TCUCFG_WC_DEPTH	8, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536	Walk cache depth, in entries.
TCUCFG_WC_BANKS	1, 2, 4	Number of banks in walk cache.
TCUCFG_WC_WAYS	4, 8, 16	Number of ways in walk cache.
TCUCFG_CC_WAYS	4, 8, 16	Number of ways in configuration cache.
TCUCFG_AGW_GC_DEPTH	8, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536	AXI Granule Protection Check (GPC) Wrapper (AGW) Granule Protection Table (GPT) cache depth, in entries.
TCUCFG_AGW_GC_BANKS	1, 2, 4	Number of banks in AGW GPT cache.
TCUCFG_AGW_GC_WAYS	4, 8, 16	Number of ways in AGW GPT cache.
TCUCFG_DGW_GC_DEPTH	8, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536	DGW GPT cache depth, in entries.
TCUCFG_DGW_GC_BANKS	1, 2, 4	Number of banks in DGW GPT cache.
TCUCFG_DGW_GC_WAYS	4, 8, 16	Number of ways in DGW GPT cache.

Parameter name	Values	Description
TCUCFG_NUM_TBU	14, 62	Maximum number of DTI requesters, that is, DTI-TBU and DTI-ATS requesters, that the TCU supports. The value is two less than 16 or 64 to better fit into system memory maps.
TCUCFG_XLATE_SLOTS	4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096	Total permitted translation requests from all DTI requesters.
TCUCFG_PTW_SLOTS	2, 4, 8, 16, 32, 64, 128, 256, 512	Number of parallel translation table walks.
TCUCFG_CTW_SLOTS	1, 2, 4.	Number of parallel configuration table walks.
TCUCFG_DGW_SLOTS	8, 16, 32, 64, 128, 256, 512	Number of GPC walks slots in DGW.
TCUCFG_WC_LKP_SLOTS	2-28	Number of lookup slots that the walk cache uses.
TCUCFG_AGW_GC_LKP_SLOTS	2-28	AXI Granule Protection Check (GPC) Wrapper (AGW) Granule Protection Table (GPT) cache lookup slots. The number of lookup slots that the AGW GPT cache uses.
TCUCFG_DGW_GC_LKP_SLOTS	2-28	DTI Granule Protection Check (GPC) Wrapper (DGW) Granule Protection Table (GPT) cache lookup slots. The number of lookup slots that the DGW GPT cache uses.
TCUCFG_LEGACY_TZ_EN	0, 1	Disable Realm Management Extension features and switch back to two security states. Secure and Non-secure remains, Realm and Root are removed. Note: The value of the TCUCFG_LEGACY_TZ_EN parameter must be the same as the value of TBUCFG_LEGACY_TZ_EN parameter
TCUCFG_PARITY_ON	0, 1	Enables parity protection on large flop structures and configuration registers.
TCUCFG_MECID_WIDTH	0, 4, 8, 12, 16	Width of the MECID that is supported, which is the number of bits of MECID that are supported internally. Note: The value of the TCUCFG_MECID_WIDTH parameter must be the same as the value of TBUCFG_MECID_WIDTH parameter.
TCUCFG_CC_IDXGEN_MODE	0, 1	Index generation mode for the configuration cache.
TCUCFG_DTI_ATS	0, 1, 2, 3, 4, 5, 6, 7, 8	Maximum number of DTI-ATS requesters.
TCUCFG_DTI_ATS_INV_MAX	2, 4, 8, 16, 32	Maximum number of ATS invalidate messages which can be outstanding per manager without a DTI_ATS_INV_COMP response.
TCUCFG_PMU_COUNTERS	4, 16, 32	Number of PMU counters.
TCUCFG_PARTID_WIDTH	1, 6, 9	Width of PARTID that is supported.
TCUCFG_HZU_DEPTH	2, 4, 8, 16, 32, 64.	We expect you to disable TCU hazarding. Therefore, to save area, set this parameter to a value of 2.
TCUCFG_PREFETCH_SUPPORTED	0, 1	Specifies whether prefetch is supported.
TCUCFG_DATARAM_TYPE	0, 1, 2	RAM type for data group of RAMs.
TCUCFG_SLOTRAM_TYPE	0, 1, 2	RAM type for slot group of RAMs.
TCUCFG_CACHERAM_TYPE	0, 1	RAM type for cache group of RAMs.

Parameter name	Values	Description
TCUCFG_DVM_VAS	49 or 53	Virtual address size used by the system. The INV block uses the TCUCFG_DVM_VAS parameter to correctly form the invalidate command. internally.

2.7.2 TCU debug configuration parameters

You can configure the Translation Control Unit (TCU) debug parameters.



For more detailed descriptions of these configuration parameters, see the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

The following table shows the TCU debug configuration parameters.

Table 2-56: TCU debug configuration parameters

Parameter name	Values	Description
TCUCFG_USE_ELA_DEBUG	0, 1	Determines whether ELA debug is used or not.

2.7.3 Common ACE-Lite and LTI TBU configuration parameters

You can configure the common Local Translation Interface (LTI) Translation Buffer Unit (TBU) and ACE-Lite TBU parameters.



For more detailed descriptions of these configuration parameters, see the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

The following table shows the ACE-Lite TBU and LTI TBU configuration parameters.

Table 2-57: ACE-Lite TBU and LTI TBU configuration parameters

Parameter name	Values	Description
TBUCFG_SID_WIDTH	8, 16, 20, 24	Stream ID width.
TBUCFG_SSID_WIDTH	1, 8, 20	SubstreamID width.
TBUCFG_DIRECT_IDX	0, 1	Direct indexing enabled.
TBUCFG_MTLB_PARTS	1, 2, 4, 8, 16	Number of Main TLB partitions.
TBUCFG_LTI_OG_WIDTH	1-5	LTI ordering group width.
TBUCFG_LA_HNDSHK_MODE	0, 1, 2, 3	Register slice configuration on address channel before TLB lookup.
TBUCFG_LR_HNDSHK_MODE	0, 1, 2, 3	Register slice configuration on translation response path.
TBUCFG_XLATE_SLOTS	2, 4, 8, 16, 32, 64, 128, 256, 512, 1024	Number of translation slots, controlling the Hit-Under-Miss capability of the TBU.

Parameter name	Values	Description
TBUCFG_MTLB_LKP_SLOTS	2-28	Number of MTLB lookup slots.
TBUCFG_MTLB_UPD_SLOTS	2, 4, 8	Number of MTLB update slots.
TBUCFG_UTLB_DEPTH	4, 8, 12, 16, 32, 64	MicroTLB depth, in entries.
TBUCFG_MTLB_DEPTH	0, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536	Main TLB (MTLB) depth, in entries.
TBUCFG_MTLB_WAYS	4, 8, 16	Number of ways in the MTLB.
TBUCFG_MTLB_BANKS	1, 2, 4	Number of banks in the MTLB.
TBUCFG_PMU_COUNTERS	4, 16, 32	Number of PMU counters.
TBUCFG_PARTID_WIDTH	1, 6, 9	Width of PARTID that is supported.
TBUCFG_MECID_WIDTH	0, 4, 8, 12, 16	Width of the MECID that is supported, which is the number of bits of MECID that are supported internally. Note: The value of the TBUCFG_MECID_WIDTH parameter must be the same as the value of TCUCFG_MECID_WIDTH parameter
TBUCFG_HZRD_ENTRIES	0, 4, 8, 16, 32, 64	Number of hazard entries.
TBUCFG_SLOTRAM_TYPE	0, 1, 2	RAM type for slot group of RAMs.
TBUCFG_CACHERAM_TYPE	0, 1	RAM type for cache group of RAMs.
TBUCFG_LEGACY_TZ_EN	0, 1	Disable Realm Management Extension (RME) features and switch back to 2-security states. Secure and Non-secure remains, Realm and Root are removed. Note: The value of the TBUCFG_LEGACY_TZ_EN parameter must be the same as the value of TCUCFG_LEGACY_TZ_EN parameter
TBUCFG_PARITY_ON	0, 1	Enables parity protection on large flop structures and configuration registers.

2.7.4 ACE-Lite TBU register slice configuration parameters

You can configure the Translation Buffer Unit (TBU) register slices.



For more detailed descriptions of these configuration parameters, see the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

The following table shows the ACE-Lite TBU register slice configuration parameters.

Table 2-58: ACE-Lite TBU register slice configuration parameters

Parameter name	Value	Description
TBUCFG_SI_AR_HNDSHK_MODE	0, 1, 2, 3	Register slice configuration parameters. You can configure the register slice to be one of the following: - Fully registered - Registered on the forward path only - Registered on the reverse path only - Bypassed
TBUCFG_SI_R_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_SI_AW_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_SI_W_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_SI_B_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_MI_AR_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_MI_R_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_MI_AW_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_MI_W_HNDSHK_MODE	0, 1, 2, 3	
TBUCFG_MI_B_HNDSHK_MODE	0, 1, 2, 3	

2.7.5 ACE-Lite TBU I/O configuration parameters

You can configure the ACE-Lite Translation Buffer Unit (TBU) I/O.



For more detailed descriptions of these configuration parameters, see the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

The following table shows the ACE-Lite TBU I/O configuration parameters that apply to both the TBS and TBM interfaces.

Table 2-59: ACE-Lite TBU I/O configuration parameters

Parameter name	Values	Description
TBUCFG_ID_WIDTH	1-32	AXI ID width
TBUCFG_DATA_WIDTH	64, 128, 256, 512	AXI data width
TBUCFG_ARUSER_WIDTH	1-128	AXI USER bus widths
TBUCFG_AWUSER_WIDTH		
TBUCFG_RUSER_WIDTH		
TBUCFG_WUSER_WIDTH		
TBUCFG_BUSER_WIDTH		
TBUCFG_STASH_SUPPORT	0, 1	Defines whether stash ID signals are used. When configured as 0, stash ID signals on the subordinate interface are ignored, stash ID signals on the manager interface are driven to 0, and AXI Cache_Stash_Transactions are not supported.
TBUCFG_LOOP_WIDTH	1-8	AXI loopback signal width

Parameter name	Values	Description
TBUCFG_WBUF_DEPTH	0, 8, 16, 32, 64, 128, 256, 512, 1024, 2048	Write buffer depth.
TBUCFG_LFIFO_DEPTH	0, 4	Latency FIFO depth.
TBUCFG_OT_TRACKER_TYPE	0, 1	Type of the outstanding transaction tracker that is used.
TBUCFG_ROT_DEPTH	4, 8, 16, 32, 64, 128, 256, 512	Number of outstanding read transactions.
TBUCFG_WOT_DEPTH	4, 8, 16, 32, 64, 128, 256, 512	Number of outstanding write transactions.
TBUCFG_DATARAM_TYPE	0, 1, 2	RAM type for data group of RAMs.

2.7.6 LTI TBU configuration parameters

You can configure the Local Translation Interface (LTI) Translation Buffer Unit (TBU).



For more detailed descriptions of these configuration parameters, see the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

The following table shows the LTI TBU configuration parameters.

Table 2-60: LTI TBU configuration parameters

Parameter name	Values	Description
TBUCFG_LTI_ID_WIDTH	1-32	LTI ID width.
TBUCFG_LTI_LOOP_WIDTH	1-256	LTI loop width.

2.7.7 Common TBU debug configuration parameters

You can configure the Translation Buffer Unit (TBU) debug parameters.



For more detailed descriptions of these configuration parameters, see the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

The following table shows the TBU debug configuration parameters.

Table 2-61: Common TBU debug configuration parameters

Parameter name	Values	Description
TBUCFG_USE_ELA_DEBUG	0, 1	Determines whether ELA debug is used or not

2.8 Debug capability

The CoreLink™ MMU-720AE System Memory Management Unit provides debug functionality using the CoreSight™ ELA-600 Embedded Logic Analyzer.

For more information about debug capability, see the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.



The CoreSight™ ELA-600 Embedded Logic Analyzer is a separate licensed product that is not included with the CoreLink™ MMU-720AE System Memory Management Unit.

Configuration options

For TCU configuration options, see [2.7.2 TCU debug configuration parameters](#) on page 105.

For TBU configuration options, see [2.7.7 Common TBU debug configuration parameters](#) on page 108.

Signals

For TCU observation signals, see [B.1 TCU observation interfaces](#) on page 368.

For ACE-Lite TBU observation signals, see [B.2 ACE-Lite TBU observation interfaces](#) on page 372.

For LTI TBU observation signals, see [B.3 LTI TBU observation interfaces](#) on page 375.

3. Fault detection and control mechanisms

MMU-720AE provides several Functional Safety (FuSa) features to detect random hardware faults, systematic faults, and latent faults.

All safety mechanisms have a Fault Detection Timeout Interval (FDTI) of <8 cycles, except interface checking of asynchronous signals which have an FDTI of <256 cycles.

There are several general mechanisms for Fault Detection and Control (FDC) that are built into the MMU-720AE architecture. For example:

- We apply protection to several interfaces within MMU-720AE. See [3.1 Interface protection](#) on page 110.
- MMU-720AE uses logic duplication as its primary method of detecting faults within its digital logic. See [3.2 Logic duplication](#) on page 111.
- Error Correcting Codes (ECC) capable of detecting double bit errors and correcting single bit errors protect the RAM memories. See [3.3 RAM protection](#) on page 113.
- MMU-720AE contains a Fault Management Unit (FMU) that records and controls the reporting of faults in both the TCU and TBU. See [3.4 Error reporting](#) on page 113.

3.1 Interface protection

We apply protection to several I/O interfaces within MMU-720AE, and the following sections describe the types of parity used. You can disable interface protection mechanisms to support connections with legacy IP without matching interface parity.

If you disable interface parity it does not affect the diagnostic coverage of the SMMU as parity is checked immediately upon ingress to the SMMU. However, the wires between the SMMU and the connecting IP are unprotected. Analyze and reduce this effect, for example, keep the wires short between the SMMU and the connecting IP.

I/O protection allows for protection on unit interfaces using, for example, parity and duplication mechanisms. [A.6 I/O Port protection](#) on page 334 describes these CHK signals and how to connect them.

IP protection is always enabled, however MMU-720AE still supports connections to interfaces without I/O protection by leaving the CHK signals disconnected and disabling the associated safety mechanism. To enable and disable safety mechanisms, see [4.20.5 TCU_FMU_SMEN register](#) on page 277 and [4.21.5 TBU_FMU_SMEN register](#) on page 286.

The main methods of protecting interfaces are as follows:

Parity Checks (synchronous)

Synchronous signals are synchronous to the clock of the receiving unit. For synchronous interfaces, parity checking is the most area and power efficient checking mechanism and is the preferred protection mechanism.

This parity is defined by the signal's associated specifications and Arm AMBA® Interface Parity Specification AHB, APB and AXI4-Stream (AES 0010). For non-AMBA® signals, parity check signals are a single bit odd-parity of the associated signal. A duplicated parity check module is included to provide latent fault diagnostic. For incoming interface signals, MMU-720AE flags an error if either the parity check module or the duplicate module detect a mismatch.

Full duplication (synchronous)

Where parity checks are not suitable, synchronous interfaces are fully duplicated. This is either because of a lack of support within the protocol, or in the case of the Distributed Translation Interface (DTI), to avoid unnecessary complication of the DTI interconnect. All duplicated interfaces are inverted with the exception of the Distributed Translation Interface (DTI) and Local Translation Interfaces (LTIs).

Asynchronous filtering

For asynchronous signals there is no requirement for these signals to be synchronized to the receiving unit's clock. Asynchronous interfaces require special consideration because the ambiguity in timing causes the previous protection techniques to incorrectly flag an error. Instead, asynchronous signals are filtered by a mechanism which does not propagate changes until the primary and CHK signals are in agreement. If primary and CHK signals disagree, a timeout signal triggers within 256 cycles, raising an interrupt. If you use the FMU_SMEN register to disable the asynchronous safety mechanisms you disable this filter and propagate only the primary copy of the signal.

Clock input protection

MMU-720AE IP includes qactive and qactivechk signals from both the TCU and TBU. These signals are asynchronous and the CHK copy is an inverted copy of the primary copy.



After a reset exit, the qactive signal requires 2 clock cycles to stabilize. The parity checks on qactive must be disabled for 2 clock cycles out of reset.

Reset input protection

The resetn and resetnchk signals are combined together, using a logical OR operation. Therefore, transient faults and stuck-at-0 faults on either resetn or resetnchk have no effect. Stuck-at-1 faults on resetn might prevent an IP reset and are not detected by the reset checking circuitry. However, stuck-at-1 faults are covered by AoU_SysInt_Reset.

DFT signal protection

MMU-720AE includes several Design For Test (DFT) related signals at the I/O of both the TCU and TBU.

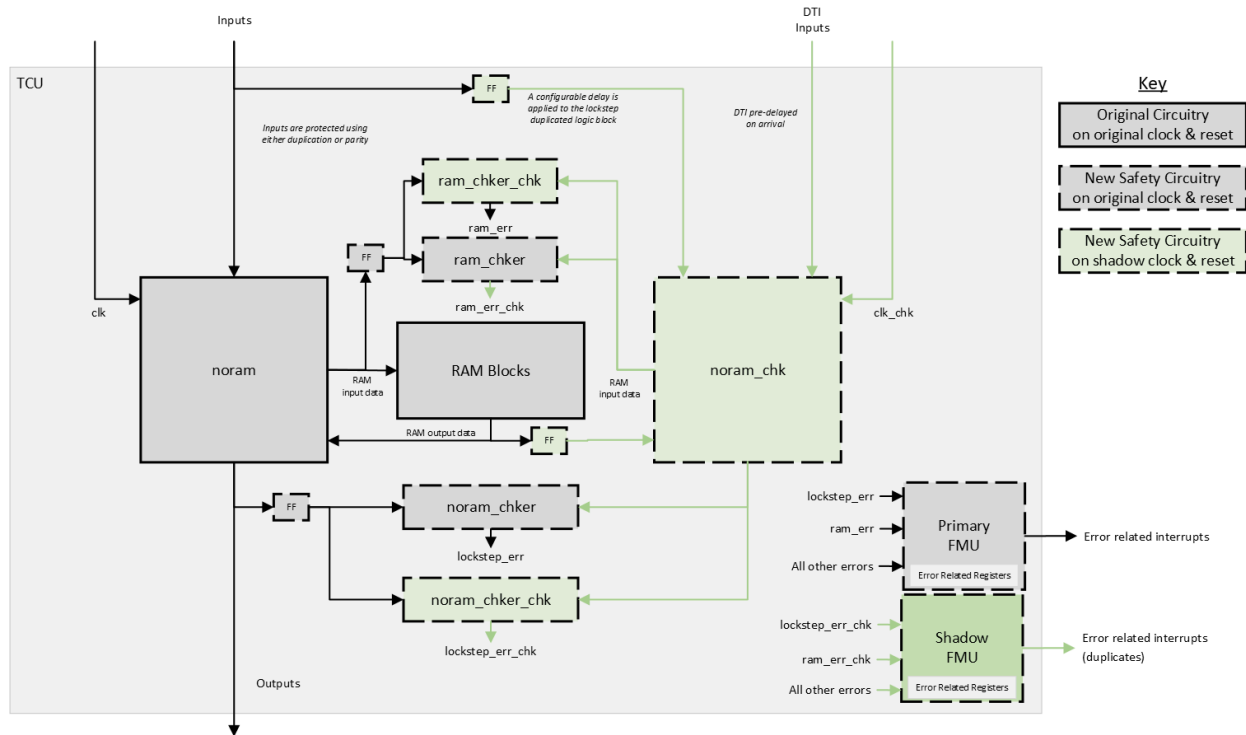
For more information about the I/O interface signals and their associated protection style, see [A.6 I/O Port protection](#) on page 334.

3.2 Logic duplication

MMU-720AE uses logic duplication as its primary method of detecting faults within its digital logic.

The following figure shows TCU duplication.

Figure 3-1: Lockstep and RAM protection in the TCU



The preceding figure is a simplified version of a TBU and TCU and it shows the following main points:

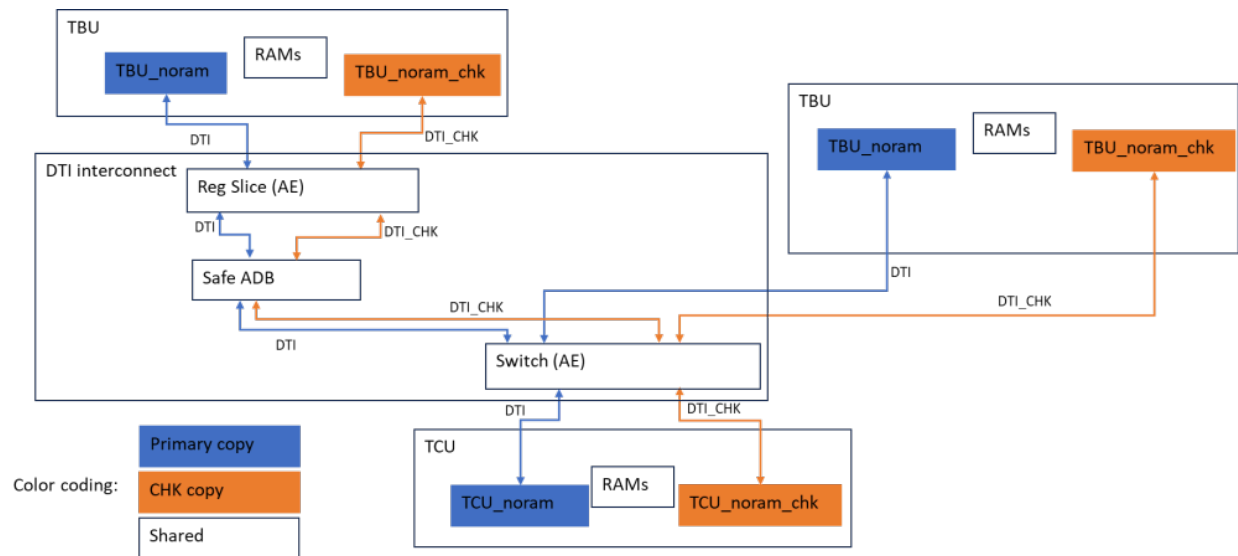
- Optionally, you can set the `FUSA_TEMPORAL_DELAY` parameter to 2 to delay the CHK logic by 2 cycles. When you use this option, it is easier to implement and physically separate the primary and CHK logic blocks. However, the power and total flop count increases slightly. See [3.5 FuSa specific configuration parameter](#) on page 118.
- The logic checkers are duplicated for latent fault protection. The FMU block is duplicated for similar reasons, although the preceding figure does not show the duplicate copy.

3.2.1 DTI duplication

Distributed Translation Interface (DTI) components, for example, register slices, switches, sizers, and domain bridges are duplicated.

The following figure shows the duplication of the DTI components.

Figure 3-2: DTI duplication



The DTI switches, sizers, and register slices are fully duplicated and form separate networks. These networks must remain in lockstep. For example, for every primary register slice, a CHK copy must exist.

Clock domain crossing bridges, for example, the AMBA® Domain Bridge (ADB) cannot be fully independent because of the asynchronous nature of clock domain crossings. These bridges pass both the primary and CHK DTI messages across the domain and re-synchronize the messages, while simultaneously checking the messages for differences.

Faults that are detected on the ADB are reported to the TCU. The Low Power Distributor (LPD) similarly has interface protection and logic duplication which it reports to the TCU.

DTI interconnect components, for example, a register slice, sizer, or switch, do not have reset synchronizers and filter protection as used in larger modules. However, you can provide this as an option. Therefore, you can add your own reset synchronizer and wire the reset synchronizer to your interconnect components. With this design you can benefit from hardware re-use.

3.3 RAM protection

Error Correcting Codes (ECC) which are capable of detecting double bit errors and correcting single bit errors, protect the RAM memories in MMU-720AE.

There is no duplication of MMU-720AE RAM modules. Instead, the RAMs use error detection and correction.

For more information about RAM protection, see [2.5.5 RAS implementation](#) on page 59.

3.4 Error reporting

MMU-720AE contains a Fault Management Unit (FMU) that records and controls the reporting of faults in both the TCU and TBU.

The error reporting implements the following functionality:

- Enable and disable the Safety Mechanisms and interrupts
- Receive and store fault information from enabled Safety Mechanisms
- Raise interrupts when faults are detected
- Allow software error injection within Safety Mechanisms

3.4.1 Access the FMU

Fault Management Unit (FMU) registers for both the TCU and TBUs can be accessed through the APB bus in the TCU.

The FMU registers adopt the highest level of security from a TrustZone® perspective. This security level can be Secure or Root depending on the system configuration. The TCU and each TBU have a separate set of interrupts. If multiple TBUs and or TCU interrupts are OR'd together at a system level, software must cycle through the error records in each TCU and TBU to determine the source of the error.

For more information about the TCU FMU registers, see [4.20 TCU FMU registers summary](#) on page 271 and for the TBU FMU registers, see [4.21 TBU FMU registers summary](#) on page 280.

3.4.2 Control which Safety Mechanisms report errors to the FMU

MMU-720AE provides a TCU_FMU_SMEN and a TBU_FMU_SMEN configuration register that control the error reporting behavior for each individual Safety Mechanism (SM) in that block.

You can use the [4.20.5 TCU_FMU_SMEN register](#) on page 277 and the [4.21.5 TBU_FMU_SMEN register](#) on page 286 to specify which individual SMs report errors to the central Fault Management Unit (FMU). You can use the [4.20.2 TCU_FMU_ERRCTLR register](#) on page 273 and the [4.21.2 TBU_FMU_ERRCTLR register](#) on page 282 to turn off all Safety Mechanisms (SMs) or none. All SMs in a TCU or a TBU are enabled at reset.

If you do not want to check a particular SM, use the software to write to the TCU_FMU_SMEN register or the TBU_FMU_SMEN register to disable that SM. By default, SMs in the SMMU are enabled at reset.



Error reporting for a Safety Mechanism through TCU_FMU_SMEN or TBU_FMU_SMEN programming must only occur prior to mission-critical behavior. Software must not reprogram this register during mission-critical operation.

Reprogramming this register during mission-critical operation can cause the error reporting mechanism to miss errors. Faults that occur while enabling safety

mechanisms and persist into the mission critical window are still detected correctly after the register write has completed. These faults include stuck-at faults that occur while using the SMEN register.

3.4.3 FMU interrupt handling

The Fault Management Unit (FMU) implements several types of external interrupt to report errors to the rest of the system. You can individually enable each type of interrupt signaling by programming the FMU configuration registers.

When software receives an interrupt, the software first reads the Global Status Register either the [4.20.4 TCU_FMU_ERRGSR register](#) on page 276 or the [4.21.4 TBU_FMU_ERRGSR register](#) on page 285 that issued the interrupt. The software can determine whether the error originated in the Single-Error-Correct-Double-Error-Detect (SECCDED) Error Correcting Code (ECC) RAMs, or the MMU-720AE specific fault detection mechanisms. The software can then read the [4.20.3 TCU_FMU_ERRSTATUS register](#) on page 274 or the [4.21.3 TBU_FMU_ERRSTATUS register](#) on page 283 and handle the error. If necessary, the software clears the error status register and proceeds with normal operation.

However, some errors in particular Critical Errors, might require a full reset to permanently clear the interrupt, especially if the internal state has been corrupted. For example, a lockstep error occurs when the primary and shadow blocks are not in lockstep. Therefore, errors might still be detected even after clearing the ERRSTATUS registers until the IP has been reset.



Note

The [4.20.4 TCU_FMU_ERRGSR register](#) on page 276 and the [4.21.4 TBU_FMU_ERRGSR register](#) on page 285 determine whether a fault is in the RAMs or other safety mechanisms and reflect the interrupts coming from the RAM and FMU units respectively. If only edge triggered interrupts are generated, the ERRGSR register might be empty. In this case, the interrupt handler reads both the [4.20.3 TCU_FMU_ERRSTATUS register](#) on page 274 and [4.8.3 TCU_ERRSTATUS register](#) on page 165 or the [4.21.3 TBU_FMU_ERRSTATUS register](#) on page 283 and [4.17.3 TBU_ERRSTATUS register](#) on page 236 to determine the source of the interrupt. When reading, the ERRSTATUS register has an overflow bit. This bit can be triggered even from a single point fault on a single cycle, as it can cause issues detected in multiple places or across multiple cycles.

When an MMU-720AE Safety Mechanism (SM) detects an error, the error is usually logged in the FMU error record.

You can configure the FMU to signal interrupts externally when an error is logged, enabling the FMU to report the error state to the system.

To provide this external signaling, the FMU implements the following interrupt types:

Error Recovery Interrupt (ERI)

Used to report Uncorrected Errors (UEs) and Deferred Errors (DEs) to the rest of the system. The interrupt signals `ras_et_eri`, `ras_et_eri_chk`, `ras_lt_eri`, and `ras_lt_eri_chk` communicate these errors to the rest of the system. You can use ERIs for error recovery, fault handling, or both.

Fault Handling Interrupt (FHI)

Used to report UEs and Corrected Errors (CEs) to the rest of the system. The interrupt signals `ras_et_fhi`, `ras_et_fhi_chk`, `ras_lt_fhi`, and `ras_lt_fhi_chk` communicate these errors to the rest of the system. You can use FHIs for fault handling.

Critical Error Interrupt (CRI)

Used to report critical error conditions to the rest of the system. The interrupt signals `ras_et_cri`, `ras_et_cri_chk`, `ras_lt_cri`, and `ras_lt_cri_chk` communicate these errors to the rest of the system. You can use CRIs for error recovery.

Set the ED bit, bit 0, of the [4.20.2 TCU_FMU_ERRCTLR register](#) on page 273 or the [4.21.2 TBU_FMU_ERRCTLR register](#) on page 282 to enable error reporting and logging. The bit which is not implemented is Reserved0 or bit 1.

MMU-720AE has three types of interrupts that are Edge triggered and four types of interrupts that are Level sensitive.

The Edge triggered interrupts are as follows:

- Fault Handling Interrupt
- Error Recovery Interrupt
- Critical Interrupt

The Level triggered interrupts are as follows:

- Fault Handling Interrupt
- Error Recovery Interrupt
- Critical Interrupt
- Reliability, Availability, and Serviceability Interrupt (`ras_lt_irpt`)

For MMU-720AE specific faults might occur unless you program them not to use the [4.20.2 TCU_FMU_ERRCTLR register](#) on page 273 or [4.21.2 TBU_FMU_ERRCTLR register](#) on page 282.

Software Generated Error Interrupts

Software can generate the error interrupts using the [4.20.6 TCU_FMU_ERRGEN register](#) on page 278 or the [4.21.6 TBU_FMU_ERRGEN register](#) on page 287. Software reads the [4.20.3 TCU_FMU_ERRSTATUS register](#) on page 274 or [4.21.3 TBU_FMU_ERRSTATUS register](#) on page 283 to verify the interrupt-triggering error.

3.4.4 FMU error injection

To inject an error into a safety mechanism, write to the [4.20.6 TCU_FMU_ERRGEN register](#) on page 278 or [4.21.6 TBU_FMU_ERRGEN register](#) on page 287 of the Fault Management Unit (FMU).

There are also FMU registers that enable fault injection and clearing for each protection mechanism.

For more information, see [4.20 TCU FMU registers summary](#) on page 271 and [4.21 TBU FMU registers summary](#) on page 280.

3.4.5 FMU warm reset

A warm reset clears the functional state but preserves the Fault Management Unit (FMU) error record by asserting only the functional resets. This operation is useful for diagnosing Functional Safety (FuSa) errors because you can reset MMU-720AE while preserving FMU error record information for later analysis.

TBU preserve reset

The TBU contains an `fmurstdisable` pin which, if asserted, prevents the reset from affecting any of the FMU registers including the [4.21.3 TBU_FMU_ERRSTATUS register](#) on page 283 and the [4.21.4 TBU_FMU_ERRGSR register](#) on page 285. Both the `fmurstdisable` and `fmurstdisablechk` ports must be stable prior to entering reset. If these ports are not stable, a glitch on one of these signals might cause a cold reset when reset is asserted.

For more information about the TBU FMU registers see, [4.21 TBU FMU registers summary](#) on page 280.

TCU preserve reset

The TCU contains an `fmurstdisable` pin which, if asserted, prevents the reset from affecting any of the FMU registers including the [4.20.3 TCU_FMU_ERRSTATUS register](#) on page 274 and the [4.20.4 TCU_FMU_ERRGSR register](#) on page 276. Both the `fmurstdisable` and `fmurstdisablechk` ports must be stable prior to entering reset. If these ports are not stable, a glitch on one of these signals might cause a cold reset when reset is asserted.

For more information about the TCU FMU registers see, [4.20 TCU FMU registers summary](#) on page 271.

Test a TCU or TBU reset disable

You can test the FMU reset disable functionality that is present in both the TCU and TBU FMUs.

There are two ways to test a TCU or TBU reset functionality. You can verify that FMU registers are cleared during a standard reset or verify that the FMU registers are preserved when asserting `fmurstdisable`. The following steps apply to both the TCU and the TBU:

1. Verify that the FMU registers are cleared during a standard reset:
 - a. Issue a reset using the `resetsn` and `resetsnchk` pins.

- b. Inject an error into the FMU registers.
 - c. Read back the error status from the FMU and confirm that the injected error is present.
 - d. Issue a reset using the resetn and resetnchk pins.
 - e. Read back the error status from the FMU and confirm that the injected error has been cleared.
2. Verify that the FMU registers are preserved when asserting fmurstdisable:
 - a. Issue a reset using the resetn and resetnchk pins.
 - b. Inject an error into the FMU registers.
 - c. Read back the error status from the FMU and confirm that the injected error is present.
 - d. Issue a reset using the resetn and resetnchk pins.
 - e. Read back the error status from the FMU and confirm that the injected error is present.

3.5 FuSa specific configuration parameter

MMU-720AE adds only one parameter, Lock-Step delay which is applicable to both the TCU and the TBUs.

We recommend that you disable the `TCUCFG_PARITY_ON` and `TBUCFG_PARITY_ON` parameters to save area because full duplication covers these parameters.

The following table describes the MMU-720AE specific configuration parameter and its permitted values.

Table 3-1: MMU-720AE safety specific parameter

Parameter name	Values	Description
<code>FUSA_TEMPORAL_DELAY</code>	0, 2	Use the <code>FUSA_TEMPORAL_DELAY</code> parameter to delay the lockstep and shadow copies of the logic by N cycles, where N is the value of the parameter. This value must be the same across all SMMU components. All components except the switches use the <code>FUSA_TEMPORAL_DELAY</code> parameter.

For more information about MMU-720AE configuration parameters, see *Configuration guidelines* in the Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual.

4. Programmers model for MMU-720AE

The Programmers model provides general information about the MMU-720AE register properties, such as access types, unused, and reserved registers.

The following information applies to the MMU-720AE registers:

- The base address is not fixed, and can be different for any particular system implementation. The offset of each register from the base address is fixed.
- Access type is described as follows:

RW	Read and write
RO	Read-only
WO	Write-only
RAZ	Read-As-Zero
WI	Writes ignored
SBZ	Should-Be-Zero

- Do not attempt to access reserved or unused address locations. Reading these locations results in RAZ and writing to these locations results in WI.
- Unless otherwise stated in the accompanying text:
 - Do not modify **UNDEFINED** register bits
 - Ignore **UNDEFINED** register bits on reads
 - All register bits have a reset value of 0
- Bit positions that are described as reserved are:
 - In an RW register, RAZ/WI
 - In an RO register, RAZ
 - In a WO register, WI

MMU-720AE registers are accessed using the PROG APB5 completer interface on the TCU, and cannot be accessed directly through any other completer interfaces.

Some registers are 64-bit registers, but the PROG APB4 interface is 32 bits wide. Because software accesses 64-bit registers 32 bits at a time, such accesses are not guaranteed to be 64-bit atomic. This behavior does not cause problems for software, because the SMMUv3 architecture does not require 64-bit atomic access to any registers.

The Programmers model contains separate TBU and TCU regions for internal control, RAS, and identification registers. Writes to unmapped or reserved registers are ignored, and reads Should-Be-Zero (SBZ). Non-secure accesses to Secure registers are RAZ/WI. MMU-720AE implements the identification register scheme that the SMMUv3 architecture defines.

MMU-720AE implements all the Performance Monitor Counter Group (PMCG) registers that the SMMUv3 architecture defines, except for:

- SMMU_PMCG_IIDR

- SMMU_PMCG_IRQ_CFG0
- SMMU_PMCG_IRQ_CFG1
- SMMU_PMCG_IRQ_CFG2
- SMMU_PMCG_IRQ_STATUS
- SMMU_PMCG_GMPAM
- SMMU_PMCG_MPAMIDR
- SMMU_PMCG_S_MPAMIDR

MMU-720AE does not implement the following SMMUv3 architectural registers, and accesses to these locations are RAZ/WI:

- SMMU_IDR4
- SMMU_S_IDR4
- SMMU_IDR6
- SMMU_S_IDR6
- SMMU_R_IDR6
- SMMU_STATUSR
- SMMU_CMDQ_CONTROL_*
- SMMU_S_CMDQ_CONTROL_*
- SMMU_R_CMDQ_CONTROL_*
- SMMU_ECMDQ_*
- SMMU_DPT_*
- SMMU_R_DPT_*
- SMMU_GATOS_*
- SMMU_S_GATOS_*
- SMMU_VATOS_*
- SMMU_S_VATOS_*

For more information about the SMMU architectural registers, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

4.1 Clearing ERRSTATUS registers

Software can clear the TCU_ERRSTATUS and TBU_ERRSTATUS registers by writing ones to fields that are set.

For more information about these registers, see the following:

- [4.8.3 TCU_ERRSTATUS register](#) on page 165

- [4.17.3 TBU_ERRSTATUS register](#) on page 236

If both of the following are true, a write to the register is ignored:

- Any of the V, UE, OF, CE, DE, fields are nonzero before the write.
- The write does not clear the nonzero V, UE, OF, CE, DE fields to zero by writing ones to the applicable field or fields.



Note

CE must be cleared by writing 2'b11 to the field.

If a valid clearing write reaches the ERRSTATUS register on the same cycle as a new error, the new record is applied as though no previous error existed.

4.1.1 Realm Management Extension Register Principles

MMU-720AE uses principles of Realm Management Extension (RME). Register ownership dictates which PAS is permitted to access the register. Register accesses with the incorrect PAS result in a RAZ/WI response.

The following table shows the permissible PAS for each register ownership.

Table 4-1: Permissible PAS for each register ownership

Register Ownership	Non-secure access	Secure access	Realm access	Root access
Non-secure	Permit	Permit	Permit	Permit
Secure	RAZ/WI	Permit	RAZ/WI	Permit
Realm	RAZ/WI	RAZ/WI	Permit	Permit
Root	RAZ/WI	RAZ/WI	RAZ/WI	Permit

4.2 SMMU architectural registers

MMU-720AE implements many of the SMMU architectural registers, that the Arm® *System Memory Management Unit Architecture Specification - SMMU architecture version 3* defines.

The following table shows the SMMUv3 architectural registers that MMU-720AE implements.



Note

All writable register fields reset to 0 unless the SMMU architecture specifies otherwise.

Table 4-2: SMMUv3 architectural registers

Register	Name	Description
SMMU_S_IDR0 - SMMU_S_IDR3	SMMU Secure feature Identification Registers	Provides information about the Secure features that the SMMU implementation supports
SMMU_S_CR0	Secure global Control Register 0	Provides global configuration of the Secure SMMU
SMMU_S_CR0ACK	Secure global Control Register 0 update Acknowledge	Provides acknowledgment of completion of updates to SMMU_S_CR0
SMMU_S_CR1 SMMU_S_CR2	Secure global Control Registers	Provides the controls for Secure table and queue access attributes
SMMU_S_INIT	Secure Initialization control register	Provides a control to invalidate all Secure SMMU caching on system initialization
SMMU_S_GBPA	Secure Global Bypass Attribute register	Controls the global bypass attributes that are used for transactions from Secure streams when the MMU is disabled
SMMU_S_IRQ_CTRL	Secure Interrupt Control register	Contains enables for SMMU interrupts
SMMU_S_IRQ_CTRLACK	Secure Interrupt Control register update Acknowledge	Provides acknowledgment of the completion of updates to SMMU_S_IRQ_CTRL
SMMU_S_GERROR	Secure Global Error status register	Provides information on Secure global programming interface errors
SMMU_S_GERRORN	Secure Global Error Acknowledgment register	Contains the acknowledgment fields for SMMU_S_GERROR errors
SMMU_S_GERROR_IRQ_CFG0 - SMMU_S_GERROR_IRQ_CFG2	Secure Global Error IRQ Configuration register	Contains the Secure MSI address configuration for the GERROR IRQ
SMMU_S_STRTAB_BASE	Secure Stream Table Base address register	Contains the base address and attributes for the Secure Stream table
SMMU_S_STRTAB_BASE_CFG	Secure Stream Table Base Configuration register	Contains configuration fields for the Secure Stream table
SMMU_S_CMDQ_BASE	Secure Command queue Base address register	Contains the base address and attributes for the Secure Command queue
SMMU_S_CMDQ_PROD	Secure Command queue Producer index register	Contains the Secure Command queue index for writes by the producer
SMMU_S_CMDQ_CONS	Secure Command queue Consumer index register	Contains the Secure Command queue index for reads by the consumer
SMMU_S_EVENTQ_BASE	Secure Event queue Base address register	Contains the base address and attributes for the Secure Event queue
SMMU_S_EVENTQ_PROD	Secure Event queue Producer index register	Contains the Secure Event queue index for writes by the producer
SMMU_S_EVENTQ_CONS	Secure Event queue Consumer index register	Contains the Secure Event queue index for reads by the consumer
SMMU_S_EVENTQ_IRQ_CFG0 - SMMU_S_EVENTQ_IRQ_CFG2	Secure Event queue IRQ Configuration registers	Contains the MSI address configuration for the Secure Event queue IRQ
SMMU_IDR0 - SMMU_IDR3 SMMU_IDR5	SMMU feature Identification Registers	Provides information about the features that the SMMU implementation supports
SMMU_IIDR	Implementation Identification Register	Provides implementer, part, and revision information for the SMMU implementation

Register	Name	Description
SMMU_AIDR	Architecture Identification Register	Identifies the SMMU architecture version to which the implementation conforms
SMMU_CR0	Non-secure global Control Register 0	Provides the controls for the global configuration of the Non-secure SMMU
SMMU_CR0ACK	Non-secure global Control Register 0 update Acknowledge register	Provides acknowledgment of completion of updates to SMMU_CR0
SMMU_CR1	Non-secure global Control Register 1	Provides the controls for Non-secure table and queue access attributes
SMMU_CR2	Non-secure global Control Register 2	Provides the controls for the configuration of the global Non-secure features
SMMU_GBPA	Non-secure Global Bypass Attribute register	Controls the global bypass attributes that are used for transactions from Non-secure streams when the MMU is disabled
SMMU_AGBPA	Alternate Global Bypass Attribute	Allows an implementation to apply additional non-architected attributes or tags to bypassing transactions.
SMMU_IRQ_CTRL	Non-secure Interrupt Control register	Provides IRQ enable flags for edge-triggered wired outputs, if implemented, and MSI writes, if implemented
SMMU_IRQ_CTRLACK	Non-secure Interrupt Control register update Acknowledge register	Provides acknowledgment of the completion of updates to SMMU_IRQ_CTRL
SMMU_GERROR	Non-secure Global Error status register	Provides information about Non-secure global programming interface errors
SMMU_GERRORN	Non-secure Global Error acknowledgment register	Contains the acknowledgment fields for SMMU_GERROR errors
SMMU_GERROR_IRQ_CFG0	Non-secure Global Error IRQ Configuration register 0	Contains the MSI address configuration for the GERROR IRQ
SMMU_GERROR_IRQ_CFG1	Non-secure Global Error IRQ Configuration register 1	Contains the MSI payload configuration for the GERROR IRQ
SMMU_GERROR_IRQ_CFG2	Non-secure Global Error IRQ Configuration register 2	Contains the MSI attribute configuration for the GERROR IRQ
SMMU_STRTAB_BASE	Non-secure Stream Table Base address register	Contains the base address and attributes for the Non-secure Stream table
SMMU_STRTAB_BASE_CFG	Non-secure Stream Table Configuration register	Contains configuration fields for the Non-secure Stream table
SMMU_CMDQ_BASE	Non-secure Command queue Base address register	Contains the base address and attributes for the Non-secure Command queue
SMMU_CMDQ_PROD	Non-secure Command queue Producer index register	Contains the Non-secure Command queue index for writes by the producer
SMMU_CMDQ_CONS	Non-secure Command queue Consumer index register	Contains the Non-secure Command queue index for reads by the consumer
SMMU_EVENTQ_BASE	Non-secure Event queue Base address register	Contains the base address and attributes for the Non-secure Event queue
SMMU_EVENTQ_PROD	Non-secure Event queue Producer index register	Contains the Non-secure Event queue index for writes by the producer
SMMU_EVENTQ_CONS	Non-secure Event queue Consumer index register	Contains the Non-secure Event queue index for reads by the consumer

Register	Name	Description
SMMU_EVENTQ_IRQ_CFG0	Non-secure Event queue IRQ Configuration register 0	Contains the MSI address configuration for the Event queue IRQ
SMMU_EVENTQ_IRQ_CFG1	Non-secure Event queue IRQ Configuration register 1	Contains the MSI payload configuration for the Event queue IRQ
SMMU_EVENTQ_IRQ_CFG2	Non-secure Event queue IRQ Configuration register 2	Contains the MSI attribute configuration for the Event queue IRQ
SMMU_PRIQ_BASE	Non-secure PRI queue Base address register	Contains the base address and attributes for the Non-secure PRI queue
SMMU_PRIQ_PROD	Non-secure PRI queue Producer index register	Contains the Non-secure PRI queue index for writes by the producer
SMMU_PRIQ_CONS	Non-secure PRI queue Consumer index register	Contains the Non-secure PRI queue index for reads by the consumer
SMMU_PRIQ_IRQ_CFG0	Non-secure PRI queue IRQ Configuration register 0	Contains the MSI address configuration for the PRI queue IRQ
SMMU_PRIQ_IRQ_CFG1	Non-secure PRI queue IRQ Configuration register 1	Contains the MSI payload configuration for the PRI queue IRQ
SMMU_PRIQ_IRQ_CFG2	Non-secure PRI queue IRQ Configuration register 2	Contains the MSI attribute configuration for the PRI queue IRQ

MMU-720AE implements an SMMUv3 Performance Monitor Counter Group (PMCG) in the TCU and in each TBU. The following table lists the registers that MMU-720AE implements in each PMCG.

Table 4-3: SMMUv3 PMCG registers

Register	Name	Description
SMMU_PMCG_EVCNTR0 - SMMU_PMCG_EVCNTR3	SMMU PMCG Event Counter registers	Contains the values of the event counters
SMMU_PMCG_EVTYPER0 - SMMU_PMCG_EVTYPER3	SMMU PMCG Event Type configuration registers	Configures the events that the corresponding counter counts
SMMU_PMCG_SVR0 - SMMU_PMCG_SVR3	SMMU PMCG Shadow Value Registers	Contains the shadow value of the corresponding event counter
SMMU_PMCG_SMRO	SMMU PMCG Stream Match filter Register	Configures the stream match filter for the corresponding event counter
SMMU_PMCG_CNTENSET0	SMMU PMCG Counter Enable Set register	Provides the set mechanism for the counter enables
SMMU_PMCG_CNTENCLR0	SMMU PMCG Counter Enable Clear register	Provides the clear mechanism for the counter enables
SMMU_PMCG_INTENSET0	SMMU PMCG Interrupt contribution Enable Set register	Provides the set mechanism for the counter interrupt contribution enables
SMMU_PMCG_INTENCLR0	SMMU PMCG Interrupt contribution Enable Clear register	Provides the clear mechanism for the counter interrupt enables
SMMU_PMCG_OVSCLR0	SMMU PMCG Overflow Status Clear register	Provides the clear mechanism for the overflow status bits and provides read access to the overflow status bit values
SMMU_PMCG_OVSSET0	SMMU PMCG Overflow Status Set register	Provides the set mechanism for the overflow status bits and provides read access to the overflow status bit values

Register	Name	Description
SMMU_PMCGR_CAPR	SMMU PMCG Counter shadow value Capture Register	Controls the counter shadow value capture mechanism
SMMU_PMCGR_SCR	SMMU PMCG Secure Control Register	Secure Control Register
SMMU_PMCGR_SCRA	SMMU PMCG Secure Control Register	Secure Control Register, alias of SMMU_PMCGR_SCR register
SMMU_PMCGR_CFGR	SMMU PMCG Configuration information Register	Provides information about the PMCG implementation
SMMU_PMCGR_CR	SMMU PMCG Control Register	Contains the Performance Monitor control flags
SMMU_PMCGR_ROOTCR	SMMU PMCG Root Control Register	Root Control Register
SMMU_PMCGR_CEID0 - SMMU_PMCGR_CEID1	SMMU PMCG Common Event ID registers	Contains the lower and upper 64 bits of the Common Event identification bitmap
SMMU_PMCGR_IRQ_CTRL	SMMU PMCG IRQ enable register	Contains the Performance Monitors IRQ enable
SMMU_PMCGR_IRQ_CTRLACK	SMMU PMCG IRQ enable Acknowledge register	Provides acknowledgment of the completion of updates to SMMU_PMCGR_IRQ_CTRL
SMMU_PMCGR_AIDR	SMMU PMCG Architecture Identification Register	Provides the Performance Monitor Architecture Identification
SMMU_PMCGR_ID_REGS	ID registers	IMPLEMENTATION DEFINED
SMMU_PMCGR_PMAUTHSTATUS	PMU Authentication Status register	Performance Monitor authentication status
SMMU_PMCGR_PMDEVARCH	PMU Device Architecture register	Performance Monitor architecture identifier
SMMU_PMCGR_PMDEVTYPE	PMU Device Type register	Performance Monitor device type

The following are the SMMU for RME registers:

- SMMU_ROOT_IDR0
- SMMU_ROOT_IIDR
- SMMU_ROOT_CRO
- SMMU_ROOT_CROACK
- SMMU_ROOT_GPT_BASE
- SMMU_ROOT_GPT_BASE2
- SMMU_ROOT_GPT_BASE_UPDATE
- SMMU_ROOT_GPT_BASE_CFG
- SMMU_ROOT_GPF_FAR
- SMMU_ROOT_GPT_CFG_FAR
- SMMU_ROOT_TLBI
- SMMU_ROOT_TLBI_CTRL

The following are the SMMU for RME-DA registers:

- SMMU_R_IDR0
- SMMU_R_IDR1
- SMMU_R_IDR2
- SMMU_R_IDR3
- SMMU_R_IDR4
- SMMU_R_AIDR
- SMMU_R_CR0
- SMMU_R_CR0ACK
- SMMU_R_CR1
- SMMU_R_CR2
- SMMU_R_GBPA
- SMMU_R_AGBPA
- SMMU_R_IRQ_CTRL
- SMMU_R_IRQ_CTRLACK
- SMMU_R_GERROR
- SMMU_R_GERRORN
- SMMU_R_GERROR_IRQ_CFG0_LO
- SMMU_R_GERROR_IRQ_CFG0_HI
- SMMU_R_GERROR_IRQ_CFG1
- SMMU_R_GERROR_IRQ_CFG2
- SMMU_R_STRTAB_BASE_LO
- SMMU_R_STRTAB_BASE_HI
- SMMU_R_STRTAB_BASE_CFG
- SMMU_R_CMDQ_BASE_LO
- SMMU_R_CMDQ_BASE_HI
- SMMU_R_CMDQ_PROD
- SMMU_R_CMDQ_CONS
- SMMU_R_EVENTQ_BASE_LO
- SMMU_R_EVENTQ_BASE_HI
- SMMU_R_EVENTQ_IRQ_CFG0_LO
- SMMU_R_EVENTQ_IRQ_CFG0_HI
- SMMU_R_EVENTQ_IRQ_CFG1
- SMMU_R_EVENTQ_IRQ_CFG2
- SMMU_R_PRIQ_BASE_LO

- SMMU_R_PRIQ_BASE_HI
- SMMU_R_PRIQ_IRQ_CFG0_LO
- SMMU_R_PRIQ_IRQ_CFG0_HI
- SMMU_R_PRIQ_IRQ_CFG1
- SMMU_R_PRIQ_IRQ_CFG2
- SMMU_R_MPAMIDR
- SMMU_R_GMPAM
- SMMU_R_MECIDR
- SMMU_R_GMECID
- SMMU_R_EVENTQ_PROD
- SMMU_R_EVENTQ_CONS
- SMMU_R_PRIQ_PROD
- SMMU_R_PRIQ_CONS

The following are the SMMU for PMU registers:

- SMMU_PMCG_SCRA
- SMMU_PMCG_ROOTCR

4.3 SMMU memory map

The MMU-720AE memory map contains all the registers and includes the Translation Control Unit (TCU) and all Translation Buffer Units (TBUs), and shows the maximum number of implemented TBUs.

This document describes all TBU and TCU register addresses relative to the base address for that component.



Note

The TBU number is assigned according to the TID value that the TBU provides in its initial Distributed Translation Interface (DTI) connection request. The registers for that TBU are accessed according to the address range for its TID, as the following table shows.

The following table shows the full memory map.

Table 4-4: Main MMU-720AE memory map

Address range	Description
0x00000000-0x00AFFFC	TCU registers

Address range	Description
0x0100000-0x017FFFC	TBU0 registers. Includes microarchitectural, RAS, ID, MPAM, and PMCG registers.
0x0180000-0x01FFFFC	TBU1 registers. Includes microarchitectural, RAS, ID, MPAM, and PMCG registers.
0x0200000-0x027FFFC	TBU2 registers. Includes microarchitectural, RAS, ID, MPAM, and PMCG registers.
...	...
0x1F00000-0x1F7FFFC	TBU60 registers. Includes microarchitectural, RAS, ID, MPAM, and PMCG registers.
0x1F80000-0x1FFFFFC	TBU61 registers. Includes microarchitectural, RAS, ID, MPAM, and PMCG registers.



Note

Where a multi-port LTI TBU is implemented, it occupies the same address map space as if it were a single-port LTI TBU. The addressing is per TBU, not per initiating interface.

4.3.1 TCU memory map

The TCU memory map contains various categories of registers.

The TCU **IMPLEMENTATION DEFINED** registers include the following:

- [4.6 TCU PMU registers](#) on page 140
- [4.7 TCU microarchitectural registers](#) on page 143 for controlling microarchitectural features
- [4.8 TCU RAS registers](#) on page 163
- [4.9 TCU system discovery registers](#) on page 175

The following registers are also included:

- [4.12 TCU PIU integration registers](#) on page 213.
- Walk cache stage and level Memory System Resource Partitioning and Monitoring (MPAM) maximum capacity registers
- Granule Protection Table (GPT) cache level MPAM maximum capacity registers
- MPAM memory-mapped registers

The following registers are on separate 64K pages to enable mapping to different pieces of software:

- Configuration

- PMU
- RAS
- MPAM

The following table shows the MMU-720AE TCU memory map.

Table 4-5: MMU-720AE TCU memory map

Address range	Description
0x0000000- 0x000FFFC	TCU SMMUv3 registers page 0, ID registers.
0x0010000- 0x001FFFC	TCU SMMUv3 registers page 1.
0x0020000- 0x002FFFC	TCU IMPLEMENTATION DEFINED registers, integration registers, and TCU system discovery registers.
0x0030000- 0x003FFFC	TCU PMCG registers, page 0.
0x0040000- 0x004FFFC	TCU PMCG registers, page 1.
0x0050000- 0x005FFFC	TCU MPAM registers.
0x0060000- 0x006FFFC	TCU RAS registers.
0x0080000- 0x008FFFC	TCU Realm Register Page 0.
0x0090000- 0x009FFFC	TCU Realm Register Page 1.
0x00A0000- 0x00AFFFC	TCU Root Control Page.

The following table shows how the TCU **IMPLEMENTATION DEFINED** PMCG, and MPAM registers are allocated to regions of the TCU address space. Other regions are reserved.

Table 4-6: TCU PMCG, RAS, and MPAM register allocation to regions of TCU address space

Address range	Description
0x00FD0- 0x00FFC	SMMU ID registers

Address range	Description
0x0020000- 0x002FFFC	4.4.4 TCU microarchitectural registers summary on page 132, 0x0028E00-0x0029910 4.4.5 TCU system discovery registers summary on page 133, 0x002A000-0x002A090 - TCU_NODE_CTRL microarchitecture register. This register provides per connected TBU configuration options. See 4.7.10 TCU_NODE_CTRLn register on page 156, 0x0029000-0x00297FC - Walk Cache, 0x0029800-0x002981C for the following: - AGW GPT cache - DGW GPT cache CMAX settings
0x0030000- 0x003FFFC	Performance Monitor, page 0, as the previous table describes.
0x0040000- 0x004FFFC	Performance Monitor, page 1, as the previous table describes.
0x0050000- 0x005FFFC	TCU MPAM registers: - Non-secure MPAM, 0x0050000-0x0050FFC - Secure MPAM, 0x0051000-0x0051FFC - Realm MPAM, 0x0052000-0x0052FFC - Root MPAM, 0x0053000-0x0053FFC
0x0060000- 0x006FFFC	4.4.3 TCU Reliability, Availability, and Serviceability registers summary on page 132

4.3.2 TBU memory map

The TBU memory map contains various categories of registers.

The TBU registers contain the following:

- IMPLEMENTATION DEFINED** 4.16 TBU microarchitectural registers on page 224 for controlling microarchitectural features
- 4.18 TBU system discovery registers on page 242
- 4.17 TBU RAS registers on page 233
- 4.15 TBU PMU registers on page 222
- Performance Monitor counter registers, on a separate 64KB page to enable it to be paged for direct access from a Guest OS

The following table shows the TBU memory map.

Table 4-7: TBU memory map

Address range	Description	TBU Page
0x00FD0-0x00FFC	ID registers	0
0x08E00-0x08F10	Microarchitectural features	0
0x09000-0x09058	TBU system discovery registers	0

Address range	Description	TBU Page
0x10000-0x10040	RAS	1
0x20000-0x20FFC	Performance Monitor page 0	2
0x30000-0x30FFC	Performance Monitor page 1	3
0x40000-0x40FFC	TBU MPAM Non-secure registers	4
0x41000-0x41FFC	TBU MPAM Secure registers	4
0x42000-0x42FFC	TBU MPAM Realm registers	4
0x43000-0x43FFC	TBU MPAM Root registers	4



Any regions that the table does not show are reserved.

4.4 SMMU registers summary

The register summary describes the MMU-720AE registers and some key characteristics.

4.4.1 TCU identification registers summary

MMU-720AE contains TCU identification registers.

The following table shows the TCU identification registers in offset order from the base memory address.

Table 4-8: TCU identification registers summary

Offset	Name	Type	Description
0x00FFC	SMMU_CIDR3	RO	4.5 TCU component and peripheral ID registers on page 139
0x00FF8	SMMU_CIDR2	RO	
0x00FF4	SMMU_CIDR1	RO	
0x00FF0	SMMU_CIDR0	RO	
0x00FEC	SMMU_PIDR3	RO	
0x00FE8	SMMU_PIDR2	RO	
0x00FE4	SMMU_PIDR1	RO	
0x00FE0	SMMU_PIDR0	RO	
0x00FDC	SMMU_PIDR7	RO	
0x00FD8	SMMU_PIDR6	RO	
0x00FD4	SMMU_PIDR5	RO	
0x00FD0	SMMU_PIDR4	RO	

4.4.2 TCU and TBU PMU identification registers summary

The TCU and the TBU use the same PMU identification registers.

The following table shows the TCU and TBU PMU identification registers in offset order from the base memory address.

Table 4-9: TCU and TBU PMU identification registers summary

Offset	Name	Type	Description
0x00FB8	SMMU_PMC_G_PMAUTHSTATUS	RO	4.6 TCU PMU registers on page 140 4.15 TBU PMU registers on page 222
0x00FD0	SMMU_PMC_G_PIDR4	RO	
0x00FD4	SMMU_PMC_G_PIDR5	RO	
0x00FD8	SMMU_PMC_G_PIDR6	RO	
0x00FDC	SMMU_PMC_G_PIDR7	RO	
0x00FE0	SMMU_PMC_G_PIDR0	RO	
0x00FE4	SMMU_PMC_G_PIDR1	RO	
0x00FE8	SMMU_PMC_G_PIDR2	RO	
0x00FEC	SMMU_PMC_G_PIDR3	RO	
0x00FF0	SMMU_PMC_G_CIDR0	RO	
0x00FF4	SMMU_PMC_G_CIDR1	RO	
0x00FF8	SMMU_PMC_G_CIDR2	RO	
0x00FFC	SMMU_PMC_G_CIDR3	RO	

4.4.3 TCU Reliability, Availability, and Serviceability registers summary

MMU-720AE contains TCU Reliability, Availability, and Serviceability (RAS) registers.

The following table shows the TCU RAS registers in offset order from the base memory address. RAS registers are 64 bits wide. The top 32 bits are Reserved and are omitted in this document for clarity.

Table 4-10: TCU RAS registers summary

Offset	Name	Type	Width	Description
0x60000	TCU_ERRFR	RO, Root	32-bit	4.8.1 TCU_ERRFR register on page 163
0x60008	TCU_ERRCTLR	RW, Root	32-bit	4.8.2 TCU_ERRCTLR register on page 164
0x60010	TCU_ERRSTATUS	RW, Root	32-bit	4.8.3 TCU_ERRSTATUS register on page 165
0x60040	TCU_ERRGEN	RW, Root	32-bit	4.8.4 TCU_ERRGEN register on page 170

4.4.4 TCU microarchitectural registers summary

MMU-720AE contains TCU microarchitectural registers.

The following table shows the TCU microarchitectural registers in offset order from the base memory address.

Table 4-11: TCU microarchitectural registers summary

Offset	Name	Type	Width	Description
0x28E00	TCU_CTRL	RW	32-bit	4.7.1 TCU_CTRL register on page 146
0x28E04	TCU_QOS	RW	32-bit	4.7.2 TCU_QOS register on page 147
0x28E08	TCU_CFG	RO	32-bit	4.7.3 TCU_CFG register on page 149
0x28E10	TCU_STATUS	RO	32-bit	4.7.4 TCU_STATUS register on page 149
0x28E18	TCU_SCR	RW, Secure	32-bit	4.7.5 TCU_SCR register on page 150
0x28E20	TCU_ITEN	RW	32-bit	4.11.1 ITEN register for the TCU on page 212
0x28E24	ITOP_PIU	RW	32-bit	4.12.1 ITOP_PIU register for the TCU Programmer Interface Unit on page 213
0x28E28	ITIN_PIU	RW	32-bit	4.12.2 ITIN_PIU register for the TCU Programmer Interface Unit on page 218
0x28E2C	ITOP_TMU	RW	32-bit	4.13.1 ITOP_TMU register for the TCU Translation Management Unit on page 219
0x28E30	ITIN_TMU	RW	32-bit	4.13.2 ITIN_TMU register for the TCU Translation Management Unit on page 220
0x28E40	TCU_CTRL2	RW	32-bit	4.7.6 TCU_CTRL2 register on page 152
0x28F00	TCU_ROOT_CTRL	RW	32-bit	4.7.7 TCU_ROOT_CTRL register on page 153
0x28F04	TCU_ROOT_CTRL2	RW	32-bit	4.7.8 TCU_ROOT_CTRL2 register on page 154
0x28F10	TCU_RCR	RW	32-bit	4.7.9 TCU_RCR register on page 155
0x29000- 0x293FC	TCU_NODE_CTRLn	RW	32-bit	4.7.10 TCU_NODE_CTRLn register on page 156
0x29400- 0x297FC	TCU_NODE_STATUSn	RO	32-bit	4.7.11 TCU_NODE_STATUSn register on page 159
0x29800- 0x2981C	TCU_WC_SxLy_CMAX	RW	32-bit	4.7.12 TCU_WC_SxLy_CMAX registers on page 161
0x29900	TCU_AGW_GC_Lx_CMAX	RW	32-bit	4.7.13 TCU_AGW_GC_Lx_CMAX registers on page 162
0x29910	TCU_DGW_GC_Lx_CMAX	RW	32-bit	4.7.14 TCU_DGW_GC_Lx_CMAX registers on page 162
0x29918	TCU_DC_LO_CMAX	RW	32-bit	
0x2991C	TCU_DC_L1_CMAX	RW	32-bit	

4.4.5 TCU system discovery registers summary

MMU-720AE contains TCU system discovery registers.

The following table shows the TCU system discovery registers in offset order from the base memory address.

Table 4-12: TCU system discovery registers summary

Offset	Name	Type	Width	Description
0x2A000	TCU_SYSDISC0	RO	32-bit	4.9.1 TCU_SYSDISC0 system discovery register on page 175
0x2A004	TCU_SYSDISC1	RO	32-bit	4.9.2 TCU_SYSDISC1 system discovery register on page 176
0x2A008	TCU_SYSDISC2	RO	32-bit	4.9.3 TCU_SYSDISC2 system discovery register on page 177
0x2A00C	TCU_SYSDISC3	RO	32-bit	4.9.4 TCU_SYSDISC3 system discovery register on page 178
0x2A010	TCU_SYSDISC4	RO	32-bit	4.9.5 TCU_SYSDISC4 system discovery register on page 179
0x2A014	TCU_SYSDISC5	RO	32-bit	4.9.6 TCU_SYSDISC5 system discovery register on page 180
0x2A018	TCU_SYSDISC6	RO	32-bit	4.9.7 TCU_SYSDISC6 system discovery register on page 181
0x2A01C	TCU_SYSDISC7	RO	32-bit	4.9.8 TCU_SYSDISC7 system discovery register on page 182
0x2A020	TCU_SYSDISC8	RO	32-bit	4.9.9 TCU_SYSDISC8 system discovery register on page 183
0x2A024	TCU_SYSDISC9	RO	32-bit	4.9.10 TCU_SYSDISC9 system discovery register on page 184
0x2A028	TCU_SYSDISC10	RO	32-bit	4.9.11 TCU_SYSDISC10 system discovery register on page 185
0x2A02C	TCU_SYSDISC11	RO	32-bit	4.9.12 TCU_SYSDISC11 system discovery register on page 186
0x2A030	TCU_SYSDISC12	RO	32-bit	4.9.13 TCU_SYSDISC12 system discovery register on page 187
0x2A034	TCU_SYSDISC13	RO	32-bit	4.9.14 TCU_SYSDISC13 system discovery register on page 188
0x2A038	TCU_SYSDISC14	RO	32-bit	4.9.15 TCU_SYSDISC14 system discovery register on page 189
0x2A03C	TCU_SYSDISC15	RO	32-bit	4.9.16 TCU_SYSDISC15 system discovery register on page 190
0x2A040	TCU_SYSDISC16	RO	32-bit	4.9.17 TCU_SYSDISC16 system discovery register on page 191
0x2A044	TCU_SYSDISC17	RO	32-bit	4.9.18 TCU_SYSDISC17 system discovery register on page 192
0x2A048	TCU_SYSDISC18	RO	32-bit	4.9.19 TCU_SYSDISC18 system discovery register on page 193
0x2A04C	TCU_SYSDISC19	RO	32-bit	4.9.20 TCU_SYSDISC19 system discovery register on page 194
0x2A050	TCU_SYSDISC20	RO	32-bit	4.9.21 TCU_SYSDISC20 system discovery register on page 195
0x2A054	TCU_SYSDISC21	RO	32-bit	4.9.22 TCU_SYSDISC21 system discovery register on page 196
0x2A058	TCU_SYSDISC22	RO	32-bit	4.9.23 TCU_SYSDISC22 system discovery register on page 197
0x2A05C	TCU_SYSDISC23	RO	32-bit	4.9.24 TCU_SYSDISC23 system discovery register on page 198
0x2A060	TCU_SYSDISC24	RO	32-bit	4.9.25 TCU_SYSDISC24 system discovery register on page 199
0x2A064	TCU_SYSDISC25	RO	32-bit	4.9.26 TCU_SYSDISC25 system discovery register on page 199
0x2A068	TCU_SYSDISC26	RO	32-bit	4.9.27 TCU_SYSDISC26 system discovery register on page 200
0x2A06C	TCU_SYSDISC27	RO	32-bit	4.9.28 TCU_SYSDISC27 system discovery register on page 201
0x2A070	TCU_SYSDISC28	RO	32-bit	4.9.29 TCU_SYSDISC28 system discovery register on page 202
0x2A074	TCU_SYSDISC29	RO	32-bit	4.9.30 TCU_SYSDISC29 system discovery register on page 203
0x2A078	TCU_SYSDISC30	RO	32-bit	4.9.31 TCU_SYSDISC30 system discovery register on page 203
0x2A07C	TCU_SYSDISC31	RO	32-bit	4.9.32 TCU_SYSDISC31 System Discovery Register on page 204
0x2A080	TCU_SYSDISC32	RO	32-bit	4.9.33 TCU_SYSDISC32 System Discovery Register on page 205

Offset	Name	Type	Width	Description
0x2A084	TCU_SYSDISC33	RO	32-bit	4.9.34 TCU_SYSDISC33 system discovery register on page 206
0x2A088	TCU_SYSDISC34	RO	32-bit	4.9.35 TCU_SYSDISC34 System Discovery Register on page 207
0x2A08C	TCU_SYSDISC35	RO	32-bit	4.9.36 TCU_SYSDISC35 system discovery register on page 208
0x2A090	TCU_SYSDISC36	RO	32-bit	4.9.37 TCU_SYSDISC36 system discovery register on page 209
0x2A094	TCU_SYSDISC37	RO	32-bit	4.9.38 TCU_SYSDISC37 system discovery register on page 210

4.4.6 TCU AUX registers summary

MMU-720AE contains TCU auxiliary registers.

The following table shows the TCU auxiliary registers in offset order from the base memory address.

Table 4-13: TCU AUX registers summary

Offset	Name	Type	Width	Description
0x2A100	TCU_CTRL_AUX0	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000000
0x2A104	TCU_CTRL_AUX1	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000001
0x2A108	TCU_CTRL_AUX2	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000002
0x2A10C	TCU_CTRL_AUX3	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000005
0x2A110	TCU_CTRL_AUX4	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000008
0x2A114	TCU_CTRL_AUX5	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000009
0x2A118	TCU_CTRL_AUX6	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000011
0x2A11C	TCU_CTRL_AUX7	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000015
0x2A120	TCU_CTRL_AUX8	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000025
0x2A124	TCU_CTRL_AUX9	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000030
0x2A128	TCU_CTRL_AUX10	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000039
0x2A12C	TCU_CTRL_AUX11	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000031
0x2A130	TCU_CTRL_AUX12	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000038
0x2A134	TCU_CTRL_AUX13	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000020
0x2A138	TCU_CTRL_AUX14	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000021
0x2A13C	TCU_CTRL_AUX15	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000004
0x2A140	TCU_CTRL_AUX16	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000014
0x2A144	TCU_CTRL_AUX17	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000028
0x2A148	TCU_CTRL_AUX18	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000007
0x2A200	TCU_CTRL_AUX19	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000000
0x2A204	TCU_CTRL_AUX20	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000001
0x2A208	TCU_CTRL_AUX21	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000002
0x2A20C	TCU_CTRL_AUX22	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000005
0x2A210	TCU_CTRL_AUX23	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000008
0x2A214	TCU_CTRL_AUX24	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000009
0x2A218	TCU_CTRL_AUX25	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000011

Offset	Name	Type	Width	Description
0x2A21C	TCU_CTRL_AUX26	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000015
0x2A220	TCU_CTRL_AUX27	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000025
0x2A224	TCU_CTRL_AUX28	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000030
0x2A228	TCU_CTRL_AUX29	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000039
0x2A22C	TCU_CTRL_AUX30	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000031
0x2A230	TCU_CTRL_AUX31	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000038
0x2A234	TCU_CTRL_AUX32	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000020
0x2A238	TCU_CTRL_AUX33	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000021
0x2A23C	TCU_CTRL_AUX34	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000004
0x2A240	TCU_CTRL_AUX35	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000014
0x2A244	TCU_CTRL_AUX36	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000028
0x2A300	TCU_CTRL_AUX37	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000000
0x2A304	TCU_CTRL_AUX38	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000001
0x2A308	TCU_CTRL_AUX39	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000002
0x2A30C	TCU_CTRL_AUX40	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000005
0x2A310	TCU_CTRL_AUX41	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000008
0x2A314	TCU_CTRL_AUX42	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000009
0x2A318	TCU_CTRL_AUX43	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000011
0x2A31C	TCU_CTRL_AUX44	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000015
0x2A320	TCU_CTRL_AUX45	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000025
0x2A324	TCU_CTRL_AUX46	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000030
0x2A328	TCU_CTRL_AUX47	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000039
0x2A32C	TCU_CTRL_AUX48	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000031
0x2A330	TCU_CTRL_AUX49	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000038
0x2A334	TCU_CTRL_AUX50	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000020
0x2A338	TCU_CTRL_AUX51	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000021
0x2A33C	TCU_CTRL_AUX52	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000004
0x2A340	TCU_CTRL_AUX53	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000014
0x2A344	TCU_CTRL_AUX54	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000028
0x2A348	TCU_CTRL_AUX55	RW	32-bit	4.10 TCU_CTRL_AUX<n> registers on page 211. Reset is 0x00000007

4.4.7 TBU identification registers summary

MMU-720AE contains TBU identification registers.

The following table shows the TBU identification registers in offset order from the base memory address.

Table 4-14: TBU identification registers summary

Offset	Name	Type	Description
0x00FFC	SMMU_CIDR3	RO	4.14 TBU component and peripheral ID registers on page 221

Offset	Name	Type	Description
0x00FF8	SMMU_CIDR2	RO	
0x00FF4	SMMU_CIDR1	RO	
0x00FF0	SMMU_CIDR0	RO	
0x00FEC	SMMU_PIDR3	RO	
0x00FE8	SMMU_PIDR2	RO	
0x00FE4	SMMU_PIDR1	RO	
0x00FE0	SMMU_PIDR0	RO	
0x00FDC	SMMU_PIDR7	RO	
0x00FD8	SMMU_PIDR6	RO	
0x00FD4	SMMU_PIDR5	RO	
0x00FD0	SMMU_PIDR4	RO	

4.4.8 TBU Reliability, Availability, and Serviceability registers summary

MMU-720AE contains TBU Reliability, Availability, and Serviceability (RAS) registers.

The following table shows the TBU RAS registers in offset order from the base memory address. RAS registers are 64 bits wide. The top 32 bits are Reserved and are omitted in this document for clarity.

Table 4-15: TBU RAS registers summary

Offset	Name	Width	Type	Description
0x10000	TBU_ERRFR	32-bit	RO, Root	4.17.1 TBU_ERRFR register on page 234
0x10008	TBU_ERRCTLR	32-bit	RW, Root	4.17.2 TBU_ERRCTLR register on page 235
0x10010	TBU_ERRSTATUS	32-bit	RW, Root	4.17.3 TBU_ERRSTATUS register on page 236
0x10040	TBU_ERRGEN	32-bit	RW, Root	4.17.4 TBU_ERRGEN register on page 239

RAS error reporting

When a Correctable Error (CE) occurs:

A CE is reported in [4.17.3 TBU_ERRSTATUS register](#) on page 236.

If TBU_ERRCTLR.FI is set, an interrupt is raised on ras_fhi. See [2.4.2.7 TBU interrupt interface](#) on page 41.

4.4.9 TBU microarchitectural registers summary

MMU-720AE contains TBU microarchitectural registers.

The following table shows the TBU microarchitectural registers in offset order from the base memory address.

Table 4-16: TBU microarchitectural registers summary

Offset	Name	Type	Width	Description
0x08E00	TBU_CTRL	RW	32-bit	4.16.1 TBU_CTRL register on page 224
0x08E04	TBU_LTI_PORT_RESOURCE_LIMIT	RW	32-bit	4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register on page 226
0x08E18	TBU_SCR	RW, Secure	32-bit	4.16.3 TBU_SCR register on page 229
0x08F00	TBU_ROOT_CTRL	RW	32-bit	4.16.4 TBU_ROOT_CTRL register on page 231
0x08F10	TBU_RCR	RW	32-bit	4.16.5 TBU_RCR register on page 231

4.4.10 TBU system discovery registers summary

MMU-720AE contains TBU system discovery registers.

The following table shows the TBU system discovery registers in offset order from the base memory address.

Table 4-17: TBU system discovery registers summary

Offset	Name	Type	Width	Description
0x09000	TBU_SYSDISC0	RO	32-bit	4.18.1 TBU_SYSDISC0 system discovery register on page 243
0x09004	TBU_SYSDISC1	RO	32-bit	4.18.2 TBU_SYSDISC1 system discovery register on page 244
0x09008	TBU_SYSDISC2	RO	32-bit	4.18.3 TBU_SYSDISC2 system discovery register on page 245
0x0900C	TBU_SYSDISC3	RO	32-bit	4.18.4 TBU_SYSDISC3 system discovery register on page 246
0x09010	TBU_SYSDISC4	RO	32-bit	4.18.5 TBU_SYSDISC4 system discovery register on page 247
0x09014	TBU_SYSDISC5	RO	32-bit	4.18.6 TBU_SYSDISC5 system discovery register on page 248
0x09018	TBU_SYSDISC6	RO	32-bit	4.18.7 TBU_SYSDISC6 system discovery register on page 249
0x0901C	TBU_SYSDISC7	RO	32-bit	4.18.8 TBU_SYSDISC7 system discovery register on page 250
0x09020	TBU_SYSDISC8	RO	32-bit	4.18.9 TBU_SYSDISC8 system discovery register on page 251
0x09024	TBU_SYSDISC9	RO	32-bit	4.18.10 TBU_SYSDISC9 system discovery register on page 252
0x09028	TBU_SYSDISC10	RO	32-bit	4.18.11 TBU_SYSDISC10 system discovery register on page 253
0x0902C	TBU_SYSDISC11	RO	32-bit	4.18.12 TBU_SYSDISC11 system discovery register on page 254
0x09030	TBU_SYSDISC12	RO	32-bit	4.18.13 TBU_SYSDISC12 system discovery register on page 255
0x09034	TBU_SYSDISC13	RO	32-bit	4.18.14 TBU_SYSDISC13 system discovery register on page 256
0x09038	TBU_SYSDISC14	RO	32-bit	4.18.15 TBU_SYSDISC14 system discovery register on page 257
0x0903C	TBU_SYSDISC15	RO	32-bit	4.18.16 TBU_SYSDISC15 system discovery register on page 258
0x09040	TBU_SYSDISC16	RO	32-bit	4.18.17 TBU_SYSDISC16 system discovery register on page 259
0x09044	TBU_SYSDISC17	RO	32-bit	4.18.18 TBU_SYSDISC17 system discovery register on page 260
0x09048	TBU_SYSDISC18	RO	32-bit	4.18.19 TBU_SYSDISC18 system discovery register on page 261
0x0904C	TBU_SYSDISC19	RO	32-bit	4.18.20 TBU_SYSDISC19 system discovery register on page 262
0x09050	TBU_SYSDISC20	RO	32-bit	4.18.21 TBU_SYSDISC20 system discovery register on page 263
0x09054	TBU_SYSDISC21	RO	32-bit	4.18.22 TBU_SYSDISC21 system discovery register on page 263
0x09058	TBU_SYSDISC22	RO	32-bit	4.18.23 TBU_SYSDISC22 system discovery register on page 264

4.4.11 TBU integration registers summary

MMU-720AE contains TBU integration registers.

The following table shows the TBU integration registers in offset order from the base memory address.

Table 4-18: TBU integration registers summary

Offset	Name	Type	Width	Description
0x08E20	TBU_ITEN	RW	32-bit	4.19.1 ITEN register for the TBU on page 266
0x08E24	TBU_ITOP	RW	32-bit	4.19.2 ITOP register for the TBU on page 267
0x08E28	TBU_ITIN	RW	32-bit	4.19.3 ITIN register for the TBU on page 270

4.5 TCU component and peripheral ID registers

The MMU-720AE TCU contains component and peripheral ID registers.

The following table shows the TCU component and peripheral ID registers.

Table 4-19: TCU component and peripheral ID registers

Offset	Name	Field	Value	Description
0x00FFC	SMMU_CIDR3, Component ID3	[7:0]	0xB1	Preamble
0x00FF8	SMMU_CIDR2, Component ID2	[7:0]	0x05	Preamble
0x00FF4	SMMU_CIDR1, Component ID1	[7:0]	0xF0	Preamble
0x00FF0	SMMU_CIDR0, Component ID0	[7:0]	0x0D	Preamble
0x00FEC	SMMU_PIDR3, Peripheral ID3	[7:4]	MAX(p_level, ecorevnum)	REVRAND, minor revision, where p_level is 1 for p1.
	SMMU_PIDR3, Peripheral ID3	[3:0]	0x00	CMOD
0x00FE8	SMMU_PIDR2, Peripheral ID2	[7:4]	0x00	REVISION, major revision
	SMMU_PIDR2, Peripheral ID2	[3]	1	JEDEC-assigned value for DES always used
	SMMU_PIDR2, Peripheral ID2	[2:0]	3	DES_1: bits [6:4] bits of the JEP106 Designer code
0x00FE4	SMMU_PIDR1, Peripheral ID1	[7:4]	0xB	DES_0: bits [3:0] of the JEP106 Designer code
	SMMU_PIDR1, Peripheral ID1	[3:0]	0x4	PART_1: bits [11:8] of the Part number

Offset	Name	Field	Value	Description
0x00FE0	SMMU_PIDR0, Peripheral ID0	[7:0]	0x9E	PART_0: bits [7:0] of the Part number
0x00FDC	SMMU_PIDR7, Peripheral ID7	-	RES0	Reserved
0x00FD8	SMMU_PIDR6, Peripheral ID6			
0x00FD4	SMMU_PIDR5, Peripheral ID5			
0x00FD0	SMMU_PIDR4, Peripheral ID4	[7:4]	0x0	SIZE = 4KB
		[3:0]	0x4	DES_2: JEP106 Designer continuation code

4.6 TCU PMU registers

The MMU-720AE TCU contains Performance Monitor Unit (PMU) registers. The Performance Monitor counter register, on a separate 64KB page from the rest of the PMU registers, enables it to be paged for direct access from a Guest OS.

4.6.1 Registers

The TBU and TCU support the same PMCG registers.

These registers follow the register layout that the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) Performance Monitor Extension describes.

The following PMCG registers, that the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) defines, are implemented:

- SMMU_PMCG_EVCNTR{0-(TCUCFG_PMU_COUNTERS-1)}
- SMMU_PMCG_EVTYPER{0-(TCUCFG_PMU_COUNTERS-1)}
- SMMU_PMCG_SVR{0-(TCUCFG_PMU_COUNTERS-1)}
- SMMU_PMCG_SMRO
 - All counters share this mask register
 - The mask is 32 bits because the TCU uses 32-bit StreamIDs
- SMMU_PMCG_CNTENSET0
- SMMU_PMCG_CNTENCLR0
- SMMU_PMCG_INTENSET0
- SMMU_PMCG_INTSENCLR0
- SMMU_PMCG_OVSCLR0
- SMMU_PMCG_OVSSET0

- SMMU_PMCG_CAPR
- SMMU_PMCG_SCR
- SMMU_PMCG_CFGR. See [4.6.3 SMMU_PMCG_CFGR](#) on page 142.
- SMMU_PMCG_CR
- SMMU_PMCG_CEID{0-1}. See [4.6.4 SMMU_PMCG_CEID{0-1} registers](#) on page 142.
- SMMU_PMCG_IRQ_CTRL
- SMMU_PMCG_IRQ_CTRLACK
- SMMU_PMCG_AIDR, indicates SMMUV3.3
- SMMU_PMCG_ID_REGS
- SMMU_PMCG_SCRA
- SMMU_PMCG_ROOTCR

The following registers are not implemented, because the PMCG does not support MSIs:

- SMMU_PMCG_IRQ_CFG0
- SMMU_PMCG_IRQ_CFG1
- SMMU_PMCG_IRQ_CFG2
- SMMU_PMCG_IRQ_STATUS

The following registers are not implemented, because the PMCG implementation does not support MPAM:

- SMMU_PMCG_GMPAM
- SMMU_PMCG_MPAMIDR
- SMMU_PMCG_S_MPAMIDR

4.6.2 Events

In this description, a translation request corresponds to a translation slot allocation.

A single DTI translation request might correspond to multiple translation request events if:

- A translation results in a stall fault event and is restarted
- A translation results in a stall fault event when the Event queue is full, and is later retried when the Event queue becomes non-full

Each event indicates:

- Whether the SMMU_PMCG_SMR0 register can filter it
- For events that cannot be filtered, whether they are only visible when Secure events are visible by:
 - SMMU_PMCG_ROOTCR.NAO&(SMMU_PMCG_SCR.SO|SMMU_PMCG_SCR.NAO) in RME mode

- SMMU_PMCGR_SCR.SO in legacy mode

For more information about the architectural and **IMPLEMENTATION DEFINED** events that are implemented, see [2.5.2.1 SMMUv3 architectural performance events](#) on page 47.

The following events are also counted for prefetch accesses:

0x80-0x90

Walk cache events.

0x92-0x94

Configuration cache events.

4.6.3 SMMU_PMCGR_CFGR

An MMU-720AE implementation has fixed values for SMMU_PMCGR_CFGR, and these values define behavioral aspects of the implementation.

For information about the SMMU_PMCGR_CFGR field values, see [2.5.2.4 SMMUv3 PMU register architectural options](#) on page 55.

4.6.4 SMMU_PMCGR_CEID{0-1} registers

The SMMU_PMCGR_CEID{0-1} registers indicate the architectural events that are supported. They are described as 64-bit registers, but are accessed 32 bits at a time through the 32-bit PROG interface.

The following table shows the SMMU_PMCGR_CEID{0-1} registers.

Table 4-20: SMMU_PMCGR_CEID{0-1} registers

Offset	Name	Value
0x30E20	SMMU_PMCGR_CEID0	0x0000007F
0x30E28	SMMU_PMCGR_CEID1	0x00000000

4.6.5 PMU ID registers

The PMU ID registers appear only in Performance Monitor Page 0. Page 1 does not contain any ID registers.

The following table shows the PMU ID registers.

Table 4-21: PMU ID registers

Offset	Name	Field	Value	Description
0x30FB8	SMMU_PMCGR_PMAUTHSTATUS	[7:0]	0x00	No authentication interface is implemented
0x30FD0	SMMU_PMCGR_PIDR4, Peripheral ID4	[7:4]	0x4	SIZE = 4KB

Offset	Name	Field	Value	Description
		[3:0]	0x0	DES_2: JEP106 Designer continuation code
0x30FD4	SMMU_PMCG_PIDR5, Peripheral ID5	-	RES0	Reserved
0x30FD8	SMMU_PMCG_PIDR6, Peripheral ID6	-	RES0	Reserved
0x30FDC	SMMU_PMCG_PIDR7, Peripheral ID7	-	RES0	Reserved
0x30FE0	SMMU_PMCG_PIDR0, Peripheral ID0	[7:0]	0x9F for TLBU PMU 0x9E for TMU PMU	PART_0: bits [7:0] of the Part number
0x30FE4	SMMU_PMCG_PIDR1, Peripheral ID1	[7:4]	0xB	DES_0: bits [3:0] of the JEP106 Designer code
		[3:0]	0x4	PART_1: bits [11:8] of the Part number
0x30FE8	SMMU_PMCG_PIDR2, Peripheral ID2	[7:4]	0x00	REVISION, major revision, 0, for r0
		[3]	1	JEDEC-assigned value for DES always used
		[2:0]	3	DES_1: bits [6:4] bits of the JEP106 Designer code
0x30FEC	SMMU_PMCG_PIDR3, Peripheral ID3	[7:4]	MAX(p_level, ecorevnum)	REVAND, minor revision, where p_level is: 1 For p1
		[3:0]	0x00	CMOD
0x30FF0	SMMU_PMCG_CIDR0, Component ID0	[7:0]	0x0D	Preamble
0x30FF4	SMMU_PMCG_CIDR1, Component ID1	[7:0]	0x90	Preamble
0x30FF8	SMMU_PMCG_CIDR2, Component ID2	[7:0]	0x05	Preamble
0x30FFC	SMMU_PMCG_CIDR3, Component ID3	[7:0]	0xB1	Preamble

The PMDEVARCH and PMDEVTYPE registers are implemented as the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) defines.

4.7 TCU microarchitectural registers

You can set the TCU microarchitectural registers at boot time to optimize TCU behavior for your system. We recommend that you use the default values for most systems.

If the `TCUCFG_LEGACY_TZ_EN` parameter is set to 1, or the `legacy_tz_en` signal is set to 1, all bits in the [4.7.9 TCU_RCR register](#) on page 155 behave as 1 and the [4.7.9 TCU_RCR register](#) on page 155 is RAZ/WI.

The [4.7.9 TCU_RCR register](#) on page 155 is root-only. Non-root access to this register is Read-As-Zero (RAZ)/Write-Ignored (WI), RAZ/WI.

The [4.7.5 TCU_SCR register](#) on page 150 is Secure or root-only. Non-secure or realm access to this register is RAZ/WI.

TCU_RCR.S_UARCH controls non-root access to the registers that this section discusses, except for the following registers:

- [4.7.7 TCU_ROOT_CTRL register](#) on page 153
- [4.7.8 TCU_ROOT_CTRL2 register](#) on page 154

- [4.7.5 TCU_SCR register](#) on page 150
- [4.7.9 TCU_RCR register](#) on page 155



If the `TCUCFG_LEGACY_TZ_EN` parameter is set to 1, or the `legacy_tz_en` signal is set to 1, the bit `TCU_RCR.S.UARCH` behaves as 1. Therefore, non-root register access is permitted for microarchitectural registers.

Non-root accesses to the above registers, when `TCU_RCR.S.UARCH == 0` are RAZ/WI.

This mechanism permits the root to disable all non-root access to these registers if necessary for security reasons. However, by default, non-root access is granted as the rest of this section describes.

`TCU_RCR.S.ROOT_UARCH` controls non-root access to the following:

- [4.7.7 TCU_ROOT_CTRL register](#) on page 153
- [4.7.8 TCU_ROOT_CTRL2 register](#) on page 154

Non-root accesses to the `TCU_ROOT_CTRL` register when `TCU_RCR.S.ROOT_UARCH == 0` is RAZ/WI.

`TCU_SCR.NS_UARCH` controls Non-secure and realm access to registers in this section other than the `TCU_SCR` and `TCU_RCR` registers.

Non-secure and realm accesses to these registers, when `TCU_SCR.NS_UARCH == 0`, are RAZ and WI.

`TCU_RCR.S.UARCH` and `TCU_RCR.S.ROOT_UARCH` overrule control. If `TCU_RCR.S.UARCH` and `TCU_RCR.S.ROOT_UARCH` are set to 0, they prevent non-root access, regardless of the setting in the `TCU_SCR` register, as this section describes.

The following table shows how SCR and RCR override the ownership of a register.

Table 4-22: SCR and RCR override of register ownership

SCR.feature	RCR.feature	Non-secure register	Secure register	Realm register	Root register
0	0	Non-secure	Secure	Realm	Root
0	1	Non-secure	Secure	Realm	Secure
1	0	Non-secure	Non-secure	Realm	Root
1	1	Non-secure	Non-secure	Realm	Non-secure

The following registers can be written only when the conditions in the list lower down occur:

- [4.7.1 TCU_CTRL register](#) on page 146
- [4.7.6 TCU_CTRL2 register](#) on page 152
- [4.7.7 TCU_ROOT_CTRL register](#) on page 153

- [4.7.8 TCU_ROOT_CTRL2 register](#) on page 154
- [4.7.2 TCU_QOS register](#) on page 147
- [4.7.10 TCU_NODE_CTRLn register](#) on page 156
- [4.7.12 TCU_WC_SxLy_CMAX registers](#) on page 161
- [4.7.13 TCU_AGW_GC_Lx_CMAX registers](#) on page 162
- [4.7.14 TCU_DGW_GC_Lx_CMAX registers](#) on page 162

The registers in the previous list can be written only when the following conditions occur:

- SMMU_CR0.SMMUEN == 0
- SMMU_CR0ACK.SMMUEN == 0
- SMMU_S_CR0.SMMUEN == 0
- SMMU_S_CR0ACK.SMMUEN == 0
- SMMU_R_CR0.SMMUEN == 0
- SMMU_R_CR0ACK.SMMUEN == 0

In addition to the above conditions, the following registers can be written only when the conditions in the list lower down occur:

- [4.7.7 TCU_ROOT_CTRL register](#) on page 153
- [4.7.8 TCU_ROOT_CTRL2 register](#) on page 154

The registers in the previous list can be written only when the following conditions occur:

- SMMU_CR0.SMMUEN == 0
- SMMU_CR0ACK.SMMUEN == 0
- SMMU_S_CR0.SMMUEN == 0
- SMMU_S_CR0ACK.SMMUEN == 0
- SMMU_R_CR0.SMMUEN == 0
- SMMU_R_CR0ACK.SMMUEN == 0
- SMMU_ROOT_CR0.GPCEN == 0
- SMMU_ROOT_CR0ACK.GPCEN == 0

After modifying these registers, software must issue an INV_ALL operation using the SMMU_S_INIT register, before it sets SMMUEN to 1. Failure to issue the operation results in **UNPREDICTABLE** behavior.

We also recommend that for any register fields that relate to caching, you change the register only when there are no ongoing invalidations.

An array of registers, TCU_CTRL_AUX<n>, where <n> is 0-55, exists. Do not modify the values of these registers unless Arm advises otherwise. See:

- [4.10 TCU_CTRL_AUX<n> registers](#) on page 211
- [4.7 TCU microarchitectural registers](#) on page 143

4.7.1 TCU_CTRL register

The TCU Control register disables TCU features. If the hit rate of an individual walk cache is too low, you can disable individual walk caches to improve performance in some systems. Do not modify the AUX bits unless we direct you to do so.

Configurations

This register is available in all configurations.

Attributes

The TCU_CTRL register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E00

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-1: TCU_CTRL register bit assignments



Table 4-23: TCU_CTRL register bit descriptions

Bits	Name	Description
[31:28]	AUX[31:28]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[27]	HAZARD_EXPANSION_DIS	Prevent the Hazard Unit (HZU) from increasing the size of the hazard to match the downstream data width. This effectively overrides the value of the hazard_width tieoff signal to be 3'h3, 64 bits.
[26]	DONT_HASH_ASID	When set to 1, ASID is not used in the hash to create walk cache indices
[25:8]	AUX[25:8]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[7]	WCS2L3_DIS	Walk cache disable.
[6]	WCS2L2_DIS	When a bit of this field is set, it disables the corresponding stage and level of walk cache. WCS2L3_DIS is in bit [7], through to WCS1L0_DIS that is in bit [0].
[5]	WCS2L1_DIS	
[4]	WCS2L0_DIS	
[3]	WCS1L3_DIS	
[2]	WCS1L2_DIS	
[1]	WCS1L1_DIS	
[0]	WCS1L0_DIS	

4.7.2 TCU_QOS register

The TCU_QOS register controls the value of the AxQOS signal.



The QoS value that is associated with each transaction does not take account of other transactions that are blocked behind it. For example, although translations with a high priority setting in the [4.7.10 TCU_NODE_CTRLn register](#) on page 156 are normally progressed before translations with a lower priority, a low-priority page table walk can block a higher priority page table walk and prevent it from being issued from the TCU.

Configurations

This register is available in all configurations.

Attributes

The TCU_QOS register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E04

Type
 RW
Reset value
 0

Bit descriptions
 The following figure and table show the bit assignments and descriptions.

Figure 4-2: TCU_QOS register bit assignments

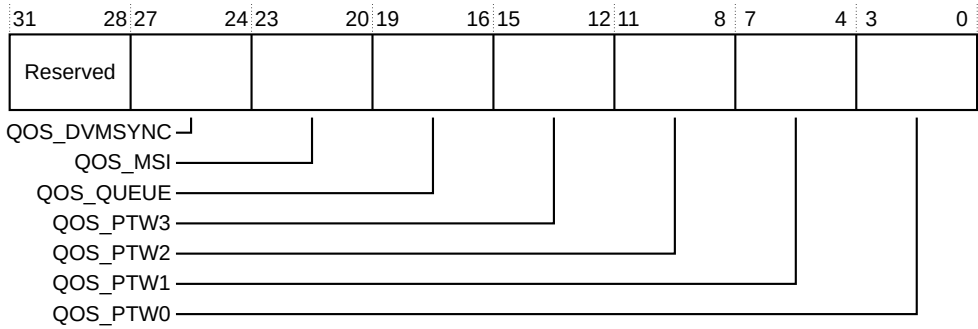


Table 4-24: TCU_QOS register bit descriptions

Bits	Name	Description
[31:28]	-	Reserved.
[27:24]	QOS_DVMSYNC	QoS level to use for DVM Sync Completion messages.
[23:20]	QOS_MSI	QoS level to use for MSIs.
[19:16]	QOS_QUEUE	QoS level to use for queue accesses.
[15:12]	QOS_PTW3	QoS level to use for the following, that are requested from nodes with TCU_NODE_CTRLn.PRIORITY = 3: - Translation table walks - Configuration Table Walks (CTWs) - Granule Protection Table (GPT) walks
[11:8]	QOS_PTW2	QoS level to use for the following, that are requested from nodes with TCU_NODE_CTRLn.PRIORITY = 2: - Translation table walks - CTWs - GPT walks
[7:4]	QOS_PTW1	QoS level to use for the following, that are requested from nodes with TCU_NODE_CTRLn.PRIORITY = 1: - Translation table walks - CTWs - GPT walks
[3:0]	QOS_PTW0	QoS level to use for the following, that are requested from nodes with TCU_NODE_CTRLn.PRIORITY = 0: - Translation table walks - CTWs - GPT walks

4.7.3 TCU_CFG register

Use the TCU_CFG register to set configuration information.

Configurations

This register is available in all configurations.

Attributes

The TCU_CFG register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E08

Type

RO

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-3: TCU_CFG register bit assignments

31		17	16			4	3	0
Reserved				XLATE_SLOTS			Reserved	

Table 4-25: TCU_CFG register bit descriptions

Bits	Name	Description
[31:17]	-	Reserved
[16:4]	XLATE_SLOTS	Number of translation slots that are available to be shared between all nodes. The value is the value of the TCUCFG_XLATE_SLOTSPARAMETER. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
[3:0]	-	Reserved

4.7.4 TCU_STATUS register

Use the TCU_STATUS register to set status information.

Configurations

This register is available in all configurations.

Attributes

The TCU_STATUS register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E10

Type

RO

Reset value

0

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-4: TCU_STATUS register bit assignments

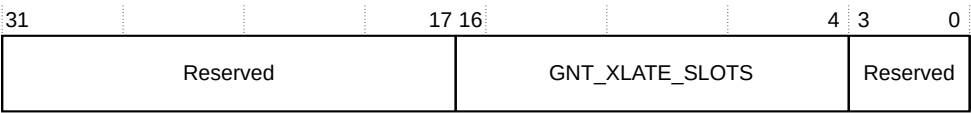


Table 4-26: TCU_STATUS register bit descriptions

Bits	Name	Description
[31:17]	-	Reserved
[16:4]	GNT_XLATE_SLOTS	Total number of translation slots that are currently allocated to all connected nodes. This information can be useful for debugging purposes.
[3:0]	-	Reserved

4.7.5 TCU_SCR register

The TCU Secure Control register controls whether Non-secure and Realm software is permitted to access each TCU register group.

This register does not control Secure access to the Performance Monitor registers. The SMMU_PMCGR_SCR register controls Secure access to these registers as the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) defines.

Configurations

This register is available in all configurations.

Attributes

The TCU_SCR register attributes are as follows:

Width

32-bit

Functional group

4.4.4 TCU microarchitectural registers summary on page 132.

Address offset

0x28E18

Type

Secure, RW

Reset value

sec_override. See A.1.12 TCU tie-off signals on page 302.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-5: TCU_SCR register bit assignments

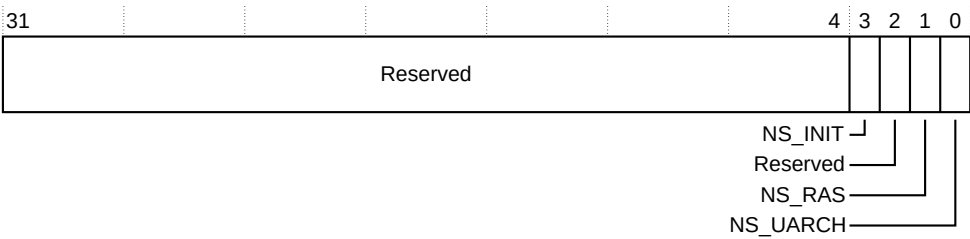


Table 4-27: TCU_SCR register bit descriptions

Bits	Name	Description
[31:4]	–	Reserved
[3]	NS_INIT	Non-secure and Realm register access is permitted to the SMMU_S_INIT register The sec_override input sets the reset value of this signal. See A.1.12 TCU tie-off signals on page 302.
[2]	–	Reserved
[1]	NS_RAS	Non-secure and Realm register access is permitted for RAS registers. When this bit is 0, Non-secure and Realm writes to the following register addresses are ignored, and Non-secure and Realm reads return zero: 0x006_0000-0x006_FFFC. The sec_override input sets the reset value of this signal. See A.1.12 TCU tie-off signals on page 302.

Bits	Name	Description
[0]	NS_UARCH	Non-secure and Realm register access is permitted for microarchitectural registers When this bit is 0, Non-secure and Realm writes to the following register addresses are ignored, and Non-secure and Realm reads return zero: 0x28E00-0x29910 The sec_override input sets the reset value of this signal. See A.1.12 TCU tie-off signals on page 302. If Secure translation might be used, we recommend that software does not set this bit.

4.7.6 TCU_CTRL2 register

The TCU Control 2 register is Reserved. Do not change these bits unless Arm advises you to.

Configurations

This register is available in all configurations.

Attributes

The TCU_CTRL2 register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E40

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-6: TCU_CTRL2 register bit assignments



Table 4-28: TCU_CTRL2 register bit descriptions

Bits	Name	Description
[31:0]	AUX[31:0]	Reserved. Do not change these bits unless Arm advises you to.

4.7.7 TCU_ROOT_CTRL register

The TCU_ROOT_CTRL register disables TCU features. In most systems, you can leave the TCU_ROOT_CTRL register unchanged.

The TCU_ROOT_CTRL register is root-only by default. It can be made accessible by Secure or Non-secure by setting various combinations of the [4.7.5 TCU_SCR register](#) on page 150 and the [4.7.9 TCU_RCR register](#) on page 155.

Configurations

This register is available only when either the TBUCFG_LEGACY_TZ_EN parameter is set to 0 or the legacy_tz_en signal is set to 0.

Attributes

The TCU_ROOT_CTRL register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28F00

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-7: TCU_ROOT_CTRL register bit assignments

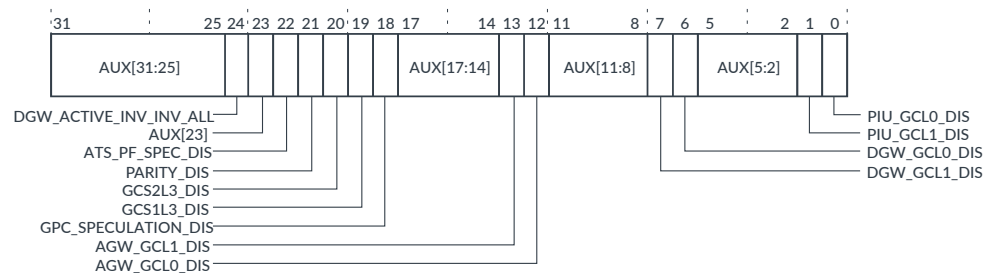


Table 4-29: TCU_ROOT_CTRL register bit descriptions

Bits	Name	Description
[31:25]	AUX[31:25]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[24]	DGW_ACTIVE_INV_INV_ALL	In the DTI GPC Wrapper (DGW), active INV causes everything to be invalidated regardless of matching with an invalidate.
[23]	AUX[23]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[22]	ATS_PF_SPEC_DIS	Split stage ATS IPA message prefetch speculation disable. The GPT cache is not updated for GPT walks at this level.
[21]	PARITY_DIS	Ignore any parity errors from registers with parity bits.
[20]	GCS2L3_DIS	Disable update of GCB resulting from a walk for S2L3.
[19]	GCS1L3_DIS	Disable update of GCB resulting from a walk for S1L3.
[18]	GPC_SPECULATION_DIS	Disables speculation on GPC checks to prevent the TCU from starting a page table walk until the GPC check for the PA of the page table entry has been performed. When set to 0, the TCU starts the GPT walk and the page table walk in parallel for improved performance.
[17:14]	AUX[17:14]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[13]	AGW_GCL1_DIS	GCB disable. When a bit of this field is set, it disables the corresponding level of GCB for the AGW GC inside the TMU.
[12]	AGW_GCL0_DIS	
[11:8]	AUX[11:8]	Set this field to 0 unless we advise otherwise.
[7]	DGW_GCL1_DIS	GCB disable. When a bit of this field is set, it disables the corresponding level of GCB for the DGW GC inside the TMU.
[6]	DGW_GCL0_DIS	
[5:2]	AUX[5:2]	Set this field to 0 unless we advise otherwise.
[1]	PIU_GCL1_DIS	GCB disable. When a bit of this field is set, it disables the corresponding level of GCB for the AGW GC inside the PIU.
[0]	PIU_GCL0_DIS	

4.7.8 TCU_ROOT_CTRL2 register

The TCU_ROOT_CTRL2 register disables TCU features. Do not change the bits in this register unless Arm advises you to.

The TCU_ROOT_CTRL2 register is root-only by default. It can be made accessible by Secure or Non-secure by setting various combinations of the [4.7.5 TCU_SCR register](#) on page 150 and the [4.7.9 TCU_RCR register](#) on page 155.

Configurations

This register is available in all configurations.

Attributes

The TCU_ROOT_CTRL2 register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28F04

Bit descriptions

Table 4-30: TCU_ROOT_CTRL2 bit descriptions

Bits	Name	Description
[31:0]	AUX31:0	Reserved. Do not change these bits unless Arm advises you to.

4.7.9 TCU_RCR register

This register is accessible only from Root software. Some values are reset to 1 to give access to the Secure world by default.

The TCU_RCR register does not control Root access to the Performance Monitor registers. The SMMU_PMCGR_SCR register controls Root access to these registers as the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) defines.

The TCU_RCR register also affects the [4.9 TCU system discovery registers](#) on page 175, [4.12 TCU PIU integration registers](#) on page 213, and [4.13 TCU TMU integration registers](#) on page 219 in the same way.

If the TCUCFG_LEGACY_TZ_EN parameter is set to 1 or the legacy_tz_en signal is set to 1, all bits in this register behave as 1 and the TBU_RCR register is RAZ/WI.

Configurations

This register is available in all configurations.

Attributes

The TCU_RCR register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28F10

Type

RW

Reset value

See 'Reset' column in the following table.

Usage constraints

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-8: TCU_RCR register bit assignments

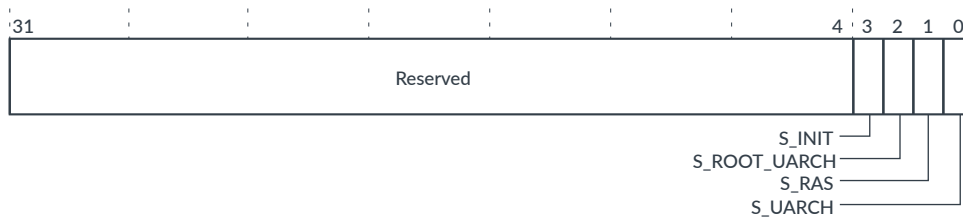


Table 4-31: TCU_RCR register bit descriptions

Bits	Name	Reset	Description
[31:4]	-	-	Reserved
[3]	S_INIT	1	Non-root register access permitted to the SMMU_S_INIT register.
[2]	S_ROOT_UARCH	0	Non-root register access permitted for the following Root-specific microarchitectural registers: 4.11.1 ITEN register for the TCU on page 212 4.12.1 ITOP_PIU register for the TCU Programmer Interface Unit on page 213 4.12.2 ITIN_PIU register for the TCU Programmer Interface Unit on page 218 4.13.1 ITOP_TMU register for the TCU Translation Management Unit on page 219 4.13.2 ITIN_TMU register for the TCU Translation Management Unit on page 220 4.7.7 TCU_ROOT_CTRL register on page 153 4.7.8 TCU_ROOT_CTRL2 register on page 154
[1]	S_RAS	1	Non-root register access permitted for RAS registers. When this bit is 0, Non-root writes to the following register addresses are ignored, and Non-secure reads return zero: 0x006_0000-0x006_FFFC
[0]	S_UARCH	1	Non-root register access permitted for microarchitectural registers. When this bit is 0, Non-root writes to some register are ignored, and Non-root reads return zero, as 4.1.1 Realm Management Extension Register Principles on page 121 describes. The TCU_RCR register also affects System discovery registers in the same way.

4.7.10 TCU_NODE_CTRLn register

The TCU_NODE_CTRLn register controls how the TCU communicates with a single DTI requester, either a TBU or a PCIe Root Port implementing ATS.

Each DTI requester, for example, a TBU has a node ID, each with TCU_NODE_CTRLn register in sequential addresses:

Node 0

TCU_NODE_CTRL0 at address 0x29000

Node 1

TCU_NODE_CTRL1 at address 0x29004

...

...

Node n

TCU_NODE_CTRLn at address 0x29000 + (4
× (TCUCFG_NUM_TBU - 1))

The number of registers that are implemented corresponds to the value of TCUCFG_NUM_TBU. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

All bits [31:16] are implemented and are readable and writable, regardless of the number of LTI ports the attached DTI node has. The following expression determines the priority that is associated with a transaction:

PRIORITY = (TCU_NODE_CTRLn.PRIORITY_SEL ?
TCU_NODE_CTRLn[(DTI_x_TRANS_REQ.QOS[2:0] × 2) + 16] :
TCU_NODE_CTRLn.DEFAULT_PRIORITY)

If a DTI node sends a translation request with an incorrect QoS value, the programmed value for the LTI port indicated in the QoS field is used. This value is used because it is not possible for the TCU to determine what is a valid or invalid port number.



Note

When the priority level is established, these are translated into QoS values by selection of the appropriate QOS_PTW* field from the TCU_QOS register, which override the value in the DTI trans_req message. This means that the transaction has its QoS value overridden but no additional information is required to be associated with the transaction. If the PRI level association is updated, the rest of the mechanism requires no alteration.

Configurations

This register is available in all configurations.

Attributes

The TCU_NODE_CTRLn register attributes are as follows:

Width

32-bit

Functional group

4.4.4 TCU microarchitectural registers summary on page 132.

Address offset

0x29000-0x293FC

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-9: TCU_NODE_CTRLn register bit assignments

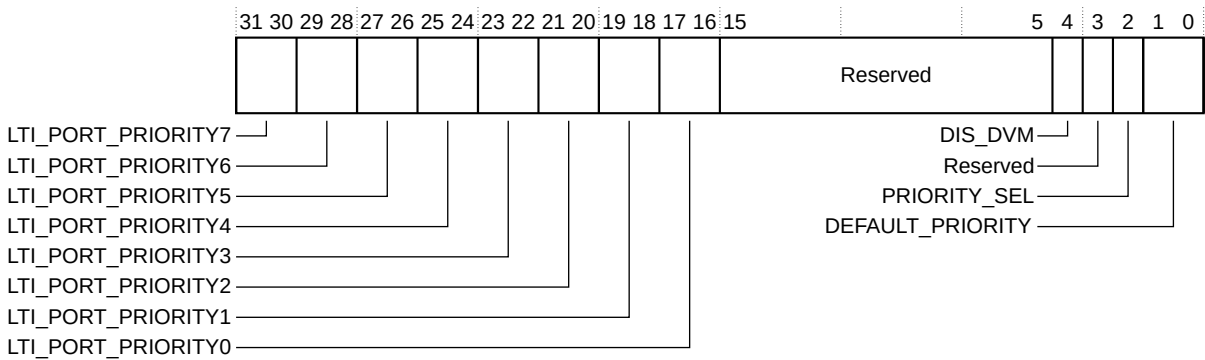


Table 4-32: TCU_NODE_CTRLn register bit descriptions

Bits	Name	Description
[31:30]	LTI_PORT_PRIORITY7	Priority level for LTI Port 7 for this node, if the port exists
[29:28]	LTI_PORT_PRIORITY6	Priority level for LTI Port 6 for this node, if the port exists
[27:26]	LTI_PORT_PRIORITY5	Priority level for LTI Port 5 for this node, if the port exists
[25:24]	LTI_PORT_PRIORITY4	Priority level for LTI Port 4 for this node, if the port exists
[23:22]	LTI_PORT_PRIORITY3	Priority level for LTI Port 3 for this node, if the port exists
[21:20]	LTI_PORT_PRIORITY2	Priority level for LTI Port 2 for this node, if the port exists
[19:18]	LTI_PORT_PRIORITY1	Priority level for LTI Port 1 for this node, if the port exists
[17:16]	LTI_PORT_PRIORITY0	Priority level for LTI Port 0 for this node
[15:5]	-	Reserved

Bits	Name	Description
[4]	DIS_DVM	Disable DVM. When this bit is set, the node does not participate in DVM invalidation. This setting can improve performance if the node can be slow to respond to invalidations issued over DTI. Note: The DIS_DVM setting does not affect invalidation by PA. This bit is only used for TBU nodes and is ignored for ATS nodes.
[3]	-	Reserved
[2]	PRIORITY_SEL	Select the priority between the DTI node ID, default and the LTI port: 0 When this bit is set to 0, the priority of all translation requests from the DTI requester, TBU, are given the priority that the DEFAULT_PRIORITY field specifies. 1 When this bit is set to 1, transactions from the DTI requester, TBU, are given the priority that is set in the LTI_PORT_PRIORITY_m field, where the QOS[2:0] bits in the DTI translation request provide the value of m. Note: LTI PORT PRIORITY is not useful for ATS connections. Software is expected to write PRIORITY_SEL to 0 for ATS managers.
[1:0]	DEFAULT_PRIORITY	Default Priority level for this DTI requester, a TBU. Translation requests from a node with a higher priority level are normally progressed before translation requests from a node with a lower priority level.

The [4.7.2 TCU_QOS register](#) on page 147 contains four QoS level fields for the following:

- Translation table walks
- Configuration Table Walks (CTWs)
- Granule Protection Table (GPT) walks



Note

These fields set the AxQOS value for each of the four levels. The TCU_NODE_CTRL_n.LTI_PORT_PRIORITY_m and TCU_NODE_CTRL_n.DEFAULT_PRIORITY fields do not set the AxQOS value directly. Instead, these fields index into the TCU_QOS register to select which of the four QoS levels to use for walks that are associated with a particular LTI port or node. See [4.7.2 TCU_QOS register](#) on page 147.

4.7.11 TCU_NODE_STATUSn register

The TCU_NODE_STATUSn register provides the status for each node. The TCU_NODE_STATUSn register and the TCU_NODE_CTRLn register each have a single status register.

The number of registers that are implemented corresponds to the value of TCUCFG_NUM_TBU. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Configurations

This register is available in all configurations.

Attributes

The TCU_NODE_STATUSn register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x29400-0x297FC

Type

RO

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-10: TCU_NODE_STATUSn register bit assignments



Table 4-33: TCU_NODE_STATUSn register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved
[3]	ATSV4	Indicates whether the node implements DTI-ATSV4. 0 The node is DTI-ATSV3, DTI-ATSV2, or DTI-ATSV1. 1 The node is DTI-ATSV4. This bit is valid only when ATS = 1. When ATS = 0, this bit is 0.

Bits	Name	Description
[2]	ATSv3	Indicates whether the node implements DTI-ATSv3. 0 The node is DTI-ATSv2 or DTI-ATSv1. 1 The node is DTI-ATSv3. This bit is valid only when ATS = 1. When ATS = 0, this bit is 0.
[1]	ATS	Indicates whether the node implements ATS: 0 The node is a TBU connected using DTI-TBU 1 The node is a PCIe Root Port supporting ATS, connected using DTI-ATS This bit is only valid when CONNECTED = 1. When CONNECTED = 0, this bit is 0.
[0]	CONNECTED	Indicates whether the DTI link for this node is in the connected state: 0 Node currently not in the connected state, including the states transitioning to and from the connected state 1 Node currently in the connected state When not connected, write accesses to TBU registers are ignored and read accesses return 0. However, the state might change between reading this register and attempting to access the TBU.

4.7.12 TCU_WC_SxLy_CMAX registers

TCU_WC_SxLy_CMAX registers enable you to set maximum capacities for the TCU walk cache RAMs, per stage and level.

The encoding of the TCU_WC_SxLy_CMAX registers is the same as the encoding for the MPAMCFG_CMAX registers that the [Arm® Memory System Resource Partitioning and Monitoring \(MPAM\) System Component Specification](#) defines. These registers are readable and writable registers.

The following table describes the TCU_WC_SxLy_CMAX registers.

Table 4-34: TCU_WC_SxLy_CMAX registers

Address	Name	Field	Position	Description
0x29800	TCU_WC_S1L0_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 1 level 0
0x29804	TCU_WC_S1L1_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 1 level 1
0x29808	TCU_WC_S1L2_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 1 level 2
0x2980C	TCU_WC_S1L3_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 1 level 3
0x29810	TCU_WC_S2L0_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 2 level 0
0x29814	TCU_WC_S2L1_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 2 level 1
0x29818	TCU_WC_S2L2_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 2 level 2
0x2981C	TCU_WC_S2L3_CMAX	CMAX	[15:0]	Maximum capacity for TCU Walk Cache stage 2 level 3



Note

The implementation defines how many bits are used, depending on the cache configuration, but not more than 8 bits. Because the value represents a fixed-point binary fraction, it is MSB aligned. Therefore, only bits [15:x] have any impact, where x can be 8, 9, 10, 11, 12, or 13.

4.7.13 TCU_AGW_GC_Lx_CMAX registers

TCU_AGW_GC_Lx_CMAX registers enable you to set maximum capacities for the TCU AGW GPTW cache RAMs, per level.

The encoding of the TCU_AGW_GC_Lx_CMAX registers is the same as the encoding for the MPAMCFG_CMAX registers that the [Arm® Memory System Resource Partitioning and Monitoring \(MPAM\) System Component Specification](#) defines. These registers are readable and writable registers.

The following table describes the TCU_AGW_GC_Lx_CMAX registers.

Table 4-35: TCU_AGW_GC_Lx_CMAX registers

Address	Name	Field	Position	Description
0x29900	TCU_AGW_GC_L0_CMAX	CMAX	[15:0]	Maximum capacity for TCU GPT Cache level 0
0x29904	TCU_AGW_GC_L1_CMAX	CMAX	[15:0]	Maximum capacity for TCU GPT Cache level 1



Note

The implementation defines how many bits are used, depending on cache configuration, but not more than 8 bits. Because the value represents a fixed-point binary fraction, it is MSB aligned, so only bits [15:x] have any impact, where x can be 8, 9, 10, 11, 12, or 13.

4.7.14 TCU_DGW_GC_Lx_CMAX registers

TCU_DGW_GC_Lx_CMAX registers enable you to set maximum capacities for the TCU DGW GPTW cache RAMs, per level.

The encoding of the TCU_DGW_GC_Lx_CMAX registers is the same as the encoding for the MPAMCFG_CMAX registers that the [Arm® Memory System Resource Partitioning and Monitoring \(MPAM\) System Component Specification](#) defines. These registers are readable and writable registers.

The following table describes the TCU_DGW_GC_Lx_CMAX registers.

Table 4-36: TCU_DGW_GC_Lx_CMAX registers

Address	Name	Field	Position	Description
0x29910	TCU_DGW_GC_L0_CMAX	CMAX	[15:0]	Maximum capacity for TCU GPT Cache level 0
0x29914	TCU_DGW_GC_L1_CMAX	CMAX	[15:0]	Maximum capacity for TCU GPT Cache level 1



Note

The implementation defines how many bits are used, depending on cache configuration, but not more than 8 bits. Because the value represents a fixed-point binary fraction, it is MSB aligned, so only bits [15:x] have any impact, where x can be 8, 9, 10, 11, 12, or 13.

4.8 TCU RAS registers

The MMU-720AE TCU contains Reliability, Availability, and Serviceability (RAS) registers.

The RAS registers implement the RAS Extension registers, single record format:

- Non-root access to these registers when TCU_RCR.S_RAS = 0 are RAZ/WI. See [4.7.9 TCU_RCR register](#) on page 155.
- Non-secure accesses to these registers, when TCU_SCR.NS_RAS = 0, are RAZ/WI. See [4.7.5 TCU_SCR register](#) on page 150.



Note

If the TCUCFG_LEGACY_TZ_EN parameter is set to 1 or the legacy_tz_en signal is set to 1:

- All bits in the [4.7.9 TCU_RCR register](#) on page 155 behave as 1.
- The [4.16.3 TBU_SCR register](#) on page 229 is RAZ/WI.

The RAS registers enable software to monitor the following classes of error:

- Corrected Errors (CEs) in the RAMs that the configuration cache uses.
- CEs in the RAMs used by the walk caches.

4.8.1 TCU_ERRFR register

To discover how the TCU handles errors, use the TCU Error Feature register.

Configurations

This register is available in all configurations.

Attributes

The TCU_ERRFR register attributes are as follows:

Width

32-bit

Functional group

[4.4.3 TCU Reliability, Availability, and Serviceability registers summary](#) on page 132.

Address offset

0x60000

Type

Root, RO

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-11: TCU_ERRFR register bit assignments

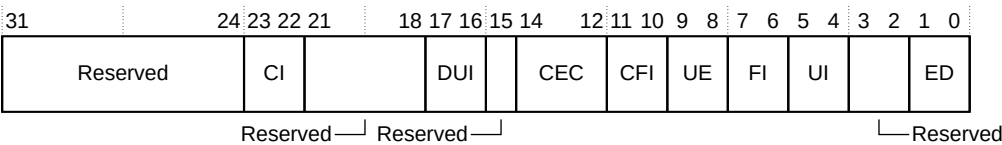


Table 4-37: TCU_ERRFR register bit descriptions

Bits	Name	Description
[31:24]	-	Reserved
[23:22]	CI	Critical Error Interrupt is always enabled. Value is 0b01.
[21:18]	-	Reserved
[17:16]	DUI	Does not support this feature. Value is 0b00.
[15]	-	Reserved
[14:12]	CEC	Does not implement the standard corrected error counter model. Value is 0b000.
[11:10]	CFI	Does not support this feature. Value is 0b00.
[9:8]	UE	In-band error signaling feature is always enabled. Value is 0b01.
[7:6]	FI	Fault handling interrupt is controllable. Value is 0b10.
[5:4]	UI	Error Recovery Interrupt always enabled for UE. Value is 0b01.
[3:2]	-	Reserved
[1:0]	ED	Error detection is always enabled. Value is 0b01.

4.8.2 TCU_ERRCTLR register

To enable fault handling interrupts, use the TCU Error Control register.

Configurations

This register is available in all configurations.

Attributes

The TCU_ERRCTLR register attributes are as follows:

Width

32-bit

Functional group

[4.4.3 TCU Reliability, Availability, and Serviceability registers summary](#) on page 132.

Address offset

0x60008

Type

Root, RW

Reset value

1

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-12: TCU_ERRCTLR register bit assignments

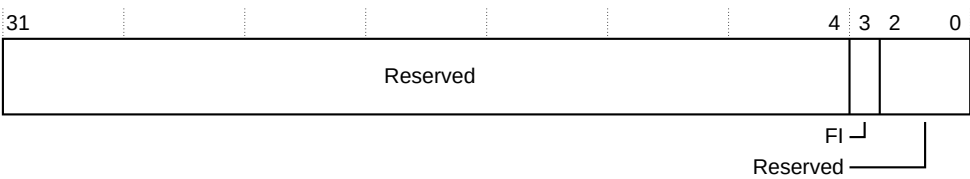


Table 4-38: TCU_ERRCTLR register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved
[3]	FI	Fault handling interrupt enable
[2:0]	-	Reserved

4.8.3 TCU_ERRSTATUS register

Use the TCU error status register to find out whether different types of error have occurred. Certain bits in this register are cleared by writing a 1 to their bit position. These writes are ignored in certain circumstances to avoid race conditions where a new error has occurred which software has not yet observed.

Configurations

This register is available in all configurations.

Attributes

The TCU_ERRSTATUS register attributes are as follows:

Width

32-bit

Functional group

[4.4.3 TCU Reliability, Availability, and Serviceability registers summary](#) on page 132.

Address offset

0x60010

Type

Root, RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-13: TCU_ERRSTATUS register bit assignments

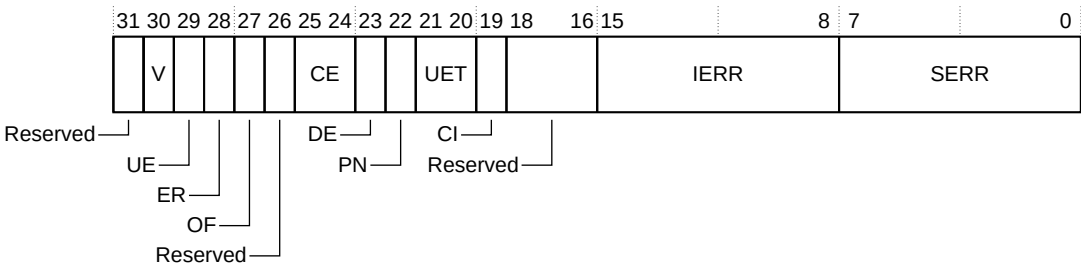


Table 4-39: TCU_ERRSTATUS register bit descriptions

Bits	Name	Description
[31]	-	Reserved
[30]	V	Status Register is valid. The possible values of this bit are as follows: 0 ERRSTATUS is not valid. 1 ERRSTATUS is valid. At least 1 error has been recorded. If any of the UE, DE, or CE bits are set to 1, and are not being cleared to 0 in the same write, direct writes to this bit are ignored. This bit is read/write-one-to-clear. This bit resets to 0 on a reset.
[29]	UE	Uncorrected error, or errors. The possible values of this bit are as follows: 0 No errors that could not be corrected or deferred. 1 At least one error detected that has not been corrected or deferred. If the OF bit is set to 1 and is not being cleared to 0 in the same write, direct writes to this bit are ignored. This bit is read/write-one-to-clear.
[28]	ER	Error Reported. The possible values of this bit are as follows: 0 No in-band error (External abort) is reported 1 The node to the requester making the access or other transaction signaled an External abort. This bit is read/write-one-to-clear.

Bits	Name	Description
[27]	OF	Overflow. Multiple errors are detected. This bit is set to 1 when: - Multiple errors are detected on the same cycle - A new error occurs when there is already a valid record in the register This bit is read/write-one-to-clear.
[26]	-	Reserved
[25:24]	CE	Correctable Error, or errors. 0b00 No correctable errors recorded 0b10 At least one Corrected error recorded Other values are Reserved. This field is cleared by writing 0b11 to it. If OF is set and not being cleared, the write is ignored. A write of any value other than 0b11 is ignored.
[23]	DE	Deferred error, or errors. The possible values of this bit are as follows: 0 No errors were deferred 1 At least one error was not corrected and deferred If the OF bit is set to 1 and is not being cleared to 0 in the same write, direct writes to this bit are ignored. This bit is read/write-one-to-clear.
[22]	PN	Poison. The possible values of this bit are as follows: 0 Uncorrected error or deferred error is recorded because a corrupt value was detected, for example, by an Error Detection Code (EDC) 1 Uncorrected error or deferred error is recorded because a poison value was detected This bit is read/write-one-to-clear.
[21:20]	UET	Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error. The possible values of this field are as follows: 0b00 Uncorrected error, Uncontainable error (UC) 0b11 Uncorrected error, Signaled or Recoverable error (UER) Accessing this field has the following behavior. This field is not valid and reads UNKNOWN if any of the following are true: - TCU_ERRSTATUS.V == 0b0 - TCU_ERRSTATUS.UE == 0b0 Otherwise, this field is read/write-ones-to-clear. Writing a value other than all-zeros or all-ones sets this field to an UNKNOWN value.
[19]	CI	Indicates whether a critical error condition has been recorded. The possible values of this bit are as follows: 0 No critical error condition 1 Critical error condition This field is read/write-one-to-clear.
[18:16]	-	Reserved

Bits	Name	Description																																								
[15:8]	IERR	IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error: <table><tr><td>80</td><td>PIU QTW FF PARITY</td></tr><tr><td>79</td><td>PIU PIB FF PARITY</td></tr><tr><td>78</td><td>PIU AGW GCB FF PARITY</td></tr><tr><td>77</td><td>PIU AGW GMB FF PARITY</td></tr><tr><td>76</td><td>PIU AGW FF PARITY</td></tr><tr><td>75</td><td>PIU INV FF PARITY</td></tr><tr><td>74</td><td>PIU CMD FF PARITY</td></tr><tr><td>73</td><td>PIU EVT FF PARITY</td></tr><tr><td>72</td><td>PIU PRI FF PARITY</td></tr><tr><td>71</td><td>PIU MGB FF PARITY</td></tr><tr><td>70</td><td>PIU CPB FF PARITY</td></tr><tr><td>69</td><td>PIU FAR FF PARITY</td></tr><tr><td>68</td><td>PIU PRG FF PARITY</td></tr><tr><td>67</td><td>PIU RAS FF PARITY</td></tr><tr><td>66</td><td>Reserved</td></tr><tr><td>65</td><td>TMU PMD FF PARITY</td></tr><tr><td>64</td><td>TMU PRG FF PARITY</td></tr><tr><td>63</td><td>TMU PMU FF PARITY</td></tr><tr><td>62</td><td>TMU WCB FF PARITY</td></tr><tr><td>61</td><td>TMU CCB FF PARITY</td></tr></table> Writes to this field are ignored.	80	PIU QTW FF PARITY	79	PIU PIB FF PARITY	78	PIU AGW GCB FF PARITY	77	PIU AGW GMB FF PARITY	76	PIU AGW FF PARITY	75	PIU INV FF PARITY	74	PIU CMD FF PARITY	73	PIU EVT FF PARITY	72	PIU PRI FF PARITY	71	PIU MGB FF PARITY	70	PIU CPB FF PARITY	69	PIU FAR FF PARITY	68	PIU PRG FF PARITY	67	PIU RAS FF PARITY	66	Reserved	65	TMU PMD FF PARITY	64	TMU PRG FF PARITY	63	TMU PMU FF PARITY	62	TMU WCB FF PARITY	61	TMU CCB FF PARITY
80	PIU QTW FF PARITY																																									
79	PIU PIB FF PARITY																																									
78	PIU AGW GCB FF PARITY																																									
77	PIU AGW GMB FF PARITY																																									
76	PIU AGW FF PARITY																																									
75	PIU INV FF PARITY																																									
74	PIU CMD FF PARITY																																									
73	PIU EVT FF PARITY																																									
72	PIU PRI FF PARITY																																									
71	PIU MGB FF PARITY																																									
70	PIU CPB FF PARITY																																									
69	PIU FAR FF PARITY																																									
68	PIU PRG FF PARITY																																									
67	PIU RAS FF PARITY																																									
66	Reserved																																									
65	TMU PMD FF PARITY																																									
64	TMU PRG FF PARITY																																									
63	TMU PMU FF PARITY																																									
62	TMU WCB FF PARITY																																									
61	TMU CCB FF PARITY																																									
[15:8]	IERR	IERR (continued) IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error: <table><tr><td>60</td><td>TMU HZU FF PARITY</td></tr><tr><td>59</td><td>TMU PIB FF PARITY</td></tr><tr><td>58</td><td>TMU DMD FF PARITY</td></tr><tr><td>57</td><td>TMU AGW GMB FF PARITY</td></tr><tr><td>56</td><td>TMU AGW GCB FF PARITY</td></tr><tr><td>55</td><td>TMU AGW FF PARITY</td></tr><tr><td>54</td><td>TMU DGW GMB FF PARITY</td></tr><tr><td>53</td><td>TMU DGW GCB FF PARITY</td></tr><tr><td>52</td><td>TMU DGW FF PARITY</td></tr><tr><td>51</td><td>TMU TWB FF PARITY</td></tr><tr><td>50-44</td><td>Reserved</td></tr><tr><td>43</td><td>PIU AGW GMB WLK STATUS</td></tr><tr><td>42</td><td>PIU AGW GMB LOOP/PIU_GMB_POISON</td></tr><tr><td>41</td><td>PIU AGW GCB TAGS+DATA</td></tr></table> Writes to this field are ignored.	60	TMU HZU FF PARITY	59	TMU PIB FF PARITY	58	TMU DMD FF PARITY	57	TMU AGW GMB FF PARITY	56	TMU AGW GCB FF PARITY	55	TMU AGW FF PARITY	54	TMU DGW GMB FF PARITY	53	TMU DGW GCB FF PARITY	52	TMU DGW FF PARITY	51	TMU TWB FF PARITY	50-44	Reserved	43	PIU AGW GMB WLK STATUS	42	PIU AGW GMB LOOP/PIU_GMB_POISON	41	PIU AGW GCB TAGS+DATA												
60	TMU HZU FF PARITY																																									
59	TMU PIB FF PARITY																																									
58	TMU DMD FF PARITY																																									
57	TMU AGW GMB FF PARITY																																									
56	TMU AGW GCB FF PARITY																																									
55	TMU AGW FF PARITY																																									
54	TMU DGW GMB FF PARITY																																									
53	TMU DGW GCB FF PARITY																																									
52	TMU DGW FF PARITY																																									
51	TMU TWB FF PARITY																																									
50-44	Reserved																																									
43	PIU AGW GMB WLK STATUS																																									
42	PIU AGW GMB LOOP/PIU_GMB_POISON																																									
41	PIU AGW GCB TAGS+DATA																																									

Bits	Name	Description	
[15:8]	IERR	IERR (continued)	
		IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error:	
		40	PIU AGW GCB REPL
		39	PIU CMD RPOISON
		38	Reserved
		37	TMU TWB WMB POISON
		36	TMU TWB CMB POISON
		35	TMU AGW ID RAM PARITY
		34	TMU AGW STATUS RAM PARITY
		33	TMU AGW RESPONSE RAM PARITY
		32	TMU AGW GMB SCRATCH
		31	TMU AGW GMB WLK STATUS
		30	TMU AGW GMB LOOP/TMU_AGW_GMB_POISON
		29	TMU DGW RAM
		28	TMU DGW GMB SCRATCH
		27	TMU DGW GMB WLK STATUS
		26	TMU DGW GMB LOOP/TMU_DGW_GMB_POISON
		25	TMU AGW GCB TAGS+DATA
		24	TMU DGW GCB TAGS+DATA
		23	TMU AGW GCB REPL
22	TMU AGW GCB PCNT		
21	TMU AGW GCB PLIM		
		Writes to this field are ignored.	
[15:8]	IERR	IERR (continued)	
		IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error:	
		20	TMU DGW GCB REPL
		19	TMU DGW GCB PCNT
		18	TMU DGW GCB PLIM
		17	TMU CCB MCC DATA
		16	TMU CCB MCC TAGS
		15	TMU WCB MWC DATA
		14	TMU WCB MWC TAGS
		13	TMU CCB MCC REPL
		12	TMU CCB MCC PCNT
		11	TMU CCB MCC PLIM
		10	TMU WCB MWC REPL
		9	TMU WCB MWC PCNT
		8	TMU WCB MWC PLIMH
		7	TMU HZU SEC PTR
		6	TMU HZU PTR
		5	TMU HTTU RAM
		4	TMU TWB WMB SCRATCH
		3	TMU TWB WMB WLK STATUS
2	TMU TWB WMB LKP STATUS		
1	Reserved		
0	TMU TWB BSU		
		Writes to this field are ignored.	

Bits	Name	Description												
[7:0]	SERR	The error code provides information about the earliest unacknowledged error. It can contain the following values: <table><tr><td>2</td><td>Single or Double Error from RAMs that are not *GW, CCB, or WCB TAGS/DATA</td></tr><tr><td>8</td><td>Single or Double Error from CCB or WCB Data</td></tr><tr><td>9</td><td>Single or Double Error from CCB or WCB Tags or *GW TAGs or DATA</td></tr><tr><td>17</td><td>Internal control register FF parity error</td></tr><tr><td>21</td><td>Poisoned data read from downstream, 0x08</td></tr><tr><td>26</td><td>Miscellaneous FF Parity Error</td></tr></table> All other values are reserved. Writes to this field are ignored.	2	Single or Double Error from RAMs that are not *GW, CCB, or WCB TAGS/DATA	8	Single or Double Error from CCB or WCB Data	9	Single or Double Error from CCB or WCB Tags or *GW TAGs or DATA	17	Internal control register FF parity error	21	Poisoned data read from downstream, 0x08	26	Miscellaneous FF Parity Error
2	Single or Double Error from RAMs that are not *GW, CCB, or WCB TAGS/DATA													
8	Single or Double Error from CCB or WCB Data													
9	Single or Double Error from CCB or WCB Tags or *GW TAGs or DATA													
17	Internal control register FF parity error													
21	Poisoned data read from downstream, 0x08													
26	Miscellaneous FF Parity Error													

4.8.4 TCU_ERRGEN register

Use the TCU Error Generation Register to test how the system responds when a RAS error occurs in the RAM.

The field locations are the same as for [4.8.3 TCU_ERRSTATUS register](#) on page 165.

When this register is updated, the TCU_ERRSTATUS register is also updated with the same value, as long as the write data generates a valid error record.

A write to the TCU_ERRGEN register is valid if all the following conditions are met:

- The TCU_ERRGEN.V bit is set.
- At least 1 of the following is true (CE is legal if CE == 0b00 or CE == 0b10):
 - The TCU_ERRGEN.UE bit is set and CE is legal
 - The TCU_ERRGEN.DE bit is set and CE is legal
 - The TCU_ERRGEN.CE bits are set to 0b10
- One of the following is true:
 - TCU_ERRGEN.UET == 0b00
 - TCU_ERRGEN.UET == 0b11 and UE == 1

The following also apply:

- All other fields can take any legal value.
- Any other write is considered to be invalid and the behavior of the SMMU is **UNPREDICTABLE**.
- If a valid error record is written, then the appropriate interrupt, or interrupts, are also generated.



This register has identical fields to the [4.8.3 TCU_ERRSTATUS register](#) on page 165.

Configurations

This register is available in all configurations.

Attributes

The TCU_ERRGEN register attributes are as follows:

Width

32-bit

Functional group

[4.4.3 TCU Reliability, Availability, and Serviceability registers summary](#) on page 132.

Address offset

0x60040

Type

Root, RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-14: TCU_ERRGEN register bit assignments

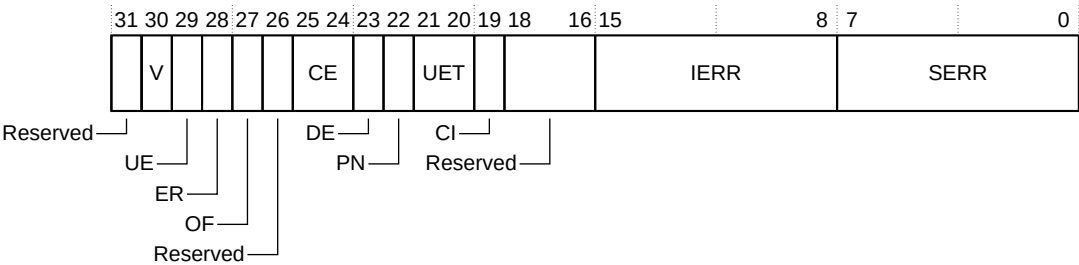


Table 4-40: TCU_ERRGEN register bit descriptions

Bits	Name	Description
[31]	-	Reserved
[30]	V	Status Register valid. The possible values of this bit are as follows: 0 ERRSTATUS is not valid 1 ERRSTATUS is valid. At least 1 error has been recorded. If any of the UE, DE, or CE bits are set to 1, and are not being cleared to 0 in the same write, direct writes to this bit are ignored. This bit is read/write-one-to-clear. This bit resets to 0 on a reset.

Bits	Name	Description
[29]	UE	Uncorrected error, or errors. The possible values of this bit are as follows: 0 No errors that could not be corrected or deferred 1 At least one error detected that has not been corrected or deferred If the OF bit is set to 1 and is not being cleared to zero in the same write, direct writes to this bit are ignored. This bit is read/write-one-to-clear.
[28]	ER	Error Reported. The possible values of this bit are as follows: 0 No in-band error (External abort) is reported. 1 The node to the requester making the access or other transaction signaled an External abort. This bit is read/write-one-to-clear.
[27]	OF	Overflow. Multiple errors are detected. This bit is set to 1 when: - Multiple errors are detected on the same cycle - A new error occurs when there is already a valid record in the register This bit is read/write-one-to-clear.
[26]	-	Reserved
[25:24]	CE	Correctable Error, or errors. 0b00 No correctable errors recorded. 0b10 At least one Corrected error recorded. Other values are Reserved. This field is cleared by writing 0b11 to it. If OF is set and not being cleared, the write is ignored. A write of any value other than 0b11 is ignored.
[23]	DE	Deferred error, or errors. The possible values of this bit are as follows: 0 No errors were deferred 1 At least one error was not corrected and deferred If the OF bit is set to 1 and is not being cleared to 0 in the same write, direct writes to this bit are ignored. This bit is read/write-one-to-clear.
[22]	PN	Poison. The possible values of this bit are as follows: 0 Uncorrected error or deferred error is recorded because a corrupt value was detected, for example, by an Error Detection Code (EDC) 1 Uncorrected error or deferred error is recorded because a poison value was detected This bit is read/write-one-to-clear.

Bits	Name	Description
[21:20]	UET	Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error. The possible values of this field are as follows: 0b00 Uncorrected error, Uncontainable error (UC) 0b11 Uncorrected error, Signaled or Recoverable error (UER) Accessing this field has the following behavior. This field is not valid and reads UNKNOWN if any of the following are true: - TCU_ERRSTATUS.V == 0b0 - TCU_ERRSTATUS.UE == 0b0 Otherwise, this field is read/write-ones-to-clear. Writing a value other than all-zeros or all-ones sets this field to an UNKNOWN value.
[19]	CI	Indicates whether a critical error condition has been recorded. The possible values of this bit are as follows: 0 No critical error condition 1 Critical error condition This field is read/write-one-to-clear.
[18:16]	-	Reserved
[15:8]	IERR	IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error: 80 PIU QTW FF PARITY 79 PIU PIB FF PARITY 78 PIU AGW GCB FF PARITY 77 PIU AGW GMB FF PARITY 76 PIU AGW FF PARITY 75 PIU INV FF PARITY 74 PIU CMD FF PARITY 73 PIU EVT FF PARITY 72 PIU PRI FF PARITY 71 PIU MGB FF PARITY 70 PIU CPB FF PARITY 69 PIU FAR FF PARITY 68 PIU PRG FF PARITY 67 PIU RAS FF PARITY 66 Reserved 65 TMU PMD FF PARITY 64 TMU PRG FF PARITY 63 TMU PMU FF PARITY 62 TMU WCB FF PARITY 61 TMU CCB FF PARITY Writes to this field are ignored.

Bits	Name	Description
[15:8]	IERR	IERR (continued)
		IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error:
		60 TMU HZU FF PARITY
		59 TMU PIB FF PARITY
		58 TMU DMD FF PARITY
		57 TMU AGW GMB FF PARITY
		56 TMU AGW GCB FF PARITY
		55 TMU AGW FF PARITY
		54 TMU DGW GMB FF PARITY
		53 TMU DGW GCB FF PARITY
		52 TMU DGW FF PARITY
		51 TMU TWB FF PARITY
		50-44 Reserved
		43 PIU AGW GMB WLK STATUS
		42 PIU AGW GMB LOOP/PIU_GMB_POISON
41 PIU AGW GCB TAGS+DATA		
		Writes to this field are ignored.
[15:8]	IERR	IERR (continued)
		IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error:
		40 PIU AGW GCB REPL
		39 PIU CMD RPOISON
		38 Reserved
		37 TMU TWB WMB POISON
		36 TMU TWB CMB POISON
		35 TMU AGW ID RAM PARITY
		34 TMU AGW STATUS RAM PARITY
		33 TMU AGW RESPONSE RAM PARITY
		32 TMU AGW GMB SCRATCH
		31 TMU AGW GMB WLK STATUS
		30 TMU AGW GMB LOOP/TMU_AGW_GMB_POISON
		29 TMU DGW RAM
		28 TMU DGW GMB SCRATCH
27 TMU DGW GMB WLK STATUS		
26 TMU DGW GMB LOOP/TMU_DGW_GMB_POISON		
25 TMU AGW GCB TAGS+DATA		
24 TMU DGW GCB TAGS+DATA		
23 TMU AGW GCB REPL		
22 TMU AGW GCB PCNT		
21 TMU AGW GCB PLIM		
		Writes to this field are ignored.

Bits	Name	Description																																										
[15:8]	IERR	IERR (continued) IMPLEMENTATION DEFINED error code. When SERR ≠ 0, the value of this field indicates the source of the error: <table><tr><td>20</td><td>TMU DGW GCB REPL</td></tr><tr><td>19</td><td>TMU DGW GCB PCNT</td></tr><tr><td>18</td><td>TMU DGW GCB PLIM</td></tr><tr><td>17</td><td>TMU CCB MCC DATA</td></tr><tr><td>16</td><td>TMU CCB MCC TAGS</td></tr><tr><td>15</td><td>TMU WCB MWC DATA</td></tr><tr><td>14</td><td>TMU WCB MWC TAGS</td></tr><tr><td>13</td><td>TMU CCB MCC REPL</td></tr><tr><td>12</td><td>TMU CCB MCC PCNT</td></tr><tr><td>11</td><td>TMU CCB MCC PLIM</td></tr><tr><td>10</td><td>TMU WCB MWC REPL</td></tr><tr><td>9</td><td>TMU WCB MWC PCNT</td></tr><tr><td>8</td><td>TMU WCB MWC PLIMH</td></tr><tr><td>7</td><td>TMU HZU SEC PTR</td></tr><tr><td>6</td><td>TMU HZU PTR</td></tr><tr><td>5</td><td>TMU HTTU RAM</td></tr><tr><td>4</td><td>TMU TWB WMB SCRATCH</td></tr><tr><td>3</td><td>TMU TWB WMB WLK STATUS</td></tr><tr><td>2</td><td>TMU TWB WMB LKP STATUS</td></tr><tr><td>1</td><td>Reserved</td></tr><tr><td>0</td><td>TMU TWB BSU</td></tr></table> Writes to this field are ignored.	20	TMU DGW GCB REPL	19	TMU DGW GCB PCNT	18	TMU DGW GCB PLIM	17	TMU CCB MCC DATA	16	TMU CCB MCC TAGS	15	TMU WCB MWC DATA	14	TMU WCB MWC TAGS	13	TMU CCB MCC REPL	12	TMU CCB MCC PCNT	11	TMU CCB MCC PLIM	10	TMU WCB MWC REPL	9	TMU WCB MWC PCNT	8	TMU WCB MWC PLIMH	7	TMU HZU SEC PTR	6	TMU HZU PTR	5	TMU HTTU RAM	4	TMU TWB WMB SCRATCH	3	TMU TWB WMB WLK STATUS	2	TMU TWB WMB LKP STATUS	1	Reserved	0	TMU TWB BSU
20	TMU DGW GCB REPL																																											
19	TMU DGW GCB PCNT																																											
18	TMU DGW GCB PLIM																																											
17	TMU CCB MCC DATA																																											
16	TMU CCB MCC TAGS																																											
15	TMU WCB MWC DATA																																											
14	TMU WCB MWC TAGS																																											
13	TMU CCB MCC REPL																																											
12	TMU CCB MCC PCNT																																											
11	TMU CCB MCC PLIM																																											
10	TMU WCB MWC REPL																																											
9	TMU WCB MWC PCNT																																											
8	TMU WCB MWC PLIMH																																											
7	TMU HZU SEC PTR																																											
6	TMU HZU PTR																																											
5	TMU HTTU RAM																																											
4	TMU TWB WMB SCRATCH																																											
3	TMU TWB WMB WLK STATUS																																											
2	TMU TWB WMB LKP STATUS																																											
1	Reserved																																											
0	TMU TWB BSU																																											
[7:0]	SERR	The error code provides information about the earliest unacknowledged error. It can contain the following values: <table><tr><td>2</td><td>Single or Double Error from RAMs that are not *GW, CCB, or WCB TAGS/DATA</td></tr><tr><td>8</td><td>Single or Double Error from CCB or WCB Data</td></tr><tr><td>9</td><td>Single or Double Error from CCB or WCB Tags or *GW TAGs or DATA</td></tr><tr><td>17</td><td>Internal control register FF parity error</td></tr><tr><td>21</td><td>Poisoned data read from downstream, 0x08</td></tr><tr><td>26</td><td>Miscellaneous FF Parity Error</td></tr></table> All other values are reserved. Writes to this field are ignored.	2	Single or Double Error from RAMs that are not *GW, CCB, or WCB TAGS/DATA	8	Single or Double Error from CCB or WCB Data	9	Single or Double Error from CCB or WCB Tags or *GW TAGs or DATA	17	Internal control register FF parity error	21	Poisoned data read from downstream, 0x08	26	Miscellaneous FF Parity Error																														
2	Single or Double Error from RAMs that are not *GW, CCB, or WCB TAGS/DATA																																											
8	Single or Double Error from CCB or WCB Data																																											
9	Single or Double Error from CCB or WCB Tags or *GW TAGs or DATA																																											
17	Internal control register FF parity error																																											
21	Poisoned data read from downstream, 0x08																																											
26	Miscellaneous FF Parity Error																																											

4.9 TCU system discovery registers

The MMU-720AE TCU contains system discovery registers.

4.9.1 TCU_SYSDISC0 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC0 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A000

Type

RO

Reset value

TCUCFG_WC_DEPTH. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-15: TCU_SYSDISC0 register bit assignments

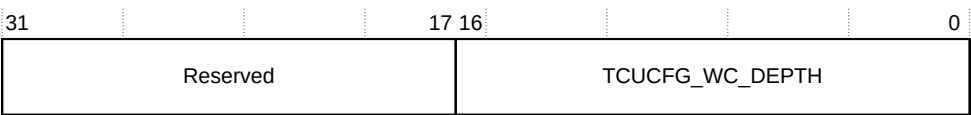


Table 4-41: TCU_SYSDISC0 register bit descriptions

Bits	Name	Description
[31:17]	-	Reserved
[16:0]	TCUCFG_WC_DEPTH	The read data reflects the chosen parameter value, for example: 17'h0_0008 : 8 17'h1_0000 : 65536

4.9.2 TCU_SYSDISC1 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC1 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A004

Type

RO

Reset value

TCUCFG_CC_DEPTH. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-16: TCU_SYSDISC1 register bit assignments



Table 4-42: TCU_SYSDISC1 register bit descriptions

Bits	Name	Description
[31:13]	-	Reserved
[12:0]	TCUCFG_CC_DEPTH	The read data reflects the chosen parameter value, for example: 13'h0004 : 4 13'h1000 : 4096

4.9.3 TCU_SYSDISC2 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC2 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A008

Type

RO

Reset value

TCUCFG_WC_WAYS. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-17: TCU_SYSDISC2 register bit assignments

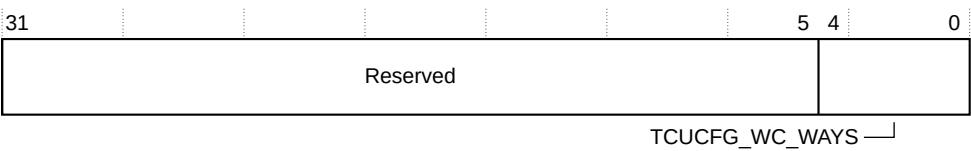


Table 4-43: TCU_SYSDISC2 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_WC_WAYS	The read data reflects the chosen parameter value, for example: 5'h04 : 4 5'h10 : 16

4.9.4 TCU_SYSDISC3 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC3 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A00C

Type

RO

Reset value

TCUCFG_WC_BANKS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-18: TCU_SYSDISC3 register bit assignments

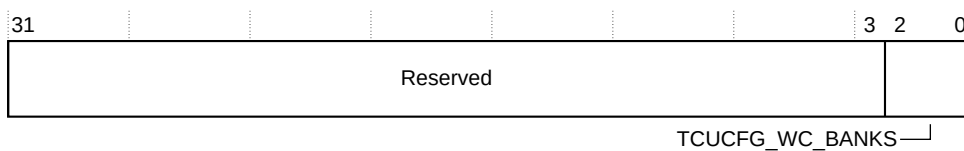


Table 4-44: TCU_SYSDISC3 register bit descriptions

Bits	Name	Description
[31:3]	-	Reserved
[2:0]	TCUCFG_WC_BANKS	The read data reflects the chosen parameter value, for example: 3'b001 : 1 3'b100 : 4

4.9.5 TCU_SYSDISC4 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC4 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A010

Type

RO

Reset value

TCUCFG_XLATE_SLOTS. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-19: TCU_SYSDISC4 register bit assignments



Table 4-45: TCU_SYSDISC4 register bit descriptions

Bits	Name	Description
[31:13]	-	Reserved
[12:0]	TCUCFG_XLATE_SLOTS	The read data reflects the chosen parameter value, for example: 13'h0004 : 4 13'h1000 : 4096

4.9.6 TCU_SYSDISC5 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC5 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A014

Type

RO

Reset value

TCUCFG_PTW_SLOTS. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-20: TCU_SYSDISC5 register bit assignments



Table 4-46: TCU_SYSDISC5 register bit descriptions

Bits	Name	Description
[31:10]	-	Reserved
[9:0]	TCUCFG_PTW_SLOTS	The read data reflects the chosen parameter value, for example: 10'h002 : 2 10'h200 : 512

4.9.7 TCU_SYSDISC6 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC6 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A018

Type

RO

Reset value

TCUCFG_CTW_SLOTS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-21: TCU_SYSDISC6 register bit assignments

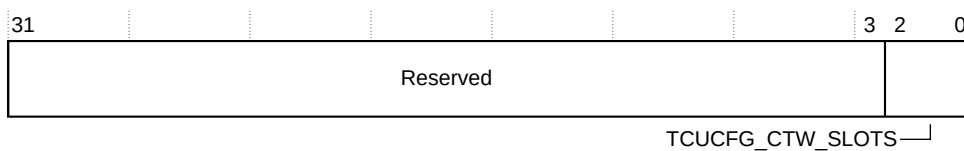


Table 4-47: TCU_SYSDISC6 register bit descriptions

Bits	Name	Description
[31:3]	-	Reserved
[2:0]	TCUCFG_CTW_SLOTS	The read data reflects the chosen parameter value, for example: 3'b001 : 1 3'b100 : 4

4.9.8 TCU_SYSDISC7 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC7 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A01C

Type

RO

Reset value

TCUCFG_CC_IDXGEN_MODE. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-22: TCU_SYSDISC7 register bit assignments

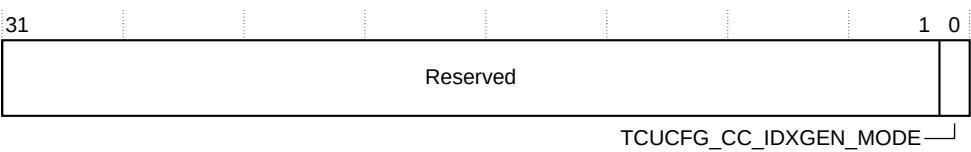


Table 4-48: TCU_SYSDISC7 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TCUCFG_CC_IDXGEN_MODE	The read data reflects the chosen parameter value, for example: 1'b0 : 0 1'b1 : 1

4.9.9 TCU_SYSDISC8 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC8 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A020

Type

RO

Reset value

TCUCFG_DTI_ATS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-23: TCU_SYSDISC8 register bit assignments

Table 4-49: TCU_SYSDISC8 register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved
[3:0]	TCUCFG_DTI_ATS	The read data reflects the chosen parameter value, for example: 4'b0000 : 0 4'b1000 : 8

4.9.10 TCU_SYSDISC9 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC9 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A024

Type

RO

Reset value

TCUCFG_NUM_TBU. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-24: TCU_SYSDISC9 register bit assignments

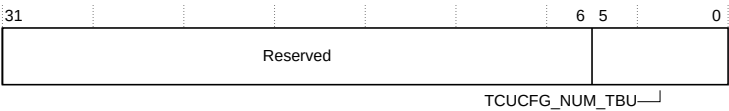


Table 4-50: TCU_SYSDISC9 register bit descriptions

Bits	Name	Description
[31:6]	-	Reserved
[5:0]	TCUCFG_NUM_TBU	The read data reflects the chosen parameter value, for example: 6'h01 : 1 6'h3E : 62 Note: Legal values are 14 and 62.

4.9.11 TCU_SYSDISC10 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC10 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A028

Type

RO

Reset value

TCUCFG_PMU_COUNTERS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-25: TCU_SYSDISC10 register bit assignments

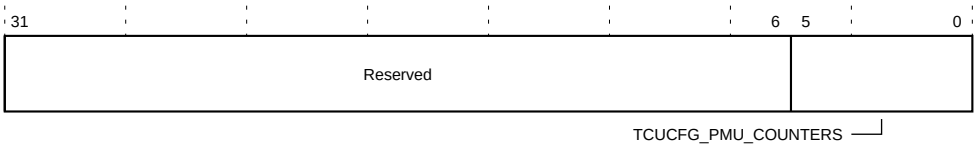


Table 4-51: TCU_SYSDISC10 register bit descriptions

Bits	Name	Description
[31:6]	-	Reserved
[5:0]	TCUCFG_PMU_COUNTERS	The read data reflects the chosen parameter value, for example: 6'h04 : 4 6'h20 : 32

4.9.12 TCU_SYSDISC11 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC11 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A02C

Type

RO

Reset value

TCUCFG_PARTID_WIDTH. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-26: TCU_SYSDISC11 register bit assignments

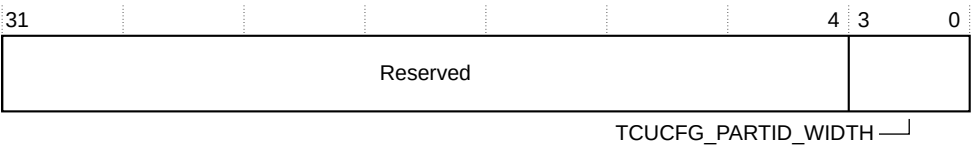


Table 4-52: TCU_SYSDISC11 register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved
[3:0]	TCUCFG_PARTID_WIDTH	The read data reflects the chosen parameter value, for example: 4'b0001 : 1 4'b0001 : 1 4'b1001 : 9

4.9.13 TCU_SYSDISC12 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC12 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A030

Type

RO

Reset value

TCUCFG_HZU_DEPTH. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-27: TCU_SYSDISC12 register bit assignments

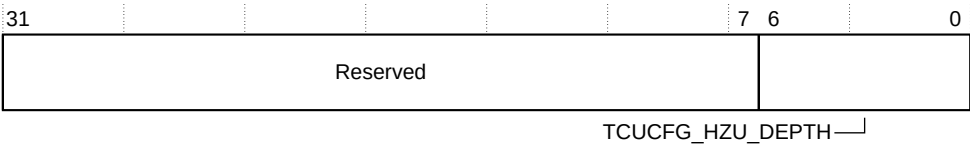


Table 4-53: TCU_SYSDISC12 register bit descriptions

Bits	Name	Description
[31:7]	-	Reserved
[6:0]	TCUCFG_HZU_DEPTH	The read data reflects the chosen parameter value, for example: 7'h02 : 2 7'h40 : 64

4.9.14 TCU_SYSDISC13 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC13 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A034

Type

RO

Reset value

TCUCFG_PREFETCH_SUPPORTED. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-28: TCU_SYSDISC13 register bit assignments

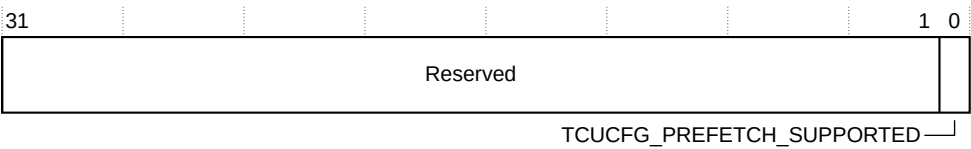


Table 4-54: TCU_SYSDISC13 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TCUCFG_PREFETCH_SUPPORTED	The read data reflects the chosen parameter value, for example: 1'b0 : 0 1'b1 : 1

4.9.15 TCU_SYSDISC14 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC14 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A038

Type

RO

Reset value

TCUCFG_DATARAM_TYPE. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-29: TCU_SYSDISC14 register bit assignments

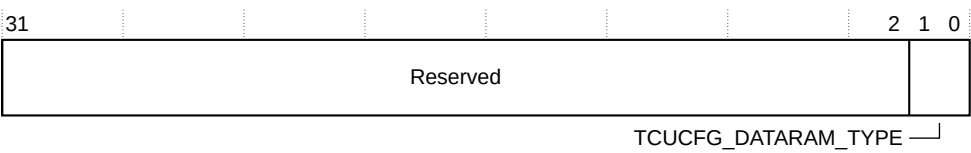


Table 4-55: TCU_SYSDISC14 register bit descriptions

Bits	Name	Description
[31:2]	-	Reserved
[1:0]	TCUCFG_DATARAM_TYPE	The read data reflects the chosen parameter value, for example: 2'b00 : 0 2'b10 : 2

4.9.16 TCU_SYSDISC15 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC15 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A03C

Type

RO

Reset value

TCUCFG_SLOTRAM_TYPE. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-30: TCU_SYSDISC15 register bit assignments

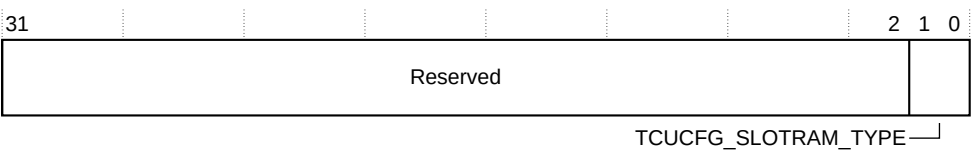


Table 4-56: TCU_SYSDISC15 register bit descriptions

Bits	Name	Description
[31:2]	-	Reserved
[1:0]	TCUCFG_SLOTRAM_TYPE	The read data reflects the chosen parameter value, for example: 2'b00 : 0 2'b01 : 1

4.9.17 TCU_SYSDISC16 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC16 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A040

Type

RO

Reset value

TCUCFG_CACHERAM_TYPE. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-31: TCU_SYSDISC16 register bit assignments

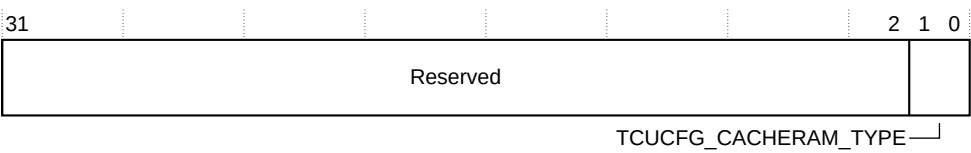


Table 4-57: TCU_SYSDISC16 register bit descriptions

Bits	Name	Description
[31:2]	-	Reserved
[1:0]	TCUCFG_CACHERAM_TYPE	The read data reflects the chosen parameter value, for example: 2'b00 : 0 2'b01 : 1

4.9.18 TCU_SYSDISC17 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC17 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A044

Type

RO

Reset value

TCUCFG_QTW_DATA_WIDTH. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-32: TCU_SYSDISC17 register bit assignments

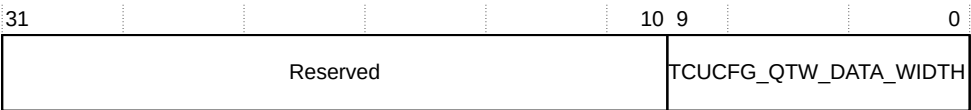


Table 4-58: TCU_SYSDISC17 register bit descriptions

Bits	Name	Description
[31:10]	-	Reserved
[9:0]	TCUCFG_QTW_DATA_WIDTH	The read data reflects the chosen parameter value, for example: 10'h040 : 64 10'h200 : 512

4.9.19 TCU_SYSDISC18 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC18 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A048

Type

RO

Reset value

TCUCFG_AGW_GC_DEPTH. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-33: TCU_SYSDISC18 register bit assignments

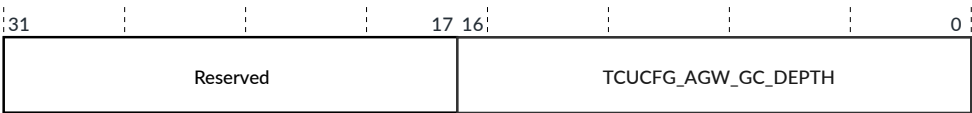


Table 4-59: TCU_SYSDISC18 register bit descriptions

Bits	Name	Description
[31:17]	-	Reserved
[16:0]	TCUCFG_AGW_GC_DEPTH	The read data reflects the chosen parameter value

4.9.20 TCU_SYSDISC19 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC19 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A04C

Type

RO

Reset value

TCUCFG_AGW_GC_WAYS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-34: TCU_SYSDISC19 register bit assignments

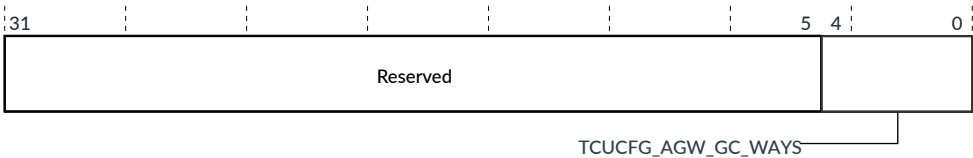


Table 4-60: TCU_SYSDISC19 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_AGW_GC_WAYS	The read data reflects the chosen parameter value

4.9.21 TCU_SYSDISC20 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC20 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A050

Type

RO

Reset value

TCUCFG_AGW_GC_BANKS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-35: TCU_SYSDISC20 register bit assignments

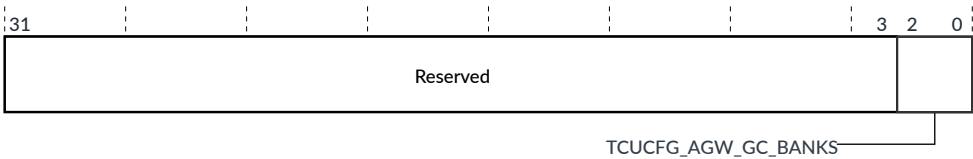


Table 4-61: TCU_SYSDISC20 register bit descriptions

Bits	Name	Description
[31:3]	-	Reserved
[2:0]	TCUCFG_AGW_GC_BANKS	The read data reflects the chosen parameter value

4.9.22 TCU_SYSDISC21 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC21 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A054

Type

RO

Reset value

TCUCFG_DGW_GC_DEPTH. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-36: TCU_SYSDISC21 register bit assignments

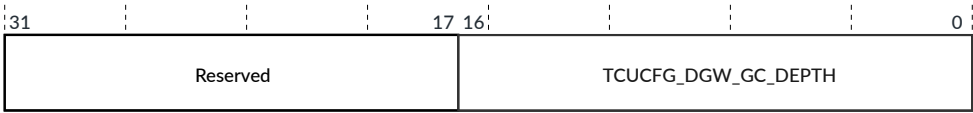


Table 4-62: TCU_SYSDISC21 register bit descriptions

Bits	Name	Description
[31:17]	-	Reserved
[16:0]	TCUCFG_DGW_GC_DEPTH	The read data reflects the chosen parameter value

4.9.23 TCU_SYSDISC22 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC22 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A058

Type

RO

Reset value

TCUCFG_DGW_GC_WAYS. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-37: TCU_SYSDISC22 register bit assignments

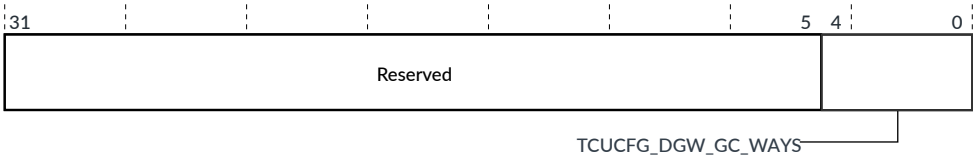


Table 4-63: TCU_SYSDISC22 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_DGW_GC_WAYS	The read data reflects the chosen parameter value

4.9.24 TCU_SYSDISC23 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC23 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A05C

Type

RO

Reset value

TCUCFG_DGW_GC_BANKS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-38: TCU_SYSDISC23 register bit assignments

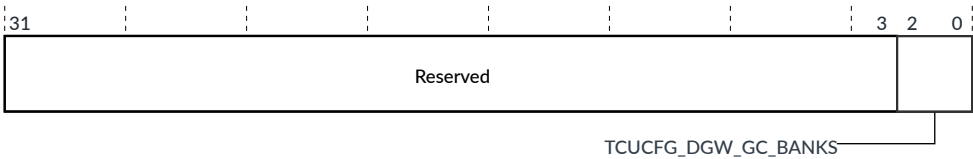


Table 4-64: TCU_SYSDISC23 register bit descriptions

Bits	Name	Description
[31:3]	-	Reserved
[2:0]	TCUCFG_DGW_GC_BANKS	The read data reflects the chosen parameter value

4.9.25 TCU_SYSDISC24 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC24 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A060

Type

RO

Reset value

TCUCFG_DGW_SLOTS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-39: TCU_SYSDISC24 register bit assignments

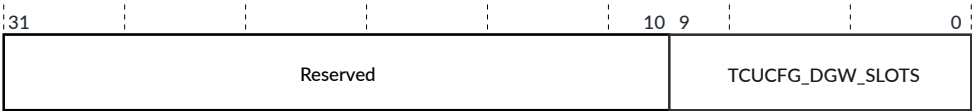


Table 4-65: TCU_SYSDISC24 register bit descriptions

Bits	Name	Description
[31:10]	-	Reserved
[9:0]	TCUCFG_DGW_SLOTS	The read data reflects the chosen parameter value

4.9.26 TCU_SYSDISC25 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC25 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A064

Type

RO

Reset value

TCUCFG_DTI_ATS_INV_MAX. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-40: TCU_SYSDISC25 register bit assignments

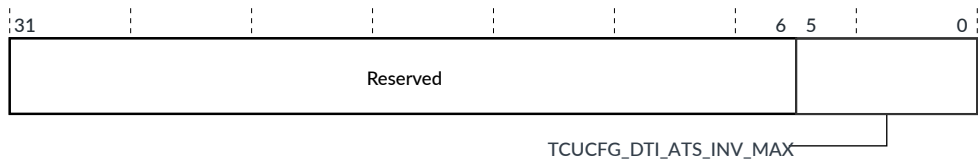


Table 4-66: TCU_SYSDISC25 register bit descriptions

Bits	Name	Description
[31:6]	-	Reserved
[5:0]	TCUCFG_DTI_ATS_INV_MAX	The read data reflects the chosen parameter value

4.9.27 TCU_SYSDISC26 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC26 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A068

Type

RO

Reset value

TCUCFG_WC_LKP_SLOTS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Table 4-67: TCU_SYSDISC26 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_WC_LKP_SLOTS	The read data reflects the chosen parameter value

4.9.28 TCU_SYSDISC27 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC27 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A06C

Type

RO

Reset value

TCUCFG_AGW_GC_LKP_SLOTS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Table 4-68: TCU_SYSDISC27 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_AGW_GC_LKP_SLOTS	The read data reflects the chosen parameter value

4.9.29 TCU_SYSDISC28 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC28 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A070

Type

RO

Reset value

TCUCFG_DGW_GC_LKP_SLOTS. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-41: TCU_SYSDISC28 register bit assignments

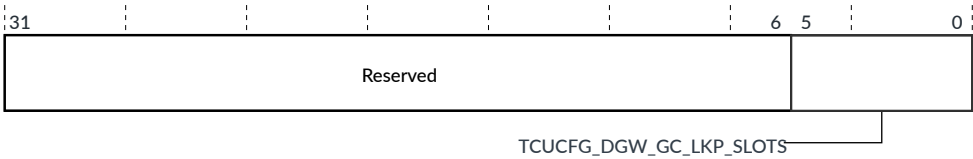


Table 4-69: TCU_SYSDISC28 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_DGW_GC_LKP_SLOTS	The read data reflects the chosen parameter value

4.9.30 TCU_SYSDISC29 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC29 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A074

Type

RO

Reset value

TCUCFG_LEGACY_TZ_EN. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-42: TCU_SYSDISC29 register bit assignments

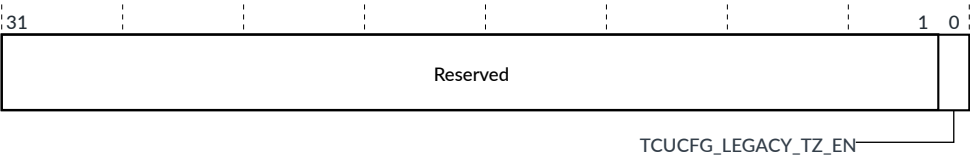


Table 4-70: TCU_SYSDISC29 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TCUCFG_LEGACY_TZ_EN	The read data reflects the chosen parameter value

4.9.31 TCU_SYSDISC30 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC30 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A078

Type

RO

Reset value

TCUCFG_USE_ELA_DEBUG. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-43: TCU_SYSDISC30 register bit assignments

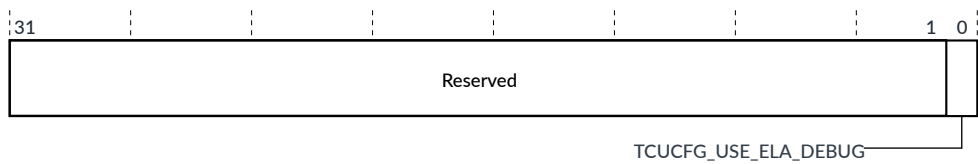


Table 4-71: TCU_SYSDISC30 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TCUCFG_USE_ELA_DEBUG	The read data reflects the chosen parameter value

4.9.32 TCU_SYSDISC31 System Discovery Register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC31 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A07C

Type

RO

Reset value

TCUCFG_PARITY_ON. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-44: TCU_SYSDISC31 register bit assignments

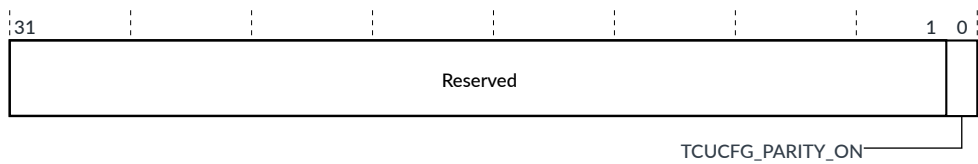


Table 4-72: TCU_SYSDISC31 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TCUCFG_PARITY_ON	The read data reflects the chosen parameter value

4.9.33 TCU_SYSDISC32 System Discovery Register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC32 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A080

Type

RO

Reset value

TCUCFG_CC_WAYS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-45: TCU_SYSDISC32 register bit assignments

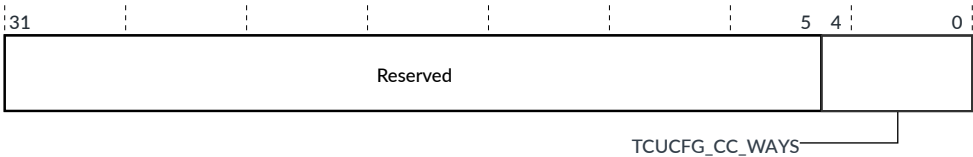


Table 4-73: TCU_SYSDISC32 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_CC_WAYS	The read data reflects the chosen parameter value

4.9.34 TCU_SYSDISC33 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC33 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A084

Type

RO

Reset value

TCUCFG_DVM_VAS. See [2.7.1 TCU I/O and buffer configuration parameters](#) on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-46: TCU_SYSDISC33 register bit assignments

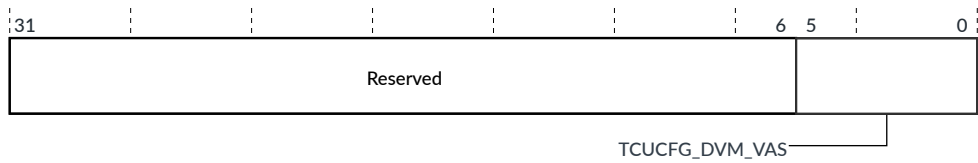


Table 4-74: TCU_SYSDISC33 register bit descriptions

Bits	Name	Description
[31:6]	-	Reserved
[5:0]	TCUCFG_DVM_VAS	The read data reflects the chosen parameter value

4.9.35 TCU_SYSDISC34 System Discovery Register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC34 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A088

Type

RO

Reset value

TCUCFG_MECID_WIDTH. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-47: TCU_SYSDISC34 register bit assignments

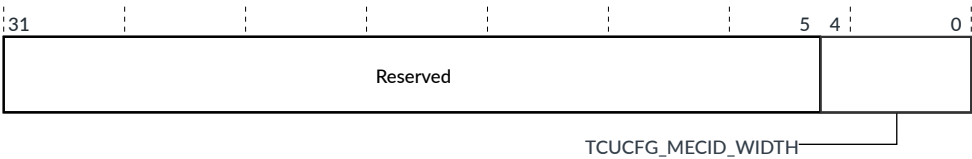


Table 4-75: TCU_SYSDISC34 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TCUCFG_MECID_WIDTH	The read data reflects the chosen parameter value

4.9.36 TCU_SYSDISC35 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC35 register attributes are as follows:

Width

32-bit

Functional group

4.4.5 TCU system discovery registers summary on page 133.

Address offset

0x2A08C

Type

RO

Reset value

TCU_X2 (2 port variant).

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-48: TCU_SYSDISC35 register bit assignments

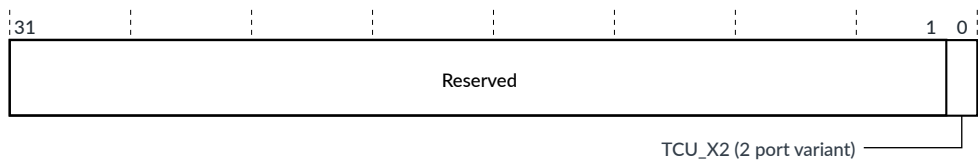


Table 4-76: TCU_SYSDISC35 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TCU_X2 (2 port variant)	The read data reflects the chosen value

4.9.37 TCU_SYSDISC36 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC36 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A090

Type

RO

Reset value

msi_addr[31:0].

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-49: TCU_SYSDISC36 register bit assignments

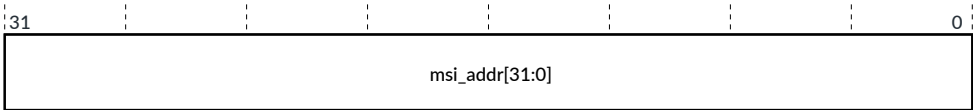


Table 4-77: TCU_SYSDISC36 register bit descriptions

Bits	Name	Description
[31:0]	msi_addr[31:0]	The read data reflects the chosen value.

4.9.38 TCU_SYSDISC37 system discovery register

The TCU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TCU_SYSDISC37 register attributes are as follows:

Width

32-bit

Functional group

[4.4.5 TCU system discovery registers summary](#) on page 133.

Address offset

0x2A094

Type

RO

Reset value

msi_addr[51:32].

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-50: TCU_SYSDISC37 register bit assignments

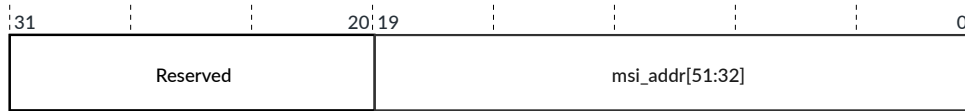


Table 4-78: TCU_SYSDISC37 register bit descriptions

Bits	Name	Description
[31:20]	-	Reserved.
[19:0]	msi_addr[51:32]	The read data reflects the chosen value.

4.10 TCU_CTRL_AUX<n> registers

The TCU Control registers, AUX<n>, where <n> can have a value of 0-55, disable TCU features. Do not modify the AUX bits unless we instruct you to do so.

Configurations

This register is available in all configurations.

Attributes

TCU_CTRL_AUX<n> register attributes are as follows:

Width

32-bit

Functional group

[4.4.6 TCU AUX registers summary](#) on page 135

Address offset

```
0x
0x2A100-0x2A348
```

Type

RW

Reset value

See the reset value in the table in [4.4.6 TCU AUX registers summary](#) on page 135.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-51: TCU_CTRL_AUX<n> register bit assignments

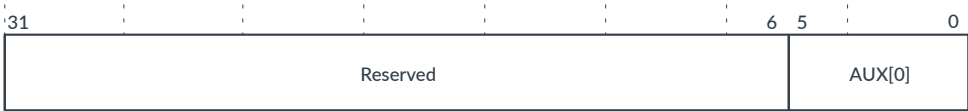


Table 4-79: TCU_CTRL_AUX<n> register bit descriptions

Bits	Name	Description
[31:6]	-	Reserved
[5:0]	AUX[0]	Reserved. Do not change these bits unless Arm advises you to.

4.11 TCU integration registers

The MMU-720AE TCU contains TCU integration registers.

4.11.1 ITEN register for the TCU

Integration mode register for the TCU. When integration mode is enabled, the values of certain TCU input pins are made visible in the ITIN register for the TCU. The values that are written to the ITOP for the TCU control the values of certain TCU output pins. This mechanism helps system integrators to integrate the SMMU into the system and perform basic connectivity checks.

See also:

- [4.13.2 ITIN_TMU register for the TCU Translation Management Unit](#) on page 220
- [4.13.1 ITOP_TMU register for the TCU Translation Management Unit](#) on page 219

Configurations

This register is available in all configurations.

Attributes

The ITEN register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E20

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-52: ITEN register bit assignments

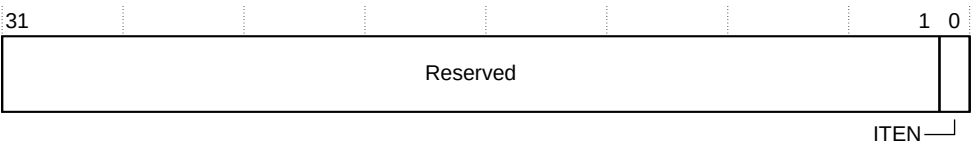


Table 4-80: ITEN register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	ITEN	0 Disable integration mode 1 Enable integration mode

4.12 TCU PIU integration registers

The MMU-720AE TCU contains Programmer Interface Unit (PIU) integration registers.

4.12.1 ITOP_PIU register for the TCU Programmer Interface Unit

The ITOP_PIU register is the integration output register for the TCU Programmer Interface Unit (PIU).

Configurations

This register is available in all configurations.

Attributes

The ITOP_PIU register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E24

Type

RW

Reset value

0

Bit descriptions

The following table shows the bit descriptions.

Table 4-81: ITOP_PIU register bit descriptions

Bits	Name	Description
[31:21]	-	Reserved, Should-Be-Zero (SBZ)
[20]	gpt_cfg_far	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[19]	gpf_far	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[18]	eventq_irpt_r	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[17]	pri_q_irpt_r	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301

Bits	Name	Description
[16]	cmd_sync_irpt_r	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[15]	global_irpt_r	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[14]	event_q_irpt_ns	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[13]	event_q_irpt_s	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[12]	pri_q_irpt_ns	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301

Bits	Name	Description
[11]	cmd_sync_irpt_ns	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[10]	cmd_sync_irpt_s	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[9]	global_irpt_ns	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[8]	global_irpt_s	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[7]	eventoreq	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301

Bits	Name	Description
[6]	ras_lt_irpt_v	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[5]	ras_lt_fhi	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[4]	ras_lt_eri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[3]	ras_lt_cri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[2]	ras_et_fhi	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301

Bits	Name	Description
[1]	ras_et_eri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301
[0]	ras_et_cri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301

4.12.2 ITIN_PIU register for the TCU Programmer Interface Unit

The ITIN_PIU register is the integration input register for the TCU Programmer Interface Unit (PIU).

Configurations

This register is available in all configurations.

Attributes

The ITIN register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E28

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-53: ITIN_PIU register bit assignments

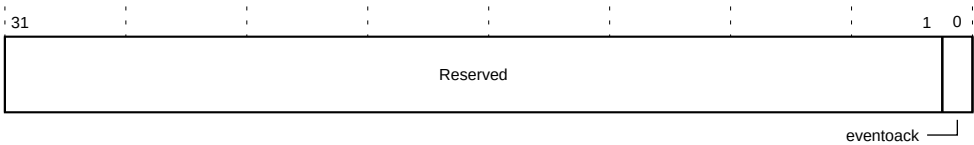


Table 4-82: ITIN_PIU register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved, Should-Be-Zero (SBZ)
[0]	eventoack	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value reflects the value of the eventoack signal See: 4.11.1 ITEN register for the TCU on page 212 A.1.9 TCU interrupt signals on page 299 A.1.11 TCU event interface on page 301

4.13 TCU TMU integration registers

The MMU-720AE TCU contains Translation Management Unit (TMU) integration registers.

4.13.1 ITOP_TMU register for the TCU Translation Management Unit

The MMU-720AE TCU contains the integration output, ITOP_TMU register for the TCU Translation Management Unit (TMU).

Configurations

This register is available in all configurations.

Attributes

The ITOP register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E2C

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-54: ITOP_TMU register bit assignments

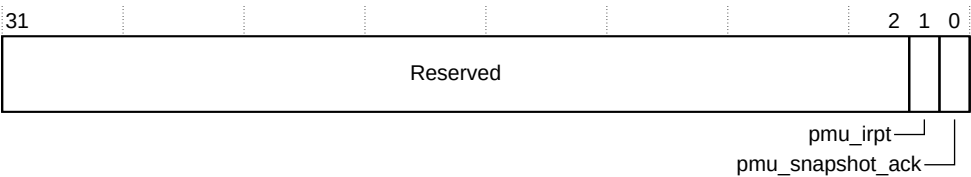


Table 4-83: ITOP_TMU register bit descriptions

Bits	Name	Description
[31:2]	-	Reserved, Should-Be-Zero (SBZ)
[1]	pmu_irpt	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified. See: 4.11.1 ITEN register for the TCU on page 212 A.3.2 PMU Snapshot signals on page 324
[0]	pmu_snapshot_ack	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified. See: 4.11.1 ITEN register for the TCU on page 212 A.3.2 PMU Snapshot signals on page 324

4.13.2 ITIN_TMU register for the TCU Translation Management Unit

The MMU-720AE TCU contains an integration input, ITIN_TMU register for the TCU Translation Management Unit (TMU).

Configurations

This register is available in all configurations.

Attributes

The ITIN register attributes are as follows:

Width

32-bit

Functional group

[4.4.4 TCU microarchitectural registers summary](#) on page 132.

Address offset

0x28E30

Type

RO

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-55: ITIN_TMU register bit assignments

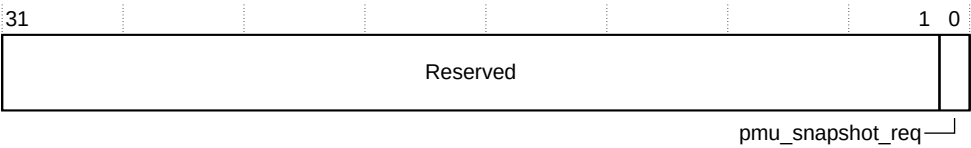


Table 4-84: ITIN_TMU register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved, Should-Be-Zero (SBZ)
[0]	pmu_snapshot_req	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1, depending on the value of the input signal that is specified See: 4.11.1 ITEN register for the TCU on page 212 A.3.2 PMU Snapshot signals on page 324

4.14 TBU component and peripheral ID registers

The MMU-720AE TBU contains component and peripheral ID registers.

The following table shows the TBU component and peripheral ID.

Table 4-85: TBU component and peripheral ID registers

Offset	Name	Field	Value	Description
0x00FFC	SMMU_CIDR3, Component ID3	[7:0]	0xB1	Preamble

Offset	Name	Field	Value	Description
0x00FF8	SMMU_CIDR2, Component ID2	[7:0]	0x05	Preamble
0x00FF4	SMMU_CIDR1, Component ID1	[7:0]	0xF0	Preamble
0x00FF0	SMMU_CIDR0, Component ID0	[7:0]	0x0D	Preamble
0x00FEC	SMMU_PIDR3, Peripheral ID3	[7:4]	MAX(p_level, ecorevnum)	REVAND, minor revision. Where p_level is 1 for p1.
	SMMU_PIDR3, Peripheral ID3	[3:0]	0x00	CMOD
0x00FE8	SMMU_PIDR2, Peripheral ID2	[7:4]	0x01	REVISION, major revision
	SMMU_PIDR2, Peripheral ID2	[3]	1	JEDEC-assigned value for DES always used
	SMMU_PIDR2, Peripheral ID2	[2:0]	3	DES_1: bits [6:4] bits of the JEP106 Designer code
0x00FE4	SMMU_PIDR1, Peripheral ID1	[7:4]	0xB	DES_0: bits [3:0] of the JEP106 Designer code
	SMMU_PIDR1, Peripheral ID1	[3:0]	0x4	PART_1: bits [11:8] of the Part number
0x00FE0	SMMU_PIDR0, Peripheral ID0	[7:0]	0x9F	PART_0: bits [7:0] of the Part number
0x00FDC	SMMU_PIDR7, Peripheral ID7	-	RES0	Reserved
0x00FD8	SMMU_PIDR6, Peripheral ID6			
0x00FD4	SMMU_PIDR5, Peripheral ID5			
0x00FD0	SMMU_PIDR4, Peripheral ID4	[7:4]	0x0	SIZE = 4KB
		[3:0]	0x4	DES_2: JEP106 Designer continuation code

4.15 TBU PMU registers

The MMU-720AE TBU contains Performance Monitor Unit (PMU) registers.

The TBU PMU registers follow the register layout that the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) Performance Monitor Extension describes.

4.15.1 Registers

The TBU and TCU support the same PMCG registers.

See [4.6 TCU PMU registers](#) on page 140.

4.15.2 Events

Each event indicates whether the SMMU_PMCG_SMRO register can filter it.

For events that cannot be filtered, you can set whether they are visible only when Secure events are visible by setting the following fields:

In RME mode

SMMU_PMCG_ROOTCR.NAO&(SMMU_PMCG_SCR.SO|SMMU_PMCG_SCR.NAO)

In legacy mode

SMMU_PMCG_SCR.SO

For more information, see the following:

- [2.5.2 Performance Monitoring Unit](#) on page 47
- [2.5.2.2 SMMU TCU events](#) on page 48

4.15.3 SMMU_PMCG_CFGR

An MMU-720AE implementation assumes fixed values for SMMU_PMCG_CFGR, and these values define behavioral aspects of the implementation.

For information about the SMMU_PMCG_CFGR fields values, see [2.5.2.4 SMMUv3 PMU register architectural options](#) on page 55.

See also [2.7 Configuration parameters and methodology](#) on page 102.

4.15.4 SMMU_PMCG_CEID{0-1} registers

The SMMU_PMCG_CEID{0-1} registers indicate the architectural events that are supported. They are described as 64-bit registers, but they are accessed 32 bits at a time through the 32-bit DTI register access messages.

The following table shows the SMMU_PMCG_CEID{0-1} registers.

Table 4-86: SMMU_PMCG_CEID{0-1} registers

Offset	Name	Value
0x20E20	SMMU_PMCG_CEID0	0x00000087
0x20E28	SMMU_PMCG_CEID1	0x00000000

4.15.5 PMU ID registers

The PMU ID registers appear only in Performance Monitor Page 0. Page 1 does not contain any ID registers.

The following table shows the PMU ID registers.

Table 4-87: PMU ID registers

Offset	Name	Field	Value	Description
0x20FFC	SMMU_PMCG_CIDR3, Component ID3	[7:0]	0xB1	Preamble
0x20FF8	SMMU_PMCG_CIDR2, Component ID2	[7:0]	0x05	Preamble
0x20FF4	SMMU_PMCG_CIDR1, Component ID1	[7:0]	0x90	Preamble
0x20FF0	SMMU_PMCG_CIDR0, Component ID0	[7:0]	0x0D	Preamble

Offset	Name	Field	Value	Description
0x20FDC	SMMU_PMC_G_PIDR7, Peripheral ID7	-	RES0	Reserved
0x20FD8	SMMU_PMC_G_PIDR6, Peripheral ID6	-	RES0	Reserved
0x20FD4	SMMU_PMC_G_PIDR5, Peripheral ID5	-	RES0	Reserved
0x20FD0	SMMU_PMC_G_PIDR4, Peripheral ID4	[7:4]	0x0	SIZE = 4KB
		[3:0]	0x4	DES_2: JEP106 Designer continuation code
0x20FEC	SMMU_PMC_G_PIDR3, Peripheral ID3	[7:4]	MAX(p_level, ecorevnum)	REVAND, minor revision, where p_level is 1 for p1.
	SMMU_PMC_G_PIDR3, Peripheral ID3	[3:0]	0x00	CMOD
0x20FE8	SMMU_PMC_G_PIDR2, Peripheral ID2	[7:4]	0x00	REVISION, major revision
	SMMU_PMC_G_PIDR2, Peripheral ID2	[3]	1	JEDEC-assigned value for DES always used
	SMMU_PMC_G_PIDR2, Peripheral ID2	[2:0]	3	DES_1: bits [6:4] bits of the JEP106 Designer code
0x20FE4	SMMU_PMC_G_PIDR1, Peripheral ID1	[7:4]	0xB	DES_0: bits [3:0] of the JEP106 Designer code
	SMMU_PMC_G_PIDR1, Peripheral ID1	[3:0]	0x4	PART_1: bits [11:8] of the Part number
0x20FE0	SMMU_PMC_G_PIDR0, Peripheral ID0	[7:0]	0x9F	PART_0: bits [7:0] of the Part number
0x20FB8	SMMU_PMC_G_PMAUTHSTATUS	[7:0]	0x00	No authentication interface is implemented

The PMAUTHSTATUS, PMDEVARCH, and PMDEVTYPE registers are implemented as the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#) defines.

4.16 TBU microarchitectural registers

You can set the microarchitectural registers at boot time to optimize TBU behavior for your system. We recommend that you use the default values for most systems.

Non-secure access to the Secure-only [4.16.3 TBU_SCR register](#) on page 229 is RAZ/WI.

Secure and Non-secure accesses to microarchitectural registers, other than the [4.16.3 TBU_SCR register](#) on page 229 are RAZ/WI except for when affected by fields in the [4.16.5 TBU_RCR register](#) on page 231 and [4.16.3 TBU_SCR register](#) on page 229.

The following table shows how SCR and RCR override the ownership of a register.

Table 4-88: SCR and RCR override of register ownership

SCR.feature	RCR.feature	Non-secure register	Secure register	Realm register	Root register
0	0	Non-secure	Secure	Realm	Root
0	1	Non-secure	Secure	Realm	Secure
1	0	Non-secure	Non-secure	Realm	Root
1	1	Non-secure	Non-secure	Realm	Non-secure

4.16.1 TBU_CTRL register

The TBU_CTRL register disables TBU features. Do not modify the bits in this register unless Arm® instructs you to do so.

Configurations

This register is available in all configurations.

Attributes

The TBU_CTRL register attributes are as follows:

Width

32-bit

Functional group

[4.4.9 TBU microarchitectural registers summary](#) on page 137.

Address offset

0x08E00

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-56: TBU_CTRL register bit assignments

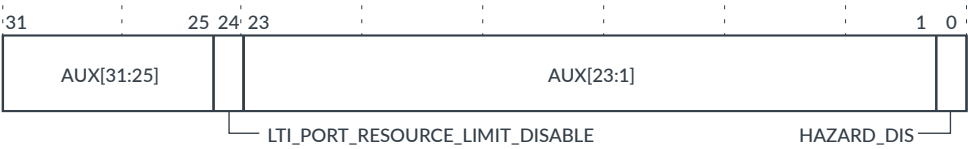


Table 4-89: TBU_CTRL register bit descriptions

Bits	Name	Description
[31:25]	AUX[31:25]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.

Bits	Name	Description
[24]	LTI_PORT_RESOURCE_LIMIT_DISABLE	This bit is used only when more than one LTI port is configured. An ACE-Lite TBU contains only one LTI port internally. 0 When 0, the 4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register on page 226 is used to control how the LTI ports use translation slots and DTI credits. 1 When 1, all LTI ports are permitted to use the maximum resource. To avoid deadlock and starvation avoidance, some slots and DTI credits are reserved for each LTI port regardless of the value of this register field and the 4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register on page 226. Therefore, if multiple LTI ports are present, it is not possible for any individual port to use all the slots or DTI credits.
[23:1]	AUX[23:1]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[0]	HAZARD_DIS	0 When this bit is clear, and multiple outstanding transactions access the same page, the TBU sends a single translation request and uses that for all the affected transactions. 1 When this bit is set, disables hazarding between translation requests from transactions in the same page. Post reset, this bit can be set to 1 once, but cannot be cleared again without a reset.

4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register

The TBU_LTI_PORT_RESOURCE_LIMIT register limits the resource usage for each LTI port.

Non-secure access to TBU_LTI_PORT_RESOURCE_LIMIT when TBU_SCR.NS_UARCH = 0 is RAZ/WI. See [4.16.3 TBU_SCR register](#) on page 229. Each 4-bit field of this register indicates the resource usage limit fraction for the LTI port number that is indicated.

If the current usage is greater than or equal to the specified fraction of total resource, an LTI port is not permitted to use extra resource.

This allocation affects the resources as follows:

- Translation slots, the total number of which the `TBUCFG_XLATE_SLOTS` parameter provides, that transactions from that LTI port can occupy. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.
- DTI translation tokens that the TCU returns in the `CONDIS_ACK` message. A transaction is considered to use a DTI translation credit from the point that it is accepted into a translation slot until it is known that it no longer requires a DTI request, other than a retry. This includes no longer requiring a translation because it has already performed one.



Note

This is different from occupancy of a translation slot because a translation that has received a cache hit, or otherwise determines that it will not require a DTI translation, is not counted in this fraction. Similarly, a transaction that has

received a DTI response, but has not returned the LR is no longer considered to use a DTI credit.

The usage of the port of each of the controlled resources is tracked separately because their usage varies separately, but the limit applies equally to all of them.

The register value expresses the number of sixteenths of the available resource that can be used. Therefore, the values encode to the fractions that the following table shows.

Table 4-90: Value encodings

Register value	Fraction
4'b0000	0
4'b0001	1/16
4'b0010	1/8
4'b0011	3/16
4'b0100	1/4
4'b0101	5/16
4'b0110	3/8
4'b0111	7/16
4'b1000	1/2
4'b1001	9/16
4'b1010	5/8
4'b1011	11/16
4'b1100	3/4
4'b1101	13/16
4'b1110	7/8
4'b1111	15/16

The greater-than-or-equal-to constraint permits a fractional credit to be used when the register fraction value multiplied by the total resource available is not a whole number.

If the sum of the allocated resources is more than 1, then a port might not achieve its maximum allocated resources. Resources are allocated on a first case first served basis for that LTI port.

To guarantee freedom from starvation and deadlock, the TLBU must receive at least $(2 \times \text{NUM_LTI_PORTS})$ DTI translation request tokens.

The minimum value of tokens that is required is considered to be part of the usage fraction when in use, but those 2 credits are not available to any other port when not in use. Two credits must be reserved for each port to use. The reserved credits are included in the computation of whether extra resource can be used.

If the minimum allocation, 2, is greater than the fractional allocation, the limit is 2.

Combining the fractional maximum and per-port reservation requirements means that the maximum number of translation slots that transactions can occupy from a given port is:

$$\min(\max(2, (\text{TBUCFG_XLATE_SLOTS} * \text{lti_port_resource_limit})), (\text{TBUCFG_XLATE_SLOTS} - (2 * (\text{NUM_LTI_PORTS} - 1))))$$



If the TBUCFG.LTI_PORT_RESOURCE_LIMIT_DISABLE register field is set to 1, this register value does not limit the usage per port. Instead, only the availability of resource once other reservations of ports are considered limits usage.

When only one port is present, this register is RAZ/WI but is treated internally as 4'hF. This register has no effect on behavior because its effective value permits the single port to use all the available credits.

Configurations

This register is available in all configurations.

Attributes

The TBU_LTI_PORT_RESOURCE_LIMIT register attributes are as follows:

Width

32-bit

Functional group

[4.4.9 TBU microarchitectural registers summary](#) on page 137.

Address offset

0x08E04

Type

RW

Reset value

4'hF

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-57: TBU_LTI_PORT_RESOURCE_LIMIT register bit assignments

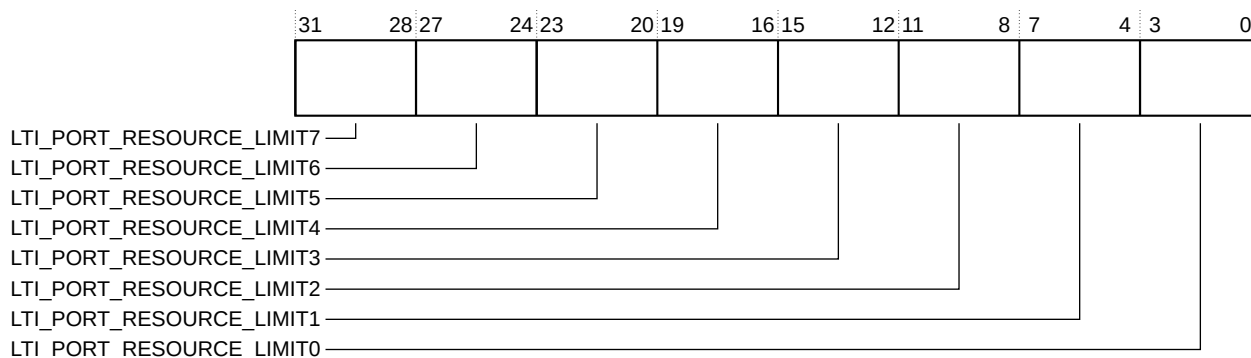


Table 4-91: TBU_LTI_PORT_RESOURCE_LIMIT bit descriptions

Bits	Name	Description
[31:28]	LTI_PORT_RESOURCE_LIMIT7	Resource limit for LTI Port 7
[27:24]	LTI_PORT_RESOURCE_LIMIT6	Resource limit for LTI Port 6
[23:20]	LTI_PORT_RESOURCE_LIMIT5	Resource limit for LTI Port 5
[19:16]	LTI_PORT_RESOURCE_LIMIT4	Resource limit for LTI Port 4
[15:12]	LTI_PORT_RESOURCE_LIMIT3	Resource limit for LTI Port 3
[11:8]	LTI_PORT_RESOURCE_LIMIT2	Resource limit for LTI Port 2
[7:4]	LTI_PORT_RESOURCE_LIMIT1	Resource limit for LTI Port 1
[3:0]	LTI_PORT_RESOURCE_LIMIT0	Resource limit for LTI Port 0

4.16.3 TBU_SCR register

The TBU Secure Control register, TBU_SCR, enables you to control Non-secure and realm register access.

The TBU_SCR register also affects [4.4.10 TBU system discovery registers summary](#) on page 138 in the same way.

Configurations

This register is available in all configurations.

Attributes

The TBU_SCR register attributes are as follows:

Width

32-bit

Functional group

[4.4.9 TBU microarchitectural registers summary](#) on page 137.

Address offset

0x08E18

Type

RW

Reset value

sec_override signal. See [A.2.9 TBU tie-off signals](#) on page 320.

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-58: TBU_SCR register bit assignments

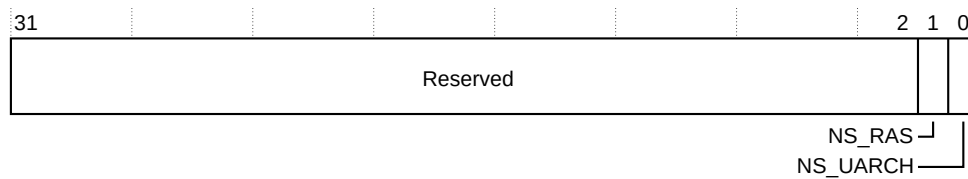


Table 4-92: TBU_SCR register bit descriptions

Bits	Name	Description
[31:2]	-	Reserved
[1]	NS_RAS	Non-secure and realm register access that is permitted for RAS registers. When this bit is 0, Non-secure writes to the following register addresses are ignored, and Non-secure reads return zero: 0x1_0000-0x1_FFFC The sec_override input sets the reset value of this signal. See A.2.9 TBU tie-off signals on page 320.
[0]	NS_UARCH	Non-secure register and realm access that is permitted for the following registers: 4.16.1 TBU_CTRL register on page 224 4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register on page 226 4.19.1 ITEN register for the TBU on page 266 4.19.2 ITOP register for the TBU on page 267 4.19.3 ITIN register for the TBU on page 270 When this bit is 0, Non-secure writes are ignored, and Non-secure reads return zero for the following registers: 4.16.1 TBU_CTRL register on page 224 4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register on page 226 4.19.1 ITEN register for the TBU on page 266 4.19.2 ITOP register for the TBU on page 267 4.19.3 ITIN register for the TBU on page 270 The TBU_SCR register also affects the 4.16.4 TBU_ROOT_CTRL register on page 231 register if TBU_RCR.S_ROOT_UARCH = 1. See 4.16.5 TBU_RCR register on page 231. The sec_override input sets the reset value of this signal. See A.2.9 TBU tie-off signals on page 320. If Secure translation might be used, we recommend that software does not set this bit.

4.16.4 TBU_ROOT_CTRL register

The TBU_ROOT_CTRL register disables TBU features. In most systems, you can leave the TBU_ROOT_CTRL register unchanged.

The TBU_ROOT_CTRL register is root only by default. It can be made accessible by Secure or Non-secure by setting various combinations of the [4.16.3 TBU_SCR register](#) on page 229 and the [4.16.5 TBU_RCR register](#) on page 231.

Configurations

This register is available in all configurations.

Attributes

The TBU_ROOT_CTRL register attributes are as follows:

Width

32-bit

Functional group

[4.4.9 TBU microarchitectural registers summary](#) on page 137.

Address offset

0x08F00

Bit descriptions

The following figure and table show the bit assignments and descriptions.



Table 4-93: TBU_ROOT_CTRL bit descriptions

Bits	Name	Description
[31:1]	AUX[31:1]	Reserved. Do not change these bits unless Arm advises you to. If writing other bits of this register, ensure that you write the current value for these bits to them, for example, by performing a read-modify-write sequence.
[0]	PARITY_DIS	Ignore any parity errors from registers with parity bits.

4.16.5 TBU_RCR register

Use the TBU_RCR register to control security for the TBU.

The TBU_RCR register also affects [4.18 TBU system discovery registers](#) on page 242 and [4.19 TBU integration registers](#) on page 265 in the same way.

If the TBU_CFG_LEGACY_TZ_EN parameter is set to 1 or the legacy_tz_en signal is set to 1, all bits in this register behave as 1 and the TBU_RCR register is RAZ/WI.

Configurations

This register is available in all configurations.

Attributes

The TBU_RCR register attributes are as follows:

Width

32-bit

Functional group

[4.4.9 TBU microarchitectural registers summary](#) on page 137.

Address offset

0x08F10

Type

RW

Reset value

sec_override. See [A.2.9 TBU tie-off signals](#) on page 320.

Bit descriptions

The following figure and table show the bit assignments and descriptions.

Figure 4-59: TBU_RCR register bit assignments



Table 4-94: TBU_RCR register bit descriptions

Bits	Name	Description
[31:3]	-	Reserved

Bits	Name	Description
[2]	S_ROOT_UARCH	Non-root register access permitted for the following root-specific microarchitectural registers: 4.16.2 TBU_LTI_PORT_RESOURCE_LIMIT register on page 226 4.19.1 ITEN register for the TBU on page 266 4.19.2 ITOP register for the TBU on page 267 4.19.3 ITIN register for the TBU on page 270 4.16.4 TBU_ROOT_CTRL register on page 231 Reset is 0.
[1]	S_RAS	Secure register access permitted for microarchitectural registers. When this bit is 0, non-root writes to the following register addresses are ignored, and non-root reads return zero: 0x1_0000-0x1_FFFC Reset is 1.
[0]	S_UARCH	Secure register access permitted for 4.16.1 TBU_CTRL register on page 224. When this bit is 0, non-root writes to TBU_CTRL are ignored, and non-root reads return zero. Reset is 1. Note: The 4.18 TBU system discovery registers on page 242 are also affected

4.17 TBU RAS registers

The MMU-720AE TBU contains Reliability, Availability, and Serviceability (RAS) registers. The RAS registers implement the RAS Extension registers, single record format.

Non-secure accesses to these registers, when TBU_SCR.NS_RAS = 0, are RAZ/WI.

The RAS registers enable software to monitor the following classes of error:

- Corrected Errors (CEs) in the RAMs that the Main TLB uses
- CEs in the RAMs, that the Write Data Buffer uses

RAS error reporting

When a CE occurs:

A CE is reported in 4.17.3 TBU_ERRSTATUS register on page 236.

If TBU_ERRCTLR.FI is set, an interrupt is raised on ras_fhi. See 2.4.2.7 TBU interrupt interface on page 41.

4.17.1 TBU_ERRFR register

Error Feature Register.

Configurations

This register is available in all configurations.

Attributes

The TBU_ERRFR register attributes are as follows:

Width

32-bit

Functional group

[4.4.8 TBU Reliability, Availability, and Serviceability registers summary](#) on page 137.

Address offset

0x10000

Type

Root, RO

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-60: TBU_ERRFR register bit assignments

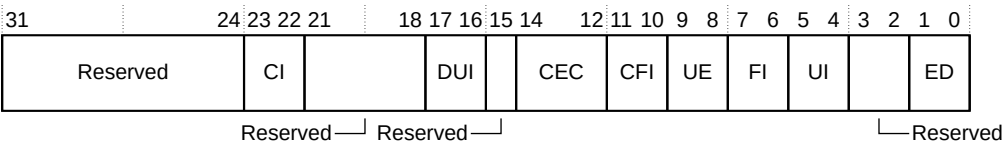


Table 4-95: TBU_ERRFR register bit descriptions

Bits	Name	Description
[31:24]	-	Reserved
[23:22]	CI	Critical Error Interrupt is always enabled. Value is 0b01.
[21:18]	-	Reserved
[17:16]	DUI	Does not support feature. Value is 0b00.
[15]	-	Reserved
[14:12]	CEC	Does not implement the standard corrected error counter model. Value is 0b000.
[11:10]	CFI	Does not support feature. Value is 0b00.
[9:8]	UE	In-band error signaling feature is not enabled. Value is 0b00.

Bits	Name	Description
[7:6]	FI	Fault handling interrupt is controllable. Value is 0b10.
[5:4]	UI	Error Recovery Interrupt always enabled for UE. Value is 0b01.
[3:2]	-	Reserved
[1:0]	ED	Error detection is always enabled. Value is 0b01.

4.17.2 TBU_ERRCTLR register

To enable fault handling interrupts, use the TBU Error Control register.

Configurations

This register is available in all configurations.

Attributes

The TBU_ERRCTLR register attributes are as follows:

Width

32-bit

Functional group

[4.4.8 TBU Reliability, Availability, and Serviceability registers summary](#) on page 137.

Address offset

0x10008

Type

Root, RW

Reset value

1

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-61: TBU_ERRCTLR register bit assignments

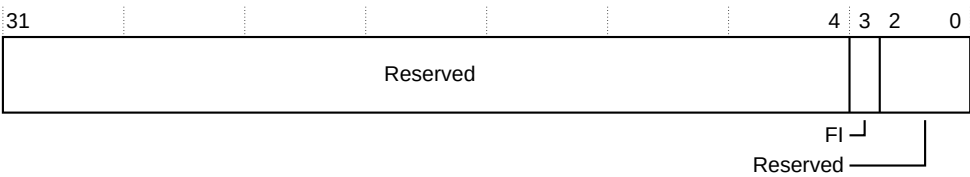


Table 4-96: TBU_ERRCTLR register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved
[3]	FI	Fault handling interrupt enable
[2:0]	-	Reserved

4.17.3 TBU_ERRSTATUS register

To find out whether different types of error have occurred, use the TBU error status register. Certain bits in this register are cleared by writing a 1 to their bit position. These writes are ignored in certain circumstances to avoid race conditions where a new error has occurred that software has not yet observed.

Configurations

This register is available in all configurations.

Attributes

The TBU_ERRSTATUS register attributes are as follows:

Width

32-bit

Functional group

[4.4.8 TBU Reliability, Availability, and Serviceability registers summary](#) on page 137.

Address offset

0x10010

Type

Root, RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-62: TBU_ERRSTATUS register bit assignments

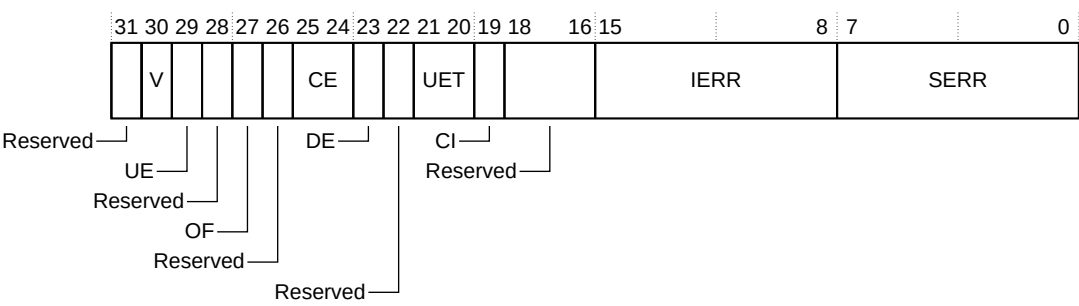


Table 4-97: TBU_ERRSTATUS register bit descriptions

Bits	Name	Description
[31]	-	Reserved

Bits	Name	Description
[30]	V	The status Register is valid. The possible values of this bit are as follows: 0 ERRSTATUS is not valid. 1 ERRSTATUS is valid, meaning that at least 1 error has been recorded. This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120. This bit resets to 0 on a reset.
[29]	UE	Uncorrected errors. The possible values of this bit are: 0 No errors that could not be corrected or deferred. 1 At least one error is detected that has not been corrected or deferred. This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[28]	-	Reserved
[27]	OF	Overflow. Multiple errors detected. This bit is set to 1 when: - Any error is received and TBU_ERRSTATUS.V is already set, and not being cleared on the same cycle - Multiple errors are received on the same cycle This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[26]	-	Reserved
[25:24]	CE	Correctable Errors: 0b00 No correctable errors recorded 0b10 At least one corrected error recorded Other values are Reserved. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[23]	DE	Deferred errors. The possible values of this bit are as follows: 0 No errors were deferred 1 At least one error was not corrected and deferred This error is raised when the Bus Interface Unit (BIU) sets wpoison. This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[22]	-	Reserved
[21:20]	UET	Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error. The possible values of this field are as follows: 0b00 Uncorrected error, UnContainable error (UC) Writes to this field are ignored.

Bits	Name	Description
[19]	CI	Indicates whether a critical error condition has been recorded. The possible values of this bit are as follows: 0 No critical error condition 1 Critical error condition Writes to this field are ignored.
[18:16]	-	Reserved
[15:8]	IERR	IMPLEMENTATION DEFINED error code. The value of this field indicates the source of the error as follows: 32 RRB FF PARITY 31 PRG FF PARITY 30 PMU FF PARITY 29 BIU CPB FF PARITY 28 BIU MIB FF PARITY 27 BIU SIB FF PARITY 26 DIB FF PARITY 25 DCU TLB FF PARITY 24 DCU DCB FF PARITY 23 TOU FF PARITY 22 Reserved 21 Reserved 20 BIU WDB ROBUFF_P 19 BIU WDB ROBUFF_C 18 BIU WDB ROBUFF_D 17 Reserved 16 Reserved 15 TLBU DCU MTLB TAGS 14 TLBU DCU MTLB REPL 13 TLBU DCU MTLB PCNT 12 TLBU DCU MTLB PLIM 11 TLBU TOU HLB_ENTRY RIGHT 10 TLBU TOU HLB_ENTRY LEFT 9 TLBU TOU HLB_PTR RIGHT 8 TLBU TOU HLB_PTR LEFT 7 Reserved 6 Reserved 5 TLBU TOU DTIQ 4 TLBU TOU UOQ 3 TLBU TOU OGQ 2 TLBU TOU LB 1 TLBU TOU RSP 0 TLBU TOU REQ Writes to this field are ignored.

Bits	Name	Description												
[7:0]	SERR	Error code. This provides information about the earliest unacknowledged Error. It can contain the following values: <table><tr><td>0</td><td>No error</td></tr><tr><td>2</td><td>Single or double error from RAMs that are not MTLB TAGS or DATA</td></tr><tr><td>8</td><td>Single or double error from MTLB Data</td></tr><tr><td>9</td><td>Single or double error from MTLB Tags</td></tr><tr><td>17</td><td>Internal control register FF parity</td></tr><tr><td>26</td><td>Miscellaneous FF Parity Error</td></tr></table> All other values are reserved. Writes to this field are ignored.	0	No error	2	Single or double error from RAMs that are not MTLB TAGS or DATA	8	Single or double error from MTLB Data	9	Single or double error from MTLB Tags	17	Internal control register FF parity	26	Miscellaneous FF Parity Error
0	No error													
2	Single or double error from RAMs that are not MTLB TAGS or DATA													
8	Single or double error from MTLB Data													
9	Single or double error from MTLB Tags													
17	Internal control register FF parity													
26	Miscellaneous FF Parity Error													

4.17.4 TBU_ERRGEN register

To test how the system responds when a RAS error occurs, use the Error Generation Register, TBU_ERRGEN.

The field locations are same as for the [4.17.3 TBU_ERRSTATUS register](#) on page 236.

When this register is updated, the [4.17.3 TBU_ERRSTATUS register](#) on page 236 is also updated with the same value, as long as the write data generates a valid error record.

A write to the TBU_ERRGEN register is valid if all the following conditions are met:

- The TBU_ERRGEN.V bit is set.
- At least 1 of the following is true (TBU_ERRGEN.CE is legal if CE == 2'b00 or CE == 2'b10):
 - The TBU_ERRGEN.UE bit is set and the CE bits have legal values
 - The TBU_ERRGEN.DE bit is set and the CE bits have legal values
 - The TBU_ERRGEN.CE bits are set to 0b10
- The TBU_ERRGEN.UET bits must be set to 0b00

The following also apply:

- All other fields can take any legal value.
- Any other write is considered to be invalid and the behavior of the SMMU is **UNPREDICTABLE**.
- If a valid error record is written, then the appropriate interrupt, or interrupts, are also generated.



Tip

This register has identical fields to the [4.17.3 TBU_ERRSTATUS register](#) on page 236.

Configurations

This register is available in all configurations.

Attributes

The TBU_ERRGEN register attributes are as follows:

Width

32-bit

Functional group

[4.4.8 TBU Reliability, Availability, and Serviceability registers summary](#) on page 137.

Address offset

0x10040

Type

Root, RW

Reset value

0

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-63: TBU_ERRGEN register bit assignments

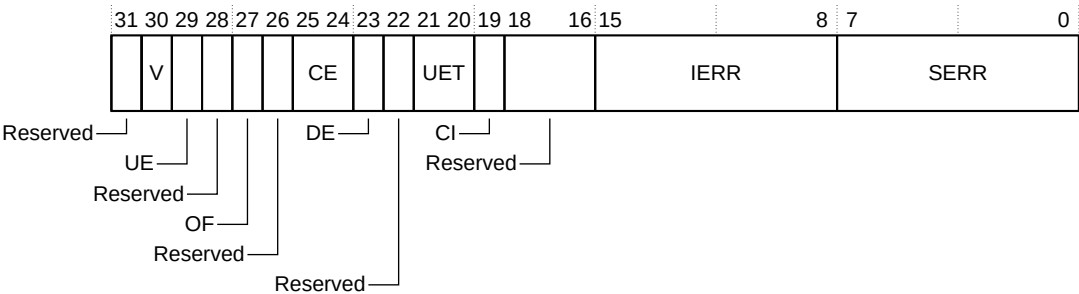


Table 4-98: TBU_ERRGEN register bit descriptions

Bits	Name	Description
[31]	-	Reserved
[30]	V	The status Register is valid. The possible values of this bit are as follows: 0 ERRSTATUS is not valid. 1 ERRSTATUS is valid, meaning that at least 1 error has been recorded. This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120. This bit resets to zero on a reset.

Bits	Name	Description
[29]	UE	Uncorrected errors. The possible values of this bit are: 0 No errors that could not be corrected or deferred 1 At least one error is detected that has not been corrected or deferred This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[28]	-	Reserved
[27]	OF	Overflow. Multiple errors detected. This bit is set to 1 when: - Any error is received and TBU_ERRSTATUS.V is already set, and not being cleared on the same cycle - Multiple errors are received on the same cycle This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[26]	-	Reserved
[25:24]	CE	Correctable Errors: 0b00 No correctable errors recorded 0b10 At least one corrected error recorded Other values are Reserved. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[23]	DE	Deferred errors. The possible values of this bit are as follows: 0 No errors were deferred. 1 At least 1 error was not corrected and deferred. This field is read/write-one-to-clear. Clearing depends on other ERRSTATUS fields. See 4.1 Clearing ERRSTATUS registers on page 120.
[22]	-	Reserved
[21:20]	UET	Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error. The possible values of this field are as follows: 0b00 Uncorrected error, UnContainable error (UC) Writes to this field are ignored.
[19]	CI	Indicates whether a critical error condition has been recorded. The possible values of this bit are as follows: 0 No critical error condition 1 Critical error condition Writes to this field are ignored.
[18:16]	-	Reserved

Bits	Name	Description
[15:8]	IERR	IMPLEMENTATION DEFINED error code. The value of this field indicates the source of the error as follows:
		32 RRB FF PARITY
		31 PRG FF PARITY
		30 PMU FF PARITY
		29 BIU CPB FF PARITY
		28 BIU MIB FF PARITY
		27 BIU SIB FF PARITY
		26 DIB FF PARITY
		25 DCU TLB FF PARITY
		24 DCU DCB FF PARITY
		23 TOU FF PARITY
		22 Reserved
		21 Reserved
		20 BIU WDB ROBUFF_P
		19 BIU WDB ROBUFF_C
		18 BIU WDB ROBUFF_D
		17 Reserved
		16 Reserved
		15 TLBU DCU MTLB TAGS
		14 TLBU DCU MTLB REPL
		13 TLBU DCU MTLB PCNT
		12 TLBU DCU MTLB PLIM
		11 TLBU TOU HLB_ENTRY RIGHT
		10 TLBU TOU HLB_ENTRY LEFT
		9 TLBU TOU HLB PTR RIGHT
		8 TLBU TOU HLB PTR LEFT
		7 Reserved
		6 Reserved
		5 TLBU TOU DTIQ
		4 TLBU TOU UOQ
		3 TLBU TOU OGQ
		2 TLBU TOU LB
		1 TLBU TOU RSP
0 TLBU TOU REQ		
		Writes to this field are ignored.
[7:0]	SERR	Error code. The error code provides information about the earliest unacknowledged Error and can contain the following values:
		0 No error
		2 Single or double error from RAMs that are not MTLB TAGS or DATA
		8 Single or double error from MTLB Data
		9 Single or double error from MTLB Tags
		17 Internal control register FF parity
		26 Miscellaneous FF Parity Error

4.18 TBU system discovery registers

The MMU-720AE TBU contains system discovery registers.

4.18.1 TBU_SYSDISC0 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC0 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09000

Type

RO

Reset value

TBUCFG_MTLB_DEPTH. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-64: TBU_SYSDISC0 register bit assignments

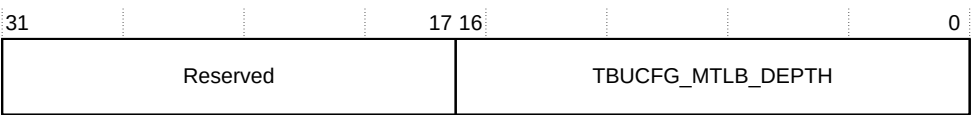


Table 4-99: TBU_SYSDISC0 register bit descriptions

Bits	Name	Description
[31:17]	-	Reserved

Bits	Name	Description
[16:0]	TBUCFG_MTLB_DEPTH	The read data reflects the chosen parameter value, for example: 17'h0_0008 : 8 17'h1_0000 : 65536

4.18.2 TBU_SYSDISC1 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC1 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09004

Type

RO

Reset value

TBUCFG_UTLB_DEPTH. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-65: TBU_SYSDISC1 register bit assignments

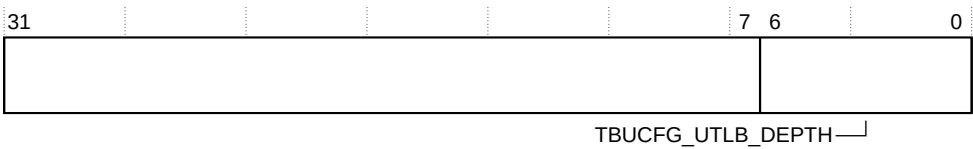


Table 4-100: TBU_SYSDISC1 register bit descriptions

Bits	Name	Description
[31:7]	-	Reserved

Bits	Name	Description
[4:0]	TBUCFG_MTLB_WAYS	The read data reflects the chosen parameter value, for example: 5'h04 : 4 5'h10 : 16

4.18.4 TBU_SYSDISC3 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC3 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x0900C

Type

RO

Reset value

TBUCFG_MTLB_BANKS. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-67: TBU_SYSDISC3 register bit assignments

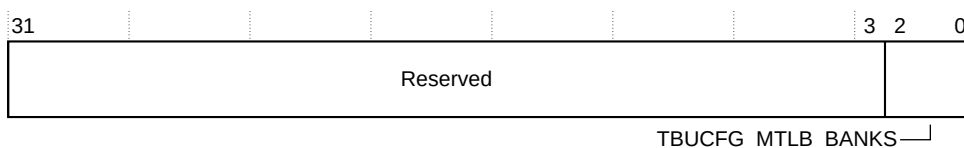


Table 4-102: TBU_SYSDISC3 register bit descriptions

Bits	Name	Description
[31:3]	-	Reserved
[2:0]	TBUCFG_MTLB_BANKS	The read data reflects the chosen parameter value, for example: 3'b001 : 1 3'b100 : 4

4.18.5 TBU_SYSDISC4 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC4 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09010

Type

RO

Reset value

TBUCFG_XLATE_SLOTS. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-68: TBU_SYSDISC4 register bit assignments

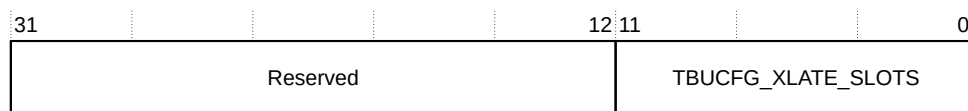


Table 4-103: TBU_SYSDISC4 register bit descriptions

Bits	Name	Description
[31:12]	-	Reserved

Bits	Name	Description
[5:0]	TBUCFG_PMU_COUNTERS	The read data reflects the chosen parameter value, for example: 6'h04 : 4 6'h20 : 32

4.18.7 TBU_SYSDISC6 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC6 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09018

Type

RO

Reset value

TBUCFG_SID_WIDTH. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-70: TBU_SYSDISC6 register bit assignments

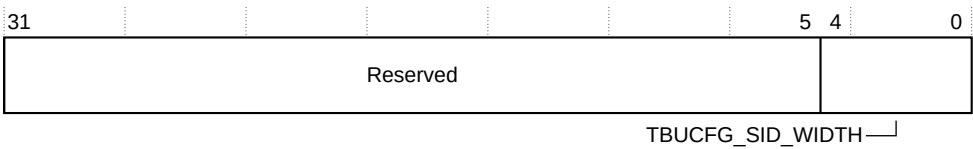


Table 4-105: TBU_SYSDISC6 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved

Bits	Name	Description
[4:0]	TBUCFG_SID_WIDTH	The read data reflects the chosen parameter value, for example: 5'h08 : 8 5'h18 : 24

4.18.8 TBU_SYSDISC7 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC7 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x0901C

Type

RO

Reset value

TBUCFG_SSID_WIDTH. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-71: TBU_SYSDISC7 register bit assignments

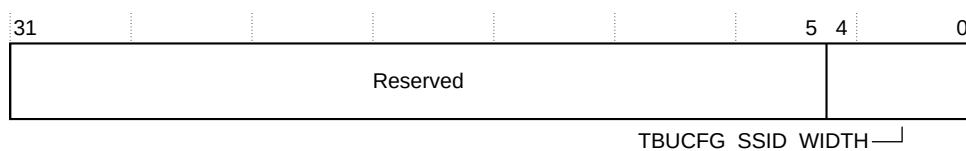


Table 4-106: TBU_SYSDISC7 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved

Bits	Name	Description
[4:0]	TBUCFG_SSID_WIDTH	The read data reflects the chosen parameter value, for example: 5'h01 : 1 5'h14 : 20

4.18.9 TBU_SYSDISC8 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC8 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09020

Type

RO

Reset value

TBUCFG_DIRECT_IDX. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-72: TBU_SYSDISC8 register bit assignments

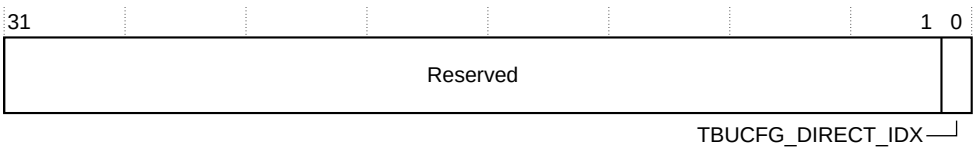


Table 4-107: TBU_SYSDISC8 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved

Bits	Name	Description
[0]	TBUCFG_DIRECT_IDX	The read data reflects the chosen parameter value, for example: 1'b0 : 0 1'b1 : 1

4.18.10 TBU_SYSDISC9 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC9 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09024

Type

RO

Reset value

TBUCFG_MTLB_PARTS. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-73: TBU_SYSDISC9 register bit assignments

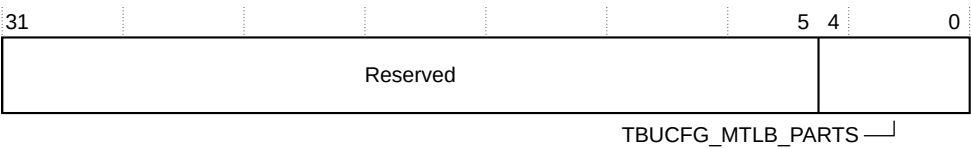


Table 4-108: TBU_SYSDISC9 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved

Bits	Name	Description
[4:0]	TBUCFG_MTLB_PARTS	The read data reflects the chosen parameter value, for example: 5'h00 : 0 5'h10 : 16

4.18.11 TBU_SYSDISC10 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC10 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09028

Type

RO

Reset value

TBUCFG_LTI_OG_WIDTH. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-74: TBU_SYSDISC10 register bit assignments

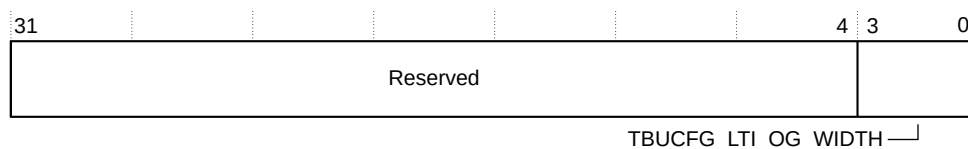


Table 4-109: TBU_SYSDISC10 register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved

Bits	Name	Description
[3:0]	TBUCFG_LTI_OG_WIDTH	The read data reflects the chosen parameter value, for example: 3'h00 : 0 3'h20 : 05

4.18.12 TBU_SYSDISC11 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC11 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x0902C

Type

RO

Reset value

TBUCFG_PARTID_WIDTH. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-75: TBU_SYSDISC11 register bit assignments

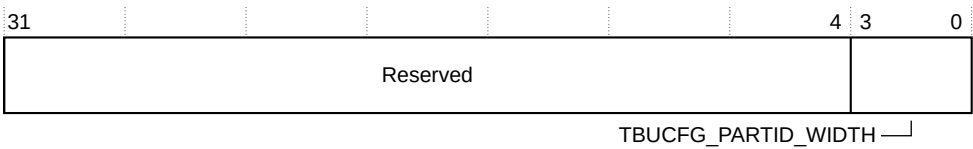


Table 4-110: TBU_SYSDISC11 register bit descriptions

Bits	Name	Description
[31:4]	-	Reserved

Bits	Name	Description
[3:0]	TBUCFG_PARTID_WIDTH	The read data reflects the chosen parameter value, for example: 4'b0001 : 1 4'b1001 : 9

4.18.13 TBU_SYSDISC12 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC12 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09030

Type

RO

Reset value

TBUCFG_HZRD_ENTRIES. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-76: TBU_SYSDISC12 register bit assignments

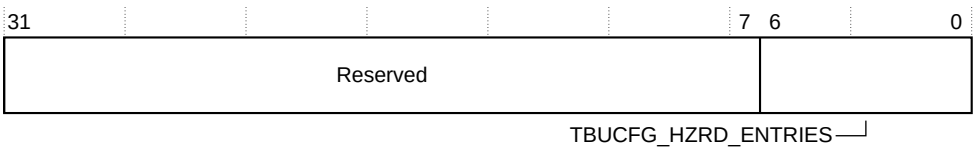


Table 4-111: TBU_SYSDISC12 register bit descriptions

Bits	Name	Description
[31:7]	-	Reserved

Bits	Name	Description
[6:0]	TBUCFG_HZRD_ENTRIES	The read data reflects the chosen parameter value, for example: 7'h00 : 0 7'h40 : 64

4.18.14 TBU_SYSDISC13 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC13 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09034

Type

RO

Reset value

TBUCFG_SLOTRAM_TYPE. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-77: TBU_SYSDISC13 register bit assignments

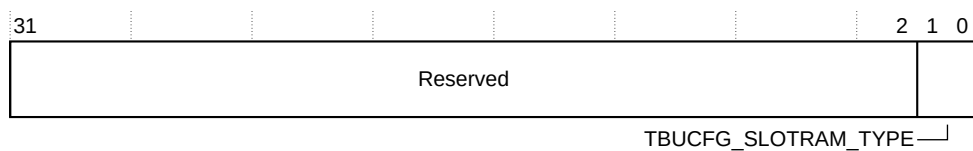


Table 4-112: TBU_SYSDISC13 register bit descriptions

Bits	Name	Description
[31:2]	-	Reserved

Bits	Name	Description
[1:0]	TBUCFG_CACHERAM_TYPE	The read data reflects the chosen parameter value, for example: 2'b00 : 0 2'b01 : 1

4.18.16 TBU_SYSDISC15 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC15 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x0903C

Type

RO

Reset value

TBUCFG_MTLB_LKP_SLOTS. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-79: TBU_SYSDISC15 register bit assignments

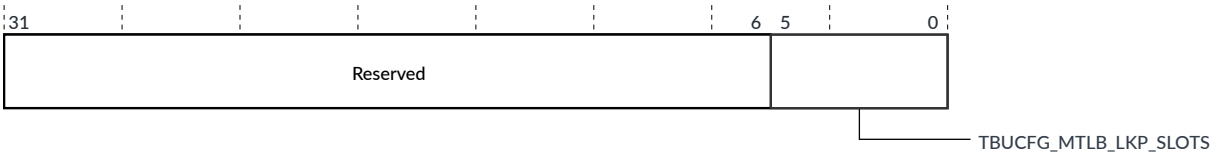


Table 4-114: TBU_SYSDISC15 register bit descriptions

Bits	Name	Description
[31:6]	-	Reserved
[5:0]	TBUCFG_MTLB_LKP_SLOTS	The read data reflects the chosen parameter value

4.18.17 TBU_SYSDISC16 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC16 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09040

Type

RO

Reset value

TBUCFG_LA_HNDSHK_MODE. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-80: TBU_SYSDISC16 register bit assignments



Table 4-115: TBU_SYSDISC16 register bit descriptions

Bits	Name	Description
[31:16]	-	Reserved

Bits	Name	Description
[15:0]	TBUCFG_LA_HNDSHK_MODE	The read data reflects the chosen parameter value. {TBUCFG_LA_HNDSHK_MODE7, ... TBUCFG_LA_HNDSHK_MODE0}

4.18.18 TBU_SYSDISC17 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC17 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09044

Type

RO

Reset value

TBUCFG_LR_HNDSHK_MODE. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-81: TBU_SYSDISC17 register bit assignments

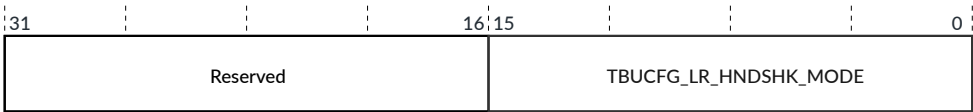


Table 4-116: TBU_SYSDISC17 register bit descriptions

Bits	Name	Description
[31:16]	-	Reserved

Bits	Name	Description
[15:0]	TBUCFG_LR_HNDSHK_MODE	The read data reflects the chosen parameter value. {TBUCFG_LR_HNDSHK_MODE7, ... TBUCFG_LR_HNDSHK_MODE0}

4.18.19 TBU_SYSDISC18 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC18 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09048

Type

RO

Reset value

TBUCFG_LEGACY_TZ_EN. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-82: TBU_SYSDISC18 register bit assignments

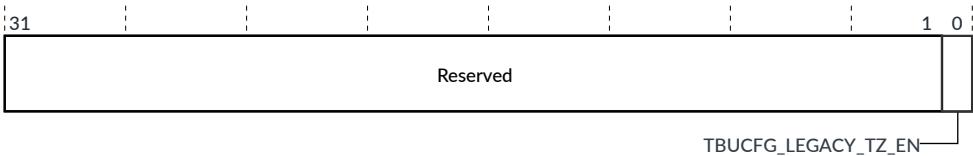


Table 4-117: TBU_SYSDISC18 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TBUCFG_LEGACY_TZ_EN	The read data reflects the chosen parameter value

4.18.20 TBU_SYSDISC19 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC19 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x0904C

Type

RO

Reset value

TCUCFG_USE_ELA_DEBUG. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-83: TBU_SYSDISC19 register bit assignments

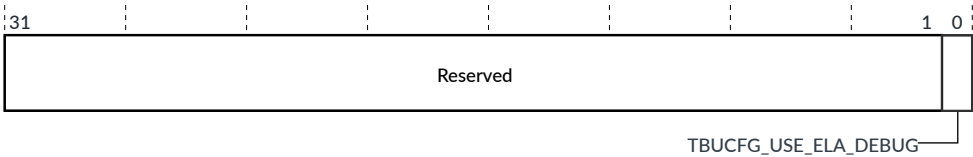


Table 4-118: TBU_SYSDISC19 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TBUCFG_USE_ELA_DEBUG	The read data reflects the chosen parameter value

4.18.21 TBU_SYSDISC20 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC20 register attributes are as follows:

Width

32-bit

Functional group

[4.4.10 TBU system discovery registers summary](#) on page 138.

Address offset

0x09050

Type

RO

Reset value

TBUCFG_PARITY_ON. See [2.7.3 Common ACE-Lite and LTI TBU configuration parameters](#) on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-84: TBU_SYSDISC20 register bit assignments

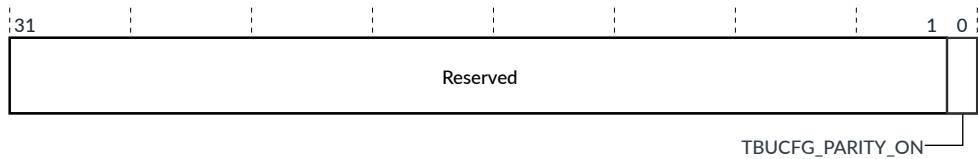


Table 4-119: TBU_SYSDISC20 register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	TBUCFG_PARITY_ON	The read data reflects the chosen parameter value

4.18.22 TBU_SYSDISC21 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC21 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09054

Type

RO

Reset value

TBUCFG_MTLB_UPD_SLOTS. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-85: TBU_SYSDISC21 register bit assignments

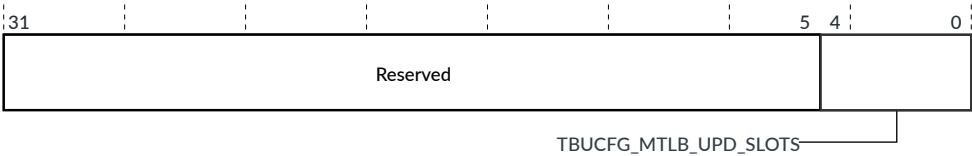


Table 4-120: TBU_SYSDISC21 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TBUCFG_MTLB_UPD_SLOTS	The read data reflects the chosen parameter value

4.18.23 TBU_SYSDISC22 system discovery register

The TBU system discovery registers discover components in the system.

Configurations

This register is available in all configurations.

Attributes

The TBU_SYSDISC22 register attributes are as follows:

Width

32-bit

Functional group

4.4.10 TBU system discovery registers summary on page 138.

Address offset

0x09058

Type

RO

Reset value

TBUCFG_MECID_WIDTH. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-86: TBU_SYSDISC22 register bit assignments

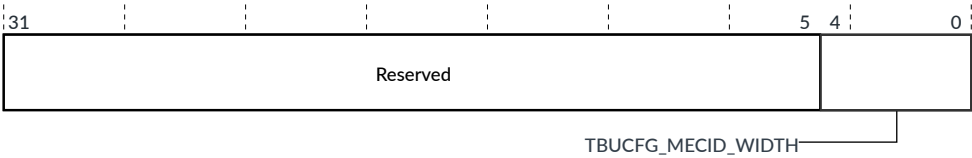


Table 4-121: TBU_SYSDISC22 register bit descriptions

Bits	Name	Description
[31:5]	-	Reserved
[4:0]	TBUCFG_MECID_WIDTH	The read data reflects the chosen parameter value

4.19 TBU integration registers

The MMU-720AE TBU contains integration registers.

4.19.1 ITEN register for the TBU

Integration mode enable register for the TBU. When integration mode is enabled, the values of certain TBU input pins are made visible in the ITIN register of the TBU. The values that are written to the ITOP register of the TBU control the values of certain TBU output pins. This mechanism helps system integrators to integrate the SMMU into the system and perform basic connectivity checks.

See also:

-
- [4.19.3 ITIN register for the TBU](#) on page 270
- [4.19.2 ITOP register for the TBU](#) on page 267

Configurations

This register is available in all configurations.

Attributes

The ITEN register attributes are as follows:

Width

32-bit

Functional group

[4.4.11 TBU integration registers summary](#) on page 138.

Address offset

0x08E20

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-87: ITEN register bit assignments

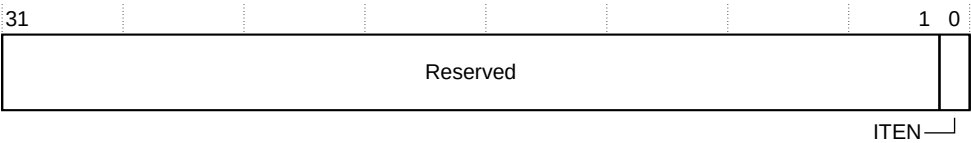


Table 4-122: ITEN register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved
[0]	ITEN	0 Disable integration mode 1 Enable integration mode

4.19.2 ITOP register for the TBU

Integration output register for the TBU.

Configurations

This register is available in all configurations.

Attributes

The ITOP_TBU register attributes are as follows:

Width

32-bit

Functional group

[4.4.11 TBU integration registers summary](#) on page 138.

Address offset

0x08E24

Type

RW

Reset value

0

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-88: ITOP_TBU register bit assignments

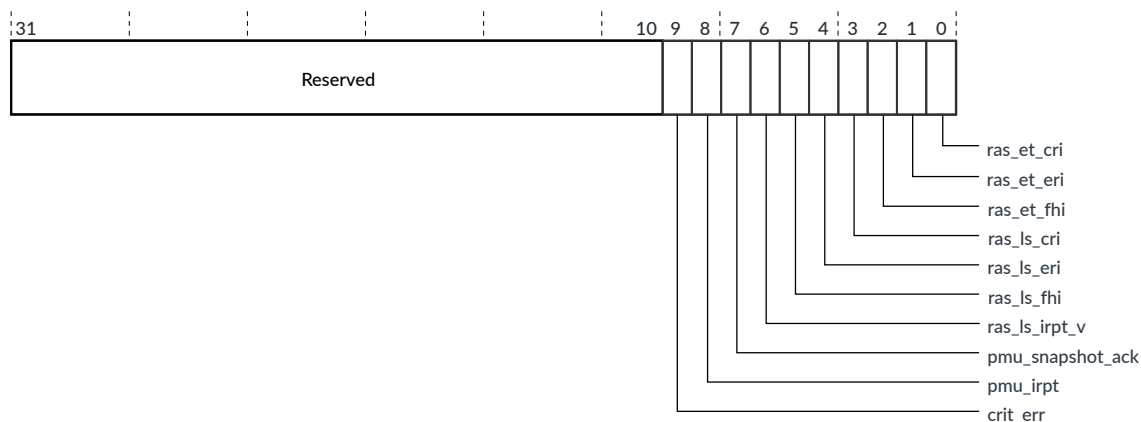


Table 4-123: ITOP_TBU register bit descriptions

Bits	Name	Description
[31:10]	-	Reserved, Should-Be-Zero (SBZ)
[9]	crit_err	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[8]	pmu_irpt	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[7]	pmu_snapshot_ack	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324

Bits	Name	Description
[6]	ras_ls_irpt_v	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[5]	ras_ls_fhi	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[4]	ras_ls_eri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[3]	ras_ls_cri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[2]	ras_et_fhi	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324

Bits	Name	Description
[1]	ras_et_eri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324
[0]	ras_et_cri	- When ITEN.ITEN == 0, the register value SBZ - When ITEN.ITEN == 1, the register value can be 0 or 1 and this value is also driven to the output signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.2.8 TBU interrupt signals on page 319 A.3.2 PMU Snapshot signals on page 324

4.19.3 ITIN register for the TBU

Integration input register for the TBU.

See [A.3.2 PMU Snapshot signals](#) on page 324.

Configurations

This register is available in all configurations.

Attributes

The ITIN_TBU register attributes are as follows:

Width

32-bit

Functional group

[4.4.11 TBU integration registers summary](#) on page 138.

Address offset

0x08E28

Type

RO

Reset value

0

Bit descriptions

The following figure and table show the register bit assignments and descriptions.

Figure 4-89: ITIN_TBU register bit assignments

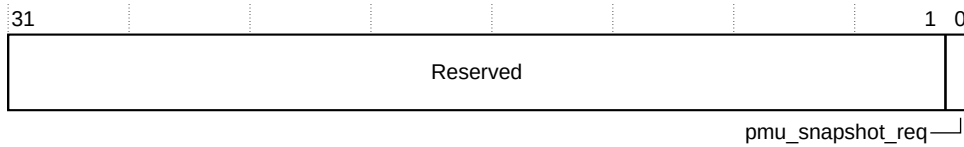


Table 4-124: ITIN_TBU register bit descriptions

Bits	Name	Description
[31:1]	-	Reserved, Should-Be-Zero (SBZ)
[0]	pmu_snapshot_req	- When ITEN.ITEN == 0, the register value SBZ. - When ITEN.ITEN == 1, the register value can be 0 or 1, depending on the value of the input signal that is specified See: 4.19.1 ITEN register for the TBU on page 266 A.3.2 PMU Snapshot signals on page 324

4.20 TCU FMU registers summary

There are several TCU registers for error records and Safety Mechanisms.

To read or write the TCU FMU registers the APB interface signal PPROT[1] must be set to zero to indicate secure access. For information on the TBU FMU registers, see [4.21 TBU FMU registers summary](#) on page 280.

The following table shows the MMU-720AE TCU registers for error records and Safety Mechanisms.

Table 4-125: TCU FMU register summary

Offset	Name	Type	Width	Description
0x60080	TCU_FMU_ERRFR	RO, Root	64-bit	4.20.1 TCU_FMU_ERRFR register on page 271
0x60088	TCU_FMU_ERRCTLR	RW, Root	64-bit	4.20.2 TCU_FMU_ERRCTLR register on page 273
0x60090	TCU_FMU_ERRSTATUS	RW, Root	64-bit	4.20.3 TCU_FMU_ERRSTATUS register on page 274
0x60E00	TCU_FMU_ERRGSR	RO, Root	64-bit	4.20.4 TCU_FMU_ERRGSR register on page 276
0x60F00	TCU_FMU_SMEN	RW, Root	32-bit	4.20.5 TCU_FMU_SMEN register on page 277
0x60F04	TCU_FMU_ERRGEN	WO, Root	64-bit	4.20.6 TCU_FMU_ERRGEN register on page 278

4.20.1 TCU_FMU_ERRFR register

The TCU_FMU_ERRFR register defines which safety features are implemented and which safety features are software programmable. This register is read-only and individual features are controlled using the ERR<n>CTRL register.

Configurations

This register is available in all configurations.

Attributes

The TCU_FMU_ERRFR register attributes are as follows:

Width

64

Functional group

[4.20 TCU FMU registers summary](#) on page 271

Address offset

0x60080

Type

RO, Root

Reset value

```
0x00000000008000a2
```

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-90: TCU_FMU_ERRFR bit assignments

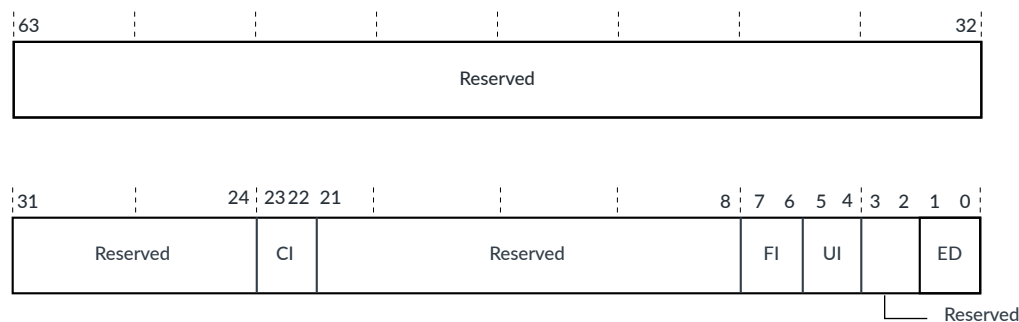


Table 4-126: TCU_FMU_ERRFR bit descriptions

Bits	Name	Description
[63:24]	-	RES0
[23:22]	CI	Critical Interrupt Use ERR<n>CTLR.CI to control this feature.
[21:8]	-	RES0

Bits	Name	Description
[7:6]	FI	Fault Handling Interrupt Use ERR<n>CTLR.FI to control this feature.
[5:4]	UI	Error Recovery Interrupt Use ERR<n>CTLR.UI to control this feature.
[3:2]	-	RES0
[1:0]	ED	Error reporting and logging Use ERR<n>CTLR.ED to control this feature.

4.20.2 TCU_FMU_ERRCTLR register

The TCU FMU Error Control register uses the ED bit to control error reporting and logging.

The TCU FMU Error Control register controls the following interrupts:

- Critical Interrupts (CI)
- Fault Handling Interrupts (FHI)
- Error Recovery Interrupts (ERI)

Configurations

This register is available in all configurations.

Attributes

The TCU_FMU_ERRCTLR register attributes are as follows:

Width

64

Functional group

[4.20 TCU FMU registers summary](#) on page 271

Address offset

0x60088

Type

RW, Root

Reset value

0x000000000000200d

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-91: TCU_FMU_ERRCTLR bit assignments

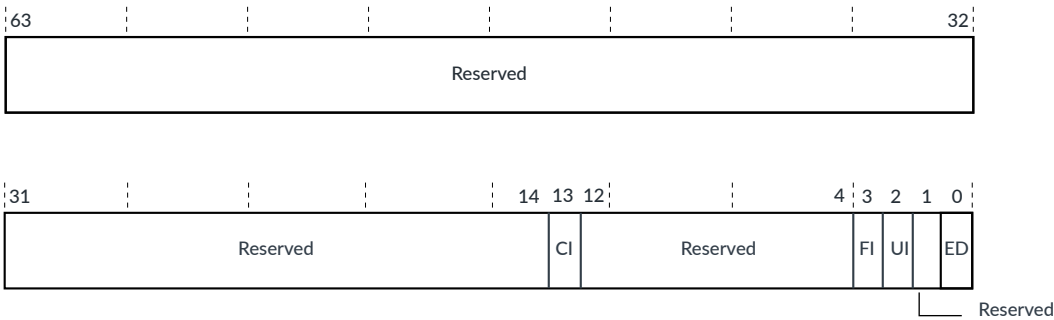


Table 4-127: TCU_FMU_ERRCTLR bit descriptions

Bits	Name	Description
[63:14]	-	Reserved0
[13]	CI	Critical Interrupt enable Use ERR<n>CTLR.CI to control this feature.
[12:4]	-	Reserved0
[3]	FI	Fault Handling Interrupt enable Use ERR<n>CTLR.FI to control this feature.
[2]	UI	Error Recovery Interrupt enable Use ERR<n>CTLR.UI to control this feature.
[1]	-	Reserved0
[0]	ED	Error reporting and logging enable Use ERR<n>CTLR.ED to control this feature.

4.20.3 TCU_FMU_ERRSTATUS register

The TCU FMU Error status register provides information about the specific types of error detected in the TCU.

All the fields in the TCU_FMU_ERRSTATUS register are write-one-to-clear which means writes of zero are ignored. A write of one, or all-ones, to the field clears the field to zero.

Configurations

This register is available in all configurations.

Attributes

The TCU_FMU_ERRSTATUS register attributes are as follows:

Width

64

Functional group

[4.20 TCU FMU registers summary](#) on page 271

Address offset

0x60090

Type

RW, Root

Reset value

0x0000000000000001

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-92: TCU_FMU_ERRSTATUS bit assignments

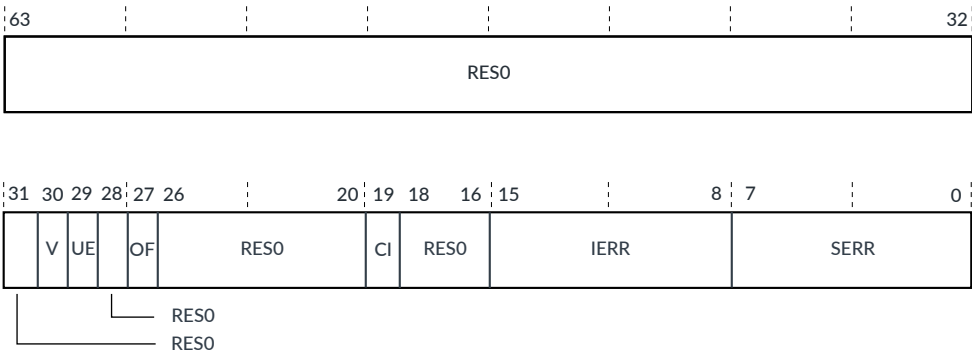


Table 4-128: TCU_FMU_ERRSTATUS bit descriptions

Bits	Name	Description
[63:31]	RES0	Unused bits are set to 0
[30]	V	Status Register Valid
[29]	UE	Uncorrected Error
[28]	RES0	Unused bits are set to 0
[27]	OF	Overflow
[26:20]	RES0	Unused bits are set to 0
[19]	CI	Critical error
[18:16]	RES0	Unused bits are set to 0

Bits	Name	Description
[15:8]	IERR	IMPLEMENTATION DEFINED error code: 0x0 Reserved 0x1 RAM Lock-Step output error 0x2 Synchronous Lock-Step output error 0x3 Asynchronous input error 0x4 F-Channel input error 0x5 APB parity input error 0x6 MSI input error 0x7 PTW input error 0x8 QTW input error A write to this field is ignored. All other values are Reserved.
[7:0]	SERR	Architecturally defined primary error code. 8'd1 represents IMPLEMENTATION DEFINED error.

4.20.4 TCU_FMU_ERRGSR register

The TCU_FMU_ERRGSR register, is a TCU's Global Status Register. MMU-720AE uses this register to determine the source of an interrupt.

In MMU-720AE there are two interrupt sources, RAM errors and FuSa errors. The FuSa errors comprise of all non-RAM errors. These sources are recorded in different error records. Therefore, when an interrupt occurs, firmware must first read this register to determine which error record to read.

Configurations

This register is available in all configurations.

Attributes

The TCU_FMU_ERRGSR register attributes are as follows:

Width

32

Functional group
4.20 TCU FMU registers summary on page 271

Address offset
0x60E00

Type
RO, Root

Reset value
0x0000000000000000

Bit descriptions
The following figure and table show the bit assignments.

Figure 4-93: TCU_FMU_ERRGSR bit assignments

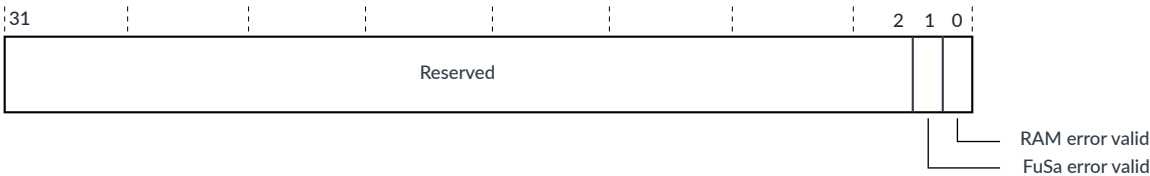


Table 4-129: TCU_FMU_ERRGSR bit descriptions

Bits	Name	Description
[63:2]	-	Reserved
[1]	FuSa Error Valid	FuSa error valid
[0]	RAM Error Valid	RAM error valid

4.20.5 TCU_FMU_SMEN register

Use the TCU_FMU_SMEN register to turn off individual Safety Mechanisms (SMs), as opposed to the TCU_FMU_ERRCTLR register which can only turn off all types of SMs, or none. All the SMs in a TCU are enabled at the reset.

Configurations
This register is available in all configurations.

Attributes
The TCU_FMU_SMEN register attributes are as follows:

Width
32

Functional group
4.20 TCU FMU registers summary on page 271

Address offset

0x60F00

Type

RW, Root

Reset value

0x000000ff

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-94: TCU_FMU_SMEN bit assignments

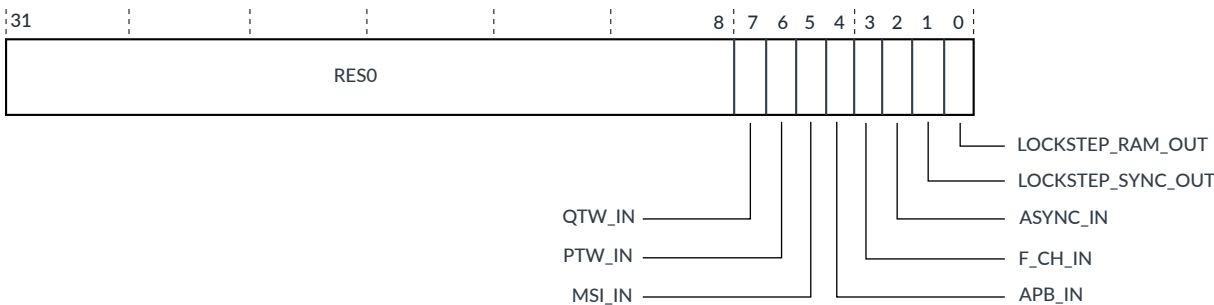


Table 4-130: TCU_FMU_SMEN bit descriptions

Bits	Name	Description
[31:8]	RES0	Unused bits are set to 0
[7]	QTW_IN	Enables checking on QTW inputs
[6]	PTW_IN	Enables checking on PTW inputs
[5]	MSI_IN	Enables checking on MSI inputs
[4]	APB_IN	Enables checking on APB inputs
[3]	F_CH_IN	Enables checking on the F-Channel
[2]	ASYNC_IN	Enables checking on the asynchronous errors
[1]	LOCKSTEP_SYNC_OUT	Enables checking on the synchronous Lock-Step checker
[0]	LOCKSTEP_RAM_OUT	Enables checking on the RAM Lock-Step checker

4.20.6 TCU_FMU_ERRGEN register

You can write to the TCU_FMU_ERRGEN register for software testing purposes. Valid writes to this register occur in the TCU_FMU_ERRSTATUS register and trigger the corresponding interrupts, if enabled, in the TCU_FMU_ERRCTLR register.

Any write to IERR error code is valid only if Status Register Valid (V) bit is set 1, Uncorrected Error (UE) bit is set 1 and Critical Error (CI) bit is set 1.

Configurations

This register is available in all configurations.

Attributes

The TCU_FMU_ERRGEN register attributes are as follows:

Width

64

Functional group

[4.20 TCU FMU registers summary](#) on page 271

Address offset

0x60F04

Type

WO, Root

Reset value

0x0000000000000000

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-95: TCU_FMU_ERRGEN bit assignments

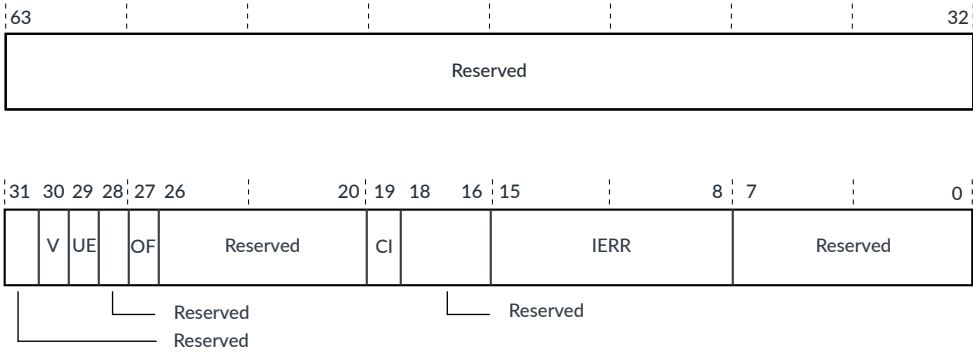


Table 4-131: TCU_FMU_ERRGEN bit descriptions

Bits	Name	Description
[63:31]	-	Reserved
[30]	V	Status register Valid
[29]	UE	Uncorrected Error
[28]	-	Reserved
[27]	OF	OverFlow
[26:20]	-	Reserved
[19]	CI	Critical error
[18:16]	-	Reserved

Bits	Name	Description
[15:8]	IERR	IMPLEMENTATION DEFINED error code: 0x0 Reserved 0x1 Lock-Step RAM output error 0x2 Lock-Step synchronous output error 0x3 Asynchronous input error 0x4 F-Channel input error 0x5 APB parity input error 0x6 MSI input error 0x7 PTW input error 0x8 QTW input error Do not use any other values for IERR. All other values are Reserved.
[7:0]	-	Reserved

4.21 TBU FMU registers summary

There are several TBU registers for error records and Safety Mechanisms.

To read or write the TCU FMU registers the APB interface signal PPROT[1] must be set to zero to indicate secure access. For information on the TCU FMU registers, see [4.20 TCU FMU registers summary](#) on page 271.

The following table shows the MMU-720AE TCU registers for error records and Safety Mechanisms.

Table 4-132: TBU FMU register summary

Offset	Name	Type	Width	Description
0x10080	TBU_FMU_ERRFR	RO, Root	64-bit	4.21.1 TBU_FMU_ERRFR register on page 281
0x10088	TBU_FMU_ERRCTLR	RW, Root	64-bit	4.21.2 TBU_FMU_ERRCTLR register on page 282

Offset	Name	Type	Width	Description
0x10090	TBU_FMU_ERRSTATUS	RW, Root	64-bit	4.21.3 TBU_FMU_ERRSTATUS register on page 283
0x10E00	TBU_FMU_ERRGSR	RO, Root	64-bit	TBU Global Status Register. See 4.21.4 TBU_FMU_ERRGSR register on page 285.
0x10F00	TBU_FMU_SMEN	RW, Root	32-bit	FMU Safety Mechanism Control Register. See 4.21.5 TBU_FMU_SMEN register on page 286.
0x10F04	TBU_FMU_ERRGEN	RW, Root	64-bit	FMU Error Generation Register. See 4.21.6 TBU_FMU_ERRGEN register on page 287.

4.21.1 TBU_FMU_ERRFR register

The TBU_FMU_ERRFR register defines which safety features are implemented and which safety features are software programmable. This register is read-only and individual features are controlled using the ERR<n>CTRL register.

Configurations

This register is available in all configurations.

Attributes

The TBU_FMU_ERRFR register attributes are as follows:

Width

64

Functional group

[4.21 TBU FMU registers summary](#) on page 280

Address offset

0x10080

Type

RO, Root

Reset value

0x00000000008000a2

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-96: TBU_FMU_ERRFR bit assignments

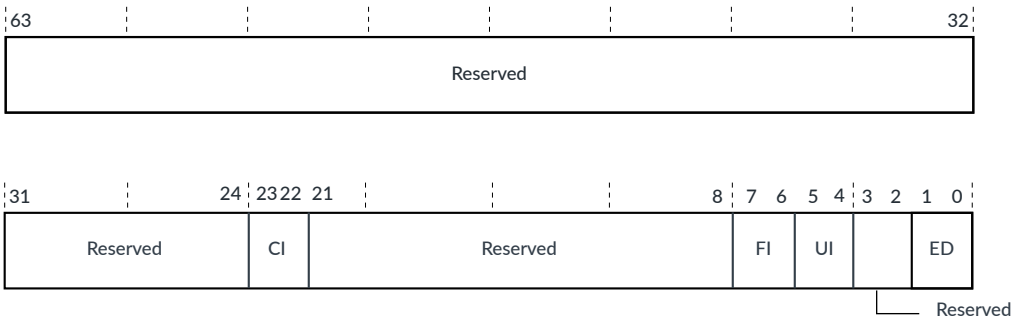


Table 4-133: TBU_FMU_ERRFR bit descriptions

Bits	Name	Description
[63:24]	-	Reserved
[23:22]	CI	Critical Interrupt Use ERR<n>CTLR.CI to control this feature.
[21:8]	-	Reserved
[7:6]	FI	Fault Handling Interrupt Use ERR<n>CTLR.FI to control this feature.
[5:4]	UI	Error Recovery Interrupt Use ERR<n>CTLR.UI to control this feature.
[3:2]	-	Reserved
[1:0]	ED	Error reporting and logging Use ERR<n>CTLR.ED to control this feature.

4.21.2 TBU_FMU_ERRCTLR register

The TBU Error Control register uses the enable bits to control interrupts.

The TBU Error Control register controls the following interrupts:

- Critical Interrupts (CI)
- Fault Handling Interrupts (FHI)
- Error Recovery Interrupts (ERI)

Configurations

This register is available in all configurations.

Attributes

The TBU_FMU_ERRCTLR register attributes are as follows:

Width

64

Functional group

4.21 TBU FMU registers summary on page 280

Address offset

0x10088

Type

RW, Root

Reset value

0x0000000000000200d

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-97: TBU_FMU_ERRCTLR bit assignments

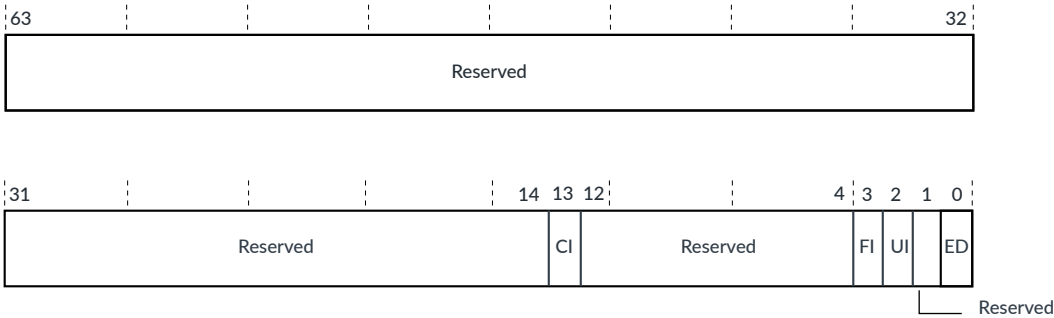


Table 4-134: TBU_FMU_ERRCTLR bit descriptions

Bits	Name	Description
[63:14]	-	Reserved
[13]	CI	Critical Interrupt enable Use ERR<n>CTLR.CI to control this feature.
[12:4]	-	Reserved
[3]	FI	Fault Handling Interrupt enable Use ERR<n>CTLR.FI to control this feature.
[2]	UI	Error Recovery Interrupt enable Use ERR<n>CTLR.UI to control this feature.
[1]	-	Reserved
[0]	ED	Error reporting and logging enable Use ERR<n>CTLR.ED to control this feature.

4.21.3 TBU_FMU_ERRSTATUS register

The TBU error status register provides information about the specific types of error detected in the TBU.

All the fields in the TBU_FMU_ERRSTATUS register are write-one-to-clear which means writes of zero are ignored. A write of one or all-ones to the field clears the field to zero.

Configurations

This register is available in all configurations.

Attributes

The TBU_FMU_ERRSTATUS register attributes are as follows:

Width

64

Functional group

[4.21 TBU FMU registers summary](#) on page 280

Address offset

0x10090

Type

RW, Root

Reset value

0x0000000000000001

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-98: TBU_FMU_ERRSTATUS bit assignments

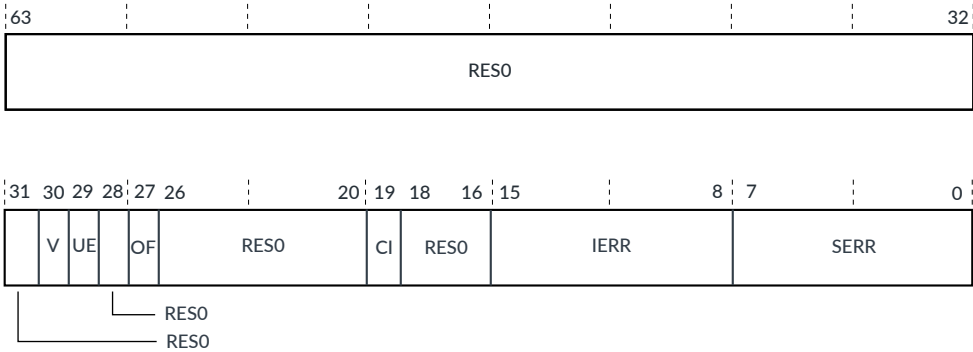


Table 4-135: TBU_FMU_ERRSTATUS bit descriptions

Bits	Name	Description
[63:31]	RES0	Unused bits are set to 0
[30]	V	Status Register Valid

Bits	Name	Description
[29]	UE	Uncorrected Error
[28]	RES0	Unused bits are set to 0
[27]	OF	Overflow This bit is asserted when more than one error is reported in the Error Record. In case of an overflow, the first error's Safety Mechanism (SM) ID is retained.
[26:20]	RES0	Unused bits are set to 0
[19]	CI	Critical error
[18:16]	RES0	Unused bits are set to 0
[15:8]	IERR	IMPLEMENTATION DEFINED error code. Encoding is: 0 Reserved 1 RAM Lock-Step output error 2 Synchronous Lock-Step output error 3 Asynchronous input error 4 ACE-Lite input error 5 F-Channel input error A write to this field is ignored. All other values are Reserved.
[7:0]	SERR	Architecturally defined primary error code. 8 'd1 represents IMPLEMENTATION DEFINED error.

4.21.4 TBU_FMU_ERRGSR register

The FMU_TBU_ERRGSR, is a TBU's Global Status Register. MMU-720AE uses this register to determine the source of an interrupt.

In MMU-720AE there are two interrupt sources, RAM errors and FuSa errors. The FuSa errors comprise of all non-RAM errors. These sources are recorded in different error records. Therefore, when an interrupt occurs, firmware may first read this register to determine which error record to read. When interrupts are low, this register does not contain meaningful values.

Configurations

This register is available in all configurations.

Attributes

The TBU_FMU_ERRGSR register attributes are as follows:

Width

64

Functional group

4.21 TBU FMU registers summary on page 280

Address offset

0x10E00

Type

RO, Root

Reset value

0x0000000000000000

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-99: TBU_FMU_ERRGSR bit assignments

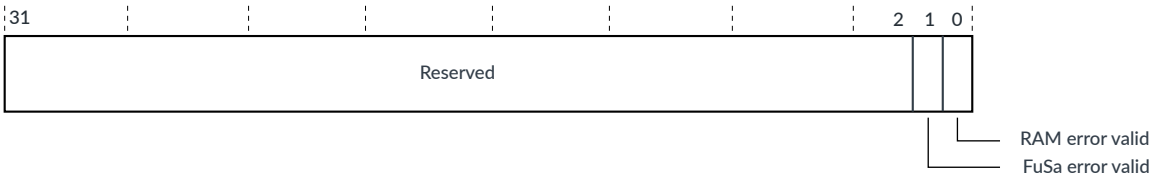


Table 4-136: TBU_FMU_ERRGSR bit descriptions

Bits	Name	Description
[63:2]	-	Reserved
[1]	FuSa Error Valid	FuSa Error Valid
[0]	RAM Error Valid	RAM Error Valid

4.21.5 TBU_FMU_SMEN register

Use the TBU_FMU_SMEN register to turn off individual Safety Mechanisms (SMs). You can use the TBU_FMU_ERRCTLR register to turn off all types of SMs or none. All the SMs in a TBU are enabled at the reset.

Configurations

This register is available in all configurations.

Attributes

The TBU_FMU_SMEN register attributes are as follows:

Width

32

Functional group

[4.21 TBU FMU registers summary](#) on page 280

Address offset

0x10F00

Type

RW, Root

Reset value

0x000001f

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-100: TBU_FMU_SMEN bit assignments

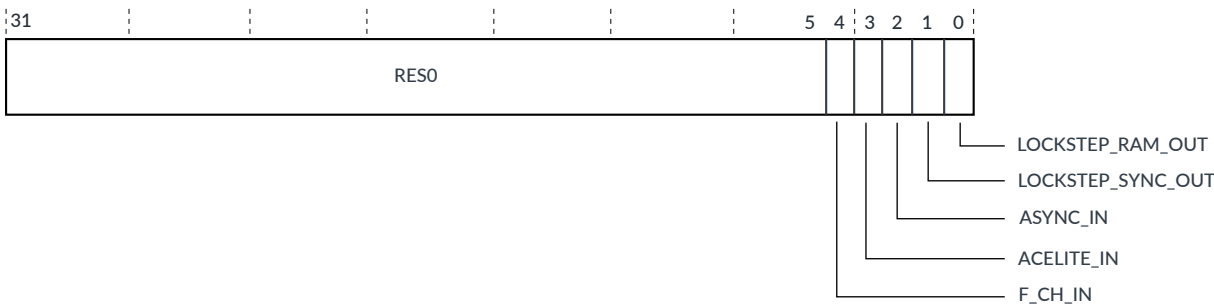


Table 4-137: TBU_FMU_SMEN bit descriptions

Bits	Name	Description
[31:5]	RES0	Unused bits are set to 0
[4]	F_CH_IN	Enables checking on the F-Channel
[3]	ACELITE_IN	Enables checking on the ACE-Lite interface
[2]	ASYNC_IN	Enables checking on the asynchronous errors
[1]	LOCKSTEP_SYNC_OUT	Enables checking on the synchronous Lock-Step checker
[0]	LOCKSTEP_RAM_OUT	Enables checking on the RAM Lock-Step checker

4.21.6 TBU_FMU_ERRGEN register

You can write to the TBU_FMU_ERRGEN register for software testing purposes. Valid writes to this register occur in the TBU_FMU_ERRSTATUS register and trigger the corresponding interrupts, if enabled, in the TBU_FMU_ERRCTLR register.

Any write to IERR error code is valid only if Status Register Valid (V) bit is set 1, Uncorrected Error (UE) bit is set 1 and Critical Error (CI) bit is set 1.

Configurations

This register is available in all configurations.

Attributes

The TBU_FMU_ERRGEN register attributes are as follows:

Width

64

Functional group

[4.21 TBU FMU registers summary](#) on page 280

Address offset

0x10F04

Type

WO, Root

Reset value

0x0000000000000000

Bit descriptions

The following figure and table show the bit assignments.

Figure 4-101: TBU_FMU_ERRGEN bit assignments

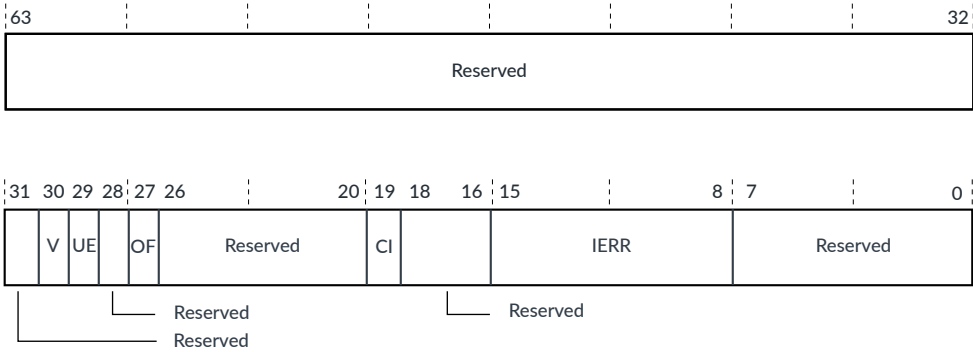


Table 4-138: TBU_FMU_ERRGEN bit descriptions

Bits	Name	Description
[63:31]	-	Reserved
[30]	V	Status register Valid
[29]	UE	Uncorrected Error
[28]	-	Reserved
[27]	OF	OverFlow
[26:20]	-	Reserved
[19]	CI	Critical error
[18:16]	-	Reserved

Bits	Name	Description
[15:8]	IERR	IMPLEMENTATION DEFINED error code: 0 Reserved 1 Lock-Step RAM output error 2 Lock-Step synchronous output error 3 Asynchronous input error 4 ACE-Lite input error 5 F-Channel input error Do not use any other values for IERR. All other values are Reserved.
[7:0]	-	Reserved

Appendix A Signal descriptions

MMU-720AE uses various external signals for the Translation Control Unit (TCU), the Translation Buffer Unit (TBU), and the Distributed Translation Interface (DTI).

MMU-720AE external signal names use the following convention:

_s	Secure signal, for example event_q_irpt_s
_ns	Non-secure signal, for example event_q_irpt_ns
_r	Realm signal, for example event_q_irpt_r

For more information about Secure, Non-secure, and Realm signals, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

A.1 TCU signals

MMU-720AE contains TCU signals.

A.1.1 TCU clock and reset signals

The TCU uses a single set of standard clock and reset signals.

Signal definitions

Table A-1: Clock and reset signals

Signal	Direction	Description
clk	Input	Global clock. Width is 1-bit.
resetrn	Input	Global reset. Width is 1-bit.

A.1.2 TCU QTW/DVM interface signals

The TCU QTW/DVM interface signals are based on the AMBA ACE5-Lite+DVM signals.

Signal definitions

Table A-2: TCU QTW/DVM interface signals

Signal	Direction	Description
acaddr_qtw	Input	Snoop address. Width is 52-bit.
acready_qtw	Output	Snoop address ready. Width is 1-bit.
acvalid_qtw	Input	Snoop address valid. Width is 1-bit.
acwakeup_qtw	Input	Snoop wakeup. Width is 1-bit.
acvmidext_qtw	Input	Snoop Extended Virtual Machine IDentifier (VMID). Width is 4-bit.

Signal	Direction	Description
arid_qtw	Output	Read address ID The width of arid_qtw is equal to QTW_ID_WIDTH. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$
araddr_qtw	Output	Read address. Width is 52-bit.
arburst_qtw	Output	Burst type. Width is 2-bit.
arcache_qtw	Output	Memory type. Width is 4-bit.
ardomain_qtw	Output	Shareability domain. Width is 2-bit.
arlen_qtw	Output	Burst length. Width is 8-bit.
arlock_qtw	Output	Lock type. Width is 1-bit.
arprot_qtw	Output	Protection type. Width is 3-bit.
arqos_qtw	Output	QoS identifier. Width is 4-bit.
arready_qtw	Input	Read address ready. Width is 1-bit.
arsize_qtw	Output	Transfer size. Width is 3-bit.
arsnoop_qtw	Output	Transaction type. Width is 4-bit.
arvalid_qtw	Output	Read address valid. Width is 1-bit.
aridunq_qtw	Output	Read address channel unique ID indicator. Width is 1-bit.
armpam_qtw	Output	Read address channel MPAM information. Width is $(\text{TCUCFG_LEGACY_TZ_EN} == 1) ? 11 : 12$
arnse_qtw	Output	Along with ARPROT[1] indicates whether the physical address space is Secure, Non-Secure, Root, or Realm. Width is 1 bit
arpbha_qtw	Output	Page-based hardware attributes for read requests. Width is 4-bit.
armecid_qtw	Output	MECID for requests on the read request channel Width is $(\text{TCUCFG_MECID_WIDTH} == 0) ? 1 : 16$
awid_qtw	Output	Write address ID Width is QTW_ID_WIDTH-bit. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$
awaddr_qtw	Output	Write address. Width is 52-bit.
awburst_qtw	Output	Burst type. Width is 2-bit.
awcache_qtw	Output	Memory type. Width is 4-bit.
awdomain_qtw	Output	Shareability domain. Width is 2-bit.

Signal	Direction	Description
awlen_qtw	Output	Burst length. Width is 8-bit.
awlock_qtw	Output	Lock type. Width is 1-bit.
awprot_qtw	Output	Protection type. Width is 3-bit.
awqos_qtw	Output	QoS identifier. Width is 4-bit.
awready_qtw	Input	Write address ready. Width is 1-bit.
awsiz_qtw	Output	Burst size. Width is 3-bit.
awsnoop_qtw	Output	Transaction type. Width is 4-bit.
awvalid_qtw	Output	Write address valid. Width is 1-bit.
awatop_qtw	Output	Atomic operation. Width is 6-bit.
awidunq_qtw	Output	Write address channel unique ID indicator. Width is 1-bit.
awmpam_qtw	Output	Write address channel MPAM information. Width is (TCUCFG_LEGACY_TZ_EN == 1) ? 11 : 12
awnse_qtw	Output	Along with ARPROT[1], indicates whether the physical address space is Secure, Non-Secure, Root, or Realm. Width is 1-bit.
awpbha_qtw	Output	Page-based hardware attributes for write requests. Width is 4-bit.
awmecid_qtw	Output	MECID for requests on the write request channel. Width is (TCUCFG_MECID_WIDTH == 0) ? 1 : 16
crready_qtw	Input	Snoop response ready. Width is 1-bit.
crvalid_qtw	Output	Snoop response valid. Width is 1-bit.
rdata_qtw	Input	Read data. The width of rdata_qtw is equal to TCUCFG_QTW_DATA_WIDTH See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
rid_qtw	Input	Read data ID Width is QTW_ID_WIDTH-bit. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$ See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
ridunq_qtw	Input	Read data channel unique ID indicator, active HIGH. Width is 1-bit.
rlast_qtw	Input	Read last. Width is 1-bit.
rpoison_qtw	Input	Read poison input to the TCU. Width is TCUCFG_QTW_DATA_WIDTH / 64
rready_qtw	Output	Read ready. Width is 1-bit.
rresp_qtw	Input	Read response. Width is 2-bit.
rvalid_qtw	Input	Read valid. Width is 1-bit.
wdata_qtw	Output	Write data The width of wdata_qtw is equal to TCUCFG_QTW_DATA_WIDTH-bit. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.

Signal	Direction	Description
wlast_qtw	Output	Write last. Width is 1-bit.
wready_qtw	Input	Write ready. Width is 1-bit.
wstrb_qtw	Output	Write strobe The width of wstrb_qtw is calculated as (TCUCFG_QTW_DATA_WIDTH/8)-bit. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
wvalid_qtw	Output	Write valid. Width is 1-bit.
wpoison_qtw	Output	Write data poison Width is TCUCFG_QTW_DATA_WIDTH / 64
bid_qtw	Input	Response ID Width is QTW_ID_WIDTH-bit. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$
bready_qtw	Output	Response ready. Width is 1-bit.
bresp_qtw	Input	Write response. Width is 2-bit.
bvalid_qtw	Input	Write response valid. Width is 1-bit.
bidunq_qtw	Input	Write response channel unique ID indicator. Width is 1-bit.
awakeup_qtw	Output	Wakeup. Width is 1-bit.

For more information, see the [AMBA® AXI Protocol Specification](#).

A.1.3 TCU PTW interface signals

The TCU Page Table Walk (PTW) interface signals are based on the AMBA ACE5-Lite signals.



Note

The TCU PTW interface signals are valid for TCUX2 only.

Signal definitions

Table A-3: TCU PTW interface signals

Signal	Direction	Description
arid_ptw	Output	Read address ID The width of arid_ptw is equal to QTW_ID_WIDTH. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$
araddr_ptw	Output	Read address. Width is 52-bit.

Signal	Direction	Description
arburst_ptw	Output	Burst type. Width is 2-bit.
arcache_ptw	Output	Memory type. Width is 4-bit.
ardomain_ptw	Output	Shareability domain. Width is 2-bit.
arlen_ptw	Output	Burst length. Width is 8-bit.
arlock_ptw	Output	Lock type. Width is 1-bit.
arprot_ptw	Output	Protection type. Width is 3-bit.
arqos_ptw	Output	QoS identifier. Width is 4-bit.
arready_ptw	Input	Read address ready. Width is 1-bit.
arsize_ptw	Output	Burst size. Width is 3-bit.
arsnoop_ptw	Output	Transaction type. Width is 4-bit.
arvalid_ptw	Output	Read address valid. Width is 1-bit.
aridunq_ptw	Output	Read address channel unique ID indicator. Width is 1-bit.
armpam_ptw	Output	Read address channel MPAM information. Width is (TCUCFG_LEGACY_TZ_EN == 1) ? 11 : 12
arnse_ptw	Output	Along with ARPROT[1] indicates whether the physical address space is Secure, Non-Secure, Root, or Realm. Width is 1 bit
arpbha_ptw	Output	Page-based hardware attributes for write requests. Width is 4-bit.
armecid_ptw	Output	MECID for requests on the write request channel Width is (TCUCFG_MECID_WIDTH == 0) ? 1 : 16
awid_ptw	Output	Write address ID Width is QTW_ID_WIDTH-bit. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$
awaddr_ptw	Output	Write address. Width is 52-bit.
awburst_ptw	Output	Burst type. Width is 2-bit.
awcache_ptw	Output	Memory type. Width is 4-bit.
awdomain_ptw	Output	Shareability domain. Width is 2-bit.
awlen_ptw	Output	Burst length. Width is 8-bit.
awlock_ptw	Output	Lock type. Width is 1-bit.
awprot_ptw	Output	Protection type. Width is 3-bit.
awqos_ptw	Output	QoS identifier. Width is 4-bit.
awready_ptw	Input	Write address ready. Width is 1-bit.
awsize_ptw	Output	Burst size. Width is 3-bit.
awsnoop_ptw	Output	Transaction type. Width is 4-bit.
awvalid_ptw	Output	Write address valid. Width is 1-bit.
awatop_ptw	Output	Atomic operation. Width is 6-bit.
awidunq_ptw	Output	Write address channel unique ID indicator. Width is 1-bit.

Signal	Direction	Description
awmpam_ptw	Output	Write address channel MPAM information. Width is $(\text{TCUCFG_LEGACY_TZ_EN} == 1) ? 11 : 12$
awnse_ptw	Output	Along with ARPROT[1], indicates whether the physical address space is Secure, Non-Secure, Root, or Realm. Width is 1-bit.
awpbha_ptw	Output	Page-based hardware attributes for write requests. Width is 4-bit.
awmecid_ptw	Output	MECID for requests on the write request channel. Width is $(\text{TCUCFG_MECID_WIDTH} == 0) ? 1 : 16$
rdata_ptw	Input	Read data. The width of rdata_ptw is equal to <code>TCUCFG_QTW_DATA_WIDTH</code> See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
rid_ptw	Input	Read data ID Width is <code>QTW_ID_WIDTH</code> -bit. <code>QTW_ID_WIDTH</code> is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_PTW_SLOTS}, 8))) + 3$ See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
ridunq_ptw	Input	Read data channel unique ID indicator, active HIGH. Width is 1-bit.
rlast_ptw	Input	Last data beat of the read. Width is 1-bit.
rpoison_ptw	Input	Read poison input to the TCU. Width is $\text{TCUCFG_QTW_DATA_WIDTH} / 64$
rready_ptw	Output	Read ready. Width is 1-bit.
rresp_ptw	Input	Read response. Width is 2-bit.
rvalid_ptw	Input	Read valid. Width is 1-bit.
wdata_ptw	Output	Write data The width of wdata_ptw is equal to <code>TCUCFG_QTW_DATA_WIDTH</code> -bit. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
wlast_ptw	Output	Write last. Width is 1-bit.
wready_ptw	Input	Write ready. Width is 1-bit.
wstrb_ptw	Output	Write strobe The width of wstrb_ptw is calculated as $(\text{TCUCFG_QTW_DATA_WIDTH} / 8)$ -bit. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
wvalid_ptw	Output	Write valid. Width is 1-bit.
wpoison_ptw	Output	Write data poison. Width is $\text{TCUCFG_QTW_DATA_WIDTH} / 64$

Signal	Direction	Description
bid_ptw	Input	Response ID Width is QTW_ID_WIDTH-bit. QTW_ID_WIDTH is calculated as follows: $\text{ceil}(\log_2(\max(\text{TCUCFG_DGW_SLOTS}, \text{TCUCFG_QTW_SLOTS}, 8))) + 3$
bready_ptw	Output	Response ready. Width is 1-bit.
bresp_ptw	Input	Write response. Width is 2-bit.
bvalid_ptw	Input	Write response valid. Width is 1-bit.
bidunq_ptw	Input	Write response channel unique ID indicator. Width is 1-bit.
awakeup_ptw	Output	Wakeup. Width is 1-bit.

See the [AMBA® AXI Protocol Specification](#).

A.1.4 TCU programming interface signals

The TCU programming interface signals are based on the AMBA APB5 signals.

Signal definitions

Table A-4: TCU programming interface signals

Signal	Direction	Description
paddr_prog	Input	Peripheral address. Width is 32-bit.
psel_prog	Input	Peripheral select. Width is 1-bit.
penable_prog	Input	Enable for transfer. Width is 1-bit.
pwrite_prog	Input	Write transaction indicator. Width is 1-bit.
pnse_prog	Input	The pnse_prog signal, in combination with the pprot_prog signal, determines the physical address space of the transaction, that is, Root or Realm. For more information, see the AMBA® APB Protocol Specification . Width is 1-bit.
pprot_prog	Input	Protection type. Width is 3-bit.
pwdata_prog	Input	Write data. Width is 32-bit.
pstrb_prog	Input	Write data strobe. Width is 4-bit.
pslverr_prog	Output	Error response. Width is 1-bit.
prdata_prog	Output	Read data. Width is 32-bit.
pready_prog	Output	Transfer ready. Width is 1-bit.
pwakeup_prog	Input	Interface wakeup. Width is 1-bit.

For more information, see the [AMBA® APB Protocol Specification](#).

A.1.5 TCU SYSCO interface signals

The following table shows the TCU SYSCO interface signals.

Signal definitions

Table A-5: TCU SYSCO interface signals

Signal	Direction	Description
syscoreq_qtw	Output	System coherency request. This output transitions: HIGH To indicate that the requester is requesting to enter the coherency domain. LOW To indicate that the requester is requesting to exit the coherency domain. Width is 1-bit.
syscoack_qtw	Input	System coherency acknowledge. The syscoack_qtw signal is an asynchronous input port which is synchronized internally. This input transitions to the same level as syscoreq_qtw when the request to enter or exit the coherency domain is complete. Width is 1-bit.

A.1.6 TCU LPI_PD interface signals

The following table shows the TCU LPI_PD interface signals.

Signal definitions

Table A-6: TCU LPI_PD interface signals

Signal	Direction	Description
qactive_pd	Output	Component active. Width is 1-bit.
qreqn_pd	Input	Quiescence request. Width is 1-bit.
qacceptn_pd	Output	Quiescence accept. Width is 1-bit.
qdeny_pd	Output	Quiescence deny. Width is 1-bit.

For more information, see the [AMBA® Low Power Interface Specification](#).

A.1.7 TCU LPI_CG interface signals

The following table shows the TCU LPI_CG interface signals.

Signal definitions

Table A-7: TCU LPI_CG interface signals

Signal	Direction	Description
qactive_cg	Output	Component active. Width is 1-bit.

Signal	Direction	Description
qreqn_cg	Input	Quiescence request. Width is 1-bit.
qacceptn_cg	Output	Quiescence accept. Width is 1-bit.
qdeny_cg	Output	Quiescence deny. Width is 1-bit.

For more information, see the [AMBA® Low Power Interface Specification](#).

A.1.8 TCU DTI interface signals

The following table shows the TCU DTI interface signals.

In the following table, the 'Direction' has the following meaning:

Input

TBU to TCU

Output

TCU to TBU

Signal definitions

Table A-8: TCU DTI interface signals

Signal	Direction	Description
tvalid_dti_dn	Input	Flow control signal on the downstream channel. Width is 1-bit.
tready_dti_dn	Output	Flow control signal on the downstream channel. Width is 1-bit.
tdata_dti_dn	Input	Message data signal. Width is 160-bit.
tkeep_dti_dn	Input	This signal indicates valid bytes. Width is 20-bit.
tlast_dti_dn	Input	Indicates the last cycle of a message. Width is 1-bit.
tid_dti_dn	Input	Identifies the requester (TBU) that initiated the message. Width is 4-bit or 6-bit. The width of tid_dti_dn is calculated as follows: If TCUCFG_NUM_TBU is 62, the width of tid_dti_dn is 6-bit. Otherwise, the width of tid_dti_dn is 4-bit. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
twakeup_dti_dn	Input	Wake up signal. Width is 1-bit.
tvalid_dti_up	Output	Flow control signal on the upstream channel. Width is 1-bit.
tready_dti_up	Input	Flow control signal on the upstream channel. Width is 1-bit.
tdata_dti_up	Output	Message data signal. Width is 160-bit or 192-bit. If TCUCFG_MECID_WIDTH is 0 or TCUCFG_LEGACY_TZ_EN is 1, then the width of this signal is 160-bit, otherwise it is 192-bit.
tkeep_dti_up	Output	Indicates valid bytes. Width is 20-bit or 24-bit. If TCUCFG_MECID_WIDTH is 0 or TCUCFG_LEGACY_TZ_EN is 1, then the width of this signal is 20-bit, otherwise it is 24-bit.
tlast_dti_up	Output	Indicates the last cycle of a message. Width is 1-bit.

Signal	Direction	Description
tdest_dti_up	Output	Identifies the requester, TBU, that is receiving the message. Width is 4-bit or 6-bit. If TCUCFG_NUM_TBU is 62, the width of tdest_dti_up is 6-bit. Otherwise, the width of tdest_dti_up is 4-bit. See 2.7.1 TCU I/O and buffer configuration parameters on page 103.
twakeup_dti_up	Output	Wake up signal. Width is 1-bit.

For more information about the DTI signals, see the [AMBA® AXI-Stream Protocol Specification](#).

For more information about DTI protocol messages, see the [AMBA® DTI Protocol Specification](#).

A.1.9 TCU interrupt signals

The TCU interrupt signals consist of edge-triggered and level-triggered interrupts. For edge triggered interrupts, the interrupt controller must detect the rising edge of these signals.

The TCU can also output the following as Message Signaled Interrupts (MSIs) on the QTW/DVM interface and the dedicated MSI delivery interface:

- Secure, Non-secure, and Realm Event queue
- SYNC complete commands
- Global interrupts
- Non-secure and Realm Page Request Interface (PRI) queue interrupts

If the system supports capturing MSIs from the TCU, there is no requirement to connect the corresponding interrupt signals in this interface.



Note

In some cases it is possible for the SMMU to enter a powered-down state and even reset after issuing an interrupt, removing the evidence of the source of the interrupt. Always ensure that you write the interrupt-handling software to handle such an occurrence.

Signal definitions

Table A-9: TCU interrupt interface signals

Signal	Direction	Description
event_q_irpt_s	Output	Edge-triggered Event queue, Secure interrupt. The event_q_irpt_s signal asserts a Secure interrupt to indicate that the Secure Event queue is not empty. Width is 1-bit.
event_q_irpt_ns	Output	Edge-triggered Event queue, Non-secure interrupt. The event_q_irpt_ns signal asserts a Non-secure interrupt to indicate that the Non-secure Event queue is not empty. Width is 1-bit.
event_q_irpt_r	Output	Edge-triggered Event queue, Realm interrupt. The event_q_irpt_ns signal asserts a Non-secure interrupt to indicate that the Realm Event queue is not empty. Width is 1-bit.
cmd_sync_irpt_ns	Output	Edge-triggered SYNC complete, Non-secure interrupt. The cmd_sync_irpt_ns signal asserts a Non-secure interrupt to indicate that the CMD_SYNC command on the Non-secure Command queue is complete. Width is 1-bit.

Signal	Direction	Description
cmd_sync_irpt_s	Output	Edge-triggered SYNC complete, Secure interrupt. The cmd_sync_irpt_s signal asserts a Secure interrupt to indicate that the CMD_SYNC command on the Secure Command queue is complete. Width is 1-bit.
cmd_sync_irpt_r	Output	Edge-triggered SYNC complete, Realm interrupt. The cmd_sync_irpt_r signal asserts a Realm interrupt to indicate that the CMD_SYNC command on the Realm Command queue is complete. Width is 1-bit.
global_irpt_ns	Output	The edge-triggered global_irpt_ns signal asserts a global Non-secure interrupt. Width is 1-bit.
global_irpt_s	Output	The edge-triggered global_irpt_s signal asserts a global Secure interrupt. Width is 1-bit.
global_irpt_r	Output	The edge-triggered global_irpt_r signal asserts a global Realm interrupt. Width is 1-bit.
ras_et_fhi	Output	Edge-triggered fault handling RAS interrupt for a contained or an uncontained error. Note: TCU_ERRCTLR.FI can also enable or disable ras_fhi. See 4.8.2 TCU_ERRCTLR register on page 164. Width is 1-bit. Note: MMU-720AE cannot output this interrupt as an MSI. You must connect this output to an interrupt controller.
ras_et_eri	Output	Edge-triggered error recovery RAS interrupt for an uncontained error. Width is 1-bit. Note: MMU-720AE cannot output this interrupt as an MSI. You must connect this output to an interrupt controller.
ras_et_cri	Output	Edge-triggered critical error interrupt, for an uncontainable uncorrected error. Width is 1-bit. Note: MMU-720AE cannot output this interrupt as an MSI. You must connect this output to an interrupt controller.
ras_lt_fhi	Output	Level-triggered fault handling RAS interrupt for a contained or an uncontained error. Note: TCU_ERRCTLR.FI can also enable or disable ras_fhi. See 4.8.2 TCU_ERRCTLR register on page 164. Width is 1-bit. Note: MMU-720AE cannot output this interrupt as an MSI. You must connect this output to an interrupt controller.
ras_lt_eri	Output	Level-triggered error recovery RAS interrupt for an uncontained error. Width is 1-bit. Note: MMU-720AE cannot output this interrupt as an MSI. You must connect this output to an interrupt controller.

Signal	Direction	Description
ras_lt_cri	Output	Level-triggered critical error interrupt, for an uncontainable uncorrected error. Width is 1-bit. Note: MMU-720AE cannot output this interrupt as an MSI. You must connect this output to an interrupt controller.
ras_lt_irpt_v	Output	Level-triggered valid output for connection to System RAS agents. Asserted when the RAS record contains at least one valid error. Connect the ras_lt_irpt_v signal, together with level-triggered versions of RAS interrupts, to a System RAS agent. Width is 1-bit.
pmu_irpt	Output	Edge-triggered PMU interrupt assertion signal. Width is 1-bit. Note: MMU-720AE cannot output PMU interrupts as MSIs. You must connect this output to an interrupt controller.
pri_q_irpt_ns	Output	Edge-triggered Non-Secure Page Request Interface (PRI) queue interrupt assertion signal. Width is 1-bit.
pri_q_irpt_r	Output	Edge triggered Realm Page Request Interface (PRI) queue interface assertion signal. Width is 1-bit.
gpf_far	Output	Edge-triggered signal to indicate that an error becomes active in SMMU_ROOT_GPF_FAR. Width is 1-bit.
gpt_cfg_far	Output	Edge-triggered signal to indicate that an error becomes active in SMMU_ROOT_GPT_CFG_FAR. Width is 1-bit.

A.1.10 TCU Message Signaled Interrupt interface signals

The MMU-720AE TCU contains a Message Signaled Interrupt (MSI) interface that follows the AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, protocol and uses signals to send MSIs.

The MSI interface uses the signals in the following table to send MSIs.

Signal definitions

Table A-10: TCU MSI interface signals

Signal	Direction	Description	Connection information
msitvalid	Output	Indicates valid data to the GIC. Width is 1-bit.	AXI-Stream signal is TVALID
msitready	Input	Indicates acceptance by the GIC. Width is 1-bit.	AXI-Stream signal is TREADY
msitdata	Output	Data being passed to the GIC. Width is 64-bit.	AXI-Stream signal is TDATA
msitwakeup	Output	Indicates that a transaction is ongoing. Width is 1-bit.	AXI-Stream signal is TWAKEUP, AMBA extension
msirtvalid	Input	Indicates that the GIC has accepted an MSI. Width is 1-bit.	AXI-Stream signal is TVALID
msirtready	Output	Indicates that the device has accepted the response packet. Width is 1-bit.	AXI-Stream signal is TREADY
msirtwakeup	Input	Indicates that a transaction is ongoing. Width is 1-bit.	AXI-Stream signal is TWAKEUP, AMBA extension

For more information about these signals, see the *GIC MSI Delivery Interface* document.

A.1.11 TCU event interface

The TCU event interface is an event handshake for connection to processors.

Signal definitions

Table A-11: TCU event interface signals

Signal	Direction	Description
eventoreq	Output	The eventoreq signal is asserted until the eventoack signal indicates an event that enables processors to wake up from the Wait For Event (WFE) low-power state. The eventoreq signal is asserted to indicate an event that enables processors to wake up from the Wait For Event (WFE) low-power state. The eventoreq signal remains asserted until the eventoack signal is asserted. The eventoreq signal can then be deasserted followed by the eventoack signal. This is a four-phase handshake. Connect the eventoreq signal of the TCU to the event interface of Arm® processors. Processors that use the DynamIQ Shared Unit (DSU) have a different event handshake mechanism. Arm® processors can use the following event mechanisms: - Some processors have an eventi input which only requires a single pulse. This can be connected to the eventoreq, with eventoack also connected to eventoreq to provide a dummy acknowledgement. - Some processors, including DSU-based systems, have a req/ack handshake mechanism that can be connected directly to the corresponding signals on the TCU. Note: You can also route the eventoreq and eventoack signals through other interconnects such as the Neoverse™ CMN-700 Coherent Mesh Network instead of connecting these signals directly to the processor. These interconnects, like the DSU, support only the req/ack handshake mechanism. In both mechanisms, in the signal names: i Represents events that are inputs to a particular component o Represents events that are outputs from a particular component The eventoreq signal is held HIGH until eventoack is observed. Width is 1-bit.
eventoack	Input	Acknowledge signal for eventoreq. The eventoack signal clears the current eventoreq output. If eventoreq is connected to a pulse interface, also connect eventoreq to eventoack to provide a dummy acknowledgment. Width is 1-bit.

For more information, see your processor or DSU documentation.

A.1.12 TCU tie-off signals

The TCU tie-off signals are sampled between exiting reset and the LPI_PD interface first entering the Q_RUN state. Ensure that the value of these signals does not change when the LPI_PD interface is in the Q_STOPPED or Q_EXIT state for the first time after exiting reset.

Signal definitions

Table A-12: TCU tie-off signals

Signal	Direction	Description														
sup_cohacc	Input	This signal indicates whether the QTW interface and the PTW interface, if present, are I/O-coherent. Tie the sup_cohacc signal HIGH when the TCU is connected to a coherent interconnect. Width is 1-bit. Note: If RME mode is enabled, the TCU must be connected to a coherent interconnect. Therefore, you must tie the sup_cohacc signal HIGH.														
sup_btm	Input	This signal indicates whether the Broadcast TLB Maintenance is supported. Tie the sup_btm signal HIGH when the TCU is connected to an interconnect that supports DVM. Width is 1-bit.														
sup_sev	Input	This signal indicates whether the Send Event mechanism is supported. Tie the sup_sev signal HIGH when the eventoreq and eventoack signals are connected. Width is 1-bit.														
sup_oas	Input	Output address size supported. The encodings for this input are as follows: <table><tr><td>0b000</td><td>32 bits</td></tr><tr><td>0b001</td><td>36 bits</td></tr><tr><td>0b010</td><td>40 bits</td></tr><tr><td>0b011</td><td>42 bits</td></tr><tr><td>0b100</td><td>44 bits</td></tr><tr><td>0b101</td><td>48 bits</td></tr><tr><td>0b110</td><td>52 bits</td></tr></table> You must not use other encodings. Other encodings are treated as 0b110. Width is 3-bit.	0b000	32 bits	0b001	36 bits	0b010	40 bits	0b011	42 bits	0b100	44 bits	0b101	48 bits	0b110	52 bits
0b000	32 bits															
0b001	36 bits															
0b010	40 bits															
0b011	42 bits															
0b100	44 bits															
0b101	48 bits															
0b110	52 bits															
sec_override	Input	When HIGH, certain registers are accessible to Non-secure accesses from reset, as the 4.7.5 TCU_SCR register on page 150 settings describe. Width is 1-bit.														
ecorevnum	Input	Tie the ecorevnum signal to 0 unless directed otherwise by Arm. Width is 4-bit.														
msi_addr	Input	If the programmed Message Signaled Interrupt (MSI) address in SMMU_(S/R)_*_IRQ_CFG0.ADDR matches msi_addr, then an MSI is generated on the TCU MSI interface. Width is 52-bit.														
tcu_sid	Input	Used as the DeviceID for TCU-generated MSIs. Note: This is only for MSIs that are issued from the dedicated AXI5-Stream, with Wakeup_Signal enabled and Check_Type not enabled, MSI delivery interface. Width is 32-bit.														

Signal	Direction	Description
sup_httu	Input	0 When set to 0, sup_httu indicates that the ACE-Lite interface that is connected to a system that cannot support atomics. The TCU cannot perform Hardware Translation Table Update (HTTU) transactions. 1 When set to 1, the sup_httu signal indicates that the ACE-Lite interface that is connected to a system that can support atomics. The TCU uses atomic transactions to perform HTTU. Note: TCUx1 If you use the TCUx1 type, the HTTU traffic uses the QTW interface. TCUx2 If you use the TCUx2 type, the HTTU traffic uses the PTW interface. The impact of the sup_httu signal on SMMU_IDR0.HTTU is as follows: sup_httu is 1'b0 SMMU_IDR0.HTTU is 2'b00 sup_httu is 1'b1 SMMU_IDR0.HTTU is 2'b10 See 2.6.1 SMMUv3 implementation on page 76. Width is 1-bit.
legacy_tz_en	Input	Disable Realm Management Extension (RME) and switch to legacy mode. Width is 1-bit.
IO_gpt_sz	Input	Set the value of Level 0 GPT entry size, LOGPTSZ, to use for RME: 0b0000 30-bits. Each entry covers 1GB of address space. 0b0100 34-bits. Each entry covers 16GB of address space. 0b0110 36-bits. Each entry covers 64GB of address space. 0b1001 39-bits. Each entry covers 512GB of address space. Width is 4-bit.

Signal	Direction	Description
hazard_width	Input	Width to create hazards in the HZU unit. Uses the same encodings as AxSIZE, as follows: 3'h3 64 3'h4 128 3'h5 256 3'h6 512 3'h7 Reserved Width is 3-bit. The decoded value must be $\leq$ TCUCFG_QTW_DATA_WIDTH. When you set the hazard_width signal to a larger size, the walk requests from the TCU are performed at a larger size. This larger size can allow the possibility of hazarding more than one page table entry with a single request on the QTW/PTW interfaces. This larger size provides more benefit for sequential workloads which tend to load adjacent pages. We recommend setting this to a size which is natural for the interconnect or memory system to use. For example, if the interconnect transfers all data at 256 bit packets regardless of AxSIZE, setting this to 256 might have very little overhead to the system but provide benefit by reducing PTW traffic.

For more information about the SMMUv3 ID signals, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

A.1.13 TCU ELA debug signals

The MMU-720AE TCU includes Embedded Logic Analyzer (ELA) debug signals.

Signal definitions

Table A-13: ELA enable signals

Signal	Direction	Description
ela_enable	Input	The ela_enable signal is an asynchronous input port. When TCUCFG_USE_ELA_DEBUG is 0, the SMMU ignores the value of the signal. When TCUCFG_USE_ELA_DEBUG is 1, ela_enable acts as a clock enable for the TCU ELA observation interface. If ELA debug is required, drive ela_enable HIGH. If ELA debug is not required, drive ela_enable LOW to reduce the dynamic power consumption of the SMMU. Width is 1-bit.
signalgrp0..11_qtw	Output	Group observed signals. The signalgrp0..11_qtw interface is present on TCUX1 and TCUX2. Width is ELA_GRP_WIDTH-bit

Signal	Direction	Description
sigqual0..11_qtw	Output	Group qualification signal. The sigqual0..11_qtw interface is present on TCUX1 and TCUX2. Width is (ELA_GRP_WIDTH/32)-bit.
sigclken0..11_qtw	Output	Group enable signal. The sigclken0..11_qtw interface is present on TCUX1 and TCUX2. Width is 1-bit.
signalgrp0..11_ptw	Output	Group observed signals. The signalgrp0..11_ptw interface is present only on TCUX2. Width is ELA_GRP_WIDTH-bit
sigqual0..11_ptw	Output	Group qualification signals. The sigqual0..11_ptw interface is present only on TCUX2. Width is (ELA_GRP_WIDTH / 32)-bit
sigclken0..11_ptw	Output	Group enable signals. The sigclken0..11_ptw interface is present only on TCUX2. Width is 1-bit

A.2 TBU signals

MMU-720AE contains TBU signals.

A.2.1 TBU clock and reset signals

The TBU uses a single set of standard clock and reset signals.

Signal definitions

Table A-14: Clock and reset signals

Signal	Direction	Description
clk	Input	Global clock. Width is 1-bit.
resetrn	Input	Global reset. Width is 1-bit.

A.2.2 TBU TBS interface signals

The TBU TBS interface signals are based on the AMBA ACE5-Lite signals. This interface applies to the ACE-Lite TBU components.

Signal definitions

Table A-15: TBU TBS interface signals

Signal	Direction	Description
araddr_s	Input	Read address. Width is 64-bit.
arburst_s	Input	Burst type. Width is 2-bit.
arcache_s	Input	Memory type. Width is 4-bit.
archunken_s	Input	Read data chunking enable. Width is 1-bit.
arpbha_s	Input	Page-based hardware attributes. Width is 4-bit.
artagop_s	Input	Indicates whether MTE tags are associated with a read transaction. Width is 2-bit.
armmuvalid_s	Input	MMU qualifier signal. When deasserted, the transaction address is a physical address and does not require translation. Width is 1-bit.
ardomain_s	Input	Shareability domain. Width is 2-bit.
arid_s	Input	Read address ID The width of arid_s is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
aridunq_s	Input	Read address channel unique ID indicator, active-HIGH. Width is 1-bit.
arlen_s	Input	Burst length. Width is 8-bit.
arlock_s	Input	Lock type. Width is 1-bit.
arloop_s	Input	Loopback value for a read transaction. Reflected back on RLOOP. The width of arloop_s is TBUCFG_LOOP_WIDTH-bit. For information about how to set the TBUCFG_LOOP_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
armecid_s	Input	MECID for requests on the read request channel. Note: This value is considered only for NoStreamID transactions. Width is 0, 12 or 16 bits depending on the TBUCFG_MECID_WIDTH parameter. For information about how to set the TBUCFG_MECID_WIDTH parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105
armmuflow_s	Input	Indicates the SMMU flow for managing translation faults. Width is 2-bit.
armmussid_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. The width of armmusid_s is TBUCFG_SSID_WIDTH-bit. For information about how to set the TBUCFG_SSID_WIDTH parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Signal	Direction	Description
armmusid_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. The width of armmusid_s is TBUCFG_SSID_WIDTH-bit. For information about how to set the TBUCFG_SSID_WIDTH parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
armmussidv_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. Width is 1-bit.
armmusecsid_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. Width is SECSID_WIDTH-bit.
arprot_s	Input	Protection type. Width is 3-bit.
arnse_s	Input	Combines with arprot_s to define the PAS. Width is 1-bit. If the TBUCFG_LEGACY_TZ_EN parameter is set to 1, the arnse_s signal is 1-bit, unused, and tied-off internally.
arqos_s	Input	Quality of Service (QoS). Width is 4-bit.
arready_s	Output	Read address ready. Width is 1-bit.
arregion_s	Input	Region identifier. Width is 4-bit.
arsize_s	Input	Burst size. Width is 3-bit.
arsnoop_s	Input	Transaction type of read transaction. Width is 4-bit.
aruser_s	Input	Read address (AR) channel User signal Calculate the width of aruser_s as follows: $(TBUCFG_ARUSER_WIDTH + LTI_TLBLOC_WIDTH_RAW -)\text{-bit.}$ Calculate the LTI_TLBLOC_WIDTH_RAW internal parameter as follows: $LTI_TLBLOC_WIDTH_RAW = ((TBUCFG_DIRECT_IDX == 1) ? (TBUCFG_MTLB_DEPTH > 0) ? \log_2(TBUCFG_MTLB_DEPTH) : \log_2(4) : \log_2(TBUCFG_MTLB_PARTS))$ For information about how to set these parameters, see 2.7 Configuration parameters and methodology on page 102.
arvalid_s	Input	Read address valid. Width is 1-bit.
rchunknum_s	Output	Read data chunk number. Width is CHUNKNUM_WIDTH-bit.
rchunkstrb_s	Output	Read data chunk strobe. Width is CHUNKSTRB_WIDTH-bit.

Signal	Direction	Description
rchunkv_s	Output	Valid signal of RCHUNKNUM and RCHUNKSTRB. Width is 1-bit.
rdata_s	Output	Read data. The width of rdata_s is TBUCFG_DATA_WIDTH-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rid_s	Output	Read ID. The width of rid_s is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
ridunq_s	Output	Read data channel unique ID indicator, active-HIGH. Width is 1-bit.
rlast_s	Output	Read last. Width is 1-bit.
rloop_s	Output	Loopback value for a read response The width of rloop_s is TBUCFG_LOOP_WIDTH-bit. For information about how to set the TBUCFG_LOOP_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rpoison_s	Output	Indicates that the read data in this transfer has been corrupted The width of rpoison_s is (TBUCFG_DATA_WIDTH / 64)-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rready_s	Input	Read ready. Width is 1-bit.
rresp_s	Output	Read response. Width is 3-bit.
rtag_s	Output	Memory tag associated with data. Width is $\text{ceil}(\text{TBUCFG_DATA_WIDTH} / 128) \times 4$.
ruser_s	Output	Read data (R) channel User signal The width of ruser_s is TBUCFG_RUSER_WIDTH-bit. For information about how to set the TBUCFG_RUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rvalid_s	Output	Read valid. Width is 1-bit.
awaddr_s	Input	Write address. Width is 64-bit.
awatop_s	Input	Atomic operation. Width is 6-bit.
awburst_s	Input	Burst type. Width is 2-bit.
awcache_s	Input	Memory type. Width is 4-bit.
awdomain_s	Input	Shareability domain. Width is 2-bit.
awid_s	Input	Write address ID The width of awid_s is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
awlen_s	Input	Burst length. Width is 8-bit.
awlock_s	Input	Lock type. Width is 1-bit.

Signal	Direction	Description
awmeced_s	Input	MECID for requests on the write request channel. Note: This value is considered only for NoStreamID transactions. Width is 0, 12, or 16 bits depending on the <code>TBUCFG_MECID_WIDTH</code> parameter. For information about how to set the <code>TBUCFG_MECID_WIDTH</code> parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
awmmuflow_s	Input	Indicates the SMMU flow for managing translation faults. Width is 2-bit.
awmmussid_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. The width of awmmusid_s is <code>TBUCFG_SSID_WIDTH</code>-bit. For information about how to set the <code>TBUCFG_SSID_WIDTH</code> parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
awmmusid_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. The width of awmmusid_s is <code>TBUCFG_SID_WIDTH</code>-bit. For information about how to set the <code>TBUCFG_SID_WIDTH</code> parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
awmmussidv_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. Width is 1-bit.
awmmusecsid_s	Input	These signals indicate the StreamID, SubstreamID, and ATS translated status of the originating transaction. The AXI5 Untranslated_Transactions extension defines these signals. For information about how to set these parameters, see 2.4.2.2 ACE-Lite TBU TBM interface on page 38 and 2.6.2 AMBA implementation on page 81. Width is <code>SECSID_WIDTH</code>-bit.
awmmuvalid_s	Input	MMU qualifier signal. When deasserted, the transaction address is a physical address and does not require translation. Width is 1-bit.
awpbha_s	Input	Page-based hardware attributes. Width is 4-bit.
awtagop_s	Input	Indicates whether MTE tags are associated with a write transaction. Width is 2-bit.
awprot_s	Input	Protection type. Width is 3-bit.
awnse_s	Input	Combines with awprot_s to define the PAS. Width is 1-bit. If the <code>TBUCFG_LEGACY_TZ_EN</code> parameter is set to 1, the awnse_s signal is 1-bit, unused, and tied-off internally.
awqos_s	Input	QoS. Width is 4-bit.
awready_s	Output	Write address ready. Width is 1-bit.

Signal	Direction	Description
awregion_s	Input	Region identifier. Width is 4-bit.
awsize_s	Input	Burst size. Width is 3-bit.
awuser_s	Input	Write address (AW) channel User signal The width of awuser_s is (TBUCFG_AWUSER_WIDTH + LTI_TLBLOC_WIDTH_RAW)-bit. Calculate the LTI_TLBLOC_WIDTH_RAW internal parameter as follows: $\text{LTI_TLBLOC_WIDTH_RAW} = (\text{TBUCFG_DIRECT_IDX} == 1) ? (\text{TBUCFG_MTLB_DEPTH} > 0) ? \log_2(\text{TBUCFG_MTLB_DEPTH}) : \log_2(4) : \log_2(\text{TBUCFG_MTLB_PARTS})$ For information about how to set these parameters, see 2.7 Configuration parameters and methodology on page 102.
awvalid_s	Input	Write address valid. Width is 1-bit.
awakeup_s	Input	Wakeup signal. Width is 1-bit.
awsnoop_s	Input	Transaction type of write transaction. Width is 5-bit.
awstashnid_s	Input	The AXI5 Cache_Stash_Transactions extension defines the awstashnid_s signal. If the TBUCFG_STASH_SUPPORT parameter is set to 0, the awstashnid_s[10:0] signal is ignored. Width is 11-bit.
awstashniden_s	Input	The AXI5 Cache_Stash_Transactions extension defines the awstashniden_s signal. If the TBUCFG_STASH_SUPPORT parameter is set to 0, the awstashniden_s signal is ignored. Width is 1-bit.
awstashlpid_s	Input	The AXI5 Cache_Stash_Transactions extension defines the awstashlpid_s[4:0] signal. If the TBUCFG_STASH_SUPPORT parameter is set to 0, the awstashlpid_s[4:0] signal is ignored. Width is 5-bit.
awstashlpiden_s	Input	The AXI5 Cache_Stash_Transactions extension defines the awstashlpiden_s signal. If the TBUCFG_STASH_SUPPORT parameter is set to 0, the awstashlpiden_s signal is ignored. Width is 1-bit.
awidunq_s	Input	Write address channel unique ID indicator, active-HIGH. Width is 1-bit.
awloop_s	Input	Loopback value for a write transaction The width of awloop_s is TBUCFG_LOOP_WIDTH-bit. For information about how to set the TBUCFG_LOOP_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
wdata_s	Input	Write data The width of wdata_s is TBUCFG_DATA_WIDTH-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
wlast_s	Input	Write last. Width is 1-bit.
wpoison_s	Input	Indicates that the write data in this transfer has been corrupted The width of wpoison_s is [(TBUCFG_DATA_WIDTH / 64)-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
wready_s	Output	Write ready. Width is 1-bit.
wstrb_s	Input	Write strobes The width of wstrb_s is (TBUCFG_DATA_WIDTH / 8)-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.

Signal	Direction	Description
wtag_s	Input	Memory tag associated with data. Width is $\text{ceil}(\text{TBUCFG_DATA_WIDTH} / 128) \times 4$.
wtagupdate_s	Input	Indicates which tags must be written to memory when AWTAGOP is Update. Width is $\text{ceil}(\text{TBUCFG_DATA_WIDTH} / 128)$.
wuser_s	Input	Write data (W) channel User signal The width of wuser_s is TBUCFG_WUSER_WIDTH-bit. For information about how to set the TBUCFG_WUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
wvalid_s	Input	Write valid. Width is 1-bit.
bid_s	Output	Response ID The width of bid_s is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bidunq_s	Output	Write response channel unique ID indicator, active-HIGH. Width is 1-bit.
bloop_s	Output	Loopback value for a write response The width of bloop_s is TBUCFG_LOOP_WIDTH-bit. For information about how to set the TBUCFG_LOOP_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bready_s	Input	Response ready. Width is 1-bit.
bresp_s	Output	Write response. Width is 3-bit.
buser_s	Output	Write response (B) channel User signal The width of buser_s is TBUCFG_BUSER_WIDTH-bit. For information about how to set the TBUCFG_BUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bvalid_s	Output	Write response valid. Width is 1-bit.

A.2.3 TBU TBM interface signals

The TBU TBM interface signals are based on the AMBA ACE5-Lite signals. This interface applies to the ACE-Lite TBU components.

Signal definitions

Table A-16: TBU TBM interface signals

Signal	Direction	Description
araddr_m	Output	Read address. Width is 52-bit.
armmuvalid_m	Output	MMU qualifier signal. Width is 1-bit.
arpbha_m	Output	Page-based hardware attributes. Width is 4-bit.
artagop_m	Output	Indicates whether MTE tags are associated with a read transaction. Width is 2-bit.
arburst_m	Output	Burst type. Width is 2-bit.
arcache_m	Output	Memory type. Width is 4-bit.
archunken_m	Output	Read data chunking enable. Width is 1-bit.
ardomain_m	Output	Shareability domain. Width is 2-bit.

Signal	Direction	Description
arid_m	Output	Read address ID. The width of arid_m is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
aridunq_m	Output	Read address channel unique ID indicator, active-HIGH. Width is 1-bit.
arlen_m	Output	Burst length. Width is 8-bit.
arlock_m	Output	Lock type. Width is 1-bit.
arloop_m	Output	Loopback value for a read transaction. Reflected back on RLOOP. Calculate the width of arloop_m as follows: If TBUCFG_OT_TRACKER_TYPE is 1, the width of arloop_m is (TBUCFG_LOOP_WIDTH + 2)-bit. Otherwise, the width of arloop_m is TBUCFG_LOOP_WIDTH-bit. For information about how to set these parameters, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
armecid_m	Output	MECID for requests on the read request channel. Width is 0, 12, or 16 bits depending on the TBUCFG_MECID_WIDTH parameter. For information about how to set the TBUCFG_MECID_WIDTH parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
armmusid_m	Output	Indicates the StreamID of the originating transaction. The width of armmusid_m is TBUCFG_SID_WIDTH-bit. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
armmusecsid_m	Output	Indicates the Secure StreamID of the originating transaction. Width is 2-bit.
armpam_m	Output	Read address channel MPAM information. If TBUCFG_LEGACY_TZ_EN = 1, then the width of armpam_m is 14-bit. Otherwise, the width of armpam_m is 15-bit.
arprot_m	Output	Protection type. Width is 3-bit.
arnse_m	Output	Combines with arprot_m to define the PAS. Width is 1-bit. When TBUCFG_LEGACY_TZ_EN is set to 1, the arnse_m signal is 1-bit, unused, and tied-off internally.
arqos_m	Output	Quality of Service (QoS). Width is 4-bit.
arready_m	Input	Read address ready. Width is 1-bit.
arregion_m	Output	Region identifier. Width is 4-bit.
arsize_m	Output	Burst size. Width is 3-bit.
arsnoop_m	Output	Transaction type of read transaction. Width is 4-bit.
aruser_m	Output	Read address (AR) channel User signal. The width of aruser_m is (TBUCFG_ARUSER_WIDTH + 1)-bit. For information about how to set the TBUCFG_ARUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107. For information about the extra aruser bit, see 2.5.12 AXI USER bits that the SMMU TBU TBM defines on page 74.
arvalid_m	Output	Read address valid. Width is 1-bit.

Signal	Direction	Description
rchunknum_m	Input	Read data chunk number. Width can be 1-bit, 5-bit, 6-bit, 7-bit, or 8-bit. Calculate the width of rchunknum_m as follows: If TBUCFG_DATA_WIDTH is 128, then the width of rchunknum_m is 8-bit. If TBUCFG_DATA_WIDTH is 256, then the width of rchunknum_m is 7-bit. If TBUCFG_DATA_WIDTH is 512, then the width of rchunknum_m is 6-bit. If TBUCFG_DATA_WIDTH is 1024, then the width of rchunknum_m is 5-bit. Otherwise, the width of rchunknum_m is 1-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rchunkstrb_m	Input	Read data chunk strobe. Calculate the width of rchunkstrb_m as follows: If TBUCFG_DATA_WIDTH is 64, then the width of rchunkstrb_m is 1-bit. Otherwise, the width of rchunkstrb_m is $(TBUCFG_DATA_WIDTH / 128)$-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rchunkv_m	Input	Valid signal of RCHUNKNUM and RCHUNKSTRB. Width is 1-bit.
rdata_m	Input	Read data. The width of rdata_m is TBUCFG_DATA_WIDTH-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rid_m	Input	Read ID. The width of rid_m is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
ridunq_m	Input	Read data channel unique ID indicator, active-HIGH. Width is 1-bit.
rlast_m	Input	Read last. Width is 1-bit.
rloop_m	Input	Loopback value for a read response. The width of rloop_m is calculated as follows: If TBUCFG_OT_TRACKER_TYPE is 1, the width of rloop_m is $(TBUCFG_LOOP_WIDTH + 2)$-bit. Otherwise, the width of rloop_m is TBUCFG_LOOP_WIDTH-bit. For information about how to set these parameters, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rpoison_m	Input	Indicates that the read data in this transfer has been corrupted. The width of rpoison_m is $(TBUCFG_DATA_WIDTH / 64)$-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rready_m	Output	Read ready. Width is 1-bit.
rresp_m	Input	Read response. Width is 2-bit.

Signal	Direction	Description
rtag_m	Input	Memory tag associated with data. Width is $\text{ceil}(\text{TBUCFG_DATA_WIDTH}/128) \times 4$.
ruser_m	Input	Read data (R) channel User signal The width of ruser_m is TBUCFG_RUSER_WIDTH-bit. For information about how to set the TBUCFG_RUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
rvalid_m	Input	Read valid. Width is 1-bit.
awmmuvalid_m	Output	MMU qualifier signal. Width is 1-bit.
awpbha_m	Output	Page-based hardware attributes. Width is 4-bit.
awtagop_m	Output	Indicates whether MTE tags are associated with a write transaction. Width is 2-bit.
awaddr_m	Output	Write address. Width is 52-bit.
awatop_m	Output	Atomic operation. Width is 6-bit.
awburst_m	Output	Burst type. Width is 2-bit.
awcache_m	Output	Memory type. Width is 4-bit.
awdomain_m	Output	Shareability domain. Width is 2-bit.
awid_m	Output	Write address ID The width of awid_m is TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
awlen_m	Output	Burst length. Width is 8-bit.
awlock_m	Output	Lock type. Width is 1-bit.
awmecid_m	Output	MECID for requests on the write request channel. Width is 0, 12 or 16 bits depending on the TBUCFG_MECID_WIDTH parameter. For information about how to set the TBUCFG_MECID_WIDTH parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105
awmmusid_m	Output	Indicates the StreamID of the originating transaction. The Generic Interrupt Controller (GIC) uses these signals to determine the DeviceID of MSIs that originate from upstream managers. The width of awmmusid_m is TBUCFG_SID_WIDTH-bit. For information about how to set the TBUCFG_SID_WIDTH parameter, see 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
awmmusecsid_m	Output	Indicates the Secure StreamID of the originating transaction. The GIC uses these signals to determine the DeviceID of MSIs that originate from upstream managers. Width is 2-bit.
awprot_m	Output	Protection type. Width is 3-bit.
awnse_m	Output	Combines with awprot_m to define the PAS. Width is 1-bit. When TBUCFG_LEGACY_TZ_EN is set to 1, the awnse_m signal is 1-bit, unused, and tied-off internally.
awqos_m	Output	QoS. Width is 4-bit.
awready_m	Input	Write address ready. Width is 1-bit.
awregion_m	Output	Region identifier. Width is 4-bit.

Signal	Direction	Description
awsize_m	Output	Burst size. Width is 3-bit.
awstashnid_m	Output	The AXI5 Cache_Stash_Transactions extension defines this signal. See AMBA® AXI Protocol Specification . If TBUCFG_STASH_SUPPORT = 0, these signals are ignored. Width is 11-bit.
awstashniden_m	Output	The AXI5 Cache_Stash_Transactions extension defines this signal. See AMBA® AXI Protocol Specification . If TBUCFG_STASH_SUPPORT = 0, these signals are ignored. Width is 1-bit.
awstashlpid_m	Output	The AXI5 Cache_Stash_Transactions extension defines this signal. See AMBA® AXI Protocol Specification . If TBUCFG_STASH_SUPPORT = 0, these signals are ignored. Width is 5-bit.
awstashlpiden_m	Output	The AXI5 Cache_Stash_Transactions extension defines this signal. See AMBA® AXI Protocol Specification . If TBUCFG_STASH_SUPPORT = 0, these signals are ignored. Width is 1-bit.
awakeup_m	Output	Wakeup signal. Width is 1-bit.
awidunq_m	Output	Write address channel unique ID indicator, active-HIGH. Width is 1-bit.
awloop_m	Output	Loopback value for a write transaction. The width of awloop_m is calculated as follows: If TBUCFG_OT_TRACKER_TYPE is 1, the width of awloop_m is (TBUCFG_LOOP_WIDTH + 2)-bit. Otherwise, the width of awloop_m is TBUCFG_LOOP_WIDTH-bit. For information about how to set these parameters, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
awmpam_m	Output	Write address channel MPAM information. If TBUCFG_LEGACY_TZ_EN = 1, then the width of awmpam_m is 14-bit. Otherwise, the width of awmpam_m is 15-bit.
awsnoop_m	Output	Transaction type of write transaction. Width is 5-bit.
awuser_m	Output	Write address (AW) channel User signal. The width of awuser_m is (TBUCFG_AWUSER_WIDTH + 1)-bit. For information about how to set the TBUCFG_AWUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107. For information about the extra awuser bit, see 2.5.12 AXI USER bits that the SMMU TBU TBM defines on page 74.
awvalid_m	Output	Write address valid. Width is 1-bit.
wtag_m	Output	Memory tag associated with data. Width is $\text{ceil}(\text{TBUCFG_DATA_WIDTH} / 128) \times 4$.
wtagupdate_m	Output	Indicates which tags must be written to memory when AWTAGOP is Update. Width is $\text{ceil}(\text{TBUCFG_DATA_WIDTH} / 128)$.
wdata_m	Output	Write data The width of wdata_m is TBUCFG_DATA_WIDTH-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.

Signal	Direction	Description
wlast_m	Output	Write last. Width is 1-bit.
wpoison_m	Output	Indicates that the write data in this transfer has been corrupted. The width of wpoison_m is (TBUCFG_DATA_WIDTH / 64)-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
wready_m	Input	Write ready. Width is 1-bit.
wstrb_m	Output	Write strobes. The width of wstrb_m is (TBUCFG_DATA_WIDTH / 8)-bit. For information about how to set the TBUCFG_DATA_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
wvalid_m	Output	Write valid. Width is 1-bit.
wuser_m	Output	Write data (W) channel User signal. The width of wuser_m is TBUCFG_WUSER_WIDTH-bit. For information about how to set the TBUCFG_WUSER_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bidunq_m	Input	Write response channel unique ID indicator, active-HIGH. Width is 1-bit.
bid_m	Input	Response ID. TBUCFG_ID_WIDTH-bit. For information about how to set the TBUCFG_ID_WIDTH parameter, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bloop_m	Input	Loopback value for a write response. If TBUCFG_OT_TRACKER_TYPE is 1, the width of awloop_m is (TBUCFG_LOOP_WIDTH + 2)-bit. Otherwise, the width of awloop_m is TBUCFG_LOOP_WIDTH-bit. For information about how to set these parameters, see 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bready_m	Output	Response ready. Width is 1-bit.
bresp_m	Input	Write response. Width is 2-bit.
buser_m	Input	Write response (B) channel User signal. The width of buser_m is TBUCFG_BUSER_WIDTH-bit. See 2.7.5 ACE-Lite TBU I/O configuration parameters on page 107.
bvalid_m	Input	Write response valid. Width is 1-bit.

A.2.4 TBU LPI_PD interface signals

The following table shows the TBU LPI_PD interface signals.

Signal definitions

Table A-17: TBU LPI_PD interface signals

Signal	Direction	Description
qactive_pd	Output	Component active. Width is 1-bit.
qreqn_pd	Input	Quiescence request. Width is 1-bit.
qacceptn_pd	Output	Quiescence accept. Width is 1-bit.
qdeny_pd	Output	Quiescence deny. Width is 1-bit.

A.2.5 TBU LPI_CG interface signals

The following table shows the TBU LPI_CG interface signals.

Signal definitions

Table A-18: TBU LPI_CG interface signals

Signal	Direction	Description
qactive_cg	Output	Component active. Width is 1-bit.
qreqn_cg	Input	Quiescence request. Width is 1-bit.
qacceptn_cg	Output	Quiescence accept. Width is 1-bit.
qdeny_cg	Output	Quiescence deny. Width is 1-bit.

A.2.6 TBU DTI interface signals

The following table shows the TBU DTI interface signals.

In the following table, the 'Direction' has the following meaning:

Input

TCU to TBU

Output

TBU to TCU

Signal definitions

Table A-19: TBU DTI interface signals

Signal	Direction	Description
tvalid_dti_dn	Output	Flow control signal. Width is 1-bit.
tready_dti_dn	Input	Flow control signal. Width is 1-bit.
tdata_dti_dn	Output	Message data signal. Width is 160-bit.
tlast_dti_dn	Output	Indicates the last cycle of a message. Width is 1-bit.
tkeep_dti_dn	Output	Indicates valid bytes. Width is 20-bit.
tvalid_dti_up	Input	Flow control signal. Width is 1-bit.

Signal	Direction	Description
tready_dti_up	Output	Flow control signal. Width is 1-bit.
tdata_dti_up	Input	Message data signal. Width is 160-bit or 192-bit. If TBUCFG_MECID_WIDTH is 0 or TBUCFG_LEGACY_TZ_EN is 1, then the width of this signal is 160-bit, otherwise it is 192-bit.
tlast_dti_up	Input	Indicates the last cycle of a message. Width is 1-bit.
tkeep_dti_up	Input	Indicates valid bytes. Width is 20-bit or 24-bit. If TBUCFG_MECID_WIDTH is 0 or TBUCFG_LEGACY_TZ_EN is 1, then the width of this signal is 20-bit, otherwise it is 24-bit.
twakeup_dti_up	Input	Wakeup signal. Width is 1-bit.
twakeup_dti_dn	Output	Wakeup signal. Width is 1-bit.

For more information about the DTI signals, see the [AMBA® AXI-Stream Protocol Specification](#).

For more information about DTI protocol messages, see the [AMBA® DTI Protocol Specification](#).

A.2.7 TBU LTI interface signals

This interface applies to the TBU LTI components.

This interface implements an LTI interface as the [AMBA® LTI Protocol Specification](#) defines. The *AMBA LTI Specification* uses several properties to define signal widths.

[2.6.3 Local Translation Interface implementation](#) on page 92 describes how MMU-720AE defines the LTI interface properties. For TBU LTI components with multiple interfaces, these have 2, 4, or 8 instances of an LTI interface. In these components, each LTI signal has the port number appended, starting from 0.

For the descriptions of the LTI request channel, response channel, and completion channel signals, see [A.5 LTI signals](#) on page 331.

A.2.8 TBU interrupt signals

The TBU interrupt signals consist of edge-triggered and level-triggered interrupts. For edge-triggered interrupts, the interrupt controller must detect the rising edge of these signals.

The MMU-720AE TBU cannot output these interrupts as Message Signaled Interrupts (MSIs). These signals must be connected to an interrupt controller.



Note

In some cases it is possible for the SMMU to enter a powered-down state and even reset after issuing an interrupt, removing the evidence of the source of the interrupt. Always ensure that you write interrupt-handling software to handle such an occurrence.

Signal definitions

Table A-20: TBU interrupt signals

Signal	Direction	Description
ras_et_fhi	Output	Edge-triggered fault handling RAS interrupt for a contained error. Width is 1-bit.
ras_et_eri	Output	Edge-triggered error recovery RAS interrupt for an uncontained error. Width is 1-bit.
ras_et_cri	Output	Edge-triggered critical error interrupt, for an uncontainable uncorrected error. Width is 1-bit.
ras_lt_fhi	Output	Level-triggered fault handling RAS interrupt for a contained error. Width is 1-bit.
ras_lt_eri	Output	Level-triggered error recovery RAS interrupt for an uncontained error. Width is 1-bit.
ras_lt_cri	Output	Level-triggered critical error interrupt, for an uncontainable uncorrected error. Width is 1-bit.
ras_lt_irpt_v	Output	Level-triggered valid output for connection to System RAS agents. Asserted when the RAS record contains at least one valid error. Connect the ras_lt_irpt_v signal, together with level triggered versions of RAS interrupts, to a System RAS agent Width is 1-bit.
pmu_irpt	Output	Edge-triggered PMU interrupt. Width is 1-bit.
crit_err	Output	Level-triggered critical error. Cannot connect to the TCU because of deny response or insufficient tokens, or max_tok_trans is smaller than required for the TBU to function. Note: max_tok_trans must be $\geq (2 \times \text{NUM_LTI_PORTS})$. Note: The crit_err interrupt is level-based to make clock domain crossings easy to manage. Note: Once triggered, software cannot clear the crit_err interrupt signal. Resetting the TBU is the only way to clear the crit_err interrupt signal. Width is 1-bit.

A.2.9 TBU tie-off signals

The TBU tie-off signals are sampled between exiting reset and the LPI_PD interface first entering the Q_RUN state. Ensure that the signal values do not change when the LPI_PD interface is in the Q_STOPPED or Q_EXIT state for the first time after exiting reset.

Signal definitions

Table A-21: TBU tie-off signals

Signal	Direction	Description
ns_sid_high	Input	Provides the high-order StreamID bits for all transactions with a Non-secure StreamID that pass through the TBU. The width of ns_sid_high is (32 - TBUCFG_SID_WIDTH)-bit. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
s_sid_high	Input	Provides the high-order StreamID bits for all transactions with a Secure StreamID that pass through the TBU. The width of s_sid_high is (32 - TBUCFG_SID_WIDTH)-bit. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
r_sid_high	Input	Provides the high-order StreamID bits for all transactions with a Realm StreamID that pass through the TBU. The width of r_sid_high is (32 - TBUCFG_SID_WIDTH)-bit. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
max_tok_trans	Input	Indicates the number of DTI translation tokens to request when connecting to the TCU, minus 1. Note: The TBU must request a minimum of two translation tokens per LTI port. However, we recommend one translation token per translation slot for most scenarios. The ACE-Lite TBU has a single internal LTI port, and the LTI TBU can have multiple LTI ports. The width of max_tok_trans is (log ₂ TBUCFG_XLATE_SLOTS)-bit. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.

Signal	Direction	Description
pcie_mode	Input	You must tie this signal HIGH when the TBU is connected to a PCIe interface. When this signal is HIGH, the TBU interprets the input AXI memory types as encoding PCI No Snoop information. For the TBU to provide correct operation, transactions from the PCIe interface must be delivered to the TBU with the following AXI memory types: Normal Non-cacheable Bufferable If No Snoop is set Normal Write-Back Allocate or Non-allocate Outer Shareable If No Snoop is not set and TH is 0 Normal Write-Back Allocate Outer Shareable If No Snoop is not set and TH is 1 It is not recommended that other memory types are used when pcie_mode is set. If this signal is HIGH, the input attribute and shareability override information in the STE fields MTCFG, SHCFG, and ALLOCCFG are ignored. That is, the transaction attributes are always calculated as if the DTI_TBU_TRANS_RESP.STRW field is EL1-S2, regardless of the actual STRW value. If this signal is HIGH, the input attribute and Shareability override information in the ATTR_OVR field of the DTI_TBU_TRANS_RESP message is ignored. pcie_mode is not compatible with the SMMUv3.2-S2FWB feature. It is not compatible because both features attempt to override the memory type of the transaction - FWB based on the page tables and pcie_mode based on the incoming memory type. pcie_mode is applied after FWB. For example, if FWB is attempting to override the memory type to Write-Back cacheable, but the incoming memory type is Non-cacheable, the final memory type will be Non-cacheable despite the FWB setting. If the system requires FWB to always determine the memory type, then pcie_mode must be set to 0 and the PCIe No Snoop feature cannot be supported. Width is 1-bit.
same_power_domain	Input	If the same_power_domain signal is HIGH, the TBU sets the DTI_TBU_CONDIS_REQ.SPD value to 1 when it attempts to connect to the TCU. The TCU then ignores the connection status of this TBU when calculating its PD_QACTIVE output. Tie the same_power_domain signal HIGH when the TBU is in the same power domain as the TCU to which it is connecting. Tying the same_power_domain signal HIGH simplifies the connection of the power management logic.
sec_override	Input	When HIGH, some registers are accessible to Non-secure accesses from reset, as the 4.16.3 TBU_SCR register on page 229 settings describe. Width is 1-bit.
ecorevnum	Input	Tie this signal to 0 unless we recommend otherwise. Width is 4-bit.

Signal	Direction	Description
utlb_roundrobin	Input	Defines the MicroTLB entry replacement policy. When LOW, the MicroTLB uses a Pseudo Least Recently Used (PLRU) replacement policy. This policy typically provides the best average performance. When HIGH, the MicroTLB uses a round-robin replacement policy. With this policy, the oldest entry is evicted when the MicroTLB is full. Tie this signal HIGH if you want to prevent newer translations from being evicted, even if older translations have been used more recently. Otherwise, tie this signal LOW. Width is 1-bit.
poison_support	Input	Note: poison_support applies only to the ACE-Lite TBU. Determines how the ACE-Lite TBU handles RAS errors in the write data buffer. When LOW, the ACE-Lite TBU does not drive the wpoison signal HIGH after detecting an uncorrectable error in the write data buffer. The ACE-Lite TBU reports an uncontrollable uncorrected RAS error. When HIGH, the ACE-Lite TBU drives the wpoison signal HIGH after detecting an uncorrectable error in the write data buffer and reports a deferred RAS error. wpoison is driven HIGH for all write data beats of the corrupted transaction. There is no effect the pass-through of the wpoison signal from the TBS interface to the TBM interface, or rpoison from the TBM interface to the TBS interface. That is, if poison_support is LOW, and wpoison is driven HIGH on the TBS interface, then the ACE-Lite TBU drives the TBM wpoison HIGH for the related transaction. However, it is expected that poison_support is a system-wide setting. It is expected that if poison_support is LOW, no components in the system can generate poison. Width is 1-bit.
legacy_tz_en	Input	Disable Realm Management Extension (RME) and switch to legacy mode. Width is 1-bit.

A.2.10 TBU ELA debug signals

The MMU-720AE TBU includes Embedded Logic Analyzer (ELA) debug signals.

Signal definitions

Table A-22: ELA enable signals

Signal	Direction	Description
ela_enable	Input	The ela_enable signal is an asynchronous input port. When TBUCFG_USE_ELA_DEBUG is 0, the SMMU ignores the value of the signal. When TBUCFG_USE_ELA_DEBUG is 1, ela_enable acts as a clock enable for the TBU ELA observation interface. If ELA debug is required, drive ela_enable HIGH. If ELA debug is not required, drive ela_enable LOW to reduce the dynamic power consumption of the SMMU. Width is 1-bit.
signalgrp0..11	Output	Group observed signals. Width is ELA_GRP_WIDTH bit.
sigqual0..11	Output	Group qualification signal. Width is (ELA_GRP_WIDTH/32) bit.
sigclken0..11	Output	Group enable signal. Width is 1-bit.

A.3 TCU and TBU shared signals

Some signals that MMU-720AE uses are shared by the TCU and TBU.

A.3.1 TCU and TBU test and debug signals

The test and debug signals are common to the TCU and TBU.

Signal definitions

Table A-23: Test and debug signals

Signal	Direction	Description
dftcgen	Input	Clock gate enable. To enable architectural clock gates for the clock, clk, set this signal HIGH during scan shift. Width is 1-bit.
dfrstdisable	Input	Reset disable. To disable reset, set this signal HIGH during scan shift. Width is 1-bit.
dftramhold	Input	Preserve RAM state. To preserve the state of the RAMs and their connected registers, set this signal HIGH during scan shift. Width is 1-bit.
nMBISTRESET	Input	MBIST mode reset. This active-LOW signal is encoded as follows: 0 Reset MBIST functional logic. 1 Normal operation. To prevent unintended reset of the functional logic, keep the MBISTRESETN signal in the inactive state, HIGH, during scan testing. Width is 1-bit.
MBISTREQ	Input	MBIST test request. This signal is encoded as follows: 0 Normal operation. 1 Enable MBIST testing. Width is 1-bit.

A.3.2 PMU Snapshot signals

The PMU snapshot interface is common to both the TCU and TBU. The PMU snapshot interface enables you to capture the values of event counters at a specific times.

Event counters can count the following events:

- Number of transactions
- Number of cache look-ups
- Number of cache misses

The event counter values can provide useful information about the performance of your system. For a full list of countable performance events, see [2.5.2.1 SMMUv3 architectural performance events](#) on page 47.

The *Key performance counter metrics* section in the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual* provides information about how you can use these performance events to obtain useful metrics for assessing the performance of your system.

To use the PMU snapshot interface, you can connect the pmusnapshot_req and pmusnapshot_ack signals to a CoreSight Cross Trigger Interface (CTI). Both signals are LOW after reset. When the pmusnapshot_req input to the TBU or TCU is set HIGH, the rising edge causes a snapshot to occur where all of the event counter values are simultaneously copied from the SMMU_PMCG_EVCNTRn event counter registers and saved to the SMMU_PMCG_SVRn shadow value registers.

Software can then read the counter values that are saved to the SMMU_PMCG_SVRn registers. The pmusnapshot_ack signal is used to acknowledge that the snapshot has occurred and that the event counter values have been saved.

Using the PMU snapshot interface to capture event counter values is optional.

An alternative approach is to use software to write a value of 1 to the CAPTURE bit of the SMMU_PMCG_CFGR register. The hardware and software approaches have the same effect and cause the event counter values to be captured and saved to the SMMU_PMCG_SVRn registers. If you do not use the PMU snapshot interface, then you can tie the pmusnapshot_req input signal to the TBU or TCU to 0. Leave the pmusnapshot_ack output signal unconnected.

The ELA debug signals interface can initiate a PMU snapshot. See [A.1.13 TCU ELA debug signals](#) on page 305 and [A.2.10 TBU ELA debug signals](#) on page 323.

The PMU snapshot interface is asynchronous:

- pmusnapshot_req is sampled via synchronizing registers.
- pmusnapshot_ack is driven from a register.

Signal definitions

Table A-24: PMU snapshot signals

Signal	Direction	Description
pmusnapshot_req	Input	PMU snapshot request. The PMU snapshot occurs on the rising edge of pmusnapshot_req. Note: Connect to the debug infrastructure, for example, CoreSight, of your SoC. Width is 1-bit.
pmusnapshot_ack	Output	PMU snapshot acknowledge. The TCU and TBU use this signal to acknowledge that the PMU snapshot has occurred. This signal is LOW after reset. Note: Connect to the debug infrastructure, for example, CoreSight, of your SoC. Width is 1-bit.

A.4 DTI signals

MMU-720AE uses DTI interconnect switch, sizer, and register slice signals.

A.4.1 DTI interconnect switch DN_Sn interface signals

The MMU-720AE DTI interconnect switch provides one DN_Sn completer downstream interface per completer interface.

Signal definitions

Table A-25: DTI interconnect switch DN_Sn interface signals

Signal	Direction	Description
tvalid_dti_dn_sn	Input	Flow control signal
tready_dti_dn_sn	Output	Flow control signal
tdata_dti_dn_sn	Input	Message data signal
tid_dti_dn_sn	Input	Indicates the DTI manager that initiated the message
tlast_dti_dn_sn	Input	Indicates the last cycle of a message
tkeep_dti_dn_sn	Input	Indicates valid bytes
twakeup_dti_dn_sn	Input	Wakeup signal

A.4.2 DTI interconnect switch UP_Sn interface signals

The MMU-720AE DTI interconnect switch provides one UP_Sn completer upstream interface per completer interface.

Signal definitions

Table A-26: DTI interconnect switch UP_Sn interface signals

Signal	Direction	Description
tvalid_dti_up_sn	Output	Flow control signal
tready_dti_up_sn	Input	Flow control signal
tdata_dti_up_sn	Output	Message data signal
tdest_dti_up_sn	Output	Indicates the DTI manager that is the target of the message
tlast_dti_up_sn	Output	Indicates the last cycle of a message
tkeep_dti_up_sn	Output	Indicates valid bytes
twakeup_dti_up_sn	Output	Wakeup signal

A.4.3 DTI interconnect switch DN_M interface signals

The MMU-720AE DTI interconnect switch provides a DN_M requester downstream interface.

Signal definitions

Table A-27: DTI interconnect switch DN_M interface signals

Signal	Direction	Description
tvalid_dti_dn_m	Output	Flow control signal
tready_dti_dn_m	Input	Flow control signal
tdata_dti_dn_m	Output	Message data signal
tid_dti_dn_m	Output	Indicates the DTI manager that initiated the message
tlast_dti_dn_m	Output	Indicates the last cycle of a message
tkeep_dti_dn_m	Output	Indicates valid bytes
twakeup_dti_dn_m	Output	Wakeup signal

A.4.4 DTI interconnect switch UP_M interface signals

The MMU-720AE DTI interconnect switch provides an UP_M requester upstream interface.

Signal definitions

Table A-28: DTI interconnect switch UP_M interface signals

Signal	Direction	Description
tvalid_dti_up_m	Input	Flow control signal
tready_dti_up_m	Output	Flow control signal
tdata_dti_up_m	Input	Message data signal
tdest_dti_up_m	Input	Indicates the DTI manager that is the target of the message
tlast_dti_up_m	Input	Indicates the last cycle of a message
tkeep_dti_up_m	Input	Indicates valid bytes
twakeup_dti_up_m	Input	Wakeup signal

A.4.5 DTI interconnect sizer LPI_CG interface signals

The MMU-720AE DTI interconnect sizer provides an LPI_CG clock gating interface.

Signal definitions

Table A-29: DTI interconnect sizer LPI_CG interface signals

Signal	Direction	Description
qactive_cg	Output	Component active
qreqn_cg	Input	Quiescence request
qacceptn_cg	Output	Quiescence accept
qdeny_cg	Output	Quiescence deny

A.4.6 DTI interconnect sizer DN_S interface signals

The MMU-720AE DTI interconnect sizer provides a DN_S completer downstream interface.

Signal definitions

Table A-30: DTI interconnect sizer DN_S interface signals

Signal	Direction	Description
tvalid_dti_dn_s	Input	Flow control signal
tready_dti_dn_s	Output	Flow control signal
tdata_dti_dn_s	Input	Message data signal
tid_dti_dn_s	Input	Indicates the DTI manager that initiated the message
tlast_dti_dn_s	Input	Indicates the last cycle of a message
tkeep_dti_dn_s	Input	Indicates valid bytes
twakeup_dti_dn_s	Input	Wakeup signal

A.4.7 DTI interconnect sizer UP_S interface signals

The MMU-720AE DTI interconnect sizer provides an UP_S completer upstream interface.

Signal definitions

Table A-31: DTI interconnect sizer UP_S interface signals

Signal	Direction	Description
tvalid_dti_up_s	Output	Flow control signal
tready_dti_up_s	Input	Flow control signal
tdata_dti_up_s	Output	Message data signal
tdest_dti_up_s	Output	Indicates the DTI manager that is the target of the message
tlast_dti_up_s	Output	Indicates the last cycle of a message
tkeep_dti_up_s	Output	Indicates valid bytes
twakeup_dti_up_s	Output	Wakeup signal

A.4.8 DTI interconnect sizer DN_M interface signals

The MMU-720AE DTI interconnect sizer provides a DN_M requester downstream interface.

Signal definitions

Table A-32: DTI interconnect sizer DN_M interface signals

Signal	Direction	Description
tvalid_dti_dn_m	Output	Flow control signal
tready_dti_dn_m	Input	Flow control signal
tdata_dti_dn_m	Output	Message data signal

Signal	Direction	Description
tid_dti_dn_m	Output	Indicates the DTI manager that initiated the message
tlast_dti_dn_m	Output	Indicates the last cycle of a message
tkeep_dti_dn_m	Output	Indicates valid bytes
twakeup_dti_dn_m	Output	Wakeup signal

A.4.9 DTI interconnect sizer UP_M interface signals

The MMU-720AE DTI interconnect sizer provides an UP_M requester upstream interface.

Signal definitions

Table A-33: DTI interconnect sizer UP_M interface signals

Signal	Direction	Description
tvalid_dti_up_m	Input	Flow control signal
tready_dti_up_m	Output	Flow control signal
tdata_dti_up_m	Input	Message data signal
tdest_dti_up_m	Input	Indicates the DTI manager that is the target of the message
tlast_dti_up_m	Input	Indicates the last cycle of a message
tkeep_dti_up_m	Input	Indicates valid bytes
twakeup_dti_up_m	Input	Wakeup signal

A.4.10 DTI interconnect register slice LPI_CG interface signals

The MMU-720AE DTI interconnect register slice provides an LPI_CG clock gating interface.

Signal definitions

Table A-34: DTI interconnect register slice LPI_CG interface signals

Signal	Direction	Description
qactive_cg	Output	Component active
qreqn_cg	Input	Quiescence request
qacceptn_cg	Output	Quiescence accept
qdeny_cg	Output	Quiescence deny

A.4.11 DTI interconnect register slice DN_S interface signals

The MMU-720AE DTI interconnect register slice provides a DN_S completer downstream interface.

Signal definitions

Table A-35: DTI interconnect register slice DN_S interface signals

Signal	Direction	Description
tvalid_dti_dn_s	Input	Flow control signal
tready_dti_dn_s	Output	Flow control signal
tdata_dti_dn_s	Input	Message data signal
tid_dti_dn_s	Input	Indicates the DTI manager that initiated the message
tlast_dti_dn_s	Input	Indicates the last cycle of a message
tkeep_dti_dn_s	Input	Indicates valid bytes
twakeup_dti_dn_s	Input	Wakeup signal

A.4.12 DTI interconnect register slice UP_S interface signals

The MMU-720AE DTI interconnect register slice provides an UP_S completer upstream interface.

Signal definitions

Table A-36: DTI interconnect register slice UP_S interface signals

Signal	Direction	Description
tvalid_dti_up_s	Output	Flow control signal
tready_dti_up_s	Input	Flow control signal
tdata_dti_up_s	Output	Message data signal
tdest_dti_up_s	Output	Indicates the requester that initiated the message
tlast_dti_up_s	Output	Indicates the last cycle of a message
tkeep_dti_up_s	Output	Indicates valid bytes
twakeup_dti_up_s	Output	Wakeup signal

A.4.13 DTI interconnect register slice DN_M interface signals

The MMU-720AE DTI interconnect register slice provides a DN_M requester downstream interface.

Signal definitions

Table A-37: DTI interconnect register slice DN_M interface signals

Signal	Direction	Description
tvalid_dti_dn_m	Output	Flow control signal
tready_dti_dn_m	Input	Flow control signal

Signal	Direction	Description
tdata_dti_dn_m	Output	Message data signal
tid_dti_dn_m	Output	Indicates the DTI manager that initiated the message
tlast_dti_dn_m	Output	Indicates the last cycle of a message
tkeep_dti_dn_m	Output	Indicates valid bytes
twakeup_dti_dn_m	Output	Wakeup signal

A.4.14 DTI interconnect register slice UP_M interface signals

The MMU-720AE DTI interconnect register slice provides an UP_M requester upstream interface.

Signal definitions

Table A-38: DTI interconnect register slice UP_M interface signals

Signal	Direction	Description
tvalid_dti_up_m	Input	Flow control signal
tready_dti_up_m	Output	Flow control signal
tdata_dti_up_m	Input	Message data signal
tdest_dti_up_m	Input	Indicates the requester that initiated the message
tlast_dti_up_m	Input	Indicates the last cycle of a message
tkeep_dti_up_m	Input	Indicates valid bytes
twakeup_dti_up_m	Input	Wakeup signal

A.5 LTI signals

MMU-720AE uses LTI request channel, response channel, and completion channel signals.

A.5.1 LTI request channel signals

The MMU-720AE LTI signals include LTI request channel signals.

Signal definitions

Table A-39: LTI request channel signals

Signal	Direction	Category	Description
lavalid	Input	Transport	Channel valid. Width is 1-bit.
lavc	Input	Transport	VC number. This is fixed to 2 virtual channels so the width of the lavc signal is 1-bit.
lacredit	Output	Transport	Channel credit grant. The width of lacredit is fixed to 2-bit because we have 2 virtual channels.
laid	Input	Flow	Translation request ID. You can configure the width of this signal using the TBUCFG_LTI_ID_WIDTH configuration parameter. See 2.7.6 LTI TBU configuration parameters on page 108.
laogv	Input	Flow	Order group valid. Width is 1-bit.

Signal	Direction	Category	Description
laog	Input	Flow	Order group. You can configure the width of this signal using the <code>TBUCFG_LTI_OG_WIDTH</code> configuration parameter. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
laflow	Input	Context	Indicates the translation flow required. Width is 2-bit.
lammuv	Input	Context	Indicates whether SMMU translation is required. Width is 1-bit.
lasecsid	Input	Context	StreamID security level. The width is calculated as follows: $LTI_GPC == True ? 2:1$
lasid	Input	Context	StreamID. You can configure the width of this signal using the <code>TBUCFG_SID_WIDTH</code> configuration parameter. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
lassidv	Input	Context	SubstreamID valid. The width is calculated as follows: $LTI_SSID_WIDTH > 0 ? 1:0$
lassid	Input	Context	SubstreamID. You can configure the width of this signal using the <code>TBUCFG_SSID_WIDTH</code> configuration parameter. See 2.7.3 Common ACE-Lite and LTI TBU configuration parameters on page 105.
laprot	Input	Transaction	Protection information. laprot uses the same encoding as the AXI AxPROT signals. The width is calculated as follows: $LTI_MMU == True ? 3:1$
lanse	Input	Transaction	Used in conjunction with laprot[1] to define the PAS of the transaction. Width is 1-bit.
laaddr	Input	Transaction	Address. The width is calculated as follows: $LTI_MMU == True: 64\text{-bit}$ $LTI_MMU == False: LTI_LRADDR_WIDTH\text{-bit}$
latrans	Input	Transaction	Type of the transaction that the LTI request is translating. Width is 4-bit.
laattr	Input	Transaction	Attributes for the untranslated transaction. Width is 4-bit.
laloop	Input	Impdef	IMPLEMENTATION DEFINED loopback signaling. You can configure the width of this signal using the <code>TBUCFG_LTI_LOOP_WIDTH</code> configuration parameter. See 2.7.6 LTI TBU configuration parameters on page 108.
latlbloc	Input	Impdef	Location to access in the TLB. The meaning of this signal is IMPLEMENTATION DEFINED . When you add TBU_LTI interface signals, you must size the latlbloc signal. Calculate the width of latlbloc as follows: $LTI_TLBLOC_WIDTH = (LTI_TLBLOC_WIDTH_RAW < 1) ? 1 : LTI_TLBLOC_WIDTH_RAW$ For information about how to set these parameters, see 2.7 Configuration parameters and methodology on page 102.
laident	Input	Context	Indicates an identity translation is required. Width is 1-bit.

For more detailed descriptions of these signals, see the [AMBA® LTI Protocol Specification](#).

For detailed descriptions of the parameters that this section mentions, see the *Configuration parameters* section in the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

A.5.2 LTI response channel signals

The MMU-720AE LTI signals include LTI response channel signals.

Signal definitions

Table A-40: LTI response channel signals

Signal	Direction	Category	Description
Invalid	Output	Transport	Channel valid. Width is 1-bit.
Irvc	Output	Transport	VC number. This is fixed to 2 virtual channels so the Irvc signal is 1-bit wide.
Ircrredit	Input	Transport	Channel credit grant. The width is fixed to 2 because we have 2 virtual channels.
Irid	Output	Flow	Translation request ID. You can configure the width of this signal using the <code>TBUCFG_LTI_ID_WIDTH</code> configuration parameter. See 2.7.6 LTI TBU configuration parameters on page 108.
Irctag	Output	Flow	Translation completion tag. Width is 1-bit.
Irresp	Output	Translation	Translation response. Width is 3-bit.
Irprot	Output	Translation	Translated protection information. Irprot uses the same encoding as laprot. Width is 3-bit.
Irns	Output	Translation	Used in conjunction with Irprot[1] to define the PAS of the translation. This signal is always output as 0 when not in RME mode. Width is 1-bit.
Iraddr	Output	Translation	Translated address. Width is <code>LTI_LRADDR_WIDTH</code> -bit.
Irattr	Output	Translation	Translated transaction attributes. Width is 4-bit.
Irhwattr	Output	Translation	IMPLEMENTATION DEFINED hardware attributes. See the <i>PBHA for translated transactions</i> section in the <i>Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual</i> . Width is 4-bit.
Irmpam	Output	Translation	MPAM information. Width is calculated as follows: - When <code>LTI_MPAM_SUPPORT</code> is <code>MPAM_9_1</code>, width is <code>(LTI_GPC == True ? 12 : 11)</code> - When <code>LTI_MPAM_SUPPORT</code> is <code>MPAM_12_1</code>, width is <code>(LTI_GPC == True ? 15 : 14)</code>
Irmeid	Output	Translation	Memory Encryption Context Identifier (MECID). Width is <code>LTI_MECID_WIDTH</code> -bit.
Irloop	Output	Translation	Loopback signaling. You can configure the width of this signal using the <code>TBUCFG_LTI_LOOP_WIDTH</code> configuration parameter. See 2.7.6 LTI TBU configuration parameters on page 108.

For more detailed descriptions of these signals, see the [AMBA® LTI Protocol Specification](#).

For detailed descriptions of the parameters that this section mentions, see the *Configuration parameters* section in the *Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual*.

A.5.3 LTI completion channel signals

The MMU-720AE LTI signals include LTI completion channel signals.

Signal definitions

Table A-41: LTI completion channel signals

Signal	Direction	Category	Description
Icvalid	Input	Transport	Channel valid. Width is 1-bit.
Iccredit	Output	Transport	Channel credit grant. Width is 1-bit.

Signal	Direction	Category	Description
lcctag	Input	Flow	Translation completion tag. Width is 1-bit.

For detailed descriptions of these signals, see the [AMBA® LTI Protocol Specification](#).

A.6 I/O Port protection

Protection is applied to all input and output ports within MMU-720AE.

I/O protection is always enabled, however MMU-720AE still supports connections to interfaces without I/O protection by leaving the CHK signals disconnected and disabling the associated safety mechanism. To enable and disable safety mechanisms, see [4.20.5 TCU_FMU_SMEN register](#) on page 277 and [4.21.5 TBU_FMU_SMEN register](#) on page 286.

Protection styles can be one of the following:

- TCU I/O port protection, see [A.6.1 TCU protection types](#) on page 334.
- TBU I/O port protection, see [A.6.2 TBU protection types](#) on page 346.
- ADB I/O port protection, see [A.6.3 ADB protection types](#) on page 360.
- LPD I/O port protection, see [A.6.4 LPD protection types](#) on page 365.

A.6.1 TCU protection types

MMU-720AE provides different protection types on the TCU interfaces.

For more information about the TCU protection types, see:

- Asynchronous port protection, see [A.6.1.1 TCU asynchronous port protection](#) on page 334.
- Clock and reset protection, see [A.6.1.2 TCU clock and reset protection](#) on page 335.
- Clock wakeup protection, see [A.6.1.3 TCU clock wake up protection](#) on page 336.
- Full duplication protection, see [A.6.1.4 TCU full duplication](#) on page 336.
- Full inverted duplication protection, see [A.6.1.5 TCU full inverted duplication](#) on page 337.
- FMU duplication inverted, see [A.6.1.6 TCU FMU inverted duplication](#) on page 340.
- DFT protection, see [A.6.1.7 TCU DFT protection](#) on page 341.
- Synchronous parity protection, see [A.6.1.8 TCU synchronous parity protection](#) on page 341.

A.6.1.1 TCU asynchronous port protection

MMU-720AE contains asynchronous I/O protection signals for the TCU interfaces.

The following table describes the port name, signal direction, signal protected, protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU asynchronous protection.

Table A-42: TCU asynchronous protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
ela_enable_chk	Input	ela_enable	ELA	None	ASYNC_IN	Inverted
eventoack_chk	Input	eventoack	Interrupts	None	ASYNC_IN	Inverted
freqchk	Input	freq	F-Channel	None	ASYNC_IN	Inverted
pmusnapshot_req_chk	Input	pmusnapshot_req	PMU snapshot	None	ASYNC_IN	Inverted
qreqchk_cg	Input	qreqn_cg	LPI_CG	None	ASYNC_IN	Inverted
qreqchk_pd	Input	qreqn_pd	LPI_PD	None	ASYNC_IN	Inverted
syscoackchk_qtw	Input	syscoack_qtw	ACE5-LiteDVM	None	ASYNC_IN	Inverted

For more information about TCU I/O port protection, see:

- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340
- [A.6.1.7 TCU DFT protection](#) on page 341
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.2 TCU clock and reset protection

MMU-720AE contains clock and reset protection signals for the TCU interfaces. The primary clock feeds the original primary logic blocks and the shadow clock feeds the duplicated logic blocks.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU clock and reset protection.

Table A-43: TCU clock and reset protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
clkchk	Input	clk	N/A	None	N/A	Non-inverted
fmurstdisablechk	Input	fmurstdisable	N/A	None	N/A	Inverted
resetsnchk	Input	resetsn	N/A	None	N/A	Non-inverted

For more information about TCU I/O port protection, see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340

- [A.6.1.7 TCU DFT protection](#) on page 341
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.3 TCU clock wake up protection

MMU-720AE contains clock wake up protection signals for the TCU interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU clock wake up protection.

Table A-44: TCU clock wake up protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
qactivechk_cg	Output	qactive_cg	LPI_cg	None	N/A	Inverted
qactivechk_pd	Output	qactive_pd	LPI_pd	None	N/A	Inverted

For more information about TCU I/O port protection, see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340
- [A.6.1.7 TCU DFT protection](#) on page 341
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.4 TCU full duplication

MMU-720AE contains full duplication I/O protection signals for the TCU interfaces.



If you set the `FUSA_TEMPORAL_DELAY` configuration parameter to 2, you must ensure that the DTI interface input signals that enter MMU-720AE have a two-cycle clock delay applied externally. All MMU-720AE TBUs apply the two-cycle clock delay before communicating with the TCU.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU full duplication.

Table A-45: TCU full duplication protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
tdatachk_dti_dn	Input	tdata_dti_dn	DTI_DN	tvalid_dti_dn	N/A	Non-inverted
tidchk_dti_dn	Input	tid_dti_dn	DTI_DN	tvalid_dti_dn	N/A	Non-inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
tkeepchk_dti_dn	Input	tkeep_dti_dn	DTI_DN	tvalid_dti_dn	N/A	Non-inverted
tlastchk_dti_dn	Input	tlast_dti_dn	DTI_DN	tvalid_dti_dn	N/A	Non-inverted
treadychk_dti_up	Input	tready_dti_up	DTI_UP	None	N/A	Non-inverted
tvalidchk_dti_dn	Input	tvalid_dti_dn	DTI_DN	None	N/A	Non-inverted
twakeupchk_dti_dn	Input	twakeup_dti_dn	DTI_DN	None	N/A	Non-inverted
tdatachk_dti_up	Output	tdata_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
tdestchk_dti_up	Output	tdest_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
tkeepchk_dti_up	Output	tkeep_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
tlastchk_dti_up	Output	tlast_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
treadychk_dti_dn	Output	tready_dti_dn	DTI_DN	None	N/A	Non-inverted
tvalidchk_dti_up	Output	tvalid_dti_up	DTI_UP	None	N/A	Non-inverted
twakeupchk_dti_up	Output	twakeup_dti_up	DTI_UP	None	N/A	Non-inverted

For more information about TCU I/O port protection, see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340
- [A.6.1.7 TCU DFT protection](#) on page 341
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.5 TCU full inverted duplication

MMU-720AE contains full inverted duplication I/O protection signals for the TCU interfaces.

The following table describes the signal port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU full inverted duplication.

Table A-46: TCU full inverted duplication protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
ecorevnum_chk	Input	ecorevnum	Tie-offs	None	N/A	Inverted
hazard_width_chk	Input	hazard_width	Tie-offs	None	N/A	Inverted
IO_gpt_sz_chk	Input	IO_gpt_sz	Tie-offs	None	N/A	Inverted
legacy_tz_en_chk	Input	legacy_tz_en	Tie-offs	None	N/A	Inverted
msi_addr_chk	Input	msi_addr	Tie-offs	None	N/A	Inverted
sec_override_chk	Input	sec_override	Tie-offs	None	N/A	Inverted
sup_btm_chk	Input	sup_btm	Tie-offs	None	N/A	Inverted
sup_cohacc_chk	Output	sup_cohacc	Tie-offs	None	N/A	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
sup_httu_chk	Output	sup_httu	Tie-offs	None	N/A	Inverted
sup_oas_chk	Output	sup_oas	Tie-offs	None	N/A	Inverted
sup_sev_chk	Output	sup_sev	Tie-offs	None	N/A	Inverted
tcu_sid_chk	Output	tcu_sid	Tie-offs	None	N/A	Inverted
cmd_sync_irpt_ns_chk	Output	cmd_sync_irpt_ns	Interrupts	None	N/A	Inverted
cmd_sync_irpt_r_chk	Output	cmd_sync_irpt_r	Interrupts	None	N/A	Inverted
cmd_sync_irpt_s_chk	Output	cmd_sync_irpt_s	Interrupts	None	N/A	Inverted
event_q_irpt_ns_chk	Output	event_q_irpt_ns	Interrupts	None	N/A	Inverted
event_q_irpt_r_chk	Output	event_q_irpt_r	Interrupts	None	N/A	Inverted
event_q_irpt_s_chk	Output	event_q_irpt_s	Interrupts	None	N/A	Inverted
eventoreq_chk	Output	eventoreq	Interrupts	None	N/A	Inverted
fackchk	Output	fack	F-Channel	None	N/A	Inverted
global_irpt_ns_chk	Output	global_irpt_ns	Interrupts	None	N/A	Inverted
global_irpt_r_chk	Output	global_irpt_r	Interrupts	None	N/A	Inverted
global_irpt_s_chk	Output	global_irpt_s	Interrupts	None	N/A	Inverted
pmu_irpt_chk	Output	pmu_irpt	Interrupts	None	N/A	Inverted
pri_q_irpt_ns_chk	Output	pri_q_irpt_ns	Interrupts	None	N/A	Inverted
pri_q_irpt_r_chk	Output	pri_q_irpt_r	Interrupts	None	N/A	Inverted
sigclken0_ptw_chk	Output	sigclken0_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken0_qtw_chk	Output	sigclken0_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken1_ptw_chk	Output	sigclken1_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken1_qtw_chk	Output	sigclken1_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken10_ptw_chk	Output	sigclken10_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken10_qtw_chk	Output	sigclken10_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken11_ptw_chk	Output	sigclken11_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken11_qtw_chk	Output	sigclken11_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken2_ptw_chk	Output	sigclken2_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken2_qtw_chk	Output	sigclken2_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken3_ptw_chk	Output	sigclken3_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken3_qtw_chk	Output	sigclken3_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken4_ptw_chk	Output	sigclken4_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken4_qtw_chk	Output	sigclken4_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken5_ptw_chk	Output	sigclken5_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken5_qtw_chk	Output	sigclken5_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken6_ptw_chk	Output	sigclken6_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken6_qtw_chk	Output	sigclken6_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken7_ptw_chk	Output	sigclken7_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken7_qtw_chk	Output	sigclken7_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigclken8_ptw_chk	Output	sigclken8_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken8_qtw_chk	Output	sigclken8_qtw	ELA (QTW)	ela_enable	N/A	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
sigclken9_ptw_chk	Output	sigclken9_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigclken9_qtw_chk	Output	sigclken9_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp0_ptw_chk	Output	signalgrp0_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp0_qtw_chk	Output	signalgrp0_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp1_ptw_chk	Output	signalgrp1_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp1_qtw_chk	Output	signalgrp1_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp10_ptw_chk	Output	signalgrp10_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp10_qtw_chk	Output	signalgrp10_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp11_ptw_chk	Output	signalgrp11_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp11_qtw_chk	Output	signalgrp11_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp2_ptw_chk	Output	signalgrp2_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp2_qtw_chk	Output	signalgrp2_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp3_ptw_chk	Output	signalgrp3_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp3_qtw_chk	Output	signalgrp3_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp4_ptw_chk	Output	signalgrp4_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp4_qtw_chk	Output	signalgrp4_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp5_ptw_chk	Output	signalgrp5_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp5_qtw_chk	Output	signalgrp5_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp6_ptw_chk	Output	signalgrp6_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp6_qtw_chk	Output	signalgrp6_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp7_ptw_chk	Output	signalgrp7_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp7_qtw_chk	Output	signalgrp7_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp8_ptw_chk	Output	signalgrp8_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp8_qtw_chk	Output	signalgrp8_qtw	ELA (QTW)	ela_enable	N/A	Inverted
signalgrp9_ptw_chk	Output	signalgrp9_ptw	ELA (PTW)	ela_enable	N/A	Inverted
signalgrp9_qtw_chk	Output	signalgrp9_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual0_ptw_chk	Output	sigqual0_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual0_qtw_chk	Output	sigqual0_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual1_ptw_chk	Output	sigqual1_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual1_qtw_chk	Output	sigqual1_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual10_ptw_chk	Output	sigqual10_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual10_qtw_chk	Output	sigqual10_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual11_ptw_chk	Output	sigqual11_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual11_qtw_chk	Output	sigqual11_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual2_ptw_chk	Output	sigqual2_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual2_qtw_chk	Output	sigqual2_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual3_ptw_chk	Output	sigqual3_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual3_qtw_chk	Output	sigqual3_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual4_ptw_chk	Output	sigqual4_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual4_qtw_chk	Output	sigqual4_qtw	ELA (QTW)	ela_enable	N/A	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
sigqual5_ptw_chk	Output	sigqual5_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual5_qtw_chk	Output	sigqual5_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual6_ptw_chk	Output	sigqual6_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual6_qtw_chk	Output	sigqual6_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual7_ptw_chk	Output	sigqual7_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual7_qtw_chk	Output	sigqual7_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual8_ptw_chk	Output	sigqual8_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual8_qtw_chk	Output	sigqual8_qtw	ELA (QTW)	ela_enable	N/A	Inverted
sigqual9_ptw_chk	Output	sigqual9_ptw	ELA (PTW)	ela_enable	N/A	Inverted
sigqual9_qtw_chk	Output	sigqual9_qtw	ELA (QTW)	ela_enable	N/A	Inverted

For more information about TCU I/O port protection, see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340
- [A.6.1.7 TCU DFT protection](#) on page 341
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.6 TCU FMU inverted duplication

The MMU-720AE FMU is duplicated for latent fault protection purposes. The FMUs are identical to one another, but the primary FMU takes the primary inputs, and the shadow FMU takes the CHK inputs.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU FMU inverted duplication. If you configure the `FUSA_TEMPORAL_DELAY` parameter =2, the CHK copies of the interrupt may be 2 cycles delayed. See, [3.5 FuSa specific configuration parameter](#) on page 118.

Table A-47: TCU FMU inverted duplication signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
ras_et_cri_chk	Output	ras_et_cri	Interrupts	None	N/A	Inverted
ras_et_eri_chk	Output	ras_et_eri	Interrupts	None	N/A	Inverted
ras_et_fhi_chk	Output	ras_et_fhi	Interrupts	None	N/A	Inverted
ras_lt_cri_chk	Output	ras_lt_cri	Interrupts	None	N/A	Inverted
ras_lt_eri_chk	Output	ras_lt_eri	Interrupts	None	N/A	Inverted
ras_lt_fhi_chk	Output	ras_lt_fhi	Interrupts	None	N/A	Inverted
ras_lt_irpt_v_chk	Output	ras_lt_irpt_v	Interrupts	None	N/A	Inverted

For more information about TCU I/O port protection, see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.7 TCU DFT protection](#) on page 341
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.7 TCU DFT protection

MMU-720AE contains DFT I/O protection signals for the TCU interfaces.

The following table describes the signal port name, signal direction, the signal protected, the protection style, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU DFT protection.

Table A-48: TCU DFT protection signal descriptions

Port name	Direction	Signals protected	Protection style	Protection architecture	Qualifier signal	Enable parameter	Polarity
dftscanmode	Input	dftcgen, dfttrstdisable, dfttramhold	IPDS_DFT_RULE_F1, IPDS_DFT_RULE_FLS3	DFT	N/A	N/A	Non-inverted
nMBISTRESETCHK	Input	nMBISTRESET	IPDS_DFT_RULE_F5, IPDS_DFT_RULE_FLS3	DFT	None	N/A	Non-inverted
MBISTREQCHK	Input	MBISTREQ	IPDS_DFT_RULE_FLS1, IPDS_DFT_RULE_FLS3	DFT	None	N/A	Non-inverted
dftscanmodechk	Input	dftcgen, dfttrstdisable, dfttramhold	IPDS_DFT_RULE_FLS2, IPDS_DFT_RULE_FLS3	DFT	N/A	N/A	Non-inverted

For more information about TCU I/O port protection, see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340
- [A.6.1.8 TCU synchronous parity protection](#) on page 341

A.6.1.8 TCU synchronous parity protection

MMU-720AE contains synchronous parity I/O protection signals for the TCU interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TCU synchronous parity protection.

Table A-49: TCU synchronous parity protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
acaddrchk_qtw	Input	acaddr_qtw	ACE5-LiteDVM	acvalid_qtw_chk	QTW_IN	Inverted
acvalidchk_qtw	Input	acvalid_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
acvmidextchk_qtw	Input	acvmidext_qtw	ACE5-LiteDVM	acvalid_qtw_chk	QTW_IN	Inverted
acwakeupchk_qtw	Input	acwakeup_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
arreadychk_ptw	Input	arready_ptw	ACE5-LiteDVM	None	PTW_IN	Inverted
arreadychk_qtw	Input	arready_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
awreadychk_ptw	Input	awready_ptw	ACE5-LiteDVM	None	PTW_IN	Inverted
awreadychk_qtw	Input	awready_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
bidchk_ptw	Input	bid_ptw bidunq_ptw	ACE5-LiteDVM	wvalid_ptw	PTW_IN	Inverted
bidchk_qtw	Input	bid_qtw bidunq_qtw	ACE5-LiteDVM	wvalid_qtw	QTW_IN	Inverted
brespchk_ptw	Input	bresp_ptw	ACE5-LiteDVM	wvalid_ptw	PTW_IN	Inverted
brespchk_qtw	Input	bresp_qtw	ACE5-LiteDVM	wvalid_qtw	QTW_IN	Inverted
bvalidchk_ptw	Input	bvalid_ptw	ACE5-LiteDVM	None	PTW_IN	Inverted
bvalidchk_qtw	Input	bvalid_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
crreadychk_qtw	Input	crready_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
msirtvalidchk	Input	msirtvalid	MSI	None	MSI_IN	Inverted
msirtwakeupchk	Input	msirtwakeup	MSI	None	MSI_IN	Inverted
msitreadychk	Input	msitready	MSI	None	MSI_IN	Inverted
paddrchk_prog	Input	paddr_prog	APB	psel	APB_IN	Inverted
pctrchk_prog	Input	pprot_prog pwrite_prog pnse_prog	APB	psel	APB_IN	Inverted
penablechk_prog	Input	penable_prog	APB	psel	APB_IN	Inverted
pselchk_prog	Input	psel_prog	APB	None	APB_IN	Inverted
pstrbchk_prog	Input	pstrb_prog	APB	psel	APB_IN	Inverted
pwakeupchk_prog	Input	pwakeup_prog	APB	None	APB_IN	Inverted
pwdatachk_prog	Input	pwdata_prog	APB	psel	APB_IN	Inverted
rdatachk_ptw	Input	rdata_ptw	ACE5-LiteDVM	rvalid_ptw	PTW_IN	Inverted
rdatachk_qtw	Input	rdata_qtw	ACE5-LiteDVM	rvalid_qtw	QTW_IN	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
ridchk_ptw	Input	rid_ptw ridunq_ptw	ACE5-LiteDVM	rvalid_ptw	PTW_IN	Inverted
ridchk_qtw	Input	rid_qtw ridunq_qtw	ACE5-LiteDVM	rvalid_qtw	QTW_IN	Inverted
rlastchk_ptw	Input	rlast_ptw	ACE5-LiteDVM	rvalid_ptw	PTW_IN	Inverted
rlastchk_qtw	Input	rlast_qtw	ACE5-LiteDVM	rvalid_qtw	QTW_IN	Inverted
rpoisonchk_ptw	Input	rpoison_ptw	ACE5-LiteDVM	rvalid_ptw	PTW_IN	Inverted
rpoisonchk_qtw	Input	rpoison_qtw	ACE5-LiteDVM	rvalid_qtw	QTW_IN	Inverted
rrespchk_ptw	Input	rresp_ptw	ACE5-LiteDVM	rvalid_ptw	PTW_IN	Inverted
rrespchk_qtw	Input	rresp_qtw	ACE5-LiteDVM	rvalid_qtw	QTW_IN	Inverted
rvalidchk_ptw	Input	rvalid_ptw	ACE5-LiteDVM	None	PTW_IN	Inverted
rvalidchk_qtw	Input	rvalid_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
wreadychk_ptw	Input	wready_ptw	ACE5-LiteDVM	None	PTW_IN	Inverted
wreadychk_qtw	Input	wready_qtw	ACE5-LiteDVM	None	QTW_IN	Inverted
acreadychk_qtw	Output	acready_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
araddrchk_ptw	Output	araddr_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
araddrchk_qtw	Output	araddr_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
arctl0chk_ptw	Output	arsize_ptw arburst_ptw arlock_ptw arprot_ptw arnse_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
arctl0chk_qtw	Output	arsize_qtw arburst_qtw arlock_qtw arprot_qtw arnse_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
arctl1chk_ptw	Output	arcache_ptw arqos_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
arctl1chk_qtw	Output	arcache_qtw arqos_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
arctl2chk_ptw	Output	ardomain_ptw arsnoop_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
arctl2chk_qtw	Output	ardomain_qtw arsnoop_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
aridchk_ptw	Output	arid_ptw aridunq_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
aridchk_qtw	Output	arid_qtw aridunq_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
arlenchk_ptw	Output	arlen_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
arlenchk_qtw	Output	arlen_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
armecidchk_ptw	Output	armecid_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
armecidchk_qtw	Output	armecid_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
armpamchk_ptw	Output	armpam_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
armpamchk_qtw	Output	armpam_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
arpbhachk_ptw	Output	arpbha_ptw	ACE5-LiteDVM	arvalid_ptw	N/A (output)	Inverted
arpbhachk_qtw	Output	arpbha_qtw	ACE5-LiteDVM	arvalid_qtw	N/A (output)	Inverted
arvalidchk_ptw	Output	arvalid_ptw	ACE5-LiteDVM	None	N/A (output)	Inverted
arvalidchk_qtw	Output	arvalid_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
awaddrchk_ptw	Output	awaddr_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awaddrchk_qtw	Output	awaddr_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awakeupchk_ptw	Output	awakeup_ptw	ACE5-LiteDVM	None	N/A (output)	Inverted
awakeupchk_qtw	Output	awakeup_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
awctl0chk_ptw	Output	awsizе_ptw awburst_ptw awlock_ptw awprot_ptw awnse_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awctl0chk_qtw	Output	awsizе_qtw awburst_qtw awlock_qtw awprot_qtw awnse_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awctl1chk_ptw	Output	awcache_ptw awqos_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awctl1chk_qtw	Output	awcache_qtw awqos_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
awctl2chk_ptw	Output	awdomain_ptw awsnoop_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awctl2chk_qtw	Output	awdomain_qtw awsnoop_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awctl3chk_ptw	Output	awatop_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awctl3chk_qtw	Output	awatop_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awidchk_ptw	Output	awid_ptw awidunq_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awidchk_qtw	Output	awid_qtw awidunq_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awlenchk_ptw	Output	awlen_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awlenchk_qtw	Output	awlen_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awmecidchk_ptw	Output	awmecid_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awmecidchk_qtw	Output	awmecid_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awmpanchk_ptw	Output	awmpam_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awmpanchk_qtw	Output	awmpam_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awpbhachk_ptw	Output	awpbha_ptw	ACE5-LiteDVM	awvalid_ptw	N/A (output)	Inverted
awpbhachk_qtw	Output	awpbha_qtw	ACE5-LiteDVM	awvalid_qtw	N/A (output)	Inverted
awvalidchk_ptw	Output	awvalid_ptw	ACE5-LiteDVM	None	N/A (output)	Inverted
awvalidchk_qtw	Output	awvalid_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
breadychk_ptw	Output	bready_ptw	ACE5-LiteDVM	None	N/A (output)	Inverted
breadychk_qtw	Output	bready_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
crvalidchk_qtw	Output	crvalid_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
gpf_far_chk	Output	gpf_far	Interrupts	None	N/A (output)	Inverted
gpt_cfg_far_chk	Output	gpt_cfg_far	Interrupts	None	N/A (output)	Inverted
msirtreadychk	Output	msirtready	MSI	None	N/A (output)	Inverted
msitdatachk	Output	msitdata	MSI	msitvalid	N/A (output)	Inverted
msitvalidchk	Output	msitvalid	MSI	None	N/A (output)	Inverted
msitwakeupchk	Output	msitwakeup	MSI	None	N/A (output)	Inverted
pmusnapshot_ack_chk	Output	pmusnapshot_ack	PMU Snapshot	None	N/A (output)	Inverted
prdatachk_prog	Output	prdata_prog	APB	psel & penable & pready	N/A (output)	Inverted
preadychk_prog	Output	pready_prog	APB	psel & penable	N/A (output)	Inverted
pslverrchk_prog	Output	pslverr_prog	APB	psel & penable & pready	N/A (output)	Inverted
qacceptchk_cg	Output	qacceptn_cg	LPI.CG	None	N/A (output)	Inverted
qacceptchk_pd	Output	qacceptn_pd	LPI_PD	None	N/A (output)	Inverted
qdenychk_cg	Output	qdeny_cg	LPI.CG	None	N/A (output)	Inverted
qdenychk_pd	Output	qdeny_pd	LPI_PD	None	N/A (output)	Inverted
rreadychk_ptw	Output	rready_ptw	ACE5-LiteDVM	None	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
rreadychk_qtw	Output	rready_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
syscoreqchk_qtw	Output	syscoreq_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted
wdatachk_ptw	Output	wdata_ptw	ACE5-LiteDVM	wvalid_ptw	N/A (output)	Inverted
wdatachk_qtw	Output	wdata_qtw	ACE5-LiteDVM	wvalid_qtw	N/A (output)	Inverted
wlastchk_ptw	Output	wlast_ptw	ACE5-LiteDVM	wvalid_ptw	N/A (output)	Inverted
wlastchk_qtw	Output	wlast_qtw	ACE5-LiteDVM	wvalid_qtw	N/A (output)	Inverted
wpoisonchk_ptw	Output	wpoison_ptw	ACE5-LiteDVM	wvalid_ptw	N/A (output)	Inverted
wpoisonchk_qtw	Output	wpoison_qtw	ACE5-LiteDVM	wvalid_qtw	N/A (output)	Inverted
wstrbchk_ptw	Output	wstrb_ptw	ACE5-LiteDVM	wvalid_ptw	N/A (output)	Inverted
wstrbchk_qtw	Output	wstrb_qtw	ACE5-LiteDVM	wvalid_qtw	N/A (output)	Inverted
wvalidchk_ptw	Output	wvalid_ptw	ACE5-LiteDVM	None	N/A (output)	Inverted
wvalidchk_qtw	Output	wvalid_qtw	ACE5-LiteDVM	None	N/A (output)	Inverted

For more information on TCU I/O port protection see:

- [A.6.1.1 TCU asynchronous port protection](#) on page 334
- [A.6.1.2 TCU clock and reset protection](#) on page 335
- [A.6.1.3 TCU clock wake up protection](#) on page 336
- [A.6.1.4 TCU full duplication](#) on page 336
- [A.6.1.5 TCU full inverted duplication](#) on page 337
- [A.6.1.6 TCU FMU inverted duplication](#) on page 340
- [A.6.1.7 TCU DFT protection](#) on page 341

A.6.2 TBU protection types

MMU-720AE provides different protection types on the TBU interfaces.

For more information on the TBU protection types, see:

- Asynchronous port protection, see [A.6.2.1 TBU asynchronous protection](#) on page 346.
- Clock and reset protection, see [A.6.2.2 TBU clock and reset protection](#) on page 347.
- Clock wakeup protection, see [A.6.2.3 TBU clock wake up protection](#) on page 348.
- FMU duplication inverted, see [A.6.2.4 TBU FMU inverted duplication](#) on page 348.
- Full duplication protection, see [A.6.2.5 TBU full duplication](#) on page 349.
- Full inverted duplication protection, see [A.6.2.6 TBU full inverted duplication](#) on page 352.
- DFT protection, see [A.6.2.7 TBU DFT protection](#) on page 354.
- Synchronous parity protection, see [A.6.2.8 TBU synchronous parity protection](#) on page 354.

A.6.2.1 TBU asynchronous protection

MMU-720AE contains asynchronous I/O protection signals for the TBU interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for TBU asynchronous protection.

Table A-50: TBU asynchronous protection signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity	Additional information
ela_enable_chk	Input	ela_enable	ELA	None	ASYNC_IN	Inverted	-
freqchk	Input	freq	F-Channel	None	ASYNC_IN	Inverted	TBU LTI only
pmusnapshot_req_chk	Input	pmusnapshot_req	PMU Snapshot	None	ASYNC_IN	Inverted	-
qreqchk_cg	Input	qreqn_cg	LPI_CG	None	ASYNC_IN	Inverted	-
qreqchk_pd	Input	qreqn_pd	LPI_PD	None	ASYNC_IN	Inverted	-

For more information about TBU I/O port protection, see:

- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.7 TBU DFT protection](#) on page 354
- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.2 TBU clock and reset protection

The primary clock feeds the original primary logic blocks and the shadow clock feeds the duplicated logic blocks.

The following table describes the port name, signal direction, the signal protected, the protection style, the protection architecture, qualifier signal, the enable parameter, and the port polarity for the TBU clock and reset protection.

Table A-51: TBU clock and reset protection signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
clkchk	Input	clk	N/A	None	N/A	Non-inverted
fmurstdisablechk	Input	fmurstdisable	N/A	None	N/A	Inverted
resetchk	Input	resetsn	N/A	None	N/A	Non-inverted

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346

- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.7 TBU DFT protection](#) on page 354
- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.3 TBU clock wake up protection

MMU-720AE contains clock wake up I/O protection signals for the TBU interfaces.

The following table describes the TBU clock wake up protection port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity.

Table A-52: TBU clock wake up protection signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
qactivechk_cg	Output	qactive_cg	LPI_CG	None	N/A (output)	Inverted
qactivechk_pd	Output	qactive_pd	LPI_PD	None	N/A (output)	Inverted

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346
- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.7 TBU DFT protection](#) on page 354
- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.4 TBU FMU inverted duplication

The FMU in MMU-720AE is duplicated for latent fault protection purposes. The FMUs are identical to one another, but the primary FMU takes the primary inputs, and the shadow FMU takes the CHK inputs.

The following table describes the TBU FMU inverted duplication port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity. If you set the `FUSA_TEMPORAL_DELAY` parameter to 2, the CHK copies of the interrupt might be delayed by 2 cycles. See, [3.5 FuSa specific configuration parameter](#) on page 118.

Table A-53: TBU FMU inverted duplication signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
ras_et_cri_chk	Output	ras_et_cri	Interrupts	None	N/A (output)	Inverted
ras_et_eri_chk	Output	ras_et_eri	Interrupts	None	N/A (output)	Inverted
ras_et_fhi_chk	Output	ras_et_fhi	Interrupts	None	N/A (output)	Inverted
ras_lt_cri_chk	Output	ras_lt_cri	Interrupts	None	N/A (output)	Inverted
ras_lt_eri_chk	Output	ras_lt_eri	Interrupts	None	N/A (output)	Inverted
ras_lt_fhi_chk	Output	ras_lt_fhi	Interrupts	None	N/A (output)	Inverted
ras_lt_irpt_v_chk	Output	ras_lt_irpt_v	Interrupts	None	N/A (output)	Inverted

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346
- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.7 TBU DFT protection](#) on page 354
- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.5 TBU full duplication

MMU-720AE contains full duplication protection signals for the TBU interfaces.



Note

If you set the `FUSA_TEMPORAL_DELAY` configuration parameter to 2, you must ensure that the MMU-720AE LTI interface inputs have a two-cycle clock delay applied externally.

The following table describes the TBU full duplication port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity.

Table A-54: TBU full duplication signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
laaddrchk	Input	laaddr	LTI	lavalid	N/A	Non-inverted
laattrchk	Input	laattr	LTI	lavalid	N/A	Non-inverted
laflowchk	Input	laflow	LTI	lavalid & lammuv	N/A	Non-inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
lahwattrchk	Input	lahwattr	LTI	lavalid	N/A	Non-inverted
laidchk	Input	laid	LTI	lavalid	N/A	Non-inverted
laidentchk	Input	laident	LTI	lavalid & lammuv	N/A	Non-inverted
laloopchk	Input	laloop	LTI	lavalid	N/A	Non-inverted
lamecidchk	Input	lamecid	LTI	lavalid	N/A	Non-inverted
lammuvchk	Input	lammuv	LTI	lavalid	N/A	Non-inverted
lansechk	Input	lanse	LTI	lavalid	N/A	Non-inverted
laogchk	Input	laog	LTI	lavalid & laogv	N/A	Non-inverted
laogvchk	Input	laogv	LTI	lavalid	N/A	Non-inverted
laprot_hk	Input	laprot	LTI	lavalid	N/A	Non-inverted
lasecsidchk	Input	lasecsid	LTI	lavalid & lammuv	N/A	Non-inverted
lasidchk	Input	lasid	LTI	lavalid & lammuv	N/A	Non-inverted
lassidchk	Input	lassid	LTI	lavalid & lammuv & lassidv	N/A	Non-inverted
lassidvchk	Input	lassidv	LTI	lavalid & lammuv	N/A	Non-inverted
latlblocchk	Input	latlbloc	LTI	lavalid	N/A	Non-inverted
latranschk	Input	latrans	LTI	lavalid	N/A	Non-inverted
lavalidchk	Input	lavalid	LTI The TBU may include up to 8 LTI interfaces. All interfaces include the same protection listed here.	None	N/A	Non-inverted
lavcchk	Input	lavc	LTI	lavalid	N/A	Non-inverted
lcctagchk	Input	lcctag	LTI	lcvalid	N/A	Non-inverted
lcvalidchk	Input	lcvalid	LTI	None	N/A	Non-inverted
lmactivechk	Input	lmactive	LTI	None	N/A	Non-inverted
lmopenreqchk	Input	lmopenreq	LTI	None	N/A	Non-inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
lrcreditchk	Input	lrcredit	LTI	None	N/A	Non-inverted
tdatachk_dti_up	Input	tdata_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
tkeepchk_dti_up	Input	tkeep_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
tlastchk_dti_up	Input	tlast_dti_up	DTI_UP	tvalid_dti_up	N/A	Non-inverted
treadychk_dti_dn	Input	tready_dti_dn	DTI_DN	None	N/A	Non-inverted
tvalidchk_dti_up	Input	tvalid_dti_up	DTI_UP	None	N/A	Non-inverted
twakeupchk_dti_up	Input	twakeup_dti_up	DTI_UP	None	N/A	Non-inverted
lacreditchk	Output	lcredit	LTI	None	N/A (output)	Non-inverted
lccreditchk	Output	lccredit	LTI	None	N/A (output)	Non-inverted
lmaskclosechk	Output	lmaskclose	LTI	None	N/A (output)	Non-inverted
lmopenackchk	Output	lmopenack	LTI	None	N/A (output)	Non-inverted
lraddrchk	Output	lraddr	LTI	linvalid	N/A (output)	Non-inverted
lrattrchk	Output	lrattr	LTI	linvalid	N/A (output)	Non-inverted
lrctagchk	Output	lrctag	LTI	linvalid	N/A (output)	Non-inverted
lrhwattrchk	Output	lrhwattr	LTI	linvalid	N/A (output)	Non-inverted
lridchk	Output	lrid	LTI	linvalid	N/A (output)	Non-inverted
lrloopchk	Output	lrloop	LTI	linvalid	N/A (output)	Non-inverted
lrmecidchk	Output	lrmecid	LTI	linvalid	N/A (output)	Non-inverted
lrmpamchk	Output	lrmpam	LTI	linvalid	N/A (output)	Non-inverted
lrnsechk	Output	lrnse	LTI	linvalid	N/A (output)	Non-inverted
lrprotchk	Output	lrprot	LTI	linvalid	N/A (output)	Non-inverted
lrrespchk	Output	lrresp	LTI	linvalid	N/A (output)	Non-inverted
linvalidchk	Output	linvalid	LTI	None	N/A (output)	Non-inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
lrvchk	Output	lrv	LTI	lvalid	N/A (output)	Non-inverted
tdatachk_dti_dn	Output	tdata_dti_dn	DTI_DN	tvalid_dti_dn	N/A (output)	Non-inverted
tkeepchk_dti_dn	Output	tkeep_dti_dn	DTI_DN	tvalid_dti_dn	N/A (output)	Non-inverted
tlastchk_dti_dn	Output	tlast_dti_dn	DTI_DN	tvalid_dti_dn	N/A (output)	Non-inverted
treadychk_dti_up	Output	tready_dti_up	DTI_UP	None	N/A (output)	Non-inverted
tvalidchk_dti_dn	Output	tvalid_dti_dn	DTI_DN	None	N/A (output)	Non-inverted
twakeupchk_dti_dn	Output	twakeup_dti_dn	DTI_DN	None	N/A (output)	Non-inverted

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346
- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.7 TBU DFT protection](#) on page 354
- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.6 TBU full inverted duplication

MMU-720AE contains full inverted duplication I/O protection signals for the TBU interfaces.

The following table describes the TBU full inverted duplication port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity.

Table A-55: TBU full inverted duplication signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity	Additional information
ecorevnum_chk	Input	ecorevnum	Tie-offs	None	N/A	Inverted	-
legacy_tz_en_chk	Input	legacy_tz_en	Tie-offs	None	N/A	Inverted	-
max_tok_trans_chk	Input	max_tok_trans	Tie-offs	None	N/A	Inverted	-
ns_sid_high_chk	Input	ns_sid_high	Tie-offs	None	N/A	Inverted	-
pcie_mode_chk	Input	pcie_mode	Tie-offs	None	N/A	Inverted	-
poison_support_chk	Input	poison_support	Tie-offs	None	N/A	Inverted	Applies to TBU_ACE_LITE only

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity	Additional information
r_sid_high_chk	Input	r_sid_high	Tie-offs	None	N/A	Inverted	-
s_sid_high_chk	Input	s_sid_high	Tie-offs	None	N/A	Inverted	-
same_power_domain_chk	Input	same_power_domain	Tie-offs	None	N/A	Inverted	-
sec_override_chk	Input	sec_override	Tie-offs	None	N/A	Inverted	-
utlb_roundrobin_chk	Input	utlb_roundrobin	Tie-offs	None	N/A	Inverted	-
crit_err_chk	Output	crit_err	Interrupts	None	N/A (output)	Inverted	-
fackchk	Output	fack	F-Channel	None	N/A	Inverted	Applies to TBU_LTI only
pmu_irpt_chk	Output	pmu_irpt	Interrupts	None	N/A (output)	Inverted	-
sigclken0_chk	Output	sigclken0	ELA	ela_enable	N/A (output)	Inverted	-
sigclken1_chk	Output	sigclken1	ELA	ela_enable	N/A (output)	Inverted	-
sigclken2_chk	Output	sigclken2	ELA	ela_enable	N/A (output)	Inverted	-
sigclken3_chk	Output	sigclken3	ELA	ela_enable	N/A (output)	Inverted	-
sigclken4_chk	Output	sigclken4	ELA	ela_enable	N/A (output)	Inverted	-
signalgrp0_chk	Output	signalgrp0	ELA	ela_enable	N/A (output)	Inverted	-
signalgrp1_chk	Output	signalgrp1	ELA	ela_enable	N/A (output)	Inverted	-
signalgrp2_chk	Output	signalgrp2	ELA	ela_enable	N/A (output)	Inverted	-
signalgrp3_chk	Output	signalgrp3	ELA	ela_enable	N/A (output)	Inverted	-
signalgrp4_chk	Output	signalgrp4	ELA	ela_enable	N/A (output)	Inverted	-
sigqual0_chk	Output	sigqual0	ELA	ela_enable	N/A (output)	Inverted	-
sigqual1_chk	Output	sigqual1	ELA	ela_enable	N/A (output)	Inverted	-
sigqual2_chk	Output	sigqual2	ELA	ela_enable	N/A (output)	Inverted	-
sigqual3_chk	Output	sigqual3	ELA	ela_enable	N/A (output)	Inverted	-
sigqual4_chk	Output	sigqual4	ELA	ela_enable	N/A (output)	Inverted	-
sigqual5_chk	Output	sigqual5	ELA	ela_enable	N/A (output)	Inverted	Applies to TBU_LTI only
sigqual6_chk	Output	sigqual6	ELA	ela_enable	N/A (output)	Inverted	Applies to TBU_LTI only
sigqual7_chk	Output	sigqual7	ELA	ela_enable	N/A (output)	Inverted	Applies to TBU_LTI only
sigqual8_chk	Output	sigqual8	ELA	ela_enable	N/A (output)	Inverted	Applies to TBU_LTI only

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346
- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.7 TBU DFT protection](#) on page 354

- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.7 TBU DFT protection

MMU-720AE contains DFT I/O protection signals for the TBU interfaces.

The following table describes the TBU DFT port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity.

Table A-56: TBU DFT protection signal details

Port name	Direction	Signals protected	Protection style	Protection architecture	Qualifier signal	Enable parameter	Polarity
dftscanmode	Input	dftcgen dfrstdisable dftramhold	IPDS_DFT_RULE_F1 IPDS_DFT_RULE_FLS3	DFT	N/A	N/A	Non-inverted
nMBISTRESETCHK	Input	nMBISTRESET	IPDS_DFT_RULE_F5 IPDS_DFT_RULE_FLS3	DFT	None	N/A	Non-inverted
MBISTREQCHK	Input	MBISTREQ	IPDS_DFT_RULE_FLS1 IPDS_DFT_RULE_FLS3	DFT	None	N/A	Non-inverted
dftscanmodechk	Input	dftcgen dfrstdisable dftramhold	IPDS_DFT_RULE_FLS2 IPDS_DFT_RULE_FLS3	DFT	N/A	N/A	Non-inverted

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346
- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.8 TBU synchronous parity protection](#) on page 354

A.6.2.8 TBU synchronous parity protection

MMU-720AE contains synchronous parity I/O protection signals for the TBU interfaces.

The following table describes the TBU synchronous parity protection port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity.

Table A-57: TBU synchronous parity protection signal details

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
araddrchk_s	Input	araddr_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arctl0chk_s	Input	arsize_s arburst_s arlock_s arprot_s arnse_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arctl1chk_s	Input	arregion_s arcache_s arqos_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arctl2chk_s	Input	ardomain_s arsnoop_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arctl3chk_s	Input	archunken_s artagop_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
aridchk_s	Input	arid_s aridunq_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arlenchk_s	Input	arlen_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arloopchk_s	Input	arloop_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
armecidchk_s	Input	armecid_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
armmuchk_s	Input	armmuflow_s armmusecsid_s armmussidv_s armmuvalid_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
armmusidchk_s	Input	armmusid_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
armmussidchk_s	Input	armmussid_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arpbhachk_s	Input	arpbha_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arreadychk_m	Input	arready_m	ACE5-LiteDVM	None	ACELITE_IN	Inverted
aruserchk_s	Input	aruser_s	ACE5-LiteDVM	arvalid_s	ACELITE_IN	Inverted
arvalidchk_s	Input	arvalid_s	ACE5-LiteDVM	None	ACELITE_IN	Inverted
awaddrchk_s	Input	awaddr_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awakeupchk_s	Input	awakeup_s	ACE5-LiteDVM	None	ACELITE_IN	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
awctl0chk_s	Input	awsize_s awburst_s awlock_s awprot_s awnse_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awctl1chk_s	Input	awregion_s awcache_s awqos_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awctl2chk_s	Input	awdomain_s awsnoop_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awctl3chk_s	Input	awatop_s awtagop_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awidchk_s	Input	awid_s awidunq_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awlenchk_s	Input	awlen_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awloopchk_s	Input	awloop_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awmecidchk_s	Input	awmecid_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awmmuchk_s	Input	awmmuflow_s awmmusecsid_s awmmussidv_s awmmuvalid_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awmmusidchk_s	Input	awmmusid_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awmmussidchk_s	Input	awmmussid_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awpbhachk_s	Input	awpbha_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awreadychk_m	Input	awready_m	ACE5-LiteDVM	None	ACELITE_IN	Inverted
awstashlpidchk_s	Input	awstashlpid_s awstashlpiden_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awstashnidchk_s	Input	awstashnid_s awstashniden_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awuserchk_s	Input	awuser_s	ACE5-LiteDVM	awvalid_s	ACELITE_IN	Inverted
awvalidchk_s	Input	awvalid_s	ACE5-LiteDVM	None	ACELITE_IN	Inverted
bidchk_m	Input	bid_m bidunq_m	ACE5-LiteDVM	bvalid_m	ACELITE_IN	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
bloopchk_m	Input	bloop_m	ACE5-LiteDVM	bvalid_m	ACELITE_IN	Inverted
breaychk_s	Input	breay_s	ACE5-LiteDVM	None	ACELITE_IN	Inverted
brespchk_m	Input	bresp_m	ACE5-LiteDVM	bvalid_m	ACELITE_IN	Inverted
buserchk_m	Input	buser_m	ACE5-LiteDVM	bvalid_m	ACELITE_IN	Inverted
bvalidchk_m	Input	bvalid_m	ACE5-LiteDVM	None	ACELITE_IN	Inverted
rchunkchk_m	Input	rchunkv_m rchunknum_m rchunkstrb_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rdatachk_m	Input	rdata_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
ridchk_m	Input	rid_m ridunq_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rlastchk_m	Input	rlast_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rloopchk_m	Input	rloop_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rpoisonchk_m	Input	rpoison_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rreadychk_s	Input	rready_s	ACE5-LiteDVM	None	ACELITE_IN	Inverted
rrespchk_m	Input	rresp_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rtagchk_m	Input	rtag_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
ruserchk_m	Input	ruser_m	ACE5-LiteDVM	rvalid_m	ACELITE_IN	Inverted
rvalidchk_m	Input	rvalid_m	ACE5-LiteDVM	None	ACELITE_IN	Inverted
wdatachk_s	Input	wdata_s	ACE5-LiteDVM	wvalid_s	ACELITE_IN	Inverted
wlastchk_s	Input	wlast_s	ACE5-LiteDVM	wvalid_s	ACELITE_IN	Inverted
wpoisonchk_s	Input	wpoison_s	ACE5-LiteDVM	wvalid_s	ACELITE_IN	Inverted
wreadychk_m	Input	wready_m	ACE5-LiteDVM	None	ACELITE_IN	Inverted
wstrbchk_s	Input	wstrb_s	ACE5-LiteDVM	wvalid_s	ACELITE_IN	Inverted
wtagchk_s	Input	wtag_s wtagupdate_s	ACE5-LiteDVM	wvalid_s	ACELITE_IN	Inverted
wuserchk_s	Input	wuser_s	ACE5-LiteDVM	wvalid_s	ACELITE_IN	Inverted
wvalidchk_s	Input	wvalid_s	ACE5-LiteDVM	None	ACELITE_IN	Inverted
araddrchk_m	Output	araddr_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arctl0chk_m	Output	arsize_m arburst_m arlock_m arprot_m arnse_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
arctl1chk_m	Output	arregion_m arcache_m arqos_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arctl2chk_m	Output	ardomain_m arsnoop_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arctl3chk_m	Output	archunken_m artagop_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
aridchk_m	Output	arid_m aridunq_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arlenchk_m	Output	arlen_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arloopchk_m	Output	arloop_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
armecidchk_m	Output	armecid_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
armmuchk_m	Output	armmusecsid_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
armmuchk_m	Output	armmuvalid_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
armmusidchk_m	Output	armmusid_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
armpamchk_m	Output	armpam_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arpbhachk_m	Output	arpbha_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arreadychk_s	Output	arready_s	ACE5-LiteDVM	None	N/A (output)	Inverted
aruserchk_m	Output	aruser_m	ACE5-LiteDVM	arvalid_m	N/A (output)	Inverted
arvalidchk_m	Output	arvalid_m	ACE5-LiteDVM	None	N/A (output)	Inverted
awaddrchk_m	Output	awaddr_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awakeupchk_m	Output	awakeup_m	ACE5-LiteDVM	None	N/A (output)	Inverted
awctl0chk_m	Output	awsize_m awburst_m awlock_m awprot_m awnse_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awctl1chk_m	Output	awregion_m awcache_m awqos_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awctl2chk_m	Output	awdomain_m awsnoop_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awctl3chk_m	Output	awatop_m awtagop_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
awidchk_m	Output	awid_m awidunq_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awlenchk_m	Output	awlen_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awloopchk_m	Output	awloop_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awmecidchk_m	Output	awmecid_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awmmuchk_m	Output	awmmusecsid_m awmmuvalid_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awmmusidchk_m	Output	awmmusid_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awmpamchk_m	Output	awmpam_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awpbhachk_m	Output	awpbha_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awreadychk_s	Output	awready_s	ACE5-LiteDVM	None	N/A (output)	Inverted
awstashlpidchk_m	Output	awstashlpid_m awstashlpiden_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awstashnidchk_m	Output	awstashnid_m awstashniden_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awuserchk_m	Output	awuser_m	ACE5-LiteDVM	awvalid_m	N/A (output)	Inverted
awvalidchk_m	Output	awvalid_m	ACE5-LiteDVM	None	N/A (output)	Inverted
bidchk_s	Output	bid_s bidunq_s	ACE5-LiteDVM	bvalid_s	N/A (output)	Inverted
bloopchk_s	Output	bloop_s	ACE5-LiteDVM	bvalid_s	N/A (output)	Inverted
breadychk_m	Output	bready_m	ACE5-LiteDVM	None	N/A (output)	Inverted
brespchk_s	Output	bresp_s	ACE5-LiteDVM	bvalid_s	N/A (output)	Inverted
buserchk_s	Output	buser_s	ACE5-LiteDVM	bvalid_s	N/A (output)	Inverted
bvalidchk_s	Output	bvalid_s	ACE5-LiteDVM	None	N/A (output)	Inverted
pmusnapshot_ack_chk	Output	pmusnapshot_ack	PMU Snapshot	None	N/A (output)	Inverted
qacceptchk_cg	Output	qacceptn_cg	LPI_CG	None	N/A (output)	Inverted
qacceptchk_pd	Output	qacceptn_pd	LPI_PD	None	N/A (output)	Inverted
qdenychk_cg	Output	qdeny_cg	LPI_CG	None	N/A (output)	Inverted
qdenychk_pd	Output	qdeny_pd	LPI_PD	None	N/A (output)	Inverted
rchunkchk_s	Output	rchunkv_s rchunknum_s rchunkstrb_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
rdatachk_s	Output	rdata_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
ridchk_s	Output	rid_s ridunq_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
rlastchk_s	Output	rlast_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
rloopchk_s	Output	rloop_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
rpoisonchk_s	Output	rpoison_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
rreadychk_m	Output	rready_m	ACE5-LiteDVM	None	N/A (output)	Inverted
rrespchk_s	Output	rresp_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
rtagchk_s	Output	rtag_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
ruserchk_s	Output	ruser_s	ACE5-LiteDVM	rvalid_s	N/A (output)	Inverted
rvalidchk_s	Output	rvalid_s	ACE5-LiteDVM	None	N/A (output)	Inverted
wdataatk_m	Output	wdata_m	ACE5-LiteDVM	wvalid_m	N/A (output)	Inverted
wlastchk_m	Output	wlast_m	ACE5-LiteDVM	wvalid_m	N/A (output)	Inverted
wpoisonchk_m	Output	wpoison_m	ACE5-LiteDVM	wvalid_m	N/A (output)	Inverted
wreadychk_s	Output	wready_s	ACE5-LiteDVM	None	N/A (output)	Inverted
wstrbchk_m	Output	wstrb_m	ACE5-LiteDVM	wvalid_m	N/A (output)	Inverted
wtagchk_m	Output	wtag_m wtagupdate_m	ACE5-LiteDVM	wvalid_m	N/A (output)	Inverted
wuserchk_m	Output	wuser_m	ACE5-LiteDVM	wvalid_m	N/A (output)	Inverted
wvalidchk_m	Output	wvalid_m	ACE5-LiteDVM	None	N/A (output)	Inverted

For more information about TBU I/O port protection, see:

- [A.6.2.1 TBU asynchronous protection](#) on page 346
- [A.6.2.2 TBU clock and reset protection](#) on page 347
- [A.6.2.3 TBU clock wake up protection](#) on page 348
- [A.6.2.4 TBU FMU inverted duplication](#) on page 348
- [A.6.2.5 TBU full duplication](#) on page 349
- [A.6.2.6 TBU full inverted duplication](#) on page 352
- [A.6.2.7 TBU DFT protection](#) on page 354

A.6.3 ADB protection types

ADBs do not have an enable parameter, as they feed into the F-Channel that feeds the TCU.

For more information on the ADB protection types, see:

- [A.6.3.1 ADB asynchronous protection](#) on page 361
- [A.6.3.2 ADB clock and reset protection](#) on page 361
- [A.6.3.3 ADB full duplication protection](#) on page 362
- [A.6.3.4 ADB full inverted duplication protection](#) on page 363
- [A.6.3.5 ADB DFT protection](#) on page 364
- [A.6.3.6 ADB synchronous parity protection](#) on page 364

A.6.3.1 ADB asynchronous protection

MMU-720AE contains asynchronous I/O protection signals for the ADB interfaces.

The following table describes the port name, signal direction, the signals protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for ADB asynchronous protection.

Table A-58: ADB asynchronous protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
adb_fackchk_m	Input	adb_fack_m	F-Channel	None	N/A	Inverted
adb_fackchk_s	Input	adb_fack_s	F-Channel	None	N/A	Inverted
clkqreqchk_m_i	Input	clkqreqnm_i	LPI_CG	None	N/A	Inverted
clkqreqchk_s_i	Input	clkqreqns_i	LPI_CG	None	N/A	Inverted
pwrqreqchk_s_i	Input	pwrqreqns_i	LPI_PD	None	N/A	Inverted
clkqacceptchk_m_o	Output	clkqacceptnm_o	LPI_CG	None	N/A (output)	Inverted
clkqactivechk_m_o	Output	clkqactivem_o	LPI_CG	None	N/A (output)	Inverted
clkqdenychk_m_o	Output	clkqdenym_o	LPI_CG	None	N/A (output)	Inverted
pwrqactivechk_m_o	Output	pwrqactivem_o	LPI_PD	None	N/A (output)	Inverted

For more information about ADB I/O port protection, see:

- [A.6.3.2 ADB clock and reset protection](#) on page 361
- [A.6.3.3 ADB full duplication protection](#) on page 362
- [A.6.3.4 ADB full inverted duplication protection](#) on page 363
- [A.6.3.5 ADB DFT protection](#) on page 364
- [A.6.3.6 ADB synchronous parity protection](#) on page 364

A.6.3.2 ADB clock and reset protection

MMU-720AE contains clock and reset I/O protection signals for the ADB interfaces.

The following table describes the port name, signal direction, the signals protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for ADB clock and reset protection.

Table A-59: ADB clock and reset protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
ackchk_m	Input	ack_m	N/A	None	N/A	Inverted
ackchk_s	Input	ack_s	N/A	None	N/A	Non-inverted
aresetnchk_m	Input	aresetnm	N/A	None	N/A	Inverted
aresetnchk_s	Input	aresetns	N/A	None	N/A	Non-inverted

For more information about ADB I/O port protection, see:

- [A.6.3.1 ADB asynchronous protection](#) on page 361
- [A.6.3.3 ADB full duplication protection](#) on page 362
- [A.6.3.4 ADB full inverted duplication protection](#) on page 363
- [A.6.3.5 ADB DFT protection](#) on page 364
- [A.6.3.6 ADB synchronous parity protection](#) on page 364

A.6.3.3 ADB full duplication protection

MMU-720AE contains full duplication I/O protection signals for the ADB interfaces.

The following table describes the port name, signal direction, the signals protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for ADB full duplication protection.

Table A-60: ADB full duplication protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
tfwdatachks	Input	tfwdatas	ADB	tfwvalids	N/A	Non-inverted
tfwdestchks	Input	tfwdests	ADB	tfwvalids	N/A	Non-inverted
tfwidchks	Input	tfwidss	ADB	tfwvalids	N/A	Non-inverted
tfwkeepchks	Input	tfwkeeps	ADB	tfwvalids	N/A	Non-inverted
tfwlstchks	Input	tfwlsts	ADB	tfwvalids	N/A	Non-inverted
tfwreadychk	Input	tfwreadym	Bridge Protection	None	N/A	Non-inverted
tfwstrbchks	Input	tfwstrbs	ADB	tfwvalids	N/A	Non-inverted
tfwuserchks	Input	tfwusers	ADB	tfwvalids	N/A	Non-inverted
tfwvalidchks	Input	tfwvalids	ADB	None	N/A	Non-inverted
trvdatachk	Input	trvdatam	Bridge Protection	trvvalidm	N/A	Non-inverted
trvdestchk	Input	trvdestm	Bridge Protection	trvvalidm	N/A	Non-inverted
trvidchk	Input	trvidm	Bridge Protection	trvvalidm	N/A	Non-inverted
trvkeepchk	Input	trvkeepm	Bridge Protection	trvvalidm	N/A	Non-inverted
trvlstchk	Input	trvlstm	Bridge Protection	trvvalidm	N/A	Non-inverted
trvreadychk	Input	trvreadys	ADB	None	N/A	Non-inverted
trvstrbchk	Input	trvstrbm	Bridge Protection	trvvalidm	N/A	Non-inverted
trvuserchk	Input	trvuserm	Bridge Protection	trvvalidm	N/A	Non-inverted
trvvalidchk	Input	trvvalidm	Bridge Protection	None	N/A	Non-inverted
wakeupchkm_i	Input	wakeupm_i	N/A	None	N/A	Non-inverted
wakeupchks_i	Input	wakeups_i	N/A	None	N/A	Non-inverted
tfwdatachk	Output	tfwdatam	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted
tfwdestchk	Output	tfwdestm	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted
tfwidchk	Output	tfwidm	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted
tfwkeepchk	Output	tfwkeepm	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
tfwlastchk	Output	tfwlastm	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted
tfwreadychks	Output	tfwreadys	ADB	None	N/A (output)	Non-inverted
tfwstrbchk	Output	tfwstrbm	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted
tfwuserchk	Output	tfwuserm	Bridge Protection	tfwvalidm	N/A (output)	Non-inverted
tfwvalidchk	Output	tfwvalidm	Bridge Protection	None	N/A (output)	Non-inverted
trvdatachks	Output	trvdatas	ADB	trvvalids	N/A (output)	Non-inverted
trvdestchks	Output	trvdests	ADB	trvvalids	N/A (output)	Non-inverted
trvidchks	Output	trvids	ADB	trvvalids	N/A (output)	Non-inverted
trvkeepchks	Output	trvkeeps	ADB	trvvalids	N/A (output)	Non-inverted
trvlastchks	Output	trvlasts	ADB	trvvalids	N/A (output)	Non-inverted
trvreadychk	Output	trvreadym	Bridge Protection	None	N/A (output)	Non-inverted
trvstrbchks	Output	trvstrbs	ADB	trvvalids	N/A (output)	Non-inverted
trvuserchks	Output	trvusers	ADB	trvvalids	N/A (output)	Non-inverted
trvvalidchks	Output	trvvalids	ADB	None	N/A (output)	Non-inverted
wakeupchkm_o	Output	wakeupm_o	N/A	None	N/A (output)	Non-inverted
wakeupchks_o	Output	wakeups_o	N/A	None	N/A (output)	Non-inverted

For more information about ADB I/O port protection, see:

- [A.6.3.1 ADB asynchronous protection](#) on page 361
- [A.6.3.2 ADB clock and reset protection](#) on page 361
- [A.6.3.4 ADB full inverted duplication protection](#) on page 363
- [A.6.3.5 ADB DFT protection](#) on page 364
- [A.6.3.6 ADB synchronous parity protection](#) on page 364

A.6.3.4 ADB full inverted duplication protection

MMU-720AE contains a full inverted duplication I/O protection signal for the ADB interfaces.

The following table describes the port name, signal direction, the signals protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for ADB full inverted duplication protection.

Table A-61: ADB full inverted duplication protection signal description

Port name	Direction	Signals protected	Protection style	Protection architecture	Qualifier signal	Enable parameter	Polarity
adb_freqchk	Output	adb_freqm	Full inverted duplication	F-Channel	None	N/A (output)	Inverted

For more information about ADB I/O port protection, see:

- [A.6.3.1 ADB asynchronous protection](#) on page 361
- [A.6.3.2 ADB clock and reset protection](#) on page 361

- [A.6.3.3 ADB full duplication protection](#) on page 362
- [A.6.3.5 ADB DFT protection](#) on page 364
- [A.6.3.6 ADB synchronous parity protection](#) on page 364

A.6.3.5 ADB DFT protection

MMU-720AE contains DFT I/O protection signals for the ADB interfaces.

The following table describes the port name, signal direction, the signals protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for ADB DFT protection.

Table A-62: ADB DFT protection signal descriptions

Port name	Direction	Signals protected	Protection style	Protection architecture	Qualifier signal	Enable parameter	Polarity
dftscanmode	Input	dftrstdisable	IPDS_DFT_RULE_F1	DFT	None	N/A	Non-inverted
dftscanmodechk	Input	dftrstdisable	IPDS_DFT_RULE_FLS2	DFT	None	N/A	Non-inverted

For more information about ADB I/O port protection, see:

- [A.6.3.1 ADB asynchronous protection](#) on page 361
- [A.6.3.2 ADB clock and reset protection](#) on page 361
- [A.6.3.3 ADB full duplication protection](#) on page 362
- [A.6.3.4 ADB full inverted duplication protection](#) on page 363
- [A.6.3.6 ADB synchronous parity protection](#) on page 364

A.6.3.6 ADB synchronous parity protection

MMU-720AE contains synchronous I/O protection signals for the ADB interfaces.

The following table describes the port name, signal direction, the signals protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for ADB synchronous parity protection.

Table A-63: ADB synchronous parity protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
pwrq_permit_deny_sar_i_chk	Input	pwrq_permit_deny_sar_i	Tieoff	None	N/A	Inverted
adb_freqchks	Output	adb_freqs	F-Channel	None	N/A (output)	Inverted
clkqacceptchks_o	Output	clkqacceptns_o	LPI_CG	None	N/A (output)	Inverted
clkqactivechks_o	Output	clkqactives_o	LPI_CG	None	N/A (output)	Inverted
clkqdenychks_o	Output	clkqdenys_o	LPI_CG	None	N/A (output)	Inverted
pwrqacceptchks_o	Output	pwrqacceptns_o	LPI_PD	None	N/A (output)	Inverted

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
pwrqactivechs_o	Output	pwrqactives_o	LPI_PD	None	N/A (output)	Inverted
pwrqdenychs_o	Output	pwrqdenys_o	LPI_PD	None	N/A (output)	Inverted

For more information about ADB I/O port protection, see:

- [A.6.3.1 ADB asynchronous protection](#) on page 361
- [A.6.3.2 ADB clock and reset protection](#) on page 361
- [A.6.3.3 ADB full duplication protection](#) on page 362
- [A.6.3.4 ADB full inverted duplication protection](#) on page 363
- [A.6.3.5 ADB DFT protection](#) on page 364

A.6.4 LPD protection types

LPDs do not have enable parameters, as their logic feeds into the TCU's F-Channel, which is controlled by ASYNC_IN.

For more information on the LPD I/O protection types, see:

- [A.6.4.1 LPD asynchronous protection](#) on page 365
- [A.6.4.2 LPD clock and reset protection](#) on page 366
- [A.6.4.3 LPD full inverted protection](#) on page 366
- [A.6.4.4 LPD DFT protection](#) on page 367

A.6.4.1 LPD asynchronous protection

MMU-720AE contains asynchronous I/O protection signals for the LPD interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for LPD asynchronous protection.

Table A-64: LPD asynchronous protection signal descriptions

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity	Additional information
ctrlreqchk	Input	ctrlreqn	LPI	None	N/A	Inverted	-
devqacceptchk	Input	devqacceptn	LPI	None	N/A	Inverted	Use NUM_QCHL to configure the number of outputs.
devqactivechk	Input	devqactive	LPI	None	N/A	Inverted	Use NUM_QCHL to configure the number of outputs.
devqdenychk	Input	devqdeny	LPI	None	N/A	Inverted	Use NUM_QCHL to configure the number of outputs.
ctrlqacceptchk	Output	ctrlqacceptn	LPI	None	N/A (output)	Inverted	-
ctrlqactivechk	Output	ctrlqactive	LPI	None	N/A (output)	Inverted	-

Port name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity	Additional information
ctrlqdenyck	Output	ctrlqdeny	LPI	None	N/A (output)	Inverted	-
devqreqchk	Output	devqreqn	LPI	None	N/A (output)	Inverted	Use NUM_QCHL to configure the number of outputs.

For more information about LPD I/O port protection, see:

- [A.6.4.2 LPD clock and reset protection](#) on page 366
- [A.6.4.4 LPD DFT protection](#) on page 367
- [A.6.4.3 LPD full inverted protection](#) on page 366

A.6.4.2 LPD clock and reset protection

MMU-720AE contains clock and reset I/O protection signals for the LPD interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for LPD clock and reset protection.

Table A-65: LPD clock and reset protection signal descriptions

Port Name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
clkchk	Input	clk	N/A	None	N/A	Non-inverted
resetrnchk	Input	resetrn	N/A	None	N/A	Non-inverted

For more information about LPD I/O port protection, see:

- [A.6.4.1 LPD asynchronous protection](#) on page 365
- [A.6.4.4 LPD DFT protection](#) on page 367
- [A.6.4.3 LPD full inverted protection](#) on page 366

A.6.4.3 LPD full inverted protection

MMU-720AE contains full inverted I/O protection signals for the LPD interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for LPD fully inverted protection.

Table A-66: LPD fully inverted protection signal description

Port Name	Direction	Signals protected	Protection architecture	Qualifier signal	Enable parameter	Polarity
lpd_freqchk	Output	lpd_freq	F-Channel	None	N/A (output)	Inverted

For more information about LPD I/O port protection, see:

- [A.6.4.1 LPD asynchronous protection](#) on page 365

- [A.6.4.2 LPD clock and reset protection](#) on page 366
- [A.6.4.4 LPD DFT protection](#) on page 367

A.6.4.4 LPD DFT protection

MMU-720AE contains DFT I/O protection signals for the LPD interfaces.

The following table describes the port name, signal direction, the signal protected, the protection architecture, qualifier signal, the enable parameter, and the port polarity for LPD DFT protection.

Table A-67: LPD DFT protection signal descriptions

Port name	Direction	Signals protected	Protection style	Protection architecture	Qualifier signal	Enable parameter	Polarity
dftscanmode	Input	dftstdisable	DFT	IPDS_DFT_RULE_F1	None	None	Non-inverted
		dftcgen		IPDS_DFT_RULE_FLS3			
dftscanmodechk	Input	dftstdisable	DFT	IPDS_DFT_RULE_FLS2	None	None	Non-inverted
		dftcgen		IPDS_DFT_RULE_FLS3			

For more information about LPD I/O port protection, see:

- [A.6.4.1 LPD asynchronous protection](#) on page 365
- [A.6.4.2 LPD clock and reset protection](#) on page 366
- [A.6.4.3 LPD full inverted protection](#) on page 366

Appendix B ELA signal descriptions

You can use the SIGNALGRP<n>, SIGQUAL<n>, and SIGCLKEN<n> signals of the TCU and TBU components to interface with an external Embedded Logic Analyzer (ELA).

The following folder contains machine-readable ELA mapping files:

mmu_720ae/logical/testbench/ela_mapping

B.1 TCU observation interfaces

The TCU observation interfaces use the SIGNALGRP<n>, SIGQUAL<n>, and SIGCLKEN<n> signals, where <n> represents the number in the signal name, to interface to an external CoreSight™ ELA-600 Embedded Logic Analyzer.

Signal group output ports are present on each component. However, only a subset is used.

The SIGCLKEN<n> signal is set to 1 for the signal groups in the 'Enabled signal groups' column in the following table. Groups that are not enabled have their SIGCLKEN<n> signals set to 0. If ela_enable is driven LOW, all SIGCLKEN<n> signals are set to 0.

The following table shows the signal group output ports that are valid for the TCU x1 and TCU x2.

Table B-1: Number of signal groups per module for the TCU

Component	SIGNALGRP Width	_qtw observation signals	_ptw observation signals	Parameter	Enabled signal groups	Total
TCU x1	128	Yes	No	TCUCFG_QTW_DATA_WIDTH <= 128	0, 1, 2, 3, 4, 5, 6, 10	8
				TCUCFG_QTW_DATA_WIDTH == 256	0, 1, 2, 3, 4, 5, 6, 7, 10, 11	10
				TCUCFG_QTW_DATA_WIDTH == 512	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11	12
TCU x2	128	Yes	Yes	TCUCFG_QTW_DATA_WIDTH <= 128	0, 1, 2, 3, 4, 5, 6, 10	8
				TCUCFG_QTW_DATA_WIDTH == 256	0, 1, 2, 3, 4, 5, 6, 7, 10, 11	10
				TCUCFG_QTW_DATA_WIDTH == 512	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11	12

The following tables show the SIGNALGRP<n> bits for the signal groups of the TCU x1 and TCU x2.

Some buses, if configured to be larger than the 128-bit signal group width, are spread across multiple groups. MMU-720AE delays sections of the signal by a cycle so that the ELA can sample 128-bit chunks of the data one cycle after another. The Number of cycles of delay column in

the table indicates the number of cycles, from when the signal is observable on an MMU-720AE interface, to when the signal is observable on the ELA observation interface.

The TCU ELA observation groups are split into two sets of 12 signal groups:

_qtw

ELA observation signals with the _qtw suffix capture the DTI and the QTW interface signals. The *_qtw observation signals are present in TCU x1 and TCU x2.

_ptw

ELA observation signals with the _ptw suffix capture the PTW interface signals. The *_ptw observation signals are present only in TCU x2.

The following table shows the TCU x1 observation interface signals.

Table B-2: TCU x1 observation interface signals

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
0	[127:0]	tdata_dti_dn[159:32]	{1'b0, (tready_dti_dn & tvalid_dti_dn), tready_dti_dn, tvalid_dti_dn}	1
1	[127:0]	tdata_dti_up[175:48]	{1'b0, (tready_dti_up & tvalid_dti_up), tready_dti_up, tvalid_dti_up}	1
2	[127:124]	Unused	-	-
	[99:68]	tdata_dti_dn[31:0]	{tvalid_dti_up, (tready_dti_up & tvalid_dti_up), tvalid_dti_dn, (tready_dti_dn & tvalid_dti_dn)}	0
	[67]	tlast_dti_dn		
	[66]	twakeup_dti_dn		
	[65]	tready_dti_dn		
	[64]	tvalid_dti_dn		
	[63:58]	tid_dti_dn		
	[57:10]	tdata_dti_up[47:0]		
	[9]	tlast_dti_up		
	[8]	twakeup_dti_up		
	[7]	tready_dti_up		
	[6]	tvalid_dti_up		
	[5:0]	tdest_dti_up		
3	[127:126]	Unused	-	-
	[125]	rpoison_qtw	{rvalid_qtw, (rready_qtw & rvalid_qtw), arvalid_qtw, (arready_qtw & arvalid_qtw)}	1
	[124]	rlast_qtw		
	[123]	rready_qtw		
	[122]	rvalid_qtw		
	[121:110]	rid_qtw		
	[109:94]	armecid_qtw		
	[93:82]	armpam_qtw		
	[81:80]	ardomain_qtw		
	[79:76]	arpbha_qtw		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
	[75:72]	arqos_qtw		
	[71]	arnse_qtw		
	[70]	arprot_qtw[1]		
	[69:66]	arcache_qtw		
	[65:63]	arsize_qtw		
	[62:14]	araddr_qtw		
	[13]	arready_qtw		
	[12]	arvalid_qtw		
	[11:0]	arid_qtw		
4	[127]	crready_qtw	{1'b0, (crready_qtw & crvalid_qtw), (bready_qtw & bvalid_qtw), (awready_qtw & awvalid_qtw)}	1
	[126]	crvalid_qtw		
	[125]	bready_qtw		
	[124]	bvalid_qtw		
	[123:112]	bid_qtw		
	[111:96]	awmecid_qtw		
	[95]	awakeup_qtw		
	[74]	awatop_qtw		
	[93:82]	awmpam_qtw		
	[81:80]	awdomain_qtw		
	[79:76]	awpbha_qtw		
	[75:72]	awqos_qtw		
	[71]	awnse_qtw		
	[70]	awprot_qtw[1]		
	[69:66]	awcache_qtw		
	[65:63]	awsize_qtw		
	[62:14]	awaddr_qtw		
	[13]	awready_qtw		
	[12]	awvalid_qtw		
	[11:0]	awid_qtw		
5	[127]	syscoack_qtw	{2'b00, (wready_qtw & wvalid_qtw), (acready_qtw & acvalid_qtw)}	1
	[126]	syscoreq_qtw		
	[125:62]	wstrb_qtw		
	[61]	wlast_qtw		
	[60]	wready_qtw		
	[59]	wvalid_qtw		
	[58]	acwakeup_qtw		
	[57:6]	acaddr_qtw		
	[5:2]	acvmidext_qtw		
	[1]	acready_qtw		
	[0]	acvalid_qtw		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
6	[127:0]	rdata_qtw[127:0]	{1'b0, (rready_qtw & rvalid_qtw), rready_qtw, rvalid_qtw}	1
7	[127:0]	rdata_qtw[255:128]	{1'b0, (rready_qtw & rvalid_qtw), rready_qtw, rvalid_qtw}	2
8	[127:0]	rdata_qtw[383:256]	{1'b0, (rready_qtw & rvalid_qtw), rready_qtw, rvalid_qtw}	3
9	[127:0]	rdata_qtw[511:384]	{1'b0, (rready_qtw & rvalid_qtw), rready_qtw, rvalid_qtw}	4
10	[127:0]	wdata_qtw[127:0]	{1'b0, (wready_qtw & wvalid_qtw), wready_qtw, wvalid_qtw}	1
11	[127:0]	wdata_qtw[255:128]	{1'b0, (wready_qtw & wvalid_qtw), wready_qtw, wvalid_qtw}	2

The following table shows the TCU x2 observation interface signals.

Table B-3: TCU x2 observation interface signals

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
0	[127:0]	Unused	-	-
1	[127:0]	Unused	-	-
2	[127:0]	Unused	-	-
3	[127:126]	Unused	-	-
	[125]	rpoison_ptw	{rvalid_ptw, (rready_ptw & rvalid_ptw), arvalid_ptw, (arready_ptw & arvalid_ptw)}	1
	[124]	rlast_ptw		
	[123]	rready_ptw		
	[122]	rvalid_ptw		
	[121:110]	rid_ptw		
	[109:94]	armecid_ptw		
	[93:82]	armpam_ptw		
	[81:80]	ardomain_ptw		
	[79:76]	arpbha_ptw		
	[75:72]	arqos_ptw		
	[71]	arnse_ptw		
	[70]	arprot_ptw[1]		
	[69:66]	arcache_ptw		
	[65:63]	arsize_ptw		
	[62:14]	araddr_ptw		
	[13]	arready_ptw		
	[12]	arvalid_ptw		
	[11:0]	arid_ptw		
4	[127:126]	Unused	-	-
	[125]	bready_ptw	{2'b00, (bready_ptw & bvalid_ptw), (awready_ptw & awvalid_ptw)}	1
	[124]	bvalid_ptw		
	[123:112]	bid_ptw		
	[111:96]	awmecid_ptw		
	[95]	awakeup_ptw		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
	[94]	awatop_ptw		
	[93:82]	awmpam_ptw		
	[81:80]	awdomain_ptw		
	[79:76]	awpbha_ptw		
	[75:72]	awqos_ptw		
	[71]	awnse_ptw		
	[70]	awprot_ptw[1]		
	[69:66]	awcache_ptw		
	[65:63]	awsize_ptw		
	[62:14]	awaddr_ptw		
	[13]	awready_ptw		
	[12]	awvalid_ptw		
	[11:0]	awid_ptw		
5	[127:67]	Unused	-	-
	[66:3]	wstrb_ptw	{1'b00, (rready_ptw & rvalid_ptw), 1'b0}	1
	[2]	wlast_ptw		
	[1]	wready_ptw		
	[0]	wvalid_ptw		
6	[127:0]	rdata_ptw[127:0]	{1'b0, (rready_ptw & rvalid_ptw), rready_ptw, rvalid_ptw}	1
7	[127:0]	rdata_ptw[255:128]	{1'b0, (rready_ptw & rvalid_ptw), rready_ptw, rvalid_ptw}	2
8	[127:0]	rdata_ptw[383:256]	{1'b0, (rready_ptw & rvalid_ptw), rready_ptw, rvalid_ptw}	3
9	[127:0]	rdata_ptw[511:384]	{1'b0, (rready_ptw & rvalid_ptw), rready_ptw, rvalid_ptw}	4
10	[127:0]	wdata_ptw[127:0]	{1'b0, (wready_ptw & wvalid_ptw), wready_ptw, wvalid_ptw}	1
11	[127:0]	wdata_ptw[255:128]	{1'b0, (wready_ptw & wvalid_ptw), wready_ptw, wvalid_ptw}	2

B.2 ACE-Lite TBU observation interfaces

The ACE-Lite TBU observation interface uses the SIGNALGRP<n>, SIGQUAL<n>, and SIGCLKEN<n> signals, where <n> represents the number in the signal name, to interface to an external CoreSight™ ELA-600 Embedded Logic Analyzer.

Signal group output ports are present on each component. However, only a subset is used.

The SIGCLKEN<n> signal is set to 1 for the signal groups in the 'Enabled signal groups' column in the following table. Groups that are not enabled have their SIGCLKEN<n> signals set to 0. If ela_enable is driven LOW, all SIGCLKEN<n> signals are set to 0.

The following table shows the signal group output ports that are valid for the ACE-Lite TBU.

Table B-4: Number of signal groups per module for the ACE-Lite TBU

Component	Parameter	Enabled signal groups	Total
ACE-Lite TBU		0, 1, 2, 3, 4	5

The following table shows the SIGNALGRP<n> bits for the signal groups of the ACE-Lite TBU.

Some buses, if configured to be larger than the 128-bit signal group width, are spread across multiple groups. MMU-720AE delays sections of the signal by a cycle so that the ELA can sample 128-bit chunks of the data one cycle after another. The Number of cycles of delay column in the table indicates the number of cycles, from when the signal is observable on an MMU-720AE interface, to when the signal is observable on the ELA observation interface.

Table B-5: ACE-Lite TBU observation interface signals

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
0	[127:119]	Unused	-	-
	[118:117]	awtagop_m	{1'b0, (awready_m & awvalid_m), awready_m, awvalid_m}	1
	[116:113]	awpbha_m		
	[112]	awuser_m		
	[111]	awidunq_m		
	[110:105]	awatop_m		
	[104:103]	awdomain_m		
	[102:99]	awqos_m		
	[98]	awnse_m		
	[97:95]	awprot_m		
	[94:91]	awcache_m		
	[90:89]	awburst_m		
	[88:86]	awsize_m		
	[85:78]	awlen_m		
	[77:74]	awregion_m		
	[73:34]	awaddr_m		
	[33]	awready_m		
	[32]	awvalid_m		
	[31:0]	awid_m		
1	[127:117]	Unused	-	-
	[116]	awstashlpiden_m	{1'b0, (awready_m & awvalid_m), awready_m, awvalid_m}	2
	[115:111]	awstashlpid_m		
	[110]	awstashniden_m		
	[109:99]	awstashnid_m		
	[98:83]	awmecid_m		
	[82:71]	awmpam_m		
	[70:47]	awmmusid_m		
	[46:45]	awmmusecsid_m		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
	[44]	awmmuvalid_m		
	[43:34]	awloop_m		
	[33]	awready_m		
	[32]	awvalid_m		
	[31:0]	awid_m		
2	[127:113]	Unused	-	-
	[112:111]	artagop_m	{1 'b0, (arready_m & arvalid_m), arready_m, arvalid_m}	1
	[110:107]	arpbha_m		
	[106]	aruser_m		
	[105]	aridunq_m		
	[104:103]	ardomain_m		
	[102:99]	arqos_m		
	[98]	arnse_m		
	[97:95]	arprot_m		
	[94:91]	arcache_m		
	[90:89]	arburst_m		
	[88:86]	arsize_m		
	[85:78]	arlen_m		
	[77:74]	arregion_m		
	[73:34]	araddr_m		
	[33]	arready_m		
	[32]	arvalid_m		
	[31:0]	arid_m		
3	[127:121]	Unused	-	-
	[99:84]	armecid_m	{1 'b0, (arready_m & arvalid_m), arready_m, arvalid_m}	2
	[83]	archunken_m		
	[82:71]	armpam_m		
	[70:47]	armmusid_m		
	[45:46]	armmusecsid_m		
	[44]	armmuvalid_m		
	[43:34]	arloop_m		
	[33]	arready_m		
	[32]	arvalid_m		
	[31:0]	arid_m		
4	[127:106]	Unused	-	-
	[105:98]	wpoison_m	{1 'b0, (wready_m & wvalid_m), (bready_m & bvalid_m), (rready_m & rvalid_m)}	1
	[97]	wlast_m		
	[96]	wready_m		
	[95]	wvalid_m		
	[94:85]	bloop_m		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
	[84]	bidunq_m		
	[83:82]	bresp_m		
	[81]	bready_m		
	[80]	bvalid_m		
	[79:48]	bid_m		
	[47]	ridunq_m		
	[46:37]	rloop_m		
	[36]	rlast_m		
	[35:34]	rresp_m		
	[33]	rready_m		
	[32]	rvalid_m		
	[31:0]	rid_m		

B.3 LTI TBU observation interfaces

The LTI TBU observation interface uses the SIGNALGRP<n>, SIGQUAL<n>, and SIGCLKEN<n> signals, where <n> represents the number in the signal name, to interface to an external CoreSight™ ELA-600 Embedded Logic Analyzer.

Signal group output ports are present on each component. However, only a subset is used.



For multi-LTI port TBUs, the signals in groups 1-6 that this interface reports are after multiple LTI interfaces are multiplexed together. The signals captured reflect the transaction that the TBU is processing in that clock cycle, regardless of which LTI port is responsible for that transaction. The LTI port to which that transaction belongs is not captured.

The SIGCLKEN<n> signal is set to 1 for the signal groups in the 'Enabled signal groups' column in the following table. Groups that are not enabled have their SIGCLKEN<n> signals set to 0. If ela_enable is driven LOW, all SIGCLKEN<n> signals are set to 0.

The following table shows the signal group output ports that are valid for the LTI TBU.

Table B-6: Number of SignalGroups per module for the LTI TBU

Component	Parameter	Enabled signal groups	Total
LTI TBU	When TBUCFG_LTI_LOOP_WIDTH ≤ 128 bits	0, 1, 2, 3, 5, 7, 8	7
	When TBUCFG_LTI_LOOP_WIDTH > 128 bit	0, 1, 2, 3, 4, 5, 6, 7, 8	9

The following table shows the SIGNALGRP<n> bits for the signal groups of the LTI TBU.

Some buses, if configured to be larger than the 128-bit signal group width, are spread across multiple groups. MMU-720AE delays sections of the signal by a cycle so that the ELA can sample 128-bit chunks of the data one cycle after another. The Number of cycles of delay column in the table indicates the number of cycles, from when the signal is observable on an MMU-720AE interface, to when the signal is observable on the ELA observation interface.

Table B-7: LTI TBU observation interface signals

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
0	[127:112]	lamecid	{3'b0, lavalid}	1
	[111:96]	latlbloc		
	[95]	laident		
	[94:93]	laflow		
	[92:89]	laattr		
	[88:85]	latrans		
	[84:33]	laaddr		
	[32]	lavalid		
	[31:0]	laid		
1	[127:119]	Unused	-	-
	[118:114]	laog	{3'b0, lavalid}	1
	[113]	laogv		
	[112:93]	lassid		
	[92]	lassidv		
	[91:68]	lasid		
	[67:66]	lasecsid		
	[65]	lammuv		
	[64:61]	lavc		
	[60:57]	latrans		
	[56]	lanse		
	[55:53]	laprot		
	[52:1]	laaddr		
	[0]	lavalid		
2	[127:117]	lrmecid	{3'b0, linvalid}	1
	[111:100]	lrmpam		
	[99:96]	lrhwattr		
	[95:92]	lrattr		
	[91:45]	lraddr		
	[44]	lrnse		
	[43:41]	lrprot		
	[40:38]	lrresp		
	[37]	lrctag		
	[36:33]	lrv		
	[32]	linvalid		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
	[31:0]	lrid		
3	[127:0]	laloop[127:0]	{3'b0, lavalid}	1
4	[127:0]	laloop[255:128]	{3'b0, lavalid}	2
5	[127:0]	lrlloop[127:0]	{3'b0, lrrvalid}	1
6	[127:0]	lrlloop[255:128]	{3'b0, lrrvalid}	2
7	[127:16]	Unused	-	-
	[15]	lcctag_7	{3'b0, lcvalid}	0
	[14]	lcvalid_7		
	[13]	lcctag_6		
	[12]	lcvalid_6		
	[11]	lcctag_5		
	[10]	lcvalid_5		
	[9]	lcctag_4		
	[8]	lcvalid_4		
	[7]	lcctag_3		
	[6]	lcvalid_3		
	[5]	lcctag_2		
	[4]	lcvalid_2		
	[3]	lcctag_1		
	[2]	lcvalid_1		
	[1]	lcctag_0		
	[0]	lcvalid_0		
8	[127:32]	Unused	-	-
	[31]	lmaskclose_7	{1'b0, lmopenreq, lmopenack, lmaskclose}	0
	[30]	lmactive_7		
	[29]	lmopenack_7		
	[28]	lmopenreq_7		
	[27]	lmaskclose_6		
	[26]	lmactive_6		
	[25]	lmopenack_6		
	[24]	lmopenreq_6		
	[23]	lmaskclose_5		
	[22]	lmactive_5		
	[21]	lmopenack_5		
	[20]	lmopenreq_5		
	[19]	lmaskclose_4		
	[18]	lmactive_4		
	[17]	lmopenack_4		
	[16]	lmopenreq_4		
	[15]	lmaskclose_3		

SIGNALGRP<n>	Bits	Signal name	SIGQUAL<n> 4'b{MSB..LSB}	Number of cycles of delay
	[14]	Imactive_3		
	[13]	Imopenack_3		
	[12]	Imopenreq_3		
	[11]	Imaskclose_2		
	[10]	Imactive_2		
	[9]	Imopenack_2		
	[8]	Imopenreq_2		
	[7]	Imaskclose_1		
	[6]	Imactive_1		
	[5]	Imopenack_1		
	[4]	Imopenreq_1		
	[3]	Imaskclose_0		
	[2]	Imactive_0		
	[1]	Imopenack_0		
	[0]	Imopenreq_0		

Appendix C Software initialization examples

We provide examples of how software can initialize and enable MMU-720AE.

C.1 Initializing the SMMU

Software must initialize MMU-720AE before you can use it.

MMU-720AE supports Secure, Realm, and Non-secure translation worlds. We define how to initialize Non-secure translation. The procedures for initializing Secure and Realm translation are similar, and require you to access the corresponding MMU-720AE Secure and Realm registers.



We do not describe how to create translation tables. For more information, see the [Arm® Architecture Reference Manual for A-profile architecture](#).

For more information about MMU-720AE initialization, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

C.1.1 Initializing Granule Protection Checks

By default, the ACESSEN control in SMMU_ROOT_CR0 disables all access. Before any transactions can pass through the SMMU, Granule Protection Checks (GPCs) must be configured.



Initialization of the GPCs is not required if the SMMU is configured to operate in legacy mode by setting the `TBUCFG_LEGACY_TZ_EN` and `TCUCFG_LEGACY_TZ_EN` parameters to 1, or by setting the `legacy_tz_en` tie-off signal to 1.

To configure GPCs, use the following procedure:

1. Configure the `SMMU_ROOT_GPT_BASE` register to point to the Physical Address of the L0 Granule Protection Table (GPT) tables
2. Configure `SMMU_ROOT_GPT_BASE_CFG` register with the appropriate settings for the GPT tables in the system, including setting the following:
 - The granule size for GPC, that is, PGS.
 - The shareability to use when accessing GPT, that is, SH.
 - The outer cacheability to use when accessing GPT, that is, ORGN.
 - The inner cacheability to use when accessing GPT, that is, IRGN.

- The range of memory protected by GPC, that is, PPS.
3. Write 1 to SMMU_S_INIT.INV_ALL and poll until the value returns to 0 to invalidate anything in the SMMU
 4. Set SMMU_ROOT_CR0.GPCEN to 1
 5. Poll SMMU_ROOT_CR0ACK until GPCEN is 1
 6. Set SMMU_ROOT_CR0.ACCESEN to 1
 7. Poll SMMU_ROOT_CR0ACK until ACCESEN is 1



Note

If you modify SMMU_ROOT_CR0.GPCEN or SMMU_ROOT_CR0.ACCESEN while translations are ongoing in the SMMU, the modifications might cause unpredictable behavior.

C.1.2 Allocating the Command queue

MMU-720AE uses the Command queue to receive commands. Software must allocate memory for the Command queue and configure the appropriate registers in the SMMU.

About this task

To allocate the Command queue, ensure that your software performs the following steps:

Procedure

1. Allocate memory for the Command queue.
2. Configure the Command queue size and base address by writing to the SMMU_CMDQ_BASE register.



Note

The queue size can affect how many bits of the SMMU_CMDQ_CONS and SMMU_CMDQ_PROD indices are writeable. It is therefore important that you perform this step before writing to SMMU_CMDQ_CONS and SMMU_CMDQ_PROD.

3. Set the queue read index in SMMU_CMDQ_CONS and the queue write index in SMMU_CMDQ_PROD to 0.



Note

Setting the queue read index and the queue write index to the same value indicates that the queue is empty.

The same process can be performed to enable the Realm and Secure event queues using the appropriate Realm or Secure equivalent registers.

C.1.3 Allocating the Event queue

MMU-720AE uses the Event queue to signal events. Software must allocate memory for the Event queue and configure the appropriate registers in the MMU.

About this task

To allocate the Event queue, ensure that your software performs the following steps:

Procedure

1. Allocate memory for the Event queue.
2. Configure the Event queue size and base address by writing to the `SMMU_EVENTQ_BASE` register.



The queue size can affect how many bits of the `SMMU_EVENTQ_CONS` and `SMMU_EVENTQ_PROD` indices are writeable. It is therefore important that you perform this step before writing to `SMMU_EVENTQ_CONS` and `SMMU_EVENTQ_PROD`.

3. Set the queue read index in `SMMU_EVENTQ_CONS` and the queue write index in `SMMU_EVENTQ_PROD` to 0.



Setting the queue read index and the queue write index to the same value indicates that the queue is empty.

The same process can be performed to enable the Realm and Secure event queues using the appropriate Realm or Secure equivalent registers.

C.1.4 Allocating the PRI queue

The PRI queue (PRIQ) can be Non-secure or Realm. PRIQ cannot be Secure. To enable the PRIQ, use `SMMU_(R_)CR0.PRIQEN`.

If `SMMU_(R_)CR0.SMMUEN` is 0, then `SMMU_(R_)CR0.PRIQEN` is RES0

The PRIQ works in a similar way to [C.1.3 Allocating the Event queue](#) on page 380.

C.1.5 Configuring the Stream table

The Stream table is a configuration structure in memory that uses a Context Descriptor (CD) to locate translation data for a transaction. Software must allocate memory for the Stream table, configure the table format, and populate the table with Stream Table Entries (STEs).

About this task

To configure the Stream table, ensure that your software performs the following steps:

Procedure

1. Allocate memory for the Stream table.
2. Configure the format and size of the Stream table by writing to SMMU_STRTAB_BASE_CFG.
3. Configure the base address for the Stream table by writing to SMMU_STRTAB_BASE.
4. Prevent uninitialized memory being interpreted as a valid configuration by setting STE.V = 0 for each STE to mark it as invalid.
5. Ensure that written data is observable to the SMMU by performing a Data Synchronization Barrier (DSB) operation.
If SMMU_IDR0.COHAAC = 0, the system does not support coherent access to memory for the TCU. In such cases, you might require extra steps to ensure that the SMMU can observe the written data.

C.1.6 Initializing the Command queue

Software must initialize the Command queue by enabling it and checking that the enable operation is complete.

About this task

To initialize the Command queue, ensure that your software performs the following steps:

Procedure

1. Enable the Command queue by setting the SMMU_CR0.CMDQEN bit to 1.
2. Check that the enable operation is complete by polling SMMU_CROACK until CMDQEN reads as 1.

C.1.7 Initializing the Event queue

Software must initialize the Event queue by enabling it and checking that the enable operation is complete.

About this task

To initialize the Event queue, ensure that your software performs the following steps:

Procedure

1. Enable the Event queue by setting the SMMU_CR0.EVENTQEN bit to 1.
2. Check that the enable operation is complete by polling SMMU_CROACK until EVENTQEN reads as 1.

C.1.8 Invalidating TLBs and configuration caches

Before use, the MMU-720AE TLBs and configuration cache structures must be invalidated by issuing commands to the Command queue. Alternatively, Secure software can invalidate all TLBs and caches with a single write.

To invalidate TLB entries, ensure that your software issues the appropriate command for the translation context.

To invalidate TLB entries for:

Non-secure EL1 contexts

Issue `CMD_TLBI_NSNH_ALL` to the appropriate command queue

Secure or Realm EL1 contexts

Issue `CMD_TLBI_NH_ALL` to the appropriate command queue

EL2 contexts

Issue `CMD_TLBI_EL2_ALL` to the appropriate command queue

EL3 contexts

Issue `CMD_TLBI_EL3_ALL` to the appropriate command queue, if in legacy mode



Note

In non-legacy mode, where RME is enabled, the EL3 stream world is not supported. The `CMD_TLBI_EL3_ALL` and `CMD_TLBI_EL3_VA` commands result in `CERROR_ILL`. The SMMU is not required to perform any invalidation on receipt of a broadcast TLBI for EL3.

GPC-only contexts



Note

Initialization of the GPCs is not required if the SMMU is configured to operate in legacy mode by setting the `TBUCFG_LEGACY_TZ_EN` and `TCUCFG_LEGACY_TZ_EN` parameters to 1, or by setting the `legacy_tz_en` tie-off signal to 1.

1. Read `SMMU_ROOT_TLBI_CTRL` to ensure that the `RUN` field reads as 0.

This indicates that no invalidates that this mechanism issues are outstanding.

2. Write 1 to the `ALL` field of `SMMU_ROOT_TLBI` and 0 to all other fields.

This issues a PA ALL invalidate into the SMMU.

3. Poll `SMMU_ROOT_TLBI_CTRL` until the `RUN` field reads as 0.

This indicates that the invalidate is complete.



Note

Commands to invalidate Secure TLB entries can only be issued through the Secure Command queue. For a system that implements two Security states, Secure software must issue the appropriate command to the Secure Command queue for the first TLB invalidation. If your system does not use Secure software, you can permit Non-secure software to access SMMU_S_INIT by using either `sec_override` or the [4.7.5 TCU_SCR register](#) on page 150.

To invalidate both the TCU configuration cache and the TBU combined configuration cache and TLB, issue the `CMD_CFGI_ALL` command.

To force all previous commands to complete, issue `CMD_SYNC`.

To invalidate all configuration caches and TLB entries for all translation regimes and Security states, ensure that Secure software:

1. Sets `SMMU_S_INIT.INV_ALL` to 1. The SMMU sets `SMMU_S_INIT.INV_ALL` to 0 after the invalidation completes.
2. Polls `SMMU_S_INIT.INV_ALL` to check it is set to 0 before continuing the SMMU configuration.

For more information about issuing commands to the Command queue, see the [Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3](#).

C.1.9 Creating a basic Context Descriptor

A Context Descriptor (CD) is a data structure in system memory. A CD defines how Stage 1 translation is performed. The SubstreamID is used to select the CD.

To create a CD, ensure that your software performs the following steps:

1. Allocate 64 bytes of memory for the CD.
2. Configure the CD fields according to the information in the following table.

Table C-1: Configuring the CD

Field	Description
AA64	Translation table format: 1 AArch64
EPD0	Enable translations for TTBO by setting EPD0 to 0.
TTB0	Base address of translation table 0.
TG0	Translation granule size for TTBO when <code>CD.AA64 = 1</code> .
IRO	Cacheability attribute to use for translation table walks to TTBO:
ORO	00 Non-cacheable. 01 Write-Back Cacheable, Read-Allocate Write-Allocate. 10 Write-through Cacheable, Read-Allocate.

Field	Description
SH0	Shareability of translation table walks to TTBO: 00 01 10 Non-shareable. Outer Shareable. Inner Shareable.
EPD1	If the StreamWorld supports split address spaces, enable table walks for TTB1.
ENDI	The endianness for the translation tables.
IPS	The IPA size when CD.AA64 = 1.
ASET	Defines whether the ASID values are shared with the ASID values of an Arm processor. Note: If you expect this context to receive broadcast TLB invalidation commands from a PE, set ASET to 0.
V	Valid CD. This field must be set to 1.

C.1.10 Creating a Stream Table Entry

Each Stream Table Entry (STE) configures how Stage 2 translation is performed, and how the Context Descriptor (CD) table can be found. The StreamID is used to select an STE.

To create an STE, ensure that your software performs the following steps:

1. Allocate 64 bytes of memory for the STE.
2. Set the STE.Config field as required for Stage 1 translation, Stage 2 translation, or translation bypass:

0b000	No traffic can pass through the MMU. An abort is returned.
0b100	Stage 1 and Stage 2 bypass.
0b101	Stage 1 translation Stage 2 bypass.
0b110	Stage 1 bypass Stage 2 translation.
0b111	Stage 1 and Stage 2 translation.

3. If Stage 1 translation is enabled, you can set the following fields:

STE.S1CDMax	Controls whether STE.S1ContextPtr points to a single CD or a CD table.
STE.S1Fmt	If STE.S1CDMax > 0, configures the format of the CD table.
STE.S1ContextPtr	Contains a pointer to either a CD or a CD table. If Stage 2 translation is enabled, this pointer is an intermediate physical address (IPA), otherwise it is an untranslated physical address PA.

4. If Stage 2 translation is enabled, you can set the following fields:

STE.S2TTB	Points to the Stage 2 translation table base address.
STE.S2PS	Contains the PA size of the stage 2 PA range.
STE.S2AA64	Indicates whether the Stage 2 tables are AArch32 or AArch64 format. MMU-720AE supports only AArch64 format.

STE.S3ENDI	Set this field to the required endianness for the stage 2 translation tables.
STE.S2AFFD	Disable Access Flag faults for Stage 2 translation.
STE.S2TG	0b00: 4KB.
STE.S2IRO	0b00: Non-cacheable.
and	
STE.S2OR0	
STE.S2SH0	
STE.S2VMID	Contains the VMID associated with these translations.

C.2 Enabling the SMMU

Software can enable the SMMU by writing to SMMU_CR0 after the Stream table is populated.

About this task

To enable the SMMU, carry out the following procedure.

Procedure

1. Ensure that all Stream table entries are populated in memory.
2. Set the SMMU_CR0.SMMUEN bit to 1.
3. Check that the enable operation is complete by polling SMMU_CR0ACK until SMMUEN reads as 1.

Next steps

The same procedure can be used to enable the SMMU for Realm and Secure security levels, when the associated configuration of STE and CD has been performed and by using the appropriate Realm or Secure equivalent registers.

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant

export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in the Arm documents.

Product status

All products and Services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is Final, that is for a developed product.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0002-04	8 August 2025	Non-Confidential	Second EAC release for r0p2
0002-03	21 March 2025	Non-Confidential	First EAC release for r0p2
0001-02	16 December 2024	Non-Confidential	First EAC release for r0p1
0000-01	14 November 2024	Confidential	First EAC release for r0p0

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in [Document release information](#) on page 389.

Table 2: Differences between issue 0002-03 and issue 0002-04

Change	Location
Updated the <i>Lockstep and RAM protection in the TCU</i> figure	3.2 Logic duplication on page 111
Corrected the text about LTI interface inputs having a two-cycle clock delay applied externally.	A.6.2.5 TBU full duplication on page 349

Change	Location
Corrected the text about DTI interface inputs having a two-cycle clock delay applied externally.	A.6.1.4 TCU full duplication on page 336

Table 3: Differences between issue 0001-02 and issue 0002-03

Change	Location
Improved descriptions	Throughout the document
Corrected the descriptions of various caches	2.1 Translation Control Unit on page 23
Improved descriptions	2.2 Translation Buffer Unit on page 26 2.4.1.2 TCU Page Table Walk interface on page 33
Added more information	2.4.1.8 TCU SYSCO signaling on page 35
Added more information about the DTI versions in the <i>DTI connections the TCU accepts</i> section	2.5.1 Distributed Translation Interface on page 45
Corrected descriptions	2.5.2.2 SMMU TCU events on page 48 2.5.2.3 SMMU TBU events on page 52
Added to the description relating to the <code>TBUCFG_DIRECT_IDX</code> configuration parameter	2.5.4 Main TLB direct indexing and main TLB direct partitioning on page 57
Added a new <i>Interface used</i> column to the <i>TCU support for read transactions</i> table	2.5.8 TCU transaction handling on page 67
Added the following properties to the 'Issue J' section of the 'AXI property support' table: - REGION_Present - WLAST_Present - WSTRB_Present	2.6.2.1 ACE-Lite property support on page 82
Added a new <i>NoStreamID</i> (<code>AxMMUVALID = 0</code>) transaction subsection	2.6.2.4 Attribute handling on page 84
Updated some property descriptions	2.6.3 Local Translation Interface implementation on page 92
Added a new <i>MPAM and NoStreamID</i> section	2.6.4 MPAM implementation on page 93
Changed various field descriptions	2.6.4.1 TCU MPAM on page 94 2.6.4.2 TBU MPAM on page 98
Changed the description of the <code>TCUCFG_HZU_DEPTH</code> parameter	2.7.1 TCU I/O and buffer configuration parameters on page 103
Text updated to reflect that parity checks on qactive must be disabled for 2 clock cycles out of reset.	3.1 Interface protection on page 110
Updated the DTI duplication image.	3.2.1 DTI duplication on page 112
Removed topic on how to Configure which errors are critical	-
Rearranged the order of some register summary categories so that everything appears in address offset order and there is no duplication with registers appearing in more than one summary table.	4. Programmers model for MMU-720AE on page 119
Updated the <code>SMMU_PIDR0</code> , Peripheral ID0 value to <code>0x9E</code>	4.5 TCU component and peripheral ID registers on page 139

Change	Location
Changed various field descriptions	4.6.5 PMU ID registers on page 142
- Corrected some values in the 'Realm register' column of the 'SCR and RCR override of register ownership' table - Improved the descriptions of the behavior of Secure and Non-secure accesses to the TCU_RCR and TCU_SCR registers.	4.7 TCU microarchitectural registers on page 143
Corrected register bit field descriptions	4.7.2 TCU_QOS register on page 147
Corrected the address offsets	4.10 TCU_CTRL_AUX<n> registers on page 211
Updated the SMMU_PIDR0, Peripheral ID0 value to 0x9F	4.14 TBU component and peripheral ID registers on page 221
Corrected various register bit field descriptions	4.14 TBU component and peripheral ID registers on page 221 4.15.5 PMU ID registers on page 223
- Improved the descriptions of the behavior of Secure and Non-secure accesses to the TBU_RCR and TBU_SCR registers. - Added a table showing the 'SCR and RCR override of register ownership'.	4.16 TBU microarchitectural registers on page 224
Corrected various signal names and their descriptions	A.1.3 TCU PTW interface signals on page 293 A.1.4 TCU programming interface signals on page 296 A.1.11 TCU event interface on page 301 A.1.12 TCU tie-off signals on page 302
- Updated the descriptions of the arnse_s and awnse_s signals - Removed some information from the aruser_s and awuser_s signal descriptions and put them in the LATLBLOC signal description. - Made other small corrections and improvements	A.2.2 TBU TBS interface signals on page 306
Corrected descriptions and removed duplicate signals	A.2.3 TBU TBM interface signals on page 312
Corrected various signal names and their descriptions	A.2.9 TBU tie-off signals on page 320
Added new LTI signals section	A.5 LTI signals on page 331
Changed protection on the dftscanmode and dftscanmodechk ports from N/A to Non-inverted.	A.6.1.7 TCU DFT protection on page 341 A.6.2.7 TBU DFT protection on page 354 A.6.3.5 ADB DFT protection on page 364 A.6.4.4 LPD DFT protection on page 367
Corrected the steps	C.1.1 Initializing Granule Protection Checks on page 379

Table 4: Differences between issue 0000-01 and issue 0001-02

Change	Location
Updated address width for this interface to 32-bit.	2.4.1.3 TCU PROG interface on page 33
Updated the TCU_FMU_ERRSATAUS and TCU_FMU_SMEN register descriptions and diagrams.	4.20.3 TCU_FMU_ERRSTATUS register on page 274 and 4.20.5 TCU_FMU_SMEN register on page 277
Reset value for the TCU_FMU_SMEN register changed from 0x000001ff to 0x000000ff.	
Removed the MTI MISC input value and renamed the 0x4 value to F-Channel input error for the TCU_FMU_ERRGEN IERR register field.	4.20.6 TCU_FMU_ERRGEN register on page 278
Changed the TBU_FMU_ERRGEN offset value from 0x10F08 to 0x10F04	4.21 TBU FMU registers summary on page 280
Updated the TBU_FMU_ERRSTATUS and TBU_FMU_SMEN register descriptions and diagrams.	4.21.3 TBU_FMU_ERRSTATUS register on page 283 and 4.21.5 TBU_FMU_SMEN register on page 286
Removed the LTI input error, LDB error, Q-Channel error, and MISC input error descriptions and values for the TBU_FMU_ERRGEN IERR register field.	4.21.6 TBU_FMU_ERRGEN register on page 287
Updated the paddr_prog signal width to 32-bit.	A.1.4 TCU programming interface signals on page 296

Table 5: Issue 0000-01

Change	Location
First release	-

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
<i>italic</i>	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
monospace	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
monospace <u>underline</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.

Convention	Use
<and>	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>
SMALL CAPITALS	Terms that have specific technical meanings as defined in the Arm® Glossary. For example, IMPLEMENTATION DEFINED , IMPLEMENTATION SPECIFIC , UNKNOWN , and UNPREDICTABLE .



We recommend the following. If you do not follow these recommendations your system might not work.



Your system requires the following. If you do not follow these requirements your system will not work.



You are at risk of causing permanent damage to your system or your equipment, or of harming yourself.



This information is important and needs your attention.



This information might help you perform a task in an easier, better, or faster way.



This information reminds you of something important relating to the current content.

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the tables below provides direct access to the online version of the document.

Arm product resources	Document ID	Confidentiality
Arm® CoreLink™ LPD-500 Low Power Distributor Technical Reference Manual	100361	Non-Confidential
Arm® CoreSight™ ELA-600 Embedded Logic Analyzer Technical Reference Manual	101088	Non-Confidential
Arm® CoreSight™ System-on-Chip SoC-600 Technical Reference Manual	100806	Non-Confidential
Arm® Neoverse™ CMN-700 Coherent Mesh Network Technical Reference Manual	102308	Non-Confidential
Arm® CoreLink™ ADB-400 AMBA® Domain Bridge User Guide	DUI 0615	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Automotive Product Security Support Supplement	109912	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Configuration and Integration Manual	109746	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Dependent Failure Analysis	110104	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Development Interface Report	109748	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Failure Modes Effects and Diagnostic Analysis Report	110105	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Release Note	109749	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Safety Manual	109747	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Security Summary Report	109913	Confidential
Arm® CoreLink™ MMU-720AE System Memory Management Unit Verification Summary Report	110106	Confidential
GIC MSI Delivery Interface	ARM AES 0019A	Confidential

Arm architecture and specifications	Document ID	Confidentiality
Arm® System Memory Management Unit Architecture Specification - SMMU architecture version 3	IHI 0070	Non-Confidential
Learn the Architecture - SMMU Software Guide	109242	Non-Confidential
AMBA® AHB Protocol Specification	IHI 0033	Non-Confidential
AMBA® DTI Protocol Specification	IHI 0088	Non-Confidential
AMBA® LTI Protocol Specification	IHI 0089	Non-Confidential
AMBA® Low Power Interface Specification	IHI 0068	Non-Confidential
Arm® Architecture Reference Manual for A-profile architecture	DDI 0487	Non-Confidential
Arm® Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification	IHI 0099A.a	Non-Confidential

Arm architecture and specifications	Document ID	Confidentiality
AMBA® AXI Protocol Specification	IHI 0022	Non-Confidential
AMBA® AXI-Stream Protocol Specification	IHI 0051	Non-Confidential
AMBA® APB Protocol Specification	IHI 0024	Non-Confidential
Arm® Server Base System Architecture 7.1	DEN 0029	Non-Confidential